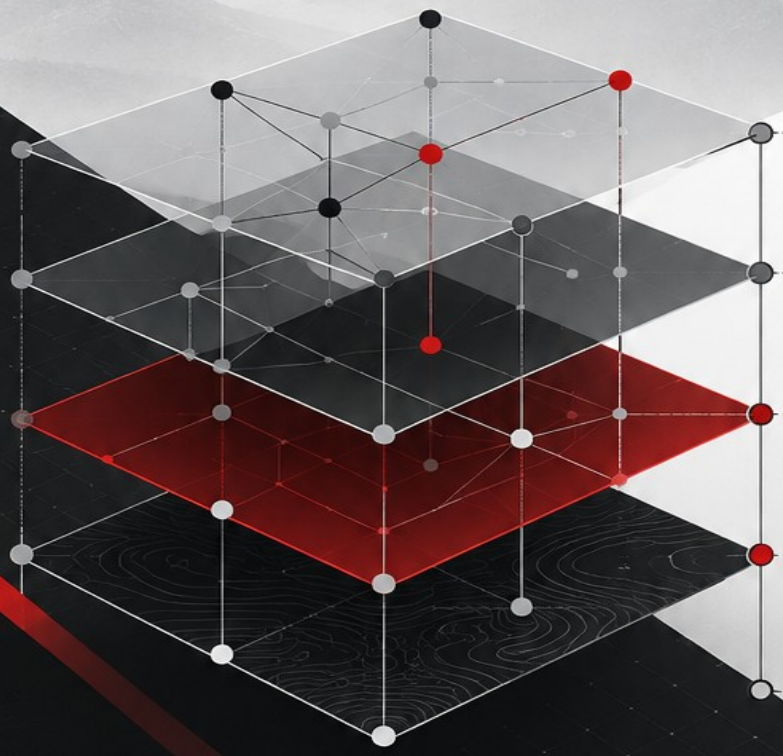
OPEN EDITION

The Security-Engineering Convergence

How Systems Security Engineering Unifies Technology, Security, and Mission Outcomes

ALIGN
INTEGRATE
ASSURE
DELIVER

PEOPLE
SYSTEMS
SECURITY
INTEGRATION
ASSURANCE
OUTCOMES



COMPLEX
SYSTEMS.
TRUSTED
CAPABILITIES.

PETER J. HILLIER

HILLIER INFORMATION PROTECTION SOLUTIONS

ENGINEERING TRUSTED CAPABILITIES

Publication and edition information

© 2026 Peter J. Hillier. Publisher: Hillier Information Protection Solutions Inc.

The Security-Engineering Convergence: How Systems Security Engineering Unifies Technology, Security, and Mission Outcomes. First digital edition — publication candidate (not yet approved for public release).

Reserved Zenodo DOI: 10.5281/zenodo.23284495 | https://doi.org/10.5281/zenodo.23284495 (the DOI will become publicly resolvable when the Zenodo record is published).

Intended licence: Creative Commons Attribution 4.0 International (CC BY 4.0) for author-owned material, subject to completion of the publication rights-clearance process. Licence terms: https://creativecommons.org/licenses/by/4.0/

Third-party materials, trademarks and separately identified content are not automatically licensed under CC BY 4.0. Their reuse is governed by their respective rights and permissions.

Citation and edition information

Suggested citation upon publication: Hillier, P. J. (2026). The Security-Engineering Convergence: How Systems Security Engineering Unifies Technology, Security, and Mission Outcomes (1st digital ed.). Hillier Information Protection Solutions Inc. https://doi.org/10.5281/zenodo.23284495

ISBN: application submitted; number pending. An ISBN is not required for this initial open-access digital edition. If assigned later, it will be added through a documented metadata and file revision.

Release controls: this is a publication candidate. Technical examples and the Appendix C Python abstraction remain educational representations, not verification evidence for a deployed system. Third-party rights clearance, final editorial approval and author sign-off remain separate requirements.

Companion material and versioning

The first approved digital edition and companion material will be published together through the GitHub release and Zenodo archival record. Edition changes, file hashes and subsequent ISBN updates will be recorded in the public version history.

Dedication

For those who build, defend, and question the systems that shape our world.

Contents

Author's preface....6

How to use this book....8

PART I — A SHARED FOUNDATION

1. Security as an engineering responsibility....10

2. Systems, boundaries and operational context....16

3. Compliance, controls, risk and engineering....22

4. One life cycle, integrated responsibilities....29

PART II — FRAME THE PROBLEM AND PLAN THE EVIDENCE

5. Mission, stakeholders, assets and protection needs....36

6. Adversity, threats and loss scenarios....44

7. Requirements, policy, constraints and allocation....52

8. Planning assurance and technical evaluation....64

PART III — ENGINEER THE SOLUTION

9. Trustworthy secure architecture and design....73

10. Interfaces, dependencies and composition....83

11. System analysis, MBSE and trade studies....92

12. Cyber resilience and controlled degradation....111

PART IV — REALIZE AND EVALUATE THE SYSTEM

13. Implementation, production and enabling systems....121

14. Integration and emergent behaviour....130

15. Verification and validation....138

16. Human-system design and transition readiness....148

PART V — ACQUIRE, ASSURE AND DECIDE

17. Acquisition and contractual engineering....157

18. Supplier contributions and supply-chain assurance....166

19. Evidence, assurance cases and technical challenge....176

20. Risk decisions, acceptance and authorization....188

PART VI — SUSTAIN TRUSTWORTHY CAPABILITY

21. Operations, response and continuing support....198

22. Configuration, change, maintenance and drift....207

23. Legacy, COTS and cloud-dependent systems....217

24. Retirement, transfer and disposal....226

PART VII — LEAD AND SCALE THE CAPABILITY

25. Technical management and engineering control....235

26. Organizational capability, integration and competence....244

27. Systems-of-systems and critical infrastructure....253

28. AI, autonomy and human–AI engineering....262

REFERENCE MATERIAL

Acknowledgements....276

About the author 277

Appendix A — MBSE languages, tools and exchange....278

Appendix B — Worked-case and record guide....281

Appendix C — Completing a bounded engineering case....286

Appendix D — A cyber-physical inspection vehicle....296

Appendix E — Protecting information flow without losing the support function....304

Appendix F — Resilience when a trusted source becomes suspect....307

Appendix G — Practitioner worksheets....309

Subject index....311

Glossary....314

References....326

Table and figure directory....334

Contents entries and table-directory entries are internal links. Page references describe this draft's rendered layout; use Word's heading navigation after repagination.

Author’s preface

A security team can implement the required controls, resolve assessment findings and maintain a disciplined monitoring programme while still being asked to protect a system whose most consequential design assumptions have never been examined. An engineering team can deliver a capable product while leaving unclear how that product should behave when a supporting service is unavailable, an administrator misuses legitimate authority, or a supplier changes an interface. Neither situation is resolved simply by asking one team to work harder. The work must connect.

This book is about that connection. Its subject is Systems Security Engineering: the security specialty within systems engineering. Its purpose is to help readers connect the useful things a system must accomplish with the protection it needs, the engineering decisions that provide that protection, and the evidence that supports reliance on the result. Security belongs in the work of defining, designing, building, acquiring, evaluating, operating and sustaining systems. It is not adequately addressed by a separate approval activity conducted after consequential choices have already been made.

My professional work has brought me into contact with the people who build capabilities, the people who acquire and govern them, and the people who must keep them functioning in operational conditions. Their perspectives are different for good reasons. A designer needs a workable specification. An operator needs a service that can be used and recovered. A security practitioner needs to understand exposure and protection. An acquisition professional needs obligations that suppliers can price, perform and demonstrate. A decision-maker needs an honest account of what is ready, what remains uncertain and what accepting that uncertainty would mean.

Those perspectives become a problem when the connections among them are left implicit. A requirement can survive in a document while disappearing from the design. A control can be implemented on one interface while another path remains outside the analysis. An operational workaround can reveal a need the original design never addressed. A supplier’s assessment can be useful without answering the question the receiving organization is actually asking. The task is not to declare one perspective superior. It is to make their contributions coherent at the level of the system. The professions also bring different vocabulary, methods and measures of success; the point is to connect their valid contributions, not require any one community to abandon its expertise.

For readers from information technology and cybersecurity, the starting material should be recognizable: service maps, identity policies, incidents, configuration baselines, control assessments, recovery plans and supplier records. These are not obstacles to engineering. They are potential inputs to it. The chapters explain how to use them to frame system questions, develop requirements, evaluate alternatives and support accountable decisions. Existing operational knowledge deserves a place in the design conversation, not merely in the lessons-learned report after deployment.

For readers from engineering, the challenge is to make the security contribution substantive. Naming security as a quality attribute is not the same as examining the ways a system could lose a critical asset or capability. The specialist work has to affect architecture, interfaces, implementation, evaluation and support. It must also be coordinated with safety, reliability,

human factors and other specialties. A protection measure that creates an unacceptable operational consequence has introduced another engineering problem; it has not completed the first one.

The book retains a demanding view of evidence, but not an impossible promise of certainty. A successful test supports a conclusion about specified conditions. An analysis depends on its model and assumptions. An assurance argument must address the limits of both. The objective is justified confidence, not a vocabulary that disguises uncertainty. Professional judgement remains necessary, including the judgement to conclude that the available support is not yet sufficient.

I intend this work to be freely accessible because the knowledge needed to build and sustain trustworthy systems should be easier to obtain. Open sharing is useful only if the material is also usable: readers need explanations they can question, examples they can adapt and sources they can examine. The book therefore offers methods and worked reasoning rather than a new claim of universal authority. It does not replace applicable standards, competent specialist review or the responsibilities of those authorized to make decisions.

The invitation is straightforward. Bring what you already know about systems, technology, operations or security, and examine how that knowledge connects to the rest of the engineering effort. We will begin with the purpose of a system, follow the work required to make it worthy of reliance, and return to earlier decisions when experience shows that the system or its context has changed. Worked engineering cases, model-based views, cyber-physical analysis and reusable worksheets show how to carry those connections into practical work.

Peter J. Hillier

How to use this book

Two entry routes, one engineering explanation

This book is written for readers who arrive with different professional experience. An IT or cybersecurity practitioner may understand identity services, vulnerability management and operational change without having used formal requirements allocation or a systems-engineering management plan. An engineer may be comfortable with interfaces, trade studies and verification but less familiar with adversarial use of administrative functions or the evidence produced by a security assessment. Neither reader needs to abandon that experience. Both need to understand what the other contribution establishes and where further work is necessary.

The opening chapters establish shared concepts; later chapters develop the methods. Central terms appear in the consolidated glossary. The chapter order is a learning sequence: acquisition, human factors, assurance and support planning begin when their contributions are needed, not when the reader reaches their dedicated chapters.

Read the examples as reasoning, not ready-made designs

The recurring example concerns remote administration of a logistics dispatch service. It develops through explicit hypothetical assumptions, candidate designs and planned evaluations. Appendix C then completes one separately identified branch using declared fictional decisions and a stipulated evidence dossier. Its small executable abstraction has its own recorded test results, which do not stand in for system or human-task evidence. Appendix D demonstrates a different, cyber-physical problem. These distinctions allow the reader to follow completion without confusing an illustrative project with one actually deployed.

Follow what changes in the case as the questions become more precise. First identify the useful service and the decision. Then examine the boundary, interfaces, operating modes and dependencies. Only then consider what an obligation requires of the system and how that requirement could be satisfied and evaluated. The value lies in the intermediate reasoning, including the points at which the information is insufficient to choose responsibly.

Exercises include explained answers that demonstrate a defensible approach, not a universally preferred architecture. For real work, replace the illustrative assumptions with evidence and decisions appropriate to the system, its users and its obligations.

Use the sources without confusing guidance with authority

Numbered citations identify public technical sources. The explanations, teaching records and examples are original synthesis unless a passage is explicitly identified otherwise. References provide publication editions and stable internal locators where practicable. A source's terminology may differ from the book's teaching vocabulary; those differences matter when applying the source itself.

Citing a publication does not make every provision in it binding on a project. Applicable obligations arise through the project's actual legal, regulatory, contractual and organizational arrangements. Conversely, a general discussion of engineering judgement does not authorize a

practitioner to waive an obligation. Resolve applicability and exception authority through the responsible organization.

The word “shall” appears in example requirements to illustrate an obligation on a specified subject. It does not make the book a standard. Terms such as “should” ordinarily express a recommended practice or a matter for reasoned consideration. Where an example lacks an agreed operating condition or acceptance threshold, that gap is identified rather than hidden behind stronger wording.

Carry the work back into an existing process

Carry the result back into an existing process: a better design-review question, a clearer requirement, a revised change record or a justified limit on an assurance claim. Reuse controlled records that preserve scope, rationale, evidence and responsibility. A separate security document is needed only when it adds a useful contribution.

Choose a route through the book

IT and cybersecurity practice. Begin with Chapters 1–4, then follow requirements in Chapter 7, architecture and MBSE in Chapters 9–12, evaluation in Chapter 15, and operations and change in Chapters 21–23. Appendix C connects these methods in one bounded completion branch.

Engineering practice. Use Chapters 1–3 to establish the security/compliance relationship, Chapters 5–12 for needs and design, and Chapters 13–20 for realization, evidence and decisions. Appendix D adds a physical-control problem whose dynamics cannot be resolved by transaction semantics alone.

Acquisition and leadership. After the shared foundation, Chapters 17–20 develop obligations, supplier contributions, assurance and authority; Chapters 25–28 develop resources, competence, shared systems and AI. Return to the detailed methods whenever a proposed decision depends on a technical assumption.

Exercises and evidence. Chapter exercises retain their explained answers. The base case, the stipulated CB-09 completion dossier, the CP-09 physical example and the actually executed Python abstraction have different evidentiary status. Appendix B and the companion README make those distinctions explicit.

Finding a topic. The subject index uses section identifiers, so its locators remain useful when the manuscript is repaginated. The glossary supplies concise meanings; the index points to the methods and examples that use them.

PART I — A SHARED FOUNDATION

1. Security as an engineering responsibility

1.1 Start with the capability people need

Consider a logistics dispatch service used to issue delivery instructions to field teams. Its value is not that its servers are running or that its administrative gateway is correctly configured. Its value is that authorized people can issue timely, correct instructions and that the recipients can act on them. Availability, information integrity, usable interfaces and dependable support all contribute to that outcome. Security concerns arise when a person, event or system condition can undermine the capability or cause another unacceptable loss.

Suppose an organization proposes a new remote-administration service for this application. The security team identifies an applicable requirement for multi-factor authentication, often abbreviated MFA. The infrastructure team proposes a gateway. The application team supplies a management interface. The service owner expects urgent maintenance to remain possible during an outage. Each contribution is reasonable. The difficulty is that their combined adequacy cannot be inferred from their individual reasonableness.

An authenticated administrator may still lack permission to alter dispatch rules. A gateway may protect the browser interface while leaving an automated management interface untreated. A response that denies new access when identity services are unavailable may preserve one protective property while delaying restoration of the dispatch service. These are questions about the behaviour and use of the combined system. They require the participants to examine their relationships, not merely complete their individual tasks.

Systems Security Engineering begins by making those relationships explicit. The engineering problem includes what must work as well as what must be prevented. A service that cannot be used is not made suitable simply because it is difficult to attack. Equally, a service that performs its useful function while permitting unacceptable manipulation is not adequate merely because it is available.

1.2 The place of the security specialty

Systems engineering integrates the technical contributions needed to develop and sustain a coherent system. It brings requirements, architecture, design, realization and evaluation into a common effort, taking account of competing needs and constraints. Security is one of the specialties that contributes to that effort. It does not replace the integrating discipline, and it does not assume the responsibilities of every other specialty. NIST's treatment of Systems Security Engineering explicitly places its principles and activities within systems engineering. [1, Section 3.10]

In this book, Systems Security Engineering (SSE) is the security-focused engineering work used to establish protection needs, develop and evaluate system responses, and support justified reliance throughout the life cycle. This explanatory formulation emphasizes contributions to understanding and realizing the system, regardless of where the security engineer sits in the organization; it is not a quotation from a standard.

That work can include examining privilege propagation across interfaces, identifying the consequences of corrupt configuration data, comparing alternative administrative architectures, allocating protective functions, reviewing implementation choices and determining what would constitute adequate evaluation. It can also include identifying what cannot presently be demonstrated. A finding that the design depends on an unexamined service can be an important engineering result even before a replacement design exists.

Specialist integration does not require everyone to become interchangeable. A safety specialist may identify a hazardous operating condition; a security specialist may examine whether an adversary can induce it; a human-factors specialist may assess whether an operator can recognize and respond to it; and a systems engineer may coordinate the competing design implications. The useful product is a connected analysis and decision, not a contest over which discipline owns the entire problem.

The same principle applies to operational cybersecurity. Incident response, security monitoring and vulnerability management remain operational capabilities with their own responsibilities. They contribute observations and knowledge to engineering and depend on engineering decisions about access, instrumentation, recovery and support. Calling every operational activity "engineering" would obscure that relationship rather than strengthen it.

1.3 Behaviour is created by relationships

A system's behaviour includes what its elements do together: the outputs they produce, the states they enter and the responses they make under particular conditions. Some of that behaviour cannot be attributed to any element acting alone. This is emergence. It is not automatically a defect. The useful dispatch capability is itself an emergent result of applications, people, communications and operating arrangements working together. NASA's systems-engineering introduction similarly emphasizes the value created by relationships among elements. [3, introductory discussion]

An unintended interaction can also emerge. Two individually reasonable retry mechanisms might repeatedly submit the same administrative request after a connection interruption. Whether the result is harmless, rejected, duplicated or applied out of order depends on the receiving service and the meaning of the request. The fact that both components passed their local tests does not answer the combined-system question.

The engineering response is to specify and evaluate the relevant interaction. What identifies a request? Can it be safely repeated? Which component determines whether it has already been applied? What happens when an acknowledgement is lost? How does the operator distinguish an unsuccessful request from one that succeeded without confirmation? Security contributes concerns about unauthorized repetition or manipulation, but the solution also depends on application semantics, interface design and human use.

Predictability therefore needs a careful meaning. It does not require every internal event to be deterministic. A distributed service can permit variable response times while satisfying a specified upper bound under an agreed load. A design can permit different recovery paths while constraining which actions each path may perform. What matters is that the behaviour on which people rely is adequately characterized and evaluated for the intended use.

A system's architecture comprises its important structures, relationships and governing principles. An architecture description or model represents selected aspects of that architecture. The two are not identical: a drawing can be incomplete or stale while the system continues to operate. Nor does a model enforce a rule merely by depicting it. The team must connect the representation to implementation, configuration and evidence about actual behaviour.

1.4 Security and resilience address related questions

For the purposes of this book, security reasoning considers how to protect assets and useful capabilities against unacceptable loss. It addresses more than disclosure of information. Corrupted commands, lost service capability, unauthorized physical action and compromised support arrangements may all be relevant, depending on the system. The significance of a loss comes from its consequences for stakeholders, not simply from the technology involved. This loss-oriented treatment is consistent with NIST's discussion of security, assets and protection needs. [1, Sections 3.1 and 3.4–3.7]

Resilience adds an important question: what useful capability can be maintained, restored or adapted when adverse conditions occur? Cyber-resilience engineering addresses that question for systems dependent on cyber resources. NIST organizes cyber-resiliency goals around anticipating, withstanding, recovering from and adapting to adversity. These goals help structure the question; they are not four products to buy or a guarantee that every system can continue every function through every disruption. [2, Section 2.1.1]

Return to the dispatch example. Preventing unauthorized modification of dispatch rules is a security concern. Maintaining a necessary dispatch capability while the identity provider is unavailable is a resilience concern. The two interact because an improvised recovery route could enable unauthorized modification, while an excessively restrictive arrangement could prevent legitimate restoration. Both concerns belong in the same requirements and design discussion.

A backup component is a design option, not a complete resilience claim. It might share the failed dependency, reproduce corrupted data or require credentials unavailable during the disruption. A manual procedure is also a design option, but it needs an examination of staffing, information, workload and error opportunities. The relevant question is whether the combined arrangement provides the required capability under the specified adversity, with acceptable consequences and sufficient supporting evidence.

SSE and cyber-resilience engineering make complementary contributions within the same systems-engineering effort. Broader mission assurance also depends on operational, organizational and other capabilities that these technical contributions do not fully encompass. A justified claim about remote administration is therefore a contribution to mission assurance, not a guarantee of logistics success. [2, Chapter 1 and Appendix A, "mission assurance"]

Safety, reliability, security and resilience overlap without becoming synonyms. A response appropriate for an information service may be unsuitable for a system controlling physical equipment. Even within one system, stopping one function may protect people while stopping another creates danger. The book will therefore use retained capability, tolerable degradation, recovery conditions and adaptation needs to make resilience expectations specific. It will not prescribe one universal "safe" response.

1.5 Separate the property, the claim and the decision

Three questions need to remain distinguishable. What protective capability does the system possess? What evidence and reasoning support a claim about that capability? Who is authorized to decide that the resulting capability and remaining uncertainty are acceptable for a particular use? Confusing these questions can turn an engineering judgement into an administrative shortcut.

Trust concerns reliance on an entity to meet an expectation. **Trustworthiness** concerns whether that reliance is merited. **Assurance** concerns the grounds for confidence in specified claims. A system may be relied upon because no alternative is immediately available, even though its trustworthiness is incompletely established. That practical decision does not fill the evidence gap. Conversely, lack of adequate evidence does not establish that every claimed property is false. It establishes a limit on what can responsibly be concluded. [1, Appendix F]

For the dispatch case, the team might propose the claim that all enumerated normal-mode management paths enforce the agreed authorization rules. Supporting it requires more than an assertion that a gateway is installed. The team must establish which paths exist, what the authorization rules require, where enforcement occurs and what applicable evidence demonstrates. Any excluded path or operating mode must remain visible. An excluded item is not automatically exempt from the underlying obligation.

An assurance case organizes such claims, supporting reasoning and evidence. It can identify a gap, expose a conflict or conclude that the support is inadequate. It should not be treated as a persuasive wrapper added to a preferred deployment decision. The protective mechanisms perform the protection; the case helps explain the basis and limits of confidence in them.

A decision-maker may authorize a limited use despite remaining uncertainty, where the governing arrangements permit it. That decision should identify its conditions, scope and reconsideration triggers. It does not convert a failed requirement into a passed one. Technical findings, compliance conclusions, contractual acceptance and permission to operate may inform one another, but they answer different questions.

1.6 Follow the reasoning, not a rigid sequence

The book uses an alignment chain to connect purpose, protection needs, requirements, design, evaluation and justified reliance. This author-developed teaching aid makes missing reasoning visible. It is neither a new standard nor a one-way development method.

Start with a stakeholder concern and the applicable obligations. Develop requirements that express what the system must provide or restrict. Evaluate alternative ways to satisfy them. Allocate functions and responsibilities, implement the selected design, and obtain evidence through appropriate evaluation. Use that evidence to support or challenge claims and decisions. Operational experience and changes can return the work to any earlier point.

The connections are often many-to-many. One protection need may lead to several requirements. A shared mechanism may contribute to multiple requirements. A test can provide evidence for several claims, while a single claim can need several kinds of evidence. The chain should not force those relationships into an artificial one-control, one-requirement, one-test pattern.

For example, a rule limiting dispatch changes to authorized administrators may depend on identity validation, permission enforcement, protected configuration, session handling and control of alternative management paths. Those contributions may span multiple suppliers and customer-operated services. The system-level conclusion concerns their combined effect, not the number of boxes completed in a traceability matrix.

Record the decision, open questions and responsibilities early. The record can sit within an existing architecture decision or project issue. Table 1.1 illustrates those contents before the engineering work is completed.

Table 1.1. Decision framing for the remote-administration example

Record field	Illustrative entry
Decision needed	Select a remote-administration approach for the dispatch application.
Intended contribution	Permit authorized maintenance while protecting dispatch-rule integrity.
Known obligation	The organization's example policy requires MFA for remote administration.
Material uncertainty	Required maintenance capability during identity-service loss has not been agreed.
Engineering work	Examine management paths, operating modes, dependencies and design alternatives.
Participants	Service owner, application and infrastructure teams, security, operations, support and the responsible decision authority.
Current conclusion	An architecture choice is premature until the continuity need and applicable constraints are clarified.

This record improves the quality of the next discussion. It prevents a gateway purchase from silently answering a question about operational recovery that the team has not yet asked. The record itself does not establish that the eventual design will be suitable.

1.7 Recognize engineering work by its contribution

The security contribution is substantive when it improves the problem definition, requirements, design choice, realization, evaluation or grounds for a decision. Renaming an existing compliance review does not achieve that result. Neither does replacing familiar control terminology with architectural language while leaving the underlying work unchanged.

An analyst who identifies an undocumented administrative path has supplied valuable system information. An operator who demonstrates that a recovery instruction cannot be performed with the available access has identified a design or support concern. A developer who determines how to reject a replayed state-changing request has contributed implementation reasoning. Their job titles do not make those contributions unimportant, and the contributions do not automatically confer authority to approve the entire system.

Professional competence includes knowing when specialist assistance is necessary. Understanding this book's method does not substitute for competence in cryptography, physical design, safety analysis, software implementation or the relevant operational domain. A generalist can frame a question and coordinate its resolution without claiming to possess all the expertise needed to answer it.

1.8 Exercise: what does the demonstration establish?

Situation. In the hypothetical dispatch project, a supplier demonstrates that an administrator can sign in to the proposed gateway using MFA. A manager proposes recording “remote administration is secure” and closing the security action. What conclusion is justified, and what should happen next?

Worked answer. The demonstration may establish that a particular authentication path worked in the demonstrated configuration and circumstances. Its precise evidentiary value depends on what was shown and recorded. It does not establish that every management path requires MFA, that authorization is correct, that an authenticated administrator cannot exceed permitted authority, or that the service can be maintained during a dependency outage. The proposed conclusion is broader than the demonstrated property.

Retain the demonstration as a potentially useful evidence item. Record its configuration, scope and limitations. Relate it to the applicable obligation, then identify the additional requirements, paths and operating conditions that need examination. The next action is not automatically “buy another security control.” It is to resolve the system questions and select the appropriate design and evaluation work.

Carry forward. Security engineering connects specialist knowledge to a shared system decision. It recognizes useful operational and compliance contributions, examines the relationships that determine system behaviour, and keeps reliance proportionate to what the evidence supports. The next chapter establishes which system and context that reasoning actually concerns.

2. Systems, boundaries and operational context

2.1 A system boundary is a modelling decision

Before deciding whether a system is adequately protected, the team needs to say which system it is discussing. That sounds simple until the same meeting uses “the system” to mean an application, a hosted service, a collection of servers, the supplier’s product and the business activity that depends on all of them. An assessment can be internally accurate yet answer the wrong question because the participants have different boundaries in mind.

A **system of interest**, abbreviated SoI, is the system selected for the engineering question under consideration. Its boundary distinguishes the elements treated as part of that system from the environment and other systems with which it interacts. The selection is made for a purpose. It is not necessarily a fence, a subnet, a procurement line item or the edge of organizational ownership.

NIST distinguishes a system of interest from interfacing systems, including systems that enable its creation or sustainment and systems with which it jointly performs a function. That distinction helps avoid treating every dependency as an internal component while still keeping the dependency visible. [1, Section 2.1.2]

The boundary must not be selected merely to make a desired claim easier to substantiate. A gateway-only boundary may be suitable for evaluating the gateway’s authentication capability. It would be insufficient by itself to establish a claim about all remote administration of the dispatch application. Narrowing the object of analysis is legitimate; silently carrying the conclusion across a wider scope is not.

2.2 Distinguish the useful service from its supporting capability

The logistics operation needs correct, timely dispatch instructions. Remote administration supports that need by allowing authorized people to configure and maintain the dispatch application. These are related capabilities at different levels. The remote-administration capability is not the entire logistics operation, and establishing its properties does not assure every aspect of delivery performance.

For the worked case, the SoI is the **remote-administration capability**: the managed administrator endpoint, gateway, application management endpoints and associated enforcement configuration, together with the administrative procedures and assigned human roles included in the proposed capability. The dispatch application’s operational processing and end users are outside this selected boundary, but their needs constrain it. The enterprise identity service is also outside the boundary and remains an explicit dependency.

This boundary is a deliberate expansion of a gateway-only product view. It includes the receiving management endpoints because enforcement there affects the remote-administration conclusion. It includes administrator work because approval, credential recovery and maintenance actions are part of the proposed operating arrangement. It does not imply that the project owns the enterprise identity service or controls every behaviour of the dispatch application.

The team also records the boundary of its authority. It can configure the gateway and the managed endpoints, negotiate changes to application interfaces, and request services from the

identity team. It cannot unilaterally redefine enterprise identity policy or waive the service owner's operational obligations. A boundary model without this authority information can produce requirements that nobody is in a position to implement.

A different decision may require a wider SoI. Evaluating whether dispatch can continue through a regional communications outage would require attention to operational users, field devices, procedures and communications services beyond remote administration. The analysis may reuse information from the narrower model, but the wider conclusion needs its own reasoning.

A single context view helps keep the recurring case boundary stable while exposing dependencies that remain relevant to the claim. The dashed boundary below identifies the selected administration capability; external services remain outside that boundary without becoming irrelevant.

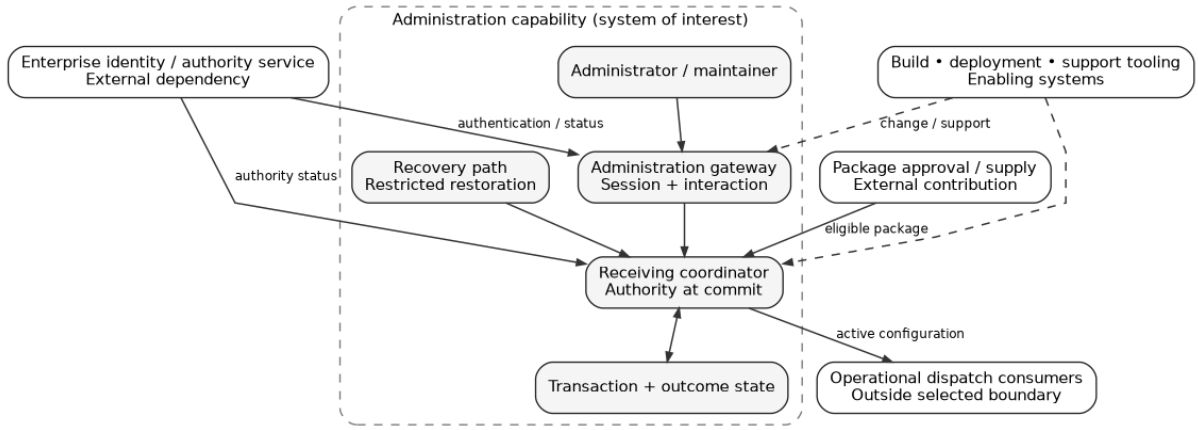


Figure 2.1. System context and consequential trust relationships

Original context schematic for the recurring administration case. The system-of-interest boundary does not erase external identity, package, operational-consumer or enabling-system dependencies; arrows identify influence and information relationships rather than automatic trust.

2.3 Start with functions and interactions, then identify elements

An inventory tells the team what items have been recorded. A system-context model explains how relevant functions are achieved and what the selected system exchanges with other entities. Both are useful. The inventory may reveal a forgotten host; the context model may reveal that a host-independent identity service is essential to a recovery action.

In the example, begin with the administrative functions: establish an identity, request authority, submit a management action, enforce the decision, communicate the result and record the action. Add support functions such as changing permissions, revoking access, restoring service and updating configuration. Only then ask how the proposed elements provide those functions and what crosses each interface.

A data exchange is not the only relevant interaction. An interface can carry commands, timing information, energy, physical material or a human decision. For a software-heavy service, the command and data relationships may dominate the first model. For a physical system, an engineering review that ignores power, mechanical access or maintenance connections may miss

an important protection concern. The correct level of detail depends on the question, not on a preference for network diagrams.

Table 2.1 records selected context for the stated system of interest. It is not exhaustive; the same dependency may take a different role when the engineering boundary or question changes.

Table 2.1. Selected context relationships for the remote-administration capability

Entity	Relationship to the selected system	Security-relevant question
Administrator and managed endpoint	Included human role and technical element initiating management actions.	How are identity, authority and endpoint conditions established?
Gateway and management endpoints	Included elements accepting, mediating or applying requests.	Can a request reach a state-changing function without the required checks?
Enterprise identity service	External dependency supplying identity and access-status information.	What happens when it is unavailable, delayed or returns information the receiving system cannot validate?
Dispatch operational service	Interfacing service affected by management actions.	Which changes can disrupt or corrupt the useful service?
Security monitoring service	External recipient of selected events and provider of response coordination.	What observation is lost if event delivery fails, and what response remains possible?
Build and deployment environment	Enabling system creating and delivering updates.	Could a change alter management paths or enforcement without the required review?
Time and name-resolution services	Supporting dependencies whose relevance must be established.	Which protective decisions actually depend on their correctness or availability?

A useful context record distinguishes facts from questions. “Uses enterprise identity service” may be an established design fact. “Existing sessions remain usable during its outage” may still be an assumption. Keeping both in the same model is useful only when their status is visible.

2.4 Model modes, states and transitions

A boundary without operating conditions is incomplete. The same interface may have different permitted uses during normal operation, maintenance, recovery and retirement. A **mode** describes an intended operating arrangement with particular functions or rules. A **state** describes a relevant condition of the system at a point in time. A **transition** is a change between such conditions. These distinctions allow the team to say more than “the system is up” or “the system is down.”

In the dispatch case, NORMAL mode might use the central identity service for administrative access. A separately designed RECOVERY mode might permit a narrower maintenance capability under different, explicitly authorized conditions. These are candidate modes, not an approval of an exception. The team must decide whether a recovery mode is necessary, what it can do, who can initiate it and how applicable authentication requirements continue to be met.

Relevant states could include an established administrative session, an expired session, a pending permission change or loss of communication with the identity service. A transition from reachable to unreachable identity service raises questions about new sessions and existing

sessions that may have different answers. Describing only the login sequence misses the behaviour of work already in progress.

Transitions deserve their own evaluation. A recovery capability that is controlled while active may still be vulnerable during activation or return to normal operation. Does old authority persist? Are queued requests re-evaluated? Can inconsistent policy versions be used simultaneously? What does the operator see, and what prevents a mistaken belief that normal protection has resumed? A mode name is not an implementation of the transition rules.

Human tasks also need operating context. A procedure that expects a second approver is incomplete unless that person can be reached, has the required information and can act within the relevant operational window. Treating the approver as a box in a workflow is not enough to establish that the arrangement will work during the disruption it is intended to address.

2.5 Give assumptions a job, an owner and a consequence

An assumption is a proposition used in reasoning before it has been established to the required degree. Assumptions are not inherently irresponsible. They allow analysis to proceed when information is incomplete. The difficulty arises when an important assumption becomes invisible, remains unexamined or is presented as a demonstrated fact.

An effective assumption record says what is being assumed, why it matters, which decision depends on it, how it will be challenged and who will obtain or evaluate the relevant information. It also identifies what the team will do when the assumption is not supported. Merely assigning an owner does not resolve the uncertainty.

Consider the proposition that every dispatch-rule change passes through the new gateway. It may be useful as an early modelling assumption, but it is dangerous as an unqualified acceptance conclusion. A supplier maintenance interface, automation account or emergency console could contradict it. The team can inspect interface documentation, examine deployed configuration and evaluate reachable management paths. It may discover that the gateway-only architecture cannot meet the proposed boundary claim.

Table 2.2 records three illustrative assumptions. Each remains open in this teaching case. The proposed resolution activities have not been performed.

Table 2.2. Assumptions that affect the architecture choice

ID and proposition	Consequence if unsupported	Planned resolution and responsible participant
A-01: all remote dispatch-rule changes use enumerated management endpoints.	A protective path may be bypassed or omitted from evaluation.	Application and infrastructure teams reconcile interface information with the deployed configuration and review coverage with security.
A-02: a permission-revocation update reaches every relevant enforcement point within the required interval.	A revoked administrator may retain usable authority longer than permitted.	Identity and application teams analyse propagation and caching, then plan a bounded timing evaluation.

ID and proposition	Consequence if unsupported	Planned resolution and responsible participant
A-03: urgent restoration can wait for central identity service recovery.	A centrally dependent design may prevent required maintenance.	The service owner and operators establish the restoration need and test the proposed operational scenario.

Some assumptions can be replaced by enforceable requirements. Others remain conditions on an analysis or agreement because the project cannot control the external environment. In that case, examine sensitivity: how much does the conclusion change if the assumption is wrong? A design with acceptable behaviour across a wider range of conditions may be preferable to one that depends on a precise external promise.

Do not confuse an assumption with a design decision or a contractual commitment. A design decision records a selected response and its rationale. A commitment records an obligation undertaken by an identified party. Either can still require evidence of effectiveness, but their management and authority are different from those of a provisional analytical assumption.

2.6 Follow dependencies to the claim that uses them

Not every dependency needs the same depth of analysis. Start with the function or claim being evaluated and ask what it relies upon. For the revocation claim, policy distribution, identity status, local caches, timing and each enforcement path may matter. For an event-retention claim, storage, transfer, access and retention arrangements may matter more. A generic list of critical suppliers does not replace this claim-specific reasoning.

Suppose the design includes two gateways. Their presence can address some gateway failures. It does not demonstrate continued administration during an identity-service outage when both gateways require that same identity service. The conclusion follows from the stated dependency, without needing a failure probability. A quantitative availability calculation would require additional assumptions and evidence about independence, modes and repair; simply multiplying component figures would conceal the shared dependency.

The same caution applies to evidence inherited from suppliers. A supplier's test may support a property of its product under stated conditions. The receiving system adds configuration, interfaces, human use and other dependencies. Reuse the evidence where those conditions support the new claim, and identify what remains to be evaluated. Component confidence does not automatically become confidence in the composition. [1, Appendix E.7]

This does not mean that one compromised component must destroy every system-level claim. A design may contain the effect or continue an essential function without relying on that component. Whether it can do so is an engineering question about the architecture and operating conditions. Neither automatic trust nor automatic mission failure is an adequate substitute for that analysis.

2.7 Reconcile the model with operational knowledge

An IT team does not need to discard its configuration management database, service catalogue or monitoring information to build a context model. Those records may provide identifiers,

dependencies, owners, interfaces and observed operating conditions. The engineering task is to reconcile them with the decision and identify where they are incomplete or inconsistent.

For example, a configuration record might identify a management server but not explain that a supplier accesses it through a separate support arrangement. A service map may show the identity provider but omit the human approval needed to restore an account. An incident record may demonstrate that operators use an undocumented recovery sequence. Each observation can change the model and the requirements; none should be dismissed because it originated outside a formal engineering tool.

Review one service scenario with the relevant participants. An operator describes the complete action; technical staff identify its functions and interfaces; security and support specialists challenge its dependencies. Record disagreements and the information needed to resolve them. NASA's stakeholder-expectations process supports identifying participants and preserving the rationale for agreed expectations. [4, Section 4.1.1.2]

The resulting model need not be elaborate. A small, controlled context register and a clear scenario can be more useful than a comprehensive-looking diagram with unresolved meaning. The acceptance test for the model is whether it supports the decision, exposes material dependencies and allows others to understand the scope of the claim.

2.8 Exercise: is the identity service outside scope?

Situation. The remote-administration supplier states that the enterprise identity service is customer-owned and therefore outside the supplier's security scope. The project proposes deleting it from the system-context model. What distinction is being missed?

Worked answer. Distinguish ownership, contractual responsibility and engineering relevance. The supplier may have no duty to build or operate the identity service, while the administration capability still depends on its behaviour. Keep that dependency in the context model and identify who supplies the required contribution.

Retain the identity service as an external dependency. Establish the interface expectations, the customer's contribution, the supplier's behaviour when those expectations are not met, and the evidence needed for the receiving-system claim. Where the customer cannot establish an assumed service property, evaluate the effect on design and assurance. Do not invent a supplier obligation that the contract does not contain, but do not let that absence masquerade as evidence that the dependency is harmless.

Further challenge. Does including the identity service mean that the project must assess the entire enterprise identity environment? Not necessarily. The appropriate scope follows the property being relied upon and the available evidence. It may require a bounded examination of the relevant service and interface, additional customer assurance or a changed design. State the limits rather than claiming comprehensive coverage.

Carry forward. A sound boundary makes the system question manageable without hiding the conditions that determine the answer. With the purpose, interfaces, operating context and responsibilities visible, compliance obligations and security controls can be connected to the actual engineering problem.

3. Compliance, controls, risk and engineering

3.1 Different instruments answer different questions

An organization may use a cybersecurity framework, a control catalogue, an information security management system, a risk management process and a certification scheme at the same time. Calling all of them “compliance frameworks” conceals important differences. They do not necessarily assess the same object, prescribe the same activities or support the same conclusions.

The NIST Cybersecurity Framework 2.0 describes high-level cybersecurity outcomes and ways to organize and communicate cybersecurity efforts. It does not prescribe the implementation that achieves those outcomes. NIST SP 800-53 provides a catalogue of security and privacy controls; the controls address both protective functionality and confidence in that functionality. ISO/IEC 27001 specifies requirements for an information security management system. None of those descriptions, by itself, is a design for the dispatch service. [7, abstract; 8, abstract; 11, overview]

Assessment and authorization also need to be distinguished from the instruments they use. NIST SP 800-53A provides control-assessment methods and procedures that can be applied across the development life cycle. The NIST Risk Management Framework includes preparation, implementation, assessment, authorization and continuing monitoring activities. Representing those approaches as inherently static or concerned only with paperwork would be inaccurate. The relevant criticism is of an implementation that disconnects them from the system, not of a simplified caricature of the source. [9, abstract; 10, abstract]

Table 3.1 compares the questions these instruments help answer. It illustrates their different purposes without making the named publications mandatory for every organization.

Table 3.1. Distinguishing the scope of familiar security instruments

Instrument or activity	Principal question it helps address	Engineering connection
Outcomes framework	What cybersecurity outcomes are sought or prioritized?	Relate the outcomes to the needs and operating context of the system.
Control catalogue or selected baseline	Which safeguards and associated expectations are relevant or required?	Resolve applicability, implementation, allocation and evaluation within the design.
Information security management system	How does the organization establish and improve its management of information-security risk?	Connect governance, resources and management decisions to the engineering effort.
Control assessment	What does the assessed evidence establish against the applicable assessment objectives?	Reuse results within scope and address system-level interactions not established by those results.
Authorization or other accountable decision	May the specified use proceed under the governing authority and conditions?	Present technical findings, evidence limits and residual risks without changing their meaning.

A programme can perform these activities well and still need additional system analysis. A well-supported finding about the gateway’s authentication mechanism may not establish that every application interface is covered. Equally, an engineering argument about the application does not

discharge an organizational requirement to manage supplier access or assess a prescribed control. Completeness depends on connecting the relevant scopes.

3.2 Keep obligations, requirements, constraints and controls connected

An **obligation** is something the project or organization is required to satisfy by an applicable source. A **requirement** expresses a necessary capability, characteristic, condition or constraint on a specified subject. A **constraint** restricts acceptable design choices, behaviour or use. A **control** is a safeguard or countermeasure used to address a security or privacy concern. In practice these categories overlap: a mandated control can be expressed as a requirement, and a technical control can enforce a behavioural constraint.

The distinction is therefore not "controls are paperwork; requirements are engineering." A catalogue control statement may already express a requirement when it is made applicable. The engineering task is to establish what it means for the particular system, resolve parameters and allocations, and demonstrate the adequacy of its implementation. Rewriting it in more technical language is useful only when the result improves meaning or evaluability.

Suppose the example policy requires MFA for remote administration. That obligation remains part of the problem. The team cannot declare MFA optional merely because it can imagine a different architecture. It can investigate alternative ways to meet the obligation, ask the responsible authority to clarify its scope, or seek a formally permitted exception. Until that authority acts, the obligation is not removed by the engineering discussion.

At the same time, "implement MFA" does not settle every relevant system requirement. Which administrative operations are covered? Which identities can perform them? What constitutes a successfully authenticated session? How is a revoked permission handled? What occurs when dependencies are unavailable? Answering these questions may lead to several allocated requirements, design decisions and evaluation activities that support the same source obligation.

A constraint is also not made unbreakable by calling it structural. Its enforcement depends on implementation and operating conditions. A separation mechanism can be defeated by an overlooked communication path; an interlock can be incorrectly installed; a software invariant can be violated through a defect. Structural protection may reduce dependence on human action, but it still needs an argument for its adequacy. Procedural and physical measures remain legitimate elements of a socio-technical design when their limitations are understood.

Requirements also express the positive capability needed by users. A design that rejects every request could satisfy an isolated prohibition while failing the intended service. Positive functions, performance, interfaces and restrictions must be considered together. The NASA requirements-development guidance provides a useful public example of this broader treatment, including the preservation of rationale and the validation of requirements against stakeholder expectations. [4, Section 4.2.1.2]

3.3 Use risk to inform the work, not to replace it

Risk management helps determine which potential consequences matter, how uncertainty and exposure affect decisions, what responses are available and who must act. Its contribution is weakened when a risk score is treated as a substitute for the engineering explanation of how a loss could occur or how a proposed measure would prevent or limit it.

For the dispatch case, “unauthorized administration: medium risk” provides little direction by itself. A more useful account identifies the affected function, a plausible sequence of events and the resulting consequence. For example, a revoked administrator might retain a usable session because an enforcement point continues to use cached permissions. That sequence could permit a dispatch-rule change after the organization believes access has been removed. The sequence is a hypothesis for examination, not evidence that the defect exists.

The hypothesis directs the work. Determine how revocation is communicated, where permissions are cached, what the receiving endpoints enforce and which timing conditions matter. The resulting information may support a different requirement, a changed architecture, a targeted test or a narrower claim. Record the risk decision alongside that reasoning so that a later reviewer can understand the connection.

Threat knowledge has a legitimate role in this analysis. It can reveal plausible ways of misusing an interface, obtaining authority, corrupting an update or exploiting a dependency. Failure analysis and operational experience can reveal different paths to related consequences. Mission and stakeholder needs establish why the consequence matters; knowledge of adversity helps determine which conditions and responses need examination. Neither contribution should be relegated to an afterthought.

Not every unknown is best represented as a vulnerability, and not every evidence gap proves that a loss scenario is likely. A team may lack test coverage even when it has found no implementation defect. That gap still limits the confidence available for a decision. Distinguish uncertainty about the evidence from an observed failure and from the consequences of relying on the system despite either condition.

The depth of analysis should match the significance and difficulty of the decision. A modest, well-understood change may need a short documented analysis. An unfamiliar mechanism controlling a high-consequence function may need models, experiments and specialist challenge. NASA’s decision-analysis guidance provides a useful reference for selecting the analytical effort and recording assumptions and uncertainty. [6, Section 6.8.1]

3.4 Worked example: develop the requirements before selecting the gateway

The case now has a defined SoI and several open dependencies. It also has an illustrative policy obligation, **POL-01**, requiring MFA for remote administration. Two preliminary protection needs guide the analysis. **PN-01** concerns preventing unauthorized modification of dispatch rules. **PN-02** concerns preserving the ability to perform necessary corrective administration during defined disruption scenarios. Neither need is a complete requirement set.

The team should first resolve which actions, users and operating conditions belong to the capability. For the teaching example, the following statements are candidate system requirements. They intentionally expose information still needed before a requirement baseline could be approved. The identifiers refer to this example only; they are not control identifiers from an external scheme.

SR-00 — Useful administrative capability. Under the administrative service conditions specified in ADM-P1, the remote-administration capability shall support the dispatch-rule maintenance actions assigned to the administrator role in AUTH-P1.

This candidate preserves the positive capability. ADM-P1 still needs to define the relevant workload, service conditions, response expectations and operating modes. AUTH-P1 must

identify the permitted actions. The statement is therefore not ready for contractual release. Its value at this stage is to make the missing definition of useful maintenance visible alongside the protective requirements.

SR-01 — Authentication. For each remote dispatch-rule modification request, the remote-administration capability shall accept the request for processing only when it is associated with an identity authenticated in accordance with MFA-P1.

MFA-P1 is the example's controlled authentication profile, not an assumed product feature. It must define what authentication evidence the receiving system accepts, its required freshness and the conditions that invalidate it. This requirement refines POL-01; it does not replace the need to examine all remote-administration functions to which POL-01 applies.

SR-02 — Authorization. For each dispatch-rule modification request, the remote-administration capability shall enforce the permissions assigned to the authenticated identity in AUTH-P1.

Authentication and authorization are deliberately separated. Evidence that an identity was authenticated does not establish that the requested action is permitted. The design must determine where the permissions are enforced and how a request reaching another management endpoint remains subject to the same applicable rules.

SR-03 — Revocation timing. With the identity service reachable under NET-P1, the remote-administration capability shall prevent further dispatch-rule modification requests by a revoked identity within 60 seconds of the revocation event recorded by the authoritative identity service.

The sixty-second interval is invented for this example, not a recommended setting. NET-P1 must define the network conditions, and the evaluation must establish event timing and measurement uncertainty. Requests accepted before revocation also need an explicit disposition: their exclusion from this early statement does not make later execution acceptable.

SR-04 — New sessions during dependency loss. In NORMAL mode, while the identity service is unreachable, the remote-administration capability shall reject requests to establish a new privileged session.

This is a candidate response, not a universal rule. It must be evaluated against PN-02 and the intended use. Existing sessions, restoration actions and any proposed RECOVERY mode require their own treatment. The team should not label the continuity problem resolved because SR-04 is straightforward to test.

These examples expose missing conditions and design implications before procurement fixes the commitment. A project must complete the definitions, establish feasibility and review the requirement set for adequacy and consistency. ISO/IEC/IEEE 29148 addresses requirements engineering and its information products; the examples here are original applications. [13, public scope]

Traceability should preserve both obligation and rationale. SR-01 relates to POL-01 and PN-01. SR-03 concerns how promptly authority is removed and therefore also needs a stakeholder-agreed tolerance. SR-04 depends on a proposed operating response and can conflict with PN-02 unless a suitable maintenance arrangement exists. A matrix should expose those relationships, not hide an unresolved conflict behind a completed reference field.

3.5 Compare genuine design alternatives

The candidate requirements now make it possible to compare alternatives rather than merely compare gateway features. One option is a centrally dependent administration capability that permits no new privileged sessions during loss of the identity service. It may simplify policy administration and reduce the number of separate credential-management arrangements. Its suitability depends, among other things, on whether the service can tolerate loss of that maintenance capability.

A second option adds a separately governed recovery capability. It could reduce dependence on the central identity service for a limited set of restoration actions. It also creates additional interfaces, credential-handling tasks, activation conditions and opportunities for misuse. It would still need to satisfy POL-01 and any other applicable obligations, or obtain a specifically permitted exception through the appropriate authority. It is not an informal password-only bypass introduced because recovery is inconvenient.

Compare the options against actual criteria: preservation of dispatch-rule integrity, necessary maintenance capability, propagation of revoked authority, dependence on common services, operator workload, support requirements, implementation feasibility and lifecycle cost. Some criteria may be binding constraints. Others may permit trade-offs within stakeholder-agreed limits. A favourable aggregate score cannot compensate for an unmet mandatory requirement unless the governing arrangements explicitly permit a change.

The comparison may lead to a third response. For example, reducing the range of administrative actions needed during an outage could make the central option adequate. Alternatively, providing a limited local maintenance function might be less complex than duplicating the complete remote-administration service. Those possibilities require further design and operational analysis; the case does not assume that more redundancy or more access paths is necessarily better.

A responsible intermediate conclusion can be “not yet enough information to select.” In this example, the unagreed restoration need and incomplete profiles prevent a justified final choice. Record the work needed to resolve them and who can decide. That is a useful outcome from a trade discussion, not a failure to be decisive.

3.6 Plan evidence around the claim

Evidence should be judged by what it establishes, not by whether it is a document, a log, a model or a test result. A policy record can establish the text and approval of POL-01. It does not establish that the implemented interfaces obey it. A configuration export can show specified settings for a captured configuration. Its contribution to a behavioural claim depends on the design and on evidence that those settings have the asserted effect.

For SR-03, relevant evaluation could combine interface analysis with timed tests of revocation propagation and receiving-end enforcement. The evaluation needs the controlled network envelope, representative configurations and an identified time reference. It should also examine the policy and cache behaviour that the tests exercise. A favourable observation in one sequence does not establish every relevant timing condition or management path.

An evidence record identifies the evaluated item and version, method, conditions, criterion, result, coverage and limitations. At this stage of the case, the evaluation is planned and the result

is unpopulated. Keep that distinction visible wherever the record supports an assurance argument.

A supplier's existing assessment can still be useful. Determine the assessed entity, configuration, objectives and conditions, then establish which conclusions transfer. Some results may be directly applicable; others may need additional receiving-system evidence. A certificate concerning an organization's management system should not silently become a product claim about revocation timing.

Crosswalks can help locate related expectations, but a mapped relationship is not equivalence. NIST explicitly cautions against assuming equivalence solely from mapping tables. A project should preserve the applicable source requirement and examine the actual scope and meaning of any proposed reuse. [8, publication-record mapping caution]

3.7 Distinguish verification, validation and accountable reliance

Verification asks whether the evaluated item satisfies specified requirements. Validation asks whether it meets stakeholder needs and intended use in the relevant context. Methods can include analysis, inspection, demonstration and testing, selected for the question and the available system representation. Early evaluation is meaningful when its claim is appropriate to its maturity; analysis of a candidate design is not proof that the delivered implementation conforms. [5, Sections 5.3–5.4]

In the case, evaluating SR-02 could establish whether enumerated management actions are permitted or rejected according to AUTH-P1. That is different from evaluating whether the authorized staff can restore a necessary dispatch capability during the agreed disruption scenario. The latter includes actual task demands, access, information, workload and support. A design could satisfy SR-04 and still fail the intended-use evaluation because needed maintenance becomes impossible.

An assurance argument connects the applicable results to a bounded claim. For example: "For configuration B1, the enumerated NORMAL-mode remote management paths enforce the agreed authentication and authorization requirements under the evaluated conditions." In this example, that statement is an **unsubstantiated candidate claim**. It must not be presented as an achieved result until the evidence has been produced and reviewed. RECOVERY mode and unexamined paths remain open matters.

The decision to proceed also considers the compliance position, unresolved deviations, operational consequences and the authority's responsibilities. A risk acceptance does not make a test pass, and a passing test does not itself authorize operation. Where conditions or temporary restrictions are permitted, record who imposed them, what capability they apply to, how they will be monitored and what would require reconsideration. The source of that authority is the governing organization, not this book.

3.8 Return the work to familiar records

The bridge to IT practice becomes useful when the engineering results improve an existing workflow. A service map can retain the selected boundary and dependency relationships. A control implementation statement can refer to allocated requirements and evaluated configurations. A change record can identify affected claims and the reason for the chosen

regression scope. An incident review can challenge a design assumption rather than end with a reminder to follow the procedure more carefully.

Table 3.2 shows several such connections. These are suggested extensions, not additional mandatory documents or claims that the existing artefacts are equivalent to one another.

Table 3.2. Putting engineering results back into existing work

Existing record	Useful engineering contribution	What remains distinct
Service map or configuration record	Boundary, functions, relevant dependencies and evidence of reconciliation with operation.	An inventory is not a complete model of system behaviour.
Control implementation statement	Source obligation, allocated requirements, enforcement points and applicable evidence.	Implementation description and demonstrated effectiveness are different claims.
Risk register	Loss scenario, assumptions, design options, evidence limits and accountable treatment.	A risk rating does not establish implementation correctness.
Change request	Affected requirements, interfaces and claims; evaluation scope; conditions for deployment and recovery.	Approval to change is not evidence that the changed system meets its requirements.
Supplier assurance record	Scope of reusable evidence, customer dependencies and remaining receiving-system evaluation.	Organizational assurance is not automatically delivered-system assurance.

Exercise. The identity provider supplies a certificate and a successful MFA demonstration. The project proposes using them to close SR-03. Is the requirement verified?

Worked answer. Not from those facts alone. SR-03 concerns revocation timing and enforcement across the specified receiving-system paths and conditions. The supplied material may support part of the authentication or service-assurance argument, but it does not by itself establish propagation, cache handling and receiving-end enforcement within the candidate interval. Retain useful evidence, identify the missing support and plan the appropriate analysis and evaluation. Do not dismiss the supplier material or extend it beyond its scope.

Carry forward. Compliance, risk management and engineering are complementary when their inputs, responsibilities and conclusions are connected accurately. The objective is not to replace a control matrix with a larger vocabulary. It is to make the system's obligations, design and evidence agree closely enough to support responsible decisions.

4. One life cycle, integrated responsibilities

4.1 Stages describe progress; processes describe work

A system can remain in use through several development projects, supplier changes, major upgrades and operating arrangements. Its life cycle is therefore not identical to the schedule of the project that first delivered it. Security and resilience concerns can originate before a contract is placed and persist after a particular product is retired, for example where retained data or continuing dependencies remain relevant.

A **life-cycle stage** is a period organized around a purpose or state of maturity, such as development, production, use or retirement. A **process** is a related set of activities that produces outcomes. Requirements definition, configuration management and verification are examples of process work that can recur within several stages. A **project** is an organized, bounded effort that may contribute to one or more stages. Keeping these concepts distinct prevents a process name from becoming a misleading one-time milestone.

ISO/IEC/IEEE 15288:2023 provides a life-cycle process framework applicable to systems, system elements and systems-of-systems. Its public scope explicitly allows iterative, concurrent and recursive application and does not prescribe a particular life-cycle model or development methodology. The chapter uses that distinction without reproducing the standard's normative provisions. [12, public scope]

In practical terms, **iteration** revisits work at a given level to resolve learning or discrepancy. **Recursion** applies relevant reasoning at different levels of the system structure. **Concurrency** allows activities to proceed in parallel where their dependencies are managed. An application team can develop a management endpoint while the identity team evaluates its service interface, but neither should assume that parallel progress has resolved the interface agreement.

The chapter sequence is not a set of barriers between processes. Feasibility studies can change requirements; an evaluation method can influence design for observability; a support constraint can eliminate an architecture. Control the work by recording these dependencies and the decisions that resolve them.

4.2 Integrate the specialty where decisions are made

A security team that attends every milestone may still be disconnected from engineering if it sees only completed decisions. Integration means participation at the points where security-relevant choices are framed, evaluated, allocated, implemented and reviewed. The important question is what the contribution changes, not how many meetings the specialty attends.

In the dispatch project, security-relevant work begins when the service owner explains the operational need and the team defines the administration boundary. It continues when application and infrastructure teams decide where authorization will be enforced, when supplier evidence is specified, when operators evaluate recovery and when maintainers propose a change. The security contribution has a different form at each point, but it remains connected to the common system model and requirements.

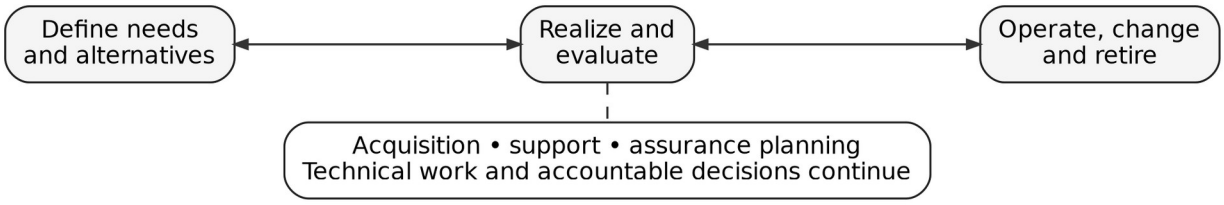
This avoids two unhelpful arrangements. In one, each specialty works separately and reports its conclusions after the design has become difficult to change. In the other, all specialist questions

are treated as something a general engineering team can resolve without appropriate depth. The first loses integration; the second loses competence. The practical alternative is shared decisions supported by distinct, accountable specialist work.

A safety concern raised during recovery analysis should remain a safety concern, with the appropriate expertise involved. A security mechanism's effect on operator workload should be evaluated with the people responsible for human-system design and use. The integrated design has to satisfy the relevant set of needs, not merely produce a separate green status for each specialty.

The following schematic brings together the relationships developed here.

Figure 4.1. Lifecycle work and continuing responsibility



Original schematic. Read across for the broad progression; the double arrows indicate iteration, not a mandatory one-pass sequence. Planning a later activity does not replace performing it.

4.3 Assign work, findings and decisions separately

Responsibility is easier to discuss when it is attached to a concrete output or decision. "Everyone owns security" may express a cultural expectation, but it does not identify who approves a requirement, resolves a supplier discrepancy, evaluates evidence or decides whether a limitation permits continued use.

Table 4.1 illustrates the allocation of responsibilities, not a universal organizational chart or a grant of authority. One person may hold several roles, or one role may span organizations. In either arrangement, identify the work, findings and decisions for which each participant is responsible.

Table 4.1. Distinguishing contributions and decision responsibilities

Participant	Principal contribution in the example	Boundary of the contribution
Mission or service owner	Defines useful outcomes, operational priorities and the consequences of lost capability.	Does not establish implementation correctness by accepting a proposed service outcome.
Systems-engineering or design authority	Integrates technical work, manages design relationships and makes or recommends technical decisions within assigned authority.	Does not silently override contractual obligations or authorities assigned elsewhere.
Security and other specialists	Supply domain analysis, requirements contributions, design challenge and evaluation of relevant evidence.	Specialist concurrence is not automatically overall acceptance or authorization.

Participant	Principal contribution in the example	Boundary of the contribution
Acquirer and supplier	Establish and perform agreed contributions, including information, interfaces, engineering work and evidence.	Supplying a conforming component does not by itself establish receiving-system suitability.
Operators and maintainers	Contribute realistic tasks, support conditions, operational evaluation and observations from use.	A successful workaround does not by itself approve a permanent design change.
Evaluators and reviewers	Examine results, methods, scope, discrepancies and support for claims.	Independence requirements and acceptance powers follow the governing arrangements.
Accountable decision authority	Decides whether a specified commitment or use may proceed under applicable conditions.	A decision to proceed does not alter the underlying technical finding.

Where responsibilities overlap, define how disagreement is handled. An engineer may conclude that a requirement has not been demonstrated while a programme manager faces a delivery deadline. The disagreement should be expressed as an unresolved technical condition and a decision requiring the appropriate authority. It should not disappear through a change to the wording of the test result.

Review independence also deserves planning. Independence may be organizational, functional or specific to an evaluation activity, depending on the applicable arrangements and the consequence of error. Whatever model applies, the reviewer needs access to sufficient information and a way to raise contrary findings. A label such as “independent assessment” is not a substitute for an examination of scope, competence and conflicting responsibilities.

4.4 Plan the integrated engineering work

A systems-engineering management plan, or an equivalent controlled planning arrangement, should explain how the technical effort will be conducted and coordinated. The security contribution belongs in that arrangement. It may be detailed in a supporting plan where useful, but the relationship to common requirements, architecture, configuration, reviews and resources needs to remain explicit. NASA’s technical-planning guidance provides a public example of organizing that integration. [6, Section 6.1]

For coverage, it is useful to consider four connected kinds of work: technical activities that define and realize the system; technical management that plans, controls and evaluates those activities; organizational arrangements that provide people, infrastructure and knowledge; and agreements that establish acquirer and supplier contributions. NIST’s process appendices provide security-related treatments across these areas. The categories are a coverage aid, not a direction to create four separate security programmes. [1, Appendices G–K]

For the dispatch example, the integrated plan needs more than a task labelled “complete security assessment.” It must provide for determining the maintenance need, reconciling management paths, establishing the profiles used by the requirements, examining identity-service failure, evaluating design options, implementing the selected response and obtaining the evidence

needed for the relevant decisions. Each activity needs an owner, dependencies, suitable competence and a usable result.

The plan should address evidence early enough to influence the work. If revocation behaviour requires a representative identity environment and controlled timing observations, those facilities cannot be assumed to materialize at acceptance. If recovery validation depends on operators with specific duties and equipment, their participation must be arranged. A design review should ask whether the proposed evaluation is feasible, not merely whether a test plan has been assigned a future delivery date.

Acquisition makes these relationships enforceable only when the obligations are clear. Distinguish the product behaviour required, the engineering work the supplier must perform, the evidence to be delivered, and the criteria used to evaluate acceptance. Also identify customer contributions: operational scenarios, approved policies, test access, representative services and timely decisions. A supplier cannot resolve an undefined customer recovery need simply by being told to provide assurance.

This work extends to the systems used to build and support the product. A development environment, deployment pipeline, signing service or maintenance facility can affect the integrity of the delivered configuration. NIST's Secure Software Development Framework provides practices that can be integrated into software-development lifecycles and used in producer–acquirer communication. Those practices complement, rather than replace, the receiving system's architecture, interface and operational analysis. [14, abstract]

Include hardware and firmware production, configuration and support in secure development. A substituted component, changed firmware build or altered maintenance tool can affect a relied-upon property. Allocate obligations and evidence according to that contribution, rather than relying solely on a generic supplier risk category.

4.5 Use incremental delivery without fragmenting the system argument

Incremental delivery can produce useful capability and evidence before the entire system is complete. It does not remove the need to understand the boundaries of each increment. A release decision should state what capability has been delivered, which requirements and operating modes it covers, what interfaces it relies on and what remains unavailable or unresolved.

The dispatch project might first deliver a central NORMAL-mode administration capability and later add an approved recovery capability. That can be a legitimate development strategy only if the earlier release is suitable for its intended use and satisfies its applicable obligations. Labelling recovery "a later increment" does not resolve PN-02 when the operational service needs that capability from the first deployment.

Evidence can also mature incrementally. A prototype may establish that a candidate permission model is feasible. A component evaluation may support the gateway's behaviour. Receiving-system evaluation may then examine the integrated management paths. Preserve the scope and limitations of each result so the project can use genuine progress without inflating it into a broader claim.

The system argument must also account for shared changes. An identity-service modification developed by another programme may alter the meaning of authentication evidence used by the dispatch release. Independently managed backlogs do not make the interface independent.

Coordinate version assumptions and change notifications at the point where the systems rely on one another.

A review gate is useful when it supports a decision with appropriate maturity criteria. It is less useful when it rewards the existence of documents without examining their content. Requirements readiness should concern whether the necessary statements, definitions and responsibilities are sufficiently established for the next commitment. Test readiness should concern the configuration, procedures, facilities, criteria and competence required for a meaningful evaluation.

4.6 Treat transition and operation as connected but distinct work

Planning for operation begins before deployment. Operators need to understand permitted actions, relevant limits and the meaning of abnormal conditions. Maintainers need access, tools, information and recovery arrangements. Support organizations need responsibilities they can actually perform. These needs can affect the design and therefore should not be postponed to a handover checklist.

Transition concerns moving the system or capability into the environment and responsibility arrangement in which it will be used. It includes readiness of the receiving organization and enabling resources, not just delivery of software or equipment. NASA's product-transition treatment is a useful public source for this distinction. [5, Section 5.5]

In the example, an operational-readiness discussion should establish who can administer the service, how a revoked identity is handled, what occurs during identity-service loss and how an anomalous management action is investigated. It should establish which recovery actions are supported and which have not been authorized or demonstrated. A procedure that exists but requires unavailable access is not a usable support capability.

Operations then perform the service, observe conditions, respond to incidents and manage routine work. Engineering supports those activities and learns from them. It is unnecessary to declare all operation to be continuous engineering. The important connection is that observations affecting requirements, design assumptions or assurance claims reach the people able to evaluate and change them.

For a legacy or commercial off-the-shelf product, that connection may be constrained by limited design access. Start with the critical use, known interfaces, available evidence and consequence of uncertainty. The response may include configuration changes, restricted use, additional receiving-system evaluation, supplier engagement or replacement. Lack of complete design information is a condition to manage, not a reason to invent confidence or demand that every legacy system be rebuilt immediately.

4.7 Re-evaluate change in proportion to its effects

Change should trigger an examination of impact, not an automatic declaration that all prior evidence is worthless. A useful analysis identifies which requirements, interfaces, assumptions, configurations and claims could be affected, then selects the necessary review, analysis, regression testing or wider evaluation. It also records why evidence remains applicable where the change does not disturb its basis.

Suppose an update changes how the gateway interprets an identity-service response. That is directly relevant to the authentication and authorization argument. The team should examine the changed semantics, relevant error conditions and all affected management paths. Existing evidence about unrelated physical installation characteristics may remain useful. The correct scope is determined by the change and its dependencies, not by a blanket rule to discard or retain everything.

Now consider a proposed correction to support text. The label “documentation only” is not enough to classify it as harmless: the text might instruct an operator to enable a recovery path or interpret a critical warning. Conversely, a verified correction to an unrelated descriptive paragraph need not force full functional retesting. Establish the affected use and information before choosing the evaluation burden.

An emergency change needs the same reasoning at a speed and depth appropriate to the situation. Define the intended result, likely effects, decision authority, feasible checks and recovery options. Record constraints on use and the follow-up work required when immediate evaluation is limited. An emergency explains why the available evidence may be incomplete; it does not turn the incomplete evidence into a complete demonstration.

A plan to reverse a change is also not sufficient by itself. Some actions cannot be cleanly undone, especially where data, external state or physical effects have changed. Where rollback is not feasible, plan another means of containing the effect or restoring capability. The choice belongs in the design and change analysis, not in a template field filled with the word “rollback.”

Continuous monitoring, periodic assessment, event-triggered evaluation and independent review are complementary. Monitoring can reveal relevant operating conditions without establishing every property of a design. Periodic work can examine questions that are not efficiently measured at runtime. A specific change or incident can justify targeted re-evaluation. The combination should support the system’s actual claims and obligations rather than a universal slogan about continuous testing.

4.8 Exercise: who owns the unresolved recovery need?

Situation. The gateway supplier is ready to deliver the NORMAL-mode design. The service owner still has not established how quickly urgent maintenance must be possible during an identity-service outage. The project manager asks the security lead to accept the recovery risk so that delivery can proceed. What should the security lead contribute?

Worked answer. The security lead should make the unresolved need, its potential design implications and the limits of the available evidence explicit. The missing decision concerns intended service capability as well as protection. It requires the service owner’s operational input, systems-engineering analysis of the alternatives, and a decision through the assigned authority. The security lead should not invent a service tolerance or assume a risk-acceptance power that has not been assigned.

The project can distinguish delivery of a specified component from acceptance of the integrated capability and permission for operational use. Those decisions may have different conditions. A supplier’s readiness does not establish the customer’s readiness, while the customer’s unresolved need does not automatically prove a supplier defect. Record the contractual and technical positions accurately, including any limited use that the proper authority may consider.

Further challenge. Would producing a separate SSE plan resolve the issue? Only if the planning process establishes the missing work, participants, dependencies and decisions and they are acted upon. The existence of another plan would not supply the absent operational requirement or the evidence needed to evaluate the design.

Carry forward. One life cycle does not mean one team performs every task. It means that requirements, technical work, evidence and decisions remain connected across organizations and over time. With that foundation, the next part develops mission and stakeholder needs, adversity analysis, requirement allocation and early assurance planning in greater depth.

PART II — FRAME THE PROBLEM AND PLAN THE EVIDENCE

5. Mission, stakeholders, assets and protection needs

5.1 Begin with the useful outcome

A service owner asks for secure remote administration. The security team identifies an authentication obligation. The infrastructure team has a gateway in mind. None of those contributions yet explains what the organization needs the administrative capability to accomplish. Before developing a protective design, the team must understand the work that makes the capability worth providing.

In the logistics example introduced in Part I, field teams need timely, correct dispatch instructions. Administrators support that outcome by maintaining the rules used by the dispatch application. Their work includes identifying the active configuration, preparing a change, activating an authorized configuration and determining whether the change took effect. During disruption, they may need to restore a previously approved configuration. These are positive capabilities, not merely opportunities for unauthorized access.

The system of interest remains the remote-administration capability defined in Chapter 2: managed administrator endpoints, the gateway, receiving management endpoints, enforcement configuration, and the human roles and procedures included in that capability. The operational dispatch service and the enterprise identity service remain outside this boundary, as relevant interfacing systems and dependencies. This chapter does not expand an administration claim into assurance of the complete logistics operation.

Mission analysis establishes the purpose and context for engineering choices. Here, a mission can be a business service, public function or operational task. Ask what stakeholders need to achieve, under which conditions, and what would make the capability unsuitable. NIST connects this work to stakeholder needs and system requirements as engineering processes. [1, Appendices H.1–H.3]

Security contributes to that analysis by examining the assets, relationships and capabilities whose loss or misuse would matter. It does not replace the service owner's explanation of the work. Nor should a security specialist invent an operational tolerance simply because the project needs a number for its specification. A well-defined protection problem begins with the people who understand the consequences and the people who understand how the system could produce them.

5.2 Distinguish a need, an objective and a selected solution

A stakeholder need expresses something necessary or valuable in the intended use. An objective provides a direction or basis for judging success. A requirement expresses an obligation on a specified subject, at an appropriate level of precision. A proposed solution describes a way to satisfy one or more of those obligations. The distinctions are useful even though organizations may use different document names for them.

"We need to restore dispatch configuration during an identity outage" is a statement about capability. "We need a second gateway" is already a proposed response. The second gateway might help with a gateway failure while doing nothing about the shared identity dependency.

Recording it as the need can prematurely remove other options, such as reducing the maintenance required during disruption or providing a narrowly scoped restoration arrangement.

Similarly, “prevent unauthorized changes” expresses a protection need but does not specify every function that makes a change authorized. The project still has to establish identity, permitted actions, target configuration, relevant operating mode and the authority to approve the change. A need is not defective because it lacks all this detail. It becomes defective when the team treats the absence of detail as if the implementation decision had already been made.

An objective such as reducing restoration time is useful only in relation to the capability restored. Restoring an administrator’s login is different from restoring the management service; restoring that service is different again from restoring correct dispatch. The remote-administration project can contribute evidence about the first two and about the configuration change it enables. It needs the operational service owner to evaluate the wider consequence.

The development work therefore moves between levels. An early operational expectation informs a candidate requirement. A feasibility study may reveal a conflict that requires the expectation to be reconsidered. That reconsideration is not permission to rewrite a mandatory obligation to match a preferred product. It is a controlled discussion with the source of the need or obligation, preserving the original concern and the reason for any agreed change. NASA’s stakeholder and technical-requirements processes provide public examples of this iterative relationship. [4, Sections 4.1–4.2]

5.3 Find stakeholders across the life cycle

The person funding the gateway is not the only stakeholder. Administrators know which maintenance tasks are necessary. Operators know when a delayed correction becomes operationally serious. The identity team knows the service’s revocation and outage behaviour. Application engineers know which interfaces can change the rules. Maintainers understand replacement, upgrades and recovery. Evaluators need a design that can be examined. The people responsible for information handling and contractual obligations identify constraints that may not emerge from a functional workshop.

Stakeholder identification is not a meeting-invitation exercise. It is a way to discover needs, knowledge and responsibilities that would otherwise be missing from the design. A stakeholder may provide crucial information without possessing approval authority. Conversely, a decision authority may be entitled to accept a limitation without personally possessing the technical knowledge needed to evaluate it. Record the contribution and the decision right separately.

People affected by the service also matter even when they do not administer it. In this case, a misleading success message can lead the dispatch operator to act on an incorrect belief about the active rules. The operator’s need for an intelligible and accurate status is therefore relevant to the administration interface. The same reasoning applies to support staff who may inherit a system after the original project team has left.

Table 5.1 links selected stakeholder concerns to engineering work. The participants illustrate the relationships to establish, not a prescribed organizational structure.

Table 5.1. Stakeholder contributions to the administration capability

Participant	Question the participant helps resolve	Result needed by engineering
Dispatch service owner and operators	What corrective action is necessary during disruption, and what delay is tolerable?	Defined service scenarios, priorities and consequences of delay.
Administrators and maintainers	What information, tools, access and assistance are needed to perform the action?	Task sequence, support conditions and recovery constraints.
Application and infrastructure teams	Which paths can inspect, stage or activate dispatch rules?	Functions, interfaces, authoritative state and integration responsibilities.
Identity and security teams	What makes authority valid, current and revocable?	Authentication and authorization expectations, dependency behaviour and adversity scenarios.
Change authority and evaluators	Who approves a package, and what would demonstrate correct activation?	Approval semantics, evaluation criteria, required records and limits on conclusions.

A project can hold a focused discussion around one operational scenario rather than ask every participant to produce a separate security document. Have the administrator explain a correction from the moment the need is recognized to the point at which the operator knows what is active. At each step, ask what information and authority are required, what can be unavailable, and what another participant assumes has happened. Record disagreement as an unresolved input. Agreement achieved by suppressing the operational problem is not requirements validation.

5.4 Identify assets by their contribution and loss

An asset inventory is a useful starting point, but replacement cost is not a sufficient measure of importance. An inexpensive configuration file can determine the instructions issued by an expensive service. A privileged identity can influence several systems without appearing as a physical item. A restoration package may be valuable because its content and provenance allow a failed service to be recovered, not because the storage occupied by the package is costly.

For this example, relevant assets include the active dispatch-rule configuration, approved restoration packages, administrative authority, management-service availability and records needed to determine what action occurred. Their significance follows from what people rely on them to do. Confidentiality can also matter: credentials, sensitive configuration details and operational information require attention even where the main worked scenario emphasizes integrity and continuity.

NIST's treatment of protection needs connects asset value, susceptibility to loss and stakeholder consequences. It also distinguishes assets recognized by stakeholders from assets introduced or revealed by the system design. A signing service, privileged intermediary or recovery credential store may become important because of the selected architecture, even though stakeholders never initially asked for that particular component. [1, Sections 3.4–3.7]

Asset analysis should therefore be revisited during design. A gateway that becomes the sole enforcement point introduces reliance on its configuration, update path and administrative authority. A recovery service introduces assets associated with recovery access and activation. Those are not reasons to reject the design automatically. They are reasons to examine the dependencies it creates and the losses that could follow.

Existing business impact analysis can contribute more than an outage-duration figure. It may identify essential functions, dependencies, time-sensitive consequences and tolerances that help prioritize protection. NIST's business-impact-analysis guidance explicitly broadens the analysis beyond availability to other forms of loss. The engineering team should reuse applicable organizational work while checking that its scope and assumptions fit the selected system. [17, abstract]

A useful asset record captures what is valued, by whom, the relevant loss and the functions that depend on it. It need not assign a numerical value to every item. Where several stakeholders attach different significance to the same asset, retain those differences until the responsible participants resolve their implications. Averaging incompatible needs can conceal a binding constraint rather than settle it.

5.5 Describe losses without claiming that every cause is known

A loss statement describes an adverse consequence that matters to stakeholders. A scenario explains how that loss could occur. A requirement specifies what a system or participant must provide in response. Keeping these different lets the team recognize a serious consequence before it knows every causal pathway, while still requiring causal analysis before it chooses a design.

For the administration capability, L-01 is dispatch-rule alteration outside the authorized change arrangements, potentially resulting in incorrect operational instructions. L-02 is inability to perform necessary corrective administration within the agreed disruption window. L-03 is an unintended or inconsistent rule configuration produced by an otherwise authorized operation. L-04 is inability to establish the authoritative outcome of an administrative action when that information is needed for recovery or investigation.

These statements overlap deliberately. A stale privilege can contribute to L-01; a duplicated activation can contribute to L-03; an inaccurate completion message can complicate both recovery and investigation. A useful model permits several paths to one loss and one event to several losses. It does not force a single cause into every register row.

The loss descriptions also need boundaries. L-03 does not make the administration project responsible for proving the correctness of every dispatch algorithm. The project can establish that the intended, approved configuration was activated consistently and that its outcome was reported accurately. Whether that approved configuration expresses a sound logistics policy remains a wider operational and application-engineering question. The interface between those responsibilities must be explicit.

Calling a loss unacceptable expresses a stakeholder position; it does not establish that the engineering can make its occurrence impossible. The team must examine achievable protection, remaining uncertainty and the authority required to decide on the resulting capability. Where a stakeholder uses absolute language, ask which consequence, conditions and decision are actually intended. Do not quietly turn "unacceptable" into a promise of zero probability.

Threat knowledge belongs in this work. It can reveal an overlooked asset, demonstrate that an assumed boundary is ineffective or expose a new way to cause a familiar loss. The point of a mission-oriented starting position is to give that knowledge consequence and purpose, not to postpone it until requirements have become fixed. The next chapter develops those causal questions.

5.6 Develop protection needs as connected engineering inputs

A protection need states what must be protected, preserved, prevented or limited in relation to the identified assets and useful capability. It should be intelligible to the stakeholder who cares about the consequence while providing direction to those who must develop requirements. It is not yet a control selection or a complete architecture.

The author-developed Mission Protection Needs method makes purpose, assets, losses, protection needs and further engineering questions visible together. It is a teaching arrangement, not an exclusive method or a substitute for specialist analysis. Established stakeholder, mission, risk and requirements processes can support equivalent reasoning.

Begin by recording the intended outcome and the selected boundary. Identify the affected stakeholders and valued capabilities. Describe the losses that matter and the context in which their consequences change. Derive protection needs that respond to those concerns. Then challenge each need for missing positive capability, hidden solution choices and dependence on an unexamined assumption. That last step is important: a clear sentence can still express an incomplete problem.

Table 5.2 develops the two needs introduced in Part I and adds two more. The new needs extend the synthetic case; they do not represent findings from an actual deployment or a completed stakeholder workshop.

Table 5.2. Protection needs for the selected teaching case

Identifier and need	Main relationship	What still has to be engineered
PN-01: prevent unauthorized modification of dispatch rules.	L-01; integrity of configuration and administrative authority.	Covered actions and paths; identity, permission and approval semantics; revocation; enforcement and evaluation.
PN-02: preserve necessary corrective administration during defined disruption.	L-02; usable maintenance and restoration capability.	Required action, disruption conditions, time window, personnel, dependencies and acceptable operating restrictions.
PN-03: prevent or limit unintended configuration effects from authorized administration.	L-03; consistency of active rules and controlled change.	Package identity, target and version checks; concurrent and repeated requests; recovery from incomplete actions.
PN-04: preserve a dependable account of administrative actions and their authoritative outcomes.	L-04; recovery, operational understanding and investigation.	State meaning, outcome records, correlation, protection, availability and reconciliation after interruption.

PN-01 does not absorb PN-03. A properly authenticated and authorized person can choose the wrong target, submit an outdated package or misunderstand a retry. Equally, PN-04 is not satisfied by collecting large volumes of logs. The record must answer the relevant question accurately, including whether an action was requested, accepted, committed, rejected or left with an unresolved outcome.

Nor is PN-02 a demand to preserve every administrative function during every outage. The required capability may be narrower than the normal service. A limited restoration action might support the mission while arbitrary rule creation remains unavailable. The distinction opens design alternatives without pretending that the disruption has no operational cost.

The set remains scoped. Credential protection and confidential information handling are continuing concerns, but this worked slice does not claim to derive every requirement associated with them. A real project must retain an explicit coverage record for those concerns. Selecting a manageable teaching example does not create an exemption from the rest of the applicable obligations.

5.7 Turn continuity aspirations into an operational envelope

Part I left the disruption tolerance unresolved. To develop the example, assume the fictional stakeholders now supply the following additional expectations. These are invented case inputs, not reported decisions or recommended service levels. During an identity-service outage lasting up to two hours, designated staff need to be able to activate a previously approved restoration package within fifteen minutes of the service owner declaring that corrective administration is required. Arbitrary creation of new rules is not required during that disruption.

The fifteen-minute interval includes mobilizing the required people, obtaining the relevant authority, accessing the maintenance capability, performing the restoration and confirming the active configuration. It does not include the time before the corrective need is declared. The wider dispatch-service analysis must examine detection and decision delay; this administration requirement cannot claim to control a clock that starts earlier than its stated trigger.

“Previously approved” also needs a meaning. The package must be identifiable, applicable to the target service and still permitted for restoration under the governing change arrangements. Its existence in an archive is not proof that it is current, uncorrupted or operationally appropriate. The designation is an input to engineering and evaluation, not a substitute for them.

This expectation gives PN-02 an observable result and a defined adverse condition. It also exposes a feasibility problem. If a candidate design requires the unavailable identity service for every possible restoration action, it cannot perform that action within the stated window during the specified longer outage. That conclusion follows from the assumed dependency and time conditions. It is not a test result or a final selection of a recovery architecture.

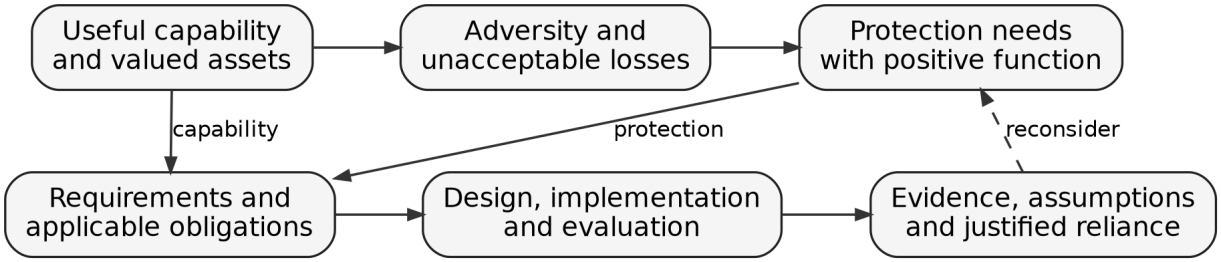
The team must now investigate alternatives. It can seek a suitable independent, governed restoration capability; reduce or relocate the maintenance function; or return to stakeholders with evidence that the expectation cannot be met within the available constraints. It cannot quietly label a password-only bypass “resilience” or assume that urgency waives POL-01. Necessary recovery authority and authentication remain engineering and governance questions.

The same discipline applies to adaptation. A need to operate under a changed mission or supporting-service arrangement requires an account of which capability changes, what remains

protected and how the modified use will be evaluated. Anticipation, withstanding, recovery and adaptation help frame resilience, but the system-specific outcome and limits must still be developed. [2, Chapter 1 and Section 2.1.1]

The following schematic brings together the relationships developed here.

Figure 5.1. From useful capability to protection and evidence



Original schematic. The two concerns converge in one requirements and design effort. Evidence can send either concern back for correction; the diagram does not prescribe a separate security lifecycle.

5.8 Resolve conflict and establish readiness for requirements work

Protection needs can conflict in implementation without being contradictory in purpose. Promptly withdrawing administrative authority protects against misuse. Preserving emergency maintenance protects service continuity. The conflict becomes concrete when the same central service supplies both the normal authority information and the only means of restoration. A list containing both needs is necessary, but not sufficient to resolve their relationship.

Before a design commitment, establish which conditions are binding, which tolerances permit trade-offs and which participants can decide. An engineer may explain that an offline component cannot learn a newly issued revocation through a channel that is unavailable. A service owner may explain the consequences of losing maintenance. An identity-policy authority may determine the permitted basis for recovery access. The architecture choice requires those contributions together.

A concise needs baseline can include the scope, stakeholder concerns, protection needs, assumptions, unresolved conflicts and planned resolution. Baseline does not mean immutable or fully demonstrated. It means that the project has an identified reference against which later development and change can be understood. If a need changes, retain its earlier meaning and the reason for revision.

For the teaching case, PN-01 through PN-04 and the new disruption expectation are sufficiently specific to guide the next analysis. They are not sufficient to release a complete product specification. The status of A-03 changes: the earlier assumption that urgent restoration can wait for identity recovery is not adopted for the new case conditions. A-01, concerning path coverage, and A-02, concerning revocation propagation, remain to be established. This is learning made traceable rather than learning hidden in a rewritten paragraph.

The result can be carried into existing IT records. The service catalogue can identify the restricted disruption capability. The business impact record can preserve the timing rationale and wider dependencies. The risk register can link the losses to scenarios. Requirements records can reference the need and source obligation. These records can share information without becoming interchangeable or requiring a new document for every concept.

Information-flow route. Appendix E applies the same needs-to-evidence reasoning to supplier diagnostics, where the positive support function and confidentiality obligation must be engineered together.

5.9 Exercise: the backup is not the need

Situation. A manager replaces PN-02 with "provide a second gateway with identical configuration." Both gateways depend on the central identity service. The manager argues that this is more precise and therefore ready for procurement. What should the project retain, and what should it challenge?

Worked answer. Retain the second gateway as a candidate design contribution, not as a replacement for the operational need. It may address some gateway failures, and that possibility deserves examination. It does not address the stated identity-service outage if restoration through either gateway requires that service. Preserve PN-02 and the fifteen-minute restoration expectation, including its starting event and restricted action. Compare the gateway option against those conditions and identify the unmet dependency question.

Precision of wording is not enough. A precisely described product can be an inadequate response to a correctly understood need. Conversely, a needs statement need not prescribe the product to be useful. The engineering work connects the two through requirements, alternatives, assumptions and evidence.

Carry forward. The chapter's outputs are an intelligible purpose, scoped assets and losses, protection needs, a bounded continuity expectation and visible unresolved assumptions. The next chapter asks how adverse events and system interactions can defeat those needs, so that the protective response is based on more than a list of desired outcomes.

6. Adversity, threats and loss scenarios

6.1 From a loss of concern to a causal question

Knowing that unauthorized rule changes are unacceptable does not establish how they could occur. The next task is to examine the circumstances, events and interactions that could produce the loss. This is where threat intelligence, vulnerability information, operational experience, failure analysis and knowledge of the architecture become practical engineering inputs.

Consider the following hypothesis. An administrator's authority is revoked at the central identity service, but a receiving management endpoint continues to use an earlier permission record. A queued activation request is then committed after the permitted withdrawal interval. The sequence connects an authority change, a dependency, internal state, timing and a consequential action. It directs work more effectively than a general entry labelled "access control weakness."

The sequence remains a hypothesis until supported by evidence about the actual system. A team can investigate a plausible pathway without claiming that it has found a vulnerability. Equally, the absence of an incident report does not establish that the pathway is impossible. The task is to identify what must be examined and what a proposed design needs to prevent, contain, detect or recover from.

Risk assessment supplies a useful discipline for establishing scope, assumptions, sources and the relationship among threats, vulnerabilities, conditions and impacts. NIST recognizes that assessments can begin from threats, assets or vulnerabilities, depending on purpose and information available. A mission-oriented engineering approach can use those different entry points without losing its connection to stakeholder consequences. [15, Sections 2.3 and 3.1–3.2]

6.2 Keep the analytical terms distinct

An adverse condition is a circumstance in which the system has to function or protect against loss. An initiating event begins a sequence of interest. A threat source can cause an adverse event; it need not be represented only by the name of an external criminal group. A vulnerability is a weakness that can be exploited or otherwise contribute to the relevant harmful outcome. A dependency condition may enable the sequence without being a vulnerability in a supplied product.

A loss scenario explains how events and conditions could produce a consequence. Identify the affected function, relevant state, interactions and point of loss. The scenario record used here is a teaching aid; retain the governing risk method and the terminology required by safety or reliability analyses.

In the revocation example, the initiating event is withdrawal of authority. Continued use of stale permissions is a candidate contributing condition. The protected function is dispatch-rule activation. The potential loss is an unauthorized change after authority should have ceased to be usable. Whether the stale state is an implementation defect depends on the requirement and circumstances. A specified, permitted propagation interval is not itself evidence of a defect.

Cause, consequence and evidence quality also differ. A missing test report is not necessarily a causal weakness in runtime enforcement. It is a weakness in the support available for a claim about enforcement. The project may therefore have both a system-risk question and an

assurance question. Conflating them can lead either to unnecessary redesign or to unjustified confidence.

Do not make a score perform the work of a scenario. A severity category can help communicate consequence. A likelihood estimate can help prioritize attention when its basis is appropriate. Neither explains where authorization is checked or how a queued request becomes effective. Those are engineering questions that require a model and relevant evidence.

6.3 Construct a scenario around an interaction

Start with a function or loss that needs examination. Identify who or what initiates the relevant action, what state the system is in, what it relies upon, and what exchange crosses an interface. Describe the sequence in enough detail to expose the condition that makes the loss possible. Then identify alternative explanations and the evidence that would support or contradict the account.

For LS-01, the stale-authority scenario, begin with NORMAL mode and an established privileged session. At time t0, the authoritative identity service records revocation. A management request has been staged or queued, but has not yet changed the active dispatch configuration. The receiving endpoint later commits that request using information that no longer establishes current authority. The engineering question is whether the actual design permits this sequence beyond the permitted interval.

Several details change the answer. Revocation may concern an identity, a role or a single permission. A session may remain open while its authority to change rules has ended. A gateway may refresh its state while an application endpoint uses a separate cache. A request accepted before revocation may not be committed until after it. None of these distinctions is captured by demonstrating that a revoked person cannot create a new login session.

The relevant observations would include the authoritative revocation record, permission distribution, cache policy, receiving-end decisions, queue handling and the configuration commit. An application log stating “request accepted” is not necessarily a record of commit. The analysis needs an explicit definition of the event that changes the protected asset. This will become a requirement and evaluation issue in Chapters 7 and 8.

The scenario also has contrary possibilities. The endpoint might re-evaluate authority immediately before commit. A bounded lease might prevent an older permission record from being used. An independent enforcement mechanism might reject the queued action. Those are candidate responses to examine, not facts to infer merely because the architecture diagram contains an authorization component.

A good scenario record therefore ends with a question and a proposed resolution activity, not always with a diagnosis. In early engineering, “determine whether queued work is re-authorized at commit” may be the correct output. Once the design is known, the record can be refined into a requirement, test condition or accepted limitation with the appropriate authority.

6.4 Examine intentional and unintentional paths

Adversarial reasoning asks how a person or other actor could deliberately induce the sequence. Could a revoked administrator keep an existing session alive, submit requests immediately before revocation or target a less visible endpoint? Could a compromised build environment

change both the enforcement code and the status reported to reviewers? Could an insider with legitimate access misuse a permitted maintenance action?

Operational and failure reasoning asks complementary questions. What happens when the acknowledgement of a successful activation is lost? Does an administrator retry? Can the system distinguish repetition of the same transaction from a new change? What happens when a process restarts between updating the active configuration and recording the outcome? Could a well-intentioned operator restore the wrong package because the displayed target is ambiguous?

These analyses often converge on the same system property. An explicit transaction identity and dependable outcome reconciliation can matter both to malicious replay and to an ordinary retry after network interruption. The shared response does not make the causal stories identical. Keep enough of each story to evaluate the relevant timing, authority and human-use conditions.

Security must also coordinate with other specialties. A security-induced service denial can produce a safety-relevant consequence in some domains; a reliability fault can create an insecure recovery condition. The security analyst should bring the interaction to the appropriate specialists rather than claim that a loss-oriented vocabulary makes every hazard a security-owned issue. Shared models and reviewed interfaces preserve integration without dissolving specialist competence.

The logistics case is intentionally limited. It does not establish a clinical, flight, industrial-control or weapon-system safety argument. Transfer to such domains requires their actual functions, physical effects, tolerances and specialist analyses. The transferable method is to connect causes, interactions and losses, not to copy the example's time values or recovery decision.

6.5 Use complementary methods for different questions

No single analytical method exposes every important feature of a system. The selection should follow the question, the available representation and the significance of a missed condition. A threat-oriented review can examine adversarial access and misuse. A function-centred failure review can examine what happens when a contribution is absent, late, incorrect or inconsistent. A control-structure analysis can examine commands, feedback and the circumstances in which otherwise legitimate actions become unsafe.

The published STPA Handbook outline organizes System-Theoretic Process Analysis around purpose, control structure, unsafe control actions and causal scenarios. This progression helps distinguish component analysis from examination of controlled interactions. The example below illustrates that distinction; it is not a completed application of STPA. [16, handbook outline, Chapter 2]

For dispatch-rule activation, the administrator issues a request, the capability evaluates it, the application management endpoint changes the configuration, and the administrator receives feedback about the result. Ask what occurs if the request should not be issued in the current state; if a needed restoration request cannot be issued; if the action occurs at the wrong time; or if feedback causes the administrator to misunderstand the active state. Each question exposes a relationship that a component inventory alone cannot answer.

A failure-oriented examination can begin with the identity service and ask what the administration capability does when the service is unavailable, delayed, inconsistent or unable to provide acceptable authority information. It should then follow the consequence through receiving-end behaviour. "Identity service unavailable" is not yet the complete loss scenario:

dispatch may continue normally until corrective administration becomes necessary, or the design may have a suitable independent restoration capability.

Table 6.1 is a method-selection aid for this book. It intentionally describes useful questions and limits rather than suggesting that a named technique automatically provides adequate coverage.

Table 6.1. Choosing an analytical view

View	Question it makes easier to examine	Important limit
Threat and misuse view	How could an actor obtain influence, exceed authority or manipulate a relevant interaction?	Actor knowledge and the model may omit accidental or unfamiliar pathways.
Function and failure view	What follows when a required contribution is absent, late, incorrect or inconsistent?	Treating failures as independent can conceal shared causes and adversarial coordination.
Control and feedback view	Which command, state or feedback relationship can produce an unacceptable outcome?	The control structure and its assumptions need validation; the diagram is not evidence of actual behaviour.
Dependency and composition view	Which external or shared contribution is necessary for a protective or recovery claim?	Supplier-local evidence may not establish the receiving-system interaction.
Operational task view	Can people perform the required action with the available information, time and resources?	A tabletop discussion does not demonstrate the implemented interface or every operating condition.

These views can share a model and a scenario register. Their findings should be reconciled at the level of needs, requirements and design, not kept in isolated reports with incompatible boundaries. A documented reason for selecting a method is more useful than claiming that every project has completed a standard set of techniques regardless of relevance.

6.5.1 Work one causal structure far enough to test coverage

A scenario list becomes more useful when several pathways to the same loss are compared through the design and evidence needed to discriminate among them. For L-01, the consequential event is an unauthorized effective configuration change. Four candidate pathways illustrate different engineering questions: stale authority, an alternative writer, over-broad recovery authority, and an enabling-system change that affects both enforcement and the evidence used to review it.

The point is not to claim that these four branches exhaust the threat space. It is to make coverage reviewable. Each branch implies a different protective contribution and a different way to discover whether that contribution is actually present. If the design review considers only stale credentials, it can miss a direct writer; if the test suite exercises only the main writer, it can miss a supply or deployment path that changes the enforcement itself.

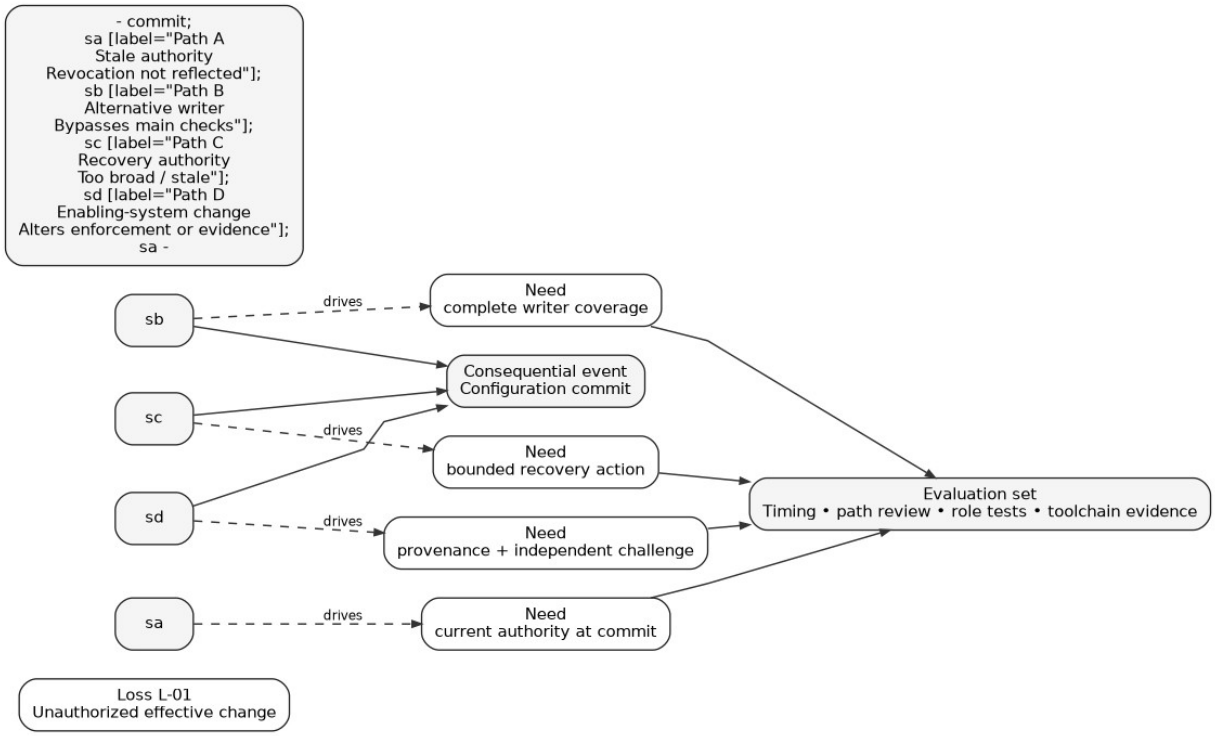


Figure 6.1. One loss, several causal pathways and discriminating evidence

Original causal-analysis schematic. The branches are candidate pathways to the same loss, not probabilities or proven defects. The lower relationships show how each pathway drives a distinct engineering and evaluation obligation.

Use the diagram as a challenge aid: for each branch, ask what condition must hold, what design contribution is supposed to prevent or contain the consequence, what evidence could distinguish a working contribution from an assumed one, and what other branch could defeat the same claim. This converts a threat or failure discussion into requirements, architecture and evaluation work without pretending that a diagram is a complete threat model.

6.6 Develop a manageable scenario set

The following selected scenario hypotheses extend the case through different causal questions. They are neither a complete threat model nor observed product defects. Their variety helps develop requirements beyond the repeated question of whether a user authenticated.

LS-01 concerns stale authority at commit following revocation. LS-02 concerns an alternative management path that can activate rules without the checks attributed to the gateway. LS-03 concerns an authorized activation repeated or applied against an unexpected configuration version after an interrupted exchange. LS-04 concerns an identity-service outage overlapping with a declared need for corrective administration. LS-05 concerns residual normal-mode authority, queued work or misleading status during entry to or exit from a proposed recovery arrangement. LS-06 concerns a change to a build or deployment enabling system that alters enforcement or hides a management path from the evidence used to assess it.

Table 6.2 connects these scenarios to their immediate engineering questions. Several share losses and needs. That is a reason to coordinate their treatment, not to delete them as duplicates.

Table 6.2. Scenario-to-work connections

Scenario	Related loss / need	Immediate engineering question
LS-01: revoked authority remains effective	L-01 / PN-01	Where is current authority established at commit, including queued and already authenticated work?
LS-02: an alternative endpoint bypasses checks	L-01 / PN-01	What is the complete relevant path set, and what enforces protection at each receiving path?
LS-03: retry or version conflict changes the wrong state	L-03, L-04 / PN-03, PN-04	How are request identity, expected configuration, commit and outcome reconciliation defined?
LS-04: identity loss prevents urgent restoration	L-02 / PN-02	Can the restricted restoration action be completed within the assumed window without the unavailable dependency?
LS-05: recovery transition preserves unintended authority	L-01, L-03, L-04 / PN-01–PN-04	Which grants, requests and state may cross the transition, and which must be re-evaluated?
LS-06: enabling-system change defeats enforcement or its evidence	L-01, L-04 / PN-01, PN-04	What configuration and provenance support both the supplied capability and the evaluation instruments?

A scenario can produce several requirements. LS-03 needs more than a generic instruction to reject duplicates. A request may have succeeded even though the reply was lost. Blindly rejecting the retry without allowing the operator to establish the authoritative outcome can leave the operational problem unresolved. The design needs a coherent account of request identity, permissible repetition, state change and feedback.

A requirement can also address several scenarios. Receiving-end authorization contributes to LS-01 and LS-02; controlled transition rules contribute to LS-01 and LS-05. The common requirement needs a rationale describing those relationships. Merely entering multiple scenario identifiers in a matrix does not explain why the response is sufficient.

6.7 Follow shared causes and evidence dependencies

Two gateways can fail together because they share an identity service, deployment pipeline, configuration source or power arrangement. Redundancy against one local failure is not independence from every common cause. Before using a component count as a resilience argument, identify which failure or compromise the duplication is meant to address and which contributions remain shared.

LS-06 adds an assurance dimension. Suppose the same deployment automation installs the enforcement rule and generates the inventory used to certify that all management paths are covered. An error or compromise in that automation could affect both the system and the

apparent evidence of its configuration. Agreement between the two records is then weaker support than it first appears.

This does not mean that all shared tooling is unacceptable or that every evidence item must be produced by a separate organization. It means that the argument must account for dependence. A receiving-system inspection, an independently obtained path observation or review of a separately controlled configuration source may provide useful challenge. The appropriate method follows the claim and consequence, not a universal rule to duplicate every tool.

NIST's assurance treatment recognizes that confidence in components does not automatically establish confidence in their composition. Interfaces, context, common conditions and the architecture remain part of the reasoning. The same discipline applies when combining evidence items: several reports can repeat one untested assumption rather than provide several independent grounds for confidence. [1, Appendix E.7 and Appendix F]

The project should record which conditions a scenario assumes. For LS-01, an available identity service and a bounded communication environment differ from a partitioned network. For LS-04, the outage begins in the identity dependency, but the restoration arrangement may also require name resolution, time information, approval personnel or a reachable credential repository. Following those secondary dependencies can change the architecture trade before expensive implementation work begins.

6.8 Prioritize without manufacturing precision

A project rarely has equal resources for every scenario. Prioritization should consider consequence, plausible exposure, uncertainty, design sensitivity and the cost of discovering a problem late. Mandatory obligations can also impose evaluation duties independently of the project's preferred risk ranking. The selection needs a recorded basis that other participants can challenge.

Avoid treating an ordinal label as a precise probability. A "high" likelihood assigned through a workshop does not become a defensible numerical failure rate when copied into an availability calculation. A vulnerability score does not determine the dispatch consequence by itself. Where quantitative analysis is useful, define the event, time interval, dependence assumptions, data source and uncertainty before doing arithmetic.

Qualitative reasoning can still be decisive. The central-only dependency conflict in LS-04 does not require a probability estimate to show that the proposed restoration action is unavailable during the stated outage. The unknown frequency of that outage affects risk and investment decisions, but it does not change the conditional behaviour of the design.

Some uncertainties justify investigation before selection. If the team does not know whether an application exposes another activation endpoint, A-01 can materially change both requirements and architecture. Resolving the path set may therefore take priority over optimizing the normal gateway's response time. The work is prioritized by its effect on the decision rather than by the ease of closing an action.

Record scenarios not selected for further analysis and the reason. A reason might be that another controlled analysis covers the same interaction, that the event is outside the agreed use with a defensible exclusion, or that the remaining uncertainty requires explicit escalation. "Out of scope" is not sufficient where the unexamined condition can invalidate an in-scope claim.

6.9 Translate findings into design and evaluation work

The output of adversity analysis is not necessarily another control. It may be a refined need, a requirement, an interface expectation, a design alternative, a test condition, a monitoring requirement or a decision about a limitation. A useful disposition says which output is needed, who will produce it and which scenario it addresses.

For LS-01, the requirement needs to refer to the event that changes the protected configuration, not simply login. The design work must identify where authority is checked and what happens between that check and commit. The evaluation work must examine revocation across queues, sessions, receiving paths and the defined time conditions. The risk record retains the consequence and any remaining limitation.

For LS-03, the response includes positive capability: the administrator must be able to determine whether a request took effect. For LS-04, the response includes human work and dependency independence as well as authentication. For LS-06, the response reaches into supplier work and enabling systems, including the provenance and applicability of evidence. These are connected engineering contributions, not a claim that the security specialty owns every activity.

Where analysis discovers a conflict, return it to the relevant requirement and decision. Do not let the scenario register become a list of concerns detached from the design. Equally, do not mark a scenario resolved solely because a requirement has been written. A requirement establishes intended behaviour; implementation and evaluation establish what support exists for believing that behaviour is provided.

6.10 Exercise: identical recovery gateways

Situation. A supplier offers two recovery gateways and successful tests of each gateway's local hardware failover. Both obtain recovery credentials from a repository reached through the unavailable enterprise identity service. The proposal states that LS-04 is closed. What has been demonstrated, and what remains unresolved?

Worked answer. The local failover results may provide useful evidence for the tested gateway-failure conditions. They do not establish restoration during the identity-service outage in LS-04. The credential-retrieval dependency prevents the proposed arrangement from being assumed independent of that outage. Retain the local evidence within its scope, examine whether another permitted credential and authority arrangement exists, and evaluate the entire restoration task against the fifteen-minute case expectation.

The issue is not that the supplier's work is worthless. It is that the claimed conclusion concerns a different adverse condition and a wider interaction. A requirement, design change or revised operational expectation may be necessary. Until the relevant evidence and decision exist, the scenario remains unresolved for the receiving-system claim.

Carry forward. Adversity analysis has produced specific causal questions, a selected scenario set, design implications and evaluation obligations. Chapter 7 translates these into requirements and allocations while preserving the useful service, source obligations and unresolved choices.

7. Requirements, policy, constraints and allocation

7.1 Requirements connect purpose to accountable work

A requirement is useful when another person can understand what is required, why it is necessary, which subject must satisfy it and how its satisfaction will be evaluated. A sentence containing “shall” is not enough. The set of requirements must also be coherent: individual statements can each be clear while collectively leaving a critical interaction undefined or demanding incompatible behaviour.

The administration example now has a purpose, a selected system boundary, four protection needs and six scenarios. It also retains POL-01, the illustrative requirement for MFA in remote administration. These inputs have different origins and authority. The engineering task is to develop a technically meaningful response without losing the original obligations or pretending that a scenario is already a design decision.

Requirements engineering identifies needs, develops requirements at appropriate levels, validates their adequacy and manages their relationships and change. ISO/IEC/IEEE 29148 addresses these processes and information products. [13, public scope] The following original case shows what those activities need to resolve.

The important shift for an IT practitioner is not from “controls” to a superior set of words. A selected control may already impose a requirement. The additional work is to establish scope, parameters, allocation, interfaces, failure behaviour and evaluation in the particular system. The output should improve the control implementation and assessment record, not create a disconnected technical specification that the operational team never sees.

7.2 Preserve levels and sources

A stakeholder requirement expresses what the capability must provide for its users and other stakeholders. A system requirement expresses an obligation on the selected system. An allocated requirement assigns a necessary contribution to an element, interface or participant. These levels are related but are not interchangeable. A component can satisfy its allocated requirement while the overall requirement remains unmet because another contribution or interaction is missing.

For the teaching case, STK-01 states that designated staff need to carry out authorized dispatch-rule maintenance with a dependable indication of the resulting configuration. STK-02 states the restricted restoration expectation developed in Chapter 5: a previously approved restoration package must be activatable within fifteen minutes of a declared corrective need during the specified identity-service outage. These are stakeholder-level teaching statements. Their derivation into technical and procedural obligations follows below.

POL-01 remains a separate source. It applies to remote administration, not merely to the dispatch-rule activation slice selected for detailed teaching. A system requirement may therefore trace both to a stakeholder need and to the policy obligation. Other requirements can arise from interface compatibility, physical limits, contractual commitments or enabling-system arrangements. Not every legitimate requirement originates in a threat or in one mission-loss scenario.

NIST distinguishes stakeholder requirements from the technical view developed for a particular system solution. That distinction helps preserve the purpose while allowing necessary design refinement. It does not require all requirements to be implementation-free at every level: a justified allocation or imposed constraint can legitimately restrict the solution. The restriction and its source should be visible. [1, Appendix C.2 and Appendices H.2–H.3]

For example, the project may be required to use an enterprise identity service in NORMAL mode. That is an imposed interface constraint, not a universal security principle. A receiving endpoint may then need to interpret a defined authority response from that service. The technical requirement can name the interface where necessary, while the record retains the constraint's origin and the need to revisit its consequences for recovery.

A design decision should not be disguised as an externally imposed requirement. If the team selects a particular gateway because it meets the evaluated criteria, record the choice and rationale. If that choice creates lower-level requirements, trace them to the decision as well as to their parent needs. This allows later engineers to distinguish what is mandatory from what can be reconsidered when technology, cost or operating conditions change.

7.3 Define the behaviour before refining the sentences

Teaching Edition TE-02 develops the profiles left incomplete in Part I, adding fictional parameters and definitions for review. It remains a candidate specification. Appendix C later selects and completes one bounded branch without changing this development history.

The selected slice is normal-mode dispatch-rule maintenance and its relationship to restricted restoration. Other remote administrative functions remain subject to POL-01 and require their own coverage. A real specification would also need complete protocol bindings, cryptographic profiles, deployment details, information-handling requirements and the requirements of the operational dispatch service. The example does not silently declare those matters satisfied.

A package is an identified collection of dispatch rules with a content identifier, intended target, version and change-approval reference. A staged package has been received for examination but has not become active. An activation request identifies the package, target, expected current configuration and a transaction identifier. Acceptance for processing means the request has entered the controlled workflow; it does not mean that activation has occurred.

Commit is the event at which the authoritative management interface designates the requested configuration as active for the target. The outcome record must distinguish that event from receipt, staging or queuing. The receiving operational service must have an agreed interface meaning for this designation. It is not, by itself, proof that every operational consumer has loaded the configuration or that the resulting dispatch decisions are correct. Those wider conditions require separate integration and intended-use evaluation.

This distinction resolves a problem already visible in SR-03. If the requirement concerns withdrawal of authority to change rules, testing only the arrival of a request does not address a queued action that changes rules later. The system must define the point of effective change and relate authorization to that point. Otherwise a timing requirement can be precise about the wrong event.

The profiles in Table 7.1 give the working semantics. They are controlled teaching definitions: changes to them can change requirement meaning even when the requirement sentence is unchanged.

Table 7.1. TE-02 definitions used by the normal-mode requirements

Profile	Working definition for this example	Scope and limitation
ADM-P1	NORMAL-mode maintenance includes inspecting the active version, staging a package, requesting activation and querying its outcome. The illustrative workload is up to five concurrent administrative sessions and ten activation attempts per minute.	A valid, non-conflicting activation has a ten-second completion target from receipt of the complete request. These values are invented, not recommended operating settings.
AUTH-P1	Authority is evaluated for the identified person, action, target and mode. The role must permit the action, and the package must have an applicable approval record. NORMAL-mode status is usable only within the withdrawal and freshness limits in SR-03 and SR-05.	Change approval and the operator's current authority are separate inputs. An approved package does not authorize every person to activate it.
MFA-P1	Each accepted request is bound to a currently valid authentication context establishing the identified person's use of MFA. The receiving capability validates the context's origin, integrity, intended recipient, validity period and binding to the session.	This does not require a new user challenge for every request. The protocol, trust material and applicable authentication-strength parameters must be bound before implementation evaluation.
NET-P1	The illustrative NORMAL envelope provides authenticated service exchanges, delivered request/response round trips of no more than two seconds and interruptions of no more than five seconds. The ADM-P1 workload applies.	Longer outages and invalid responses require separate evaluation; they are not assumed away by a NORMAL-mode performance claim.
TIME-P1	The example assumes a demonstrable upper bound of two seconds on uncertainty when relating source and receiving timestamps. Freshness decisions use a conservative upper bound on elapsed time.	A system unable to establish the required time relationship cannot treat an uncertain authority record as demonstrably fresh. Local monotonic-time behaviour also needs evaluation.
PKG-P1	An activation identifies a unique transaction, package content, intended target, expected active version and applicable approval reference. Reuse of an identifier for different activation content is invalid.	Package approval and content identity do not prove that the package's operational policy is correct.

The ten-second completion target applies to a valid, authorized, non-conflicting activation within the defined service envelope. It does not require invalid requests to succeed. However, the system cannot satisfy the positive capability requirement merely by rejecting every request. Evaluation needs examples that should succeed as well as examples that should be rejected.

NET-P1 qualifies the performance environment; it does not assume that every submitted security input is trustworthy. Invalid, manipulated or stale information remains part of the scenario set, and the security checks cannot be assumed successful simply because a delivered exchange meets a timing bound.

The profile definitions also expose remaining work. MFA-P1 identifies required authentication properties but does not select a protocol or cryptographic implementation. That binding is a recorded implementation and interface task, not an invisible assumption that any supplier mechanism will be adequate. Similarly, the NET-P1 timing envelope requires a representative evaluation environment; placing a bound in a table does not demonstrate that the enterprise service provides it.

REC-P1, the recovery authority and operating profile, is intentionally not treated as complete. The functional restoration target is known for the example, but the permitted independent authority arrangement, personnel access and transition mechanism require the design and policy work described in Section 7.6. This is a visible blocker to releasing a complete recovery specification, not a reason to omit the need from the system requirements.

7.4 Write requirements with an identifiable obligation

A practical requirement identifies the subject, relevant condition, required behaviour or property, and the criterion or controlled definition that makes evaluation possible. The rationale belongs with the record but should not obscure the obligation. References to other controlled definitions can keep a statement readable, provided the referenced information is available, versioned and sufficiently precise.

A useful drafting test is to ask whether two competent readers could disagree about what constitutes satisfaction. “The system shall quickly revoke access” leaves the starting event, affected authority, enforcement paths and allowable interval unclear. “The system shall be resilient” leaves both the useful capability and adverse condition unspecified. Adding a numerical target helps only after the measured quantity and circumstances are defined.

Requirements also need to be feasible and necessary at their level. An instruction to prevent all possible future losses is not made useful by an exact grammatical form. An instruction to use a particular mechanism may be legitimate when imposed or justified, but may unnecessarily eliminate better alternatives when inserted without rationale. NASA’s public requirements guidance provides checks for clarity, traceability, feasibility and verification; those checks are used here as writing aids rather than as an assertion of compliance with a NASA process. [18, Appendix C]

The following candidate requirements develop the Part I identifiers. They describe the selected teaching slice. Their status remains “candidate for technical review”; no design has been shown to satisfy them.

SR-00 — Useful maintenance. Under ADM-P1 and NET-P1, the remote-administration capability shall commit a valid, authorized, non-conflicting dispatch-rule activation and make its authoritative outcome available to the requesting administrator within ten seconds of receiving the complete request.

This refines the broad positive-capability statement in Part I. It now states an observable result and interval. Its feasibility depends on the selected implementation and receiving application

interface. The phrase “non-conflicting” refers to the expected-version check in PKG-P1, not a permission to classify unexplained failures as conflicts after the fact.

SR-01 — Authentication. The remote-administration capability shall accept a dispatch-rule modification request for processing only when the request is associated with an authentication context satisfying MFA-P1.

SR-02 — Authorization at effective change. The remote-administration capability shall commit a dispatch-rule activation only when authority for the identified person, action, target and current mode satisfies AUTH-P1 at commit.

The separation matters. SR-01 concerns admission to the workflow. SR-02 concerns authority when the consequential action occurs. A design may implement several checks, but a successful admission check does not eliminate the need to establish that authority remains applicable at commit. The implementation must address the interval between any check and the resulting state change.

SR-03 — Revocation deadline. Under NET-P1 and TIME-P1, the remote-administration capability shall prevent every dispatch-rule activation commit using a revoked identity at or after sixty seconds from t0, where t0 is the revocation event recorded by the authoritative identity service.

The sixty-second case tolerance now covers requests admitted, staged or queued before t0 but committed afterwards. It does not undo a commit already completed before the deadline; investigation, containment and restoration need their own requirements and operational arrangements. This requirement concerns withdrawal of an identity. Withdrawal of an individual permission needs an explicit scope and cannot inherit the result without analysis.

SR-04 — New sessions during dependency loss. In NORMAL mode, while the identity service cannot provide a valid authentication response, the remote-administration capability shall reject establishment of a new privileged session.

This refines “unreachable” to include responses that cannot establish the required authentication context. A received but invalid response is not evidence that the dependency is usable. The behaviour is specific to the example’s NORMAL mode, not a universal rule for every system or for a later, separately defined recovery arrangement.

SR-05 — Existing authority without fresh status. In NORMAL mode, the remote-administration capability shall reject a dispatch-rule activation commit when the conservative upper bound on the age of its authoritative authorization-status record exceeds thirty seconds.

The thirty-second tolerance is fictional, not a recommended setting. Measure age from the authoritative status observation, not receipt or replay of a cached copy, and apply TIME-P1 to uncertainty. SR-05 restricts state-changing authority. Diagnostic displays and established sessions need separate permitted-use rules.

SR-03 and SR-05 are complementary, not interchangeable. SR-03 bounds the use of an identity after a specific recorded revocation under the stated communication conditions. SR-05 limits reliance on an old status record, including when a fresh one cannot be obtained. A design must explain how source delay, delivery, caching, processing and commit interact. Neither a short cache setting nor a revocation message alone proves the combined property.

7.5 Specify integrity, repetition and outcome behaviour

Authentication and revocation do not exhaust the administration problem. LS-03 showed that an authorized action can create an unintended result through repetition, stale assumptions about the active configuration or misleading feedback. Requirements need to define those interactions without assuming that every implementation uses the same internal transaction mechanism.

SR-06 — Package and approval binding. The remote-administration capability shall commit an activation only when the package content, target and approval reference match the applicable authorized package record defined by PKG-P1 and AUTH-P1.

This binds what is activated to what was approved. It does not require a particular signature product, repository or workflow. Those choices require their own design and assurance reasoning. Nor does the requirement imply that approval guarantees the operational correctness of the package.

SR-07 — Repeated transaction. For repeated activation requests with the same transaction identifier and identical PKG-P1 activation content, the remote-administration capability shall produce no more than one commit and shall provide an outcome referring to that same transaction.

SR-07a — Conflicting reuse of an identifier. The remote-administration capability shall reject an activation request that reuses a known transaction identifier with different PKG-P1 activation content, without committing the conflicting request.

The pair distinguishes legitimate repetition from conflicting use. It does not claim to solve every distributed transaction problem. The design still has to establish the retention and recovery conditions under which transaction identity remains meaningful. If deduplication history is lost after restart, the scope of SR-07 is not demonstrated by a test conducted only before restart. A complete implementation profile must define transaction-record persistence and retention against the required retry and recovery scenarios.

SR-08 — Configuration conflict. The remote-administration capability shall reject an activation without changing the active configuration when the expected active version in the request differs from the authoritative active version at commit evaluation.

A preflight comparison is useful, but it is insufficient if another operation can alter the active version before commit. The requirement concerns the consequential decision. The design must provide appropriate coordination rather than assume that two checks made at different times describe the same state. This is the practical meaning of a check-to-use concern in the example.

SR-09 — Consistent activation. Following interruption or completion of an activation, the authoritative configuration exposed by the management capability shall identify either the complete preceding package or the complete requested package, and shall not identify a mixture of their rule content as a successful activation.

SR-09 expresses a system-level property, not a mandate for a particular database or storage algorithm. The set also needs the outcome semantics in SR-10: preserving complete configuration content is not enough if the administrator is falsely told which package is active. Where the receiving application cannot provide this activation model, the project must revisit the interface or choose a different, explicitly specified model. It must not claim that the requirement is satisfied merely because the gateway behaves consistently in isolation.

SR-10 — Authoritative outcome. On an authorized outcome query, the remote-administration capability shall return the transaction identifier, target, package content identifier, authoritative active version and transaction state, distinguishing committed, rejected, pending and outcome-not-yet-established conditions.

The design must define which observations establish those states and how uncertain outcomes are reconciled. A success response based only on receipt or queuing does not satisfy a claim of committed activation. “Outcome not yet established” is an honest temporary condition, but it cannot become an unlimited excuse for failing the positive completion target within its defined envelope. Performance, reconciliation and failure behaviour must be reviewed as a set.

These requirements show why the protective design includes positive functions. The administrator needs an intelligible way to discover the outcome, not only a prohibition against unsafe retries. Similarly, a configuration conflict needs a useful response that allows the task to continue under the right authority. Human-interface and task requirements will refine that interaction; a technically correct error code is not automatically a usable operational response.

7.6 Preserve the recovery requirement and expose its unresolved authority

SR-11 — Restricted restoration. During an enterprise identity-service outage of up to two hours, the remote-administration capability shall enable designated staff to activate a previously approved restoration package and confirm the authoritative active version within fifteen minutes of the service owner’s declaration of a corrective need, under REC-P1.

SR-11 makes the positive recovery outcome visible but is not ready for contractual release while REC-P1 is unresolved. In particular, “designated staff” cannot stand in for an undefined authority mechanism. The package, personnel, access, authentication, approval and operational resources needed to meet the requirement must be available under the same outage conditions. The fifteen-minute result includes those human and technical contributions.

An offline enforcement point cannot learn a new central revocation through a communication path that is unavailable. This is a dependency fact, not an argument against revocation or recovery. A proposed recovery design has to explain its own authority: for example, whether independently controlled recovery roles can authorize a narrowly defined action, whether another qualified channel supplies current status, or whether a bounded pre-authorization is permitted. Each option has different risks, dependencies and limits. None is approved by being mentioned here.

A pre-authorization that expires before necessary restoration cannot support the full two-hour outage case. An independent approval process that requires the same unavailable identity service is not independent in the relevant sense. A local recovery identity that remains usable after central withdrawal may create an unacceptable inconsistency unless the policy explicitly defines the authority relationship and its limitations. The architecture and governing policy therefore need to be developed together.

POL-01 remains in force. A different recovery authentication arrangement can meet the policy only if its interpretation and implementation satisfy the applicable obligation; otherwise the designated authority must decide whether any exception is permitted. Engineers can explain feasibility and risk, but cannot manufacture exception authority. For this teaching case, no architecture is selected and no exception is assumed.

SR-12 — Mode transition. On a transition between NORMAL mode and a defined RECOVERY mode, the remote-administration capability shall prevent an existing grant or queued activation from being used in the destination mode unless it has been evaluated under that mode's applicable authority and activation rules.

The requirement prevents mode names from being treated as protection. Its full evaluation depends on completing REC-P1, including entry, active-use and return conditions. A transaction committed before the transition retains its actual outcome; the design must reconcile it rather than erase the history. Work not yet committed requires a clear decision about cancellation or renewed evaluation. Returning to NORMAL mode must not silently reactivate obsolete authority.

The distinction between SR-11 and SR-12 is useful. One establishes the desired recovery capability; the other establishes a protective transition relationship. A design could meet the latter by preventing all work from crossing modes and still fail the former because staff cannot perform restoration. The requirement set must satisfy useful capability and protection together.

7.7 Allocate contributions without losing the system-level property

Allocation assigns required contributions to elements or participants. It is not satisfied by placing the same sentence in every supplier's specification. Each recipient needs an obligation that it can implement and evaluate, while the integrator retains the relationship among those contributions and the system-level requirement.

For SR-03, identify the authoritative source of revocation, the distribution path, receiving enforcement points, cached state and the interval between authorization evaluation and commit. The identity service is external to the selected SoI. Its relevant behaviour must therefore be established through an interface commitment, applicable evidence or a design that limits dependence on the uncertain behaviour. Writing an internal requirement against a service the project cannot control does not create that commitment.

Table 7.2 shows an illustrative allocation. These are candidate contributions for analysis; they are not measured performance or executed agreements.

Table 7.2. Contributions needed to support SR-03

Contribution	Candidate responsibility	Why the system-level obligation remains
Record and expose authoritative revocation	Enterprise identity-service provider, through an agreed interface contribution.	Receiving-system timing depends on what t0 means and when relevant status is available.
Distribute status to all relevant enforcement paths	Identity integration and infrastructure teams.	A timely message to one gateway does not establish receiving-end coverage.
Evaluate applicable authority and invalidate stale state	Management-endpoint implementation and configuration owners.	Local caches, restarts and alternate paths can change the effective withdrawal interval.
Bound the interval between authority evaluation and commit	Application management-interface design.	A queued or delayed state change can occur after an otherwise timely check.

Contribution	Candidate responsibility	Why the system-level obligation remains
Establish and measure the end-to-end property	Receiving-system integrator and evaluators.	Component results need composition, configuration and timing evidence.

A simple timing budget can test a candidate allocation. Suppose the case assigns ten seconds to source processing and publication, twenty seconds to distribution and update propagation, five seconds to receiving enforcement processing, eight seconds to the maximum interval between the final authority evaluation and commit, and two seconds as an allowance for timing uncertainty. The proposed bound is:

10 + 20 + 5 + 8 + 2 = 45 seconds; 60 – 45 = 15 seconds of unallocated margin.

This is arithmetic over invented engineering allocations, not a demonstration that SR-03 is satisfied. The distribution allowance includes such effects as polling, replication and retry; it is not the two-second delivered round-trip bound in NET-P1. The analysis assumes that the identified intervals form a complete relevant sequence and that the bounds can be justified. Missing paths, overlapping definitions or unbounded queuing can invalidate the conclusion.

Use justified bounds, not averages. A twenty-second average propagation delay does not establish a twenty-second maximum. Account for the relationship between measurement and enforcement clocks. A different strategy, such as expiring authority records, needs its own complete source-to-commit analysis before this budget can be reused.

The margin is not a spare interval any supplier can independently consume. Its management belongs to the integrated technical effort. A change to the distribution approach may require reallocation, a different enforcement design or reconsideration of the system requirement. Record the rationale and the resulting effect on other claims, including SR-05 freshness and SR-00 useful performance.

This is also where decomposition can reveal new requirements. The endpoint may need a reliable indication of record age, a bounded authorization-to-commit interval, protected transaction state and a defined response when required timing cannot be established. Those derived requirements are legitimate when traced to the parent result and design reasoning. They should not be presented as though the original stakeholder had specified each mechanism.

7.8 Trace meaning, not just identifiers

Traceability records the relationships needed to explain why a requirement exists, where it is allocated and how its satisfaction contributes to a higher-level result. A filled matrix is helpful only when those relationships have defensible meaning. Linking every requirement to every protection need creates the appearance of completeness while removing the ability to identify what depends on what.

Table 7.3 illustrates selected relationships. The evaluation entries are planned methods, not actual evidence or completed verification. Several requirements need more than one kind of evaluation, and several scenarios contribute to one requirement.

Table 7.3. Selected trace relationships for TE-02

Requirement group	Source and causal relationship	Planned evaluation connection
SR-00	STK-01 and useful maintenance; constrained by the ADM-P1 service envelope.	Verification of valid activation and timing; validation of the staff task and outcome interpretation.
SR-01, SR-02	POL-01, PN-01; LS-01 and LS-02 distinguish admission from authority at commit.	Authentication-context and authorization analysis; path coverage; positive and negative receiving-end tests.
SR-03, SR-05	PN-01; LS-01 identifies revocation, stale records and delayed commit.	End-to-end timing and freshness analysis; queued-work, outage, clock and restart scenarios.
SR-06–SR-10, including SR-07a	PN-03 and PN-04; LS-03 links package, target, repetition, version and outcome.	State and interface analysis; conflict, retry and interruption evaluation; outcome reconciliation.
SR-11, SR-12	STK-02 and PN-02 with PN-01, PN-03 and PN-04; LS-04 and LS-05.	Recovery feasibility and authority review, followed by mode-transition verification and operational validation.

LS-06 is not closed by this product table. It creates work and evidence obligations concerning build and deployment provenance, configuration control and the trustworthiness of evaluation information. Some of those obligations will become product or enabling-system requirements; others belong in supplier work and controlled project processes. Record them at the appropriate level instead of forcing every obligation into a runtime product requirement.

Traceability should expose orphaned evidence as well as orphaned requirements. A successful test with no identifiable requirement or claim may still reveal useful behaviour, but its acceptance purpose needs explanation. A critical requirement with no feasible evaluation method is a planning problem, not a completed row with “test” inserted. A supplier result linked to several requirements still needs an applicability argument for each use.

Changes to profile definitions must participate in this trace. Increasing the normal workload, changing what “active” means or adding a recovery role can alter the behaviour expected by unchanged SR identifiers. Version the profile, record the affected requirements and revisit the evaluation plan. Stable identifiers support continuity; they do not freeze meaning without controlled content.

7.9 Separate product obligations from work, evidence and decisions

A supplier asked to “provide assurance” may not know whether the customer expects a design feature, a study, an evidence package or a certificate. Those expectations have different costs, timing and acceptance conditions. A coherent acquisition package distinguishes them and identifies the customer contributions on which they depend.

A product requirement might specify the SR-03 revocation behaviour. An engineering-work obligation might require the supplier to analyse the relevant authority and commit paths, compare candidate mechanisms and maintain traceability as the design changes. An evidence

deliverable might contain the versioned analysis, applicable test procedures and results, coverage, limitations and unresolved findings. An acceptance criterion describes what will be evaluated before a particular contractual commitment is accepted.

The acquirer has work as well. It must provide the relevant policy interpretation, approved service scenarios, interface information, representative identity environment and access to suitable personnel. A supplier cannot determine what counts as an authorized recovery role merely by promising to meet a response time. NIST's acquisition and supply treatments provide a public basis for keeping agreements, contributions and acceptance arrangements explicit. [1, Appendix K]

For the example, a supplier may be responsible for a gateway while the customer maintains receiving management endpoints. The gateway contract can require evidence of its contribution, but a gateway-local test cannot close the integrated revocation requirement. The customer or nominated integrator must plan the receiving-system work and ensure that supplier information is available early enough to support it.

A contractual acceptance can also differ from permission to operate. A component may satisfy the contracted scope while the wider operational capability remains unready because REC-P1 is unresolved or the receiving environment is unavailable. Conversely, a customer's desire to deploy does not make a supplier discrepancy disappear. Keep the contractual position, engineering finding, compliance conclusion and operational decision distinct.

For a commercial product, requirements still inform selection and integration even when the supplier will not change its internal design. The options may include configuration, an additional receiving-system measure, restricted use, a different product or an explicit decision about a remaining limitation. The team should not invent source-code access or evidence rights that the agreement does not provide. It should not invent confidence in their absence either.

7.10 Review the set, then control its change

A requirements review should examine more than grammar. Does the set cover the positive capability, protection needs, relevant modes and transitions? Are the referenced profiles sufficiently defined for the next commitment? Do source obligations remain intact? Are allocations feasible and supported by the necessary agreements? Can the planned methods establish the required characteristics with adequate coverage?

Challenge contradictions through scenarios. SR-00 calls for successful, timely normal activation. SR-05 denies commits using stale authority. Can the intended environment provide current status rapidly enough to satisfy both? SR-11 requires restoration during identity loss. Does the proposed recovery authority still depend on that same identity service? SR-09 preserves complete configurations, while SR-10 describes the outcome. Can a restart preserve one property while falsifying the other? These questions connect requirements to design before implementation fixes a conflict in an undocumented way.

The status of this teaching set is deliberately bounded. It is materially more developed than Part I: request and commit semantics, normal service conditions, package/version relationships, freshness and evaluation targets are now visible. It is not a complete release specification. REC-P1, the authentication protocol binding, transaction-history retention, full path reconciliation

and receiving-application interface behaviour remain explicit completion items. Their owners and required decisions should be recorded before design selection.

When those items change a requirement, preserve the earlier version and rationale. SR-03 has been sharpened from request prevention to commit prevention because the scenario exposed a timing gap. SR-05 is newly introduced in Part II to address authoritative-status freshness. That history allows a reader to see why the specification developed rather than assume that Part I contained the final answer.

The same discipline applies to Agile or incremental work. A user story can identify valuable capability and an acceptance conversation, while linked requirements define security-relevant obligations that span several stories or releases. The team can use its existing backlog and configuration tools. What matters is that cross-cutting requirements, unresolved dependencies and the evidence for a release do not disappear when work is divided into smaller items.

7.11 Exercise: the perfectly worded but incomplete requirement

Situation. The team proposes “The gateway shall prevent revoked users from logging in within sixty seconds.” The sentence is clear, a supplier has agreed to implement it, and a test can be written. Is it an adequate replacement for SR-03?

Worked answer. It is a potentially useful allocated requirement, but not an adequate replacement for the system-level obligation. Its subject is only the gateway, its event is login and its behaviour does not address existing sessions, other management endpoints or requests queued before revocation. A system could pass that test while still committing a rule change under revoked authority after the permitted interval.

Retain the system-level outcome and determine how the gateway contribution supports it. Add the receiving-end, queue, freshness and timing obligations that the design needs. Clarify which revocation event starts the interval and which commit observation ends the consequential action. Then evaluate the allocation as a set. A supplier commitment establishes responsibility for the stated contribution; it does not expand the contribution’s technical scope.

Further challenge. Does the forty-five-second budget prove SR-03? No. It shows that one proposed allocation has numerical room within the sixty-second case tolerance if its bounds, coverage and assumptions hold. Those conditions require evidence. A budget is an engineering hypothesis to justify and manage, not a test result.

Carry forward. Requirements now connect the case’s needs, obligations, scenarios and candidate allocations. Chapter 8 plans the evidence needed to challenge those connections and support decisions, while design alternatives are still open enough to change.

Completion route. Appendix C.2–C.6 develops one selected branch's workload, authority, state and recovery definitions. Read it after the analysis and design chapters; its new assumptions do not retrospectively complete the earlier candidate profiles.

8. Planning assurance and technical evaluation

8.1 Plan for the decision, not the document deadline

An evaluation plan is most useful before consequential design choices become expensive to change. If the project needs to demonstrate revocation at receiving-end commit, the architecture must make that event observable. If recovery depends on human approval, the validation plan needs representative people, access and workload. If an external identity interface is essential to the claim, the project needs applicable information and a usable evaluation environment. These are design and planning inputs, not administrative details to settle after the product arrives.

The purpose of early assurance planning is to establish what confidence is needed for which decision, what claims would support it and what work could provide or challenge the required evidence. It does not require a fully populated assurance case before requirements are stable. It requires enough structure to prevent future tests, supplier promises or incomplete analyses from being mistaken for present support.

NIST treats assurance as something planned, established and maintained through the engineering effort. Its assurance treatment includes confidence in requirements, design, implementation and use, not merely the presence of a final test report. That supports beginning with claims appropriate to the current maturity and refining them as the system develops. [1, Appendix F.2]

For the administration example, the next decision is not authorization of a deployed recovery service. It is whether the problem definition and candidate requirements are sufficiently coherent to compare architectures and commit to further development. Useful early evidence therefore includes reviewed interface meaning, reasoned feasibility, resolved policy questions and a credible plan for obtaining later implementation evidence. A passing prototype test could contribute to that decision without establishing production readiness.

8.2 Separate claims about adequacy, implementation and use

A claim is a proposition for which support is sought. “SR-03 has been written” is different from “SR-03 adequately expresses the necessary revocation behaviour.” Both differ from “the delivered configuration satisfies SR-03.” A credible assurance plan should distinguish these propositions because they require different evidence and can fail for different reasons.

Adequacy of the requirement depends on stakeholder needs, authority semantics, scenarios and the chosen boundary. Implementation satisfaction depends on the actual configuration, interfaces, timing and enforcement. Suitability for intended use depends on how the capability functions in operational tasks, including disruption. Successful evidence at one level should not silently substitute for another.

Table 8.1 organizes candidate claims about the selected administration contribution to dispatch-rule maintenance. It neither substantiates those claims for a real system nor extends them to all logistics outcomes.

Table 8.1. Candidate claims and the questions they require

Claim	Intended proposition	Main challenge to be addressed
C-01: adequate problem definition	The selected needs and scenarios provide an adequate basis for the stated administration decision.	Missing stakeholders, incorrect boundary, incomplete operational conditions or unexamined obligations.
C-02: covered normal-mode authority	Every in-scope normal management path enforces the defined authentication, authorization and withdrawal behaviour.	Undiscovered paths, stale authority, incomplete protocol binding or unsupported composition.
C-03: controlled state and outcome	The activation interaction preserves the specified package, version, repetition and authoritative-outcome properties.	Interruption, concurrency, restart, misleading status or inconsistent receiving-system behaviour.
C-04: usable restricted restoration	Designated staff can complete the defined restoration task under the agreed disruption conditions while satisfying applicable protection obligations.	Unavailable authority or credentials, excessive task time, unqualified personnel assumptions or unresolved REC-P1.
C-05: evidence applicable to the decision	The evidence used for C-01–C-04 applies to the identified configuration, conditions and scope.	Unknown provenance, inadequate method, common unsupported assumptions or unexamined changes.

An argument explains why the evidence supports the proposition and what could defeat the conclusion. It should include the assumptions and relevant contrary findings, not simply list reports under a preferred claim. If the path inventory is incomplete, C-02 remains limited even when every enumerated path passes. If the recovery authority is unresolved, C-04 cannot become supported through a gateway timing test.

An early claim may concern feasibility. Under the fictional outage assumptions, an analysis can show that a central-only concept cannot perform the required restoration. That negative result supports rejecting or revising the concept. Each alternative still needs its own feasibility argument.

8.3 Identify evidence appropriate to maturity

Early evidence can include stakeholder records, operational scenarios, interface analyses, reviewed models, requirements validation results and prototype observations. Later evidence can include implementation reviews, configuration inspection, integration results and intended-use evaluation. Operational observations can subsequently support or challenge continued applicability. The form of the artefact does not establish its quality or the maturity of the claim it can support.

A model of revocation propagation can help identify necessary interfaces and timing bounds. It does not demonstrate that a supplier’s endpoint implements them. A prototype can reveal that a proposed recovery workflow is too slow, while its simplified authentication service limits

conclusions about security. A configuration inspection can establish that a particular setting was present, but may need behavioural evidence to support the effect attributed to it.

The plan should identify the difference between an information need and a promised deliverable. “Receive supplier report” is not yet a method for evaluating revocation. The project needs to know what configuration the report covers, what was evaluated, how time was measured, whether receiving paths and queued requests were included, and what results and limitations must be available for review.

For every planned evidence item, record its intended claim or requirement, production method, item or representation, conditions, expected criterion, responsible participant and dependencies. When the work is performed, add the actual result, configuration, date, coverage, limitations, reviewer and disposition. A field for “actual result” should remain visibly unpopulated or marked not performed until there is a result. A future test is planned support, not current evidence.

This case contains no experimental results. Its records illustrate analysis and planning. Numerical measurement examples later in the chapter are explicitly hypothetical observations used to explain interpretation, not results produced by testing a logistics system.

8.4 Select methods that answer the question

Verification evaluates specified requirements; validation evaluates stakeholder needs and intended use. Both can employ analysis, inspection, demonstration and testing, selected for the property and available representation. A method name is not enough: the plan needs to identify what the method examines and what would constitute a meaningful result. NASA’s product-realization guidance distinguishes these purposes and methods at different levels of system realization. [5, Sections 5.3–5.4]

Inspection can establish that the controlled path inventory and deployed configuration agree to the examined extent. Analysis can examine the meaning of an authority response, possible state transitions or a timing budget. A demonstration can show that a defined task can be performed in the demonstrated arrangement. A test can challenge behaviour with controlled inputs and measured observations. Each has limitations; their combination should address the claim rather than follow a ritual of completing four boxes.

For SR-03, an end-to-end test needs the authoritative revocation event and the commit event. Analysis is needed to identify the paths, relevant caching and the effect of requests already in progress. Inspection is needed to establish the tested configuration and profile versions. A test result without those connections may describe an observation accurately while providing insufficient support for the required system-wide conclusion.

For SR-09, state and interface analysis can identify interruption points that deserve evaluation. Tests can then interrupt activation before, during and after the relevant state changes, including recovery from restart. The evidence must establish the authoritative configuration and outcome, not just the absence of an error message. A limited set of interruption tests still needs an argument for the represented states and the limits of coverage.

For SR-11, a workflow discussion can identify the human and technical dependencies. A prototype rehearsal can expose missing access or confusing feedback. Later validation needs representative staff, resources and disruption conditions. Demonstrating a restoration command

on an engineer's laptop does not establish that the intended operator can obtain authority and complete the task within the full fifteen-minute case window.

8.5 Build a verification plan with visible dependencies

Table 8.2 is a compact plan for the selected slice. The identifiers VP-01 through VP-05 refer to proposed evaluations. They do not identify completed tests. Detailed procedures, inputs and expected observations would be developed as the architecture and interfaces become available.

Table 8.2. Selected planned verification work

Evaluation and target	Proposed method and important conditions	Prerequisite and scope limit
VP-01: SR-01 and SR-02, normal authority	Analyse authentication-context validation and receiving enforcement; inspect configuration; test permitted and prohibited identity/action/target combinations on each covered path.	Complete MFA-P1 binding and AUTH-P1 mapping. Does not establish path completeness unless A-01 is resolved separately.
VP-02: SR-03 and SR-05, withdrawal and freshness	Analyse propagation and time; test new, established, staged and queued work around revocation, stale status, interruption and restart.	Qualified clocks and observable commit; NET-P1 conditions; all relevant enforcement paths. No generalization to untested recovery authority.
VP-03: SR-06–SR-10, activation integrity	Analyse transaction and version states; test matching and conflicting requests, duplicate identifiers, interruption, concurrency and outcome reconciliation.	Agreed receiving-application semantics and transaction-history persistence. Does not prove the operational policy in the approved package is correct.
VP-04: SR-00, useful performance	Measure valid, authorized, non-conflicting activations at the defined workload and service envelope.	Representative configuration and workload. Invalid-request rejection alone cannot demonstrate positive capability.
VP-05: SR-12, transitions	Evaluate grants and pending transactions across mode entry, active use and return; challenge misleading status and resumed obsolete authority.	REC-P1 and transition design must first be complete. The present plan is blocked, not passed.

A plan should identify blocked work honestly. VP-05 cannot proceed meaningfully when the destination mode's authority rules are unspecified. The correct project action is to resolve the definition and its policy basis, not to run a convenient gateway test and label the transition requirement complete.

The plan should also identify what must be supplied by someone else. VP-02 requires a representative identity interface, controlled revocation events and access to receiving-end observations. VP-03 may require application-level instrumentation or a supplier explanation of authoritative state. These dependencies need schedules, access arrangements and responsible participants. A test team cannot generate evidence from interfaces it is not permitted to observe.

The existing security assessment plan can reference this work where its objectives overlap. NIST control-assessment guidance permits tailored assessment planning across the development life

cycle. Reuse of an evaluation is useful when the evidence satisfies both purposes, but the governing assessment criteria remain distinct from a broader system claim. [9, abstract]

8.6 Make validation a different plan, not a different label

The validation question is whether the capability is suitable for the intended task in its context. This includes the relationship between technical behaviour and the people who rely on it. A system can reject all unauthorized requests and still leave an administrator unable to perform a necessary authorized restoration. A technically accurate response can still be misunderstood when its terminology or presentation does not match operational work.

For VA-01, the planned normal-maintenance validation, representative administrators and dispatch operators would follow the intended task from a declared change need to an intelligible, confirmed outcome. The evaluation would examine whether they select the right target and package, understand conflict and pending states, and determine what action is appropriate after an interrupted exchange. It would use the actual intended roles, access and support resources rather than silently supply an engineer to bridge every gap.

For VA-02, the planned disruption validation, staff would perform the restricted restoration task under the stated identity-service outage conditions. The measured interval would begin with the service owner's declaration and include mobilization, authority, access, action and confirmation. The environment would preserve the outage for the relevant task; restoring the dependency immediately before the difficult step would change the scenario rather than demonstrate it.

The plan needs both success criteria and constraints on how success is achieved. Completion through an unauthorized password-only bypass is not success under POL-01. Completion by extra personnel or information unavailable in the intended arrangement is not representative without a justified change to that arrangement. Conversely, an unsuccessful rehearsal can reveal a valid design or support need rather than prove operator incompetence.

NASA's validation planning guidance connects stakeholder expectations, operational concepts, measures and methods. The useful lesson for this case is to plan representative use explicitly and keep it linked to the needs, not to recast the verification matrix as a validation record. [18, Appendix E]

Suppose, as a hypothetical rehearsal finding, the recovery approver cannot access the approval record because that access requires the failed identity service. The correct conclusion is that the tested arrangement did not establish the required restoration capability under those conditions. The finding points to a dependency and perhaps an authority or support design problem. Repeating the rehearsal with an engineer's unrestricted account would not resolve it unless that account and role became part of an authorized, evaluated operating arrangement.

8.7 Specify what the observation means

An evaluation needs a trustworthy basis for deciding whether the observed behaviour satisfies the criterion. This is sometimes called a test oracle: the rule or reference used to distinguish an acceptable result from an unacceptable one. An oracle can be an independently observable state, an authoritative transaction record, a validated model or another justified reference. It needs its own scrutiny.

For SR-09 and SR-10, observe the target's authoritative configuration and transaction state. A gateway's "success" message is insufficient if it reports only receipt. Where one defect could corrupt both the action and its reported outcome, use a means of challenge capable of revealing that shared error.

The oracle need not always be a completely separate system. The plan should explain why the chosen observation represents the required state and how its limitations are addressed. Review of interface semantics, controlled readback, configuration correlation and a second observation method may collectively provide adequate support. The depth follows consequence and uncertainty; it is not a command to duplicate every implementation.

Timing adds a measurement problem. SR-03 compares the authoritative revocation event with a later commit. The evaluation must identify clock relationships, instrumentation delays and the uncertainty in the measured interval. A timestamp with many decimal places is not evidence of corresponding accuracy. A copied log entry can preserve a time value without establishing how accurately it represents the physical or logical event.

The following observations are assumed to measure the relevant enforcement transition, not merely the arrival of the first rejected probe. The evaluation method would have to justify that interpretation.

Consider a hypothetical observed withdrawal interval of 58.8 seconds with a justified conservative uncertainty bound of plus or minus 1.5 seconds. The compatible interval is 57.3 to 60.3 seconds. Because that interval crosses the sixty-second criterion, this observation does not establish satisfaction of the bound. It also does not establish definite violation. The result is inconclusive for that criterion without further information.

For comparison, a hypothetical observation of 56.0 seconds with the same bound gives 54.5 to 57.5 seconds. That supports a passing conclusion for the observed configuration and test condition under the stated measurement assumptions. It does not prove that every path, queue state, workload or future operation meets the requirement. The uncertainty bound here is an invented measurement assumption for teaching, not an unreported confidence interval or a result from calibrated equipment.

An evaluation plan should define this decision rule before results are available. Otherwise the project can be tempted to reinterpret borderline observations to preserve its schedule. Where uncertainty is too large to distinguish satisfaction, improve the measurement method, increase design margin or record the limit on the conclusion. Do not resolve the uncertainty by rounding a preferred number.

8.8 Plan coverage and challenge assumptions

Coverage needs a model. For the administration capability, relevant dimensions include management path, permitted action, identity and authority state, operating mode, transition, transaction state, workload, dependency condition and time relationship. Listing these dimensions does not imply that every possible combination can be exhaustively tested. It provides a basis for selecting representative and consequential combinations and explaining the remaining limits.

For revocation, evaluate more than an idle session after authority withdrawal. Relevant cases include a request staged before t0, one waiting in a queue, one evaluated just before the deadline

and one delayed between authority evaluation and commit. Combine those states with receiving-path differences and the important dependency conditions. Analysis can help identify equivalence classes and boundary conditions, but the equivalence assumption itself needs a basis.

A project may test one of several identical endpoint configurations and argue that the result applies to the others. That argument needs configuration identity, relevant environmental similarity and evidence that differences do not affect the property. A common product name or version label is insufficient where role mapping, plugins or interface settings differ. Where evidence is reused, record why it applies and what receiving-system work remains.

Requirements coverage and scenario coverage answer related but different questions. A requirement-by-test matrix helps identify specified obligations with no planned evaluation. A scenario-by-response review helps identify causal pathways that the requirements or design have not adequately addressed. A project can achieve high apparent requirement coverage while missing a requirement needed for an overlooked scenario.

NIST's assurance treatment emphasizes evidence sufficient for the claim, including the adequacy of the requirements and the reasoning connecting the evidence. Accordingly, coverage should not be reduced to the percentage of green rows. A critical missing mode or unexamined common dependency can outweigh numerous routine passes. [1, Appendix F.2.1–F.2.3]

Contrary evidence deserves a deliberate route into the argument. A test that passes on one endpoint but fails on another should not be averaged into a partial success for "the system." Record the configurations, causal investigation and affected claim. An inspection that reveals an extra management path can invalidate the asserted coverage without proving that the path is insecure. Both findings require action, but they are not the same technical conclusion.

8.9 Resource and protect the evaluation work

An ambitious plan without resources is another unsubstantiated claim. Identify the environments, representative services, data, tools, instrumentation, people and access needed for each evaluation. Include time for resolving discrepancies and repeating affected work. The programme needs to know not only when a report is due, but what makes it possible to produce a useful report.

Evaluation environments are themselves enabling systems. They can introduce risk through privileged test interfaces, copied credentials, sensitive data, temporary instrumentation or connections to operational services. The plan should specify how tests are isolated or otherwise controlled, who can authorize intrusive actions and how the environment will be restored. A production-like condition does not require uncontrolled experimentation on a live mission service.

Synthetic data can reduce some exposure, but its relevance needs examination. A test using trivial package content may not represent the size, state relationships or processing demands that affect activation. A simulated identity service may permit precise fault injection while omitting the real service's timing and response semantics. State what the representation preserves and what it does not.

Competence and independence also need planning. The person who designed a mechanism may provide essential explanations and support, while another qualified participant challenges the

evidence or decision. The required degree of separation follows the governing arrangements and significance of error. A nominally independent evaluator who lacks access to relevant configuration or cannot raise an adverse finding provides little assurance value.

Technical planning and decision analysis help connect these resources to the next commitment. The analytical effort should be proportionate to the consequence, complexity, novelty and uncertainty of the decision. The project should record the rationale for its chosen depth rather than invent a universal assurance level for every system. [6, Sections 6.1 and 6.8]

8.10 Define readiness and preserve honest conclusions

A requirements-readiness decision asks whether the selected requirements and their definitions are adequate for the proposed commitment. A design-readiness decision asks whether the chosen approach and outstanding work support proceeding. A test-readiness decision asks whether the configuration, procedures, criteria, tools, environment and competence allow meaningful evaluation. These decisions need different information even when the same review board considers them.

For TE-02, the early readiness record should state that normal-mode request and commit semantics are developed, selected protective and positive requirements are defined, and planned methods expose the necessary evidence. It should also state the open items: path reconciliation; authentication protocol binding; transaction-history persistence and retention; receiving-application activation semantics; the recovery authority profile; and applicable interface contributions from the external identity service. No completed-looking matrix should conceal these dependencies.

The next architecture work can proceed with those items as explicit questions and constraints, provided the commitment being made is appropriate to their uncertainty. A final production release or a claim of demonstrated recovery cannot proceed on the same evidentiary basis. The maturity of the decision should match the maturity of the support.

The project also needs a rule for changes affecting evidence. A revised endpoint, altered profile, changed identity interface or new recovery procedure can change the applicability of earlier results. The response should identify the affected requirements, scenarios and claims, then select the necessary analysis or re-evaluation. Unaffected evidence can remain useful when its continued applicability is justified. Neither universal invalidation nor universal reuse is defensible.

An assurance conclusion can be conditional or negative. “The normal-mode paths tested meet the specified criteria, but path completeness and recovery suitability remain unestablished” is more useful than an unqualified “secure.” It preserves real progress while exposing what the next decision must address. An authorized decision to accept a limitation does not convert an inconclusive measurement into a passing test or turn an incomplete recovery profile into an implemented capability.

8.11 Exercise: all planned tests are green

Situation. A project dashboard marks every planned gateway test green. The tests used a simulated identity service, a single management endpoint and an engineer’s account. The team

proposes claiming that designated operational staff can perform the fifteen-minute restoration during the identity outage. What is missing from that conclusion?

Worked answer. The proposed claim concerns intended use and a wider dependency and authority arrangement than the test scope. The evidence may support specified gateway behaviours in the simulated configuration, but it does not establish the complete path set, actual receiving-service interaction, designated personnel's access or the recovery authority profile. It also does not establish the full restoration interval, including mobilization and approval.

Retain the useful results and record their configuration, methods and limitations. Identify which claims they support. Plan the missing interface evaluation, authority resolution and operational validation before asserting the wider outcome. The correct response is not necessarily to repeat every gateway test with different labels; it is to obtain evidence relevant to the missing relationships.

Further challenge. The hypothetical 58.8-second observation remains inconclusive against the sixty-second bound with the stated uncertainty. Improve the measurement or design margin. An authority's decision about the limitation does not change the technical result.

Carry forward. Part II connects protection needs, loss scenarios, requirements, allocations and evidence planning. The next part compares architecture, interfaces and resilience options against that problem. A clearer specification supports designing and evaluating a system; it is not evidence that the system already exists.

PART III — ENGINEER THE SOLUTION

9. Trustworthy secure architecture and design

9.1 Move from obligations to a reasoned solution

A requirements set does not assemble itself into a system. The logistics team now knows that authorized maintenance must remain useful, that authority must be current when a configuration changes, and that a restricted restoration capability is needed during an identity-service outage. Those statements still permit several designs. They do not determine where enforcement belongs, how state is protected, which dependencies can be shared or what a maintainer must do when normal access is unavailable.

Architecture work makes those choices explicit. It establishes how functions, responsibilities, interactions and resources are organized to produce the required capability. Detailed design then develops the information needed to realize the selected approach. The two activities inform one another: a promising allocation can prove impractical when timing, storage, physical packaging or operational tasks are examined. A useful architecture is therefore neither a picture of purchased products nor an unchangeable abstraction above implementation. It is a set of consequential choices with a reviewable rationale. NASA's design-solution guidance describes this iterative development of alternatives and their evaluation. [4, Section 4.4]

For an IT practitioner, a deployment diagram, identity flow or infrastructure definition can already contain important architectural information. The additional work is to connect that information to the required system behaviour. Two gateways behind a load balancer tell us something about deployment. They do not yet explain whether a revoked administrator can commit queued work or whether restoration remains possible when both gateways require the same unavailable identity service.

The security specialty contributes loss-oriented reasoning to the common engineering effort. It examines how design choices can introduce, eliminate, restrict or expose paths to unacceptable outcomes. It does not select an architecture solely by counting security features. Performance, human use, maintainability, cost, implementation feasibility and the other specialties remain part of the decision. Nor does it substitute its own risk appetite for the authority responsible for the service.

TE-03 retains Part II's boundary, obligations, needs and requirements while developing architecture alternatives and interface proposals. REC-P1 remains unresolved at this point. The purpose is to expose the work needed to choose a design; Appendix C follows one later completion branch.

9.2 Describe functions before assigning products

Start with the work the capability has to perform. In NORMAL mode, an administrator identifies a target, stages an approved package, requests activation and obtains its authoritative outcome. The capability establishes authentication, evaluates authority, checks package and target relationships, coordinates the configuration change and preserves enough state to resolve repetition and interruption. Supporting functions provide time, protected records, communication and operational feedback.

These are logical responsibilities, not necessarily separate servers. One implementation might place several in a receiving application; another might use a gateway and a small target-side enforcement component. Creating one process for every function can add unnecessary interfaces and failure opportunities. Combining everything in one privileged process can instead increase the consequence of a defect. The architecture should explain the chosen grouping rather than equate either distribution or consolidation with security.

The allocation must reach the point of consequential action. If the gateway checks a request but the receiving endpoint can activate rules independently, the gateway cannot alone provide the system-level authority property. The team must either remove or constrain the alternative path, provide equivalent receiving-end enforcement, or revise the claim to expose the missing coverage. A-01, the assumption about relevant path coverage, is not discharged by giving the main path a more detailed diagram.

The same discipline applies to human functions. Someone defines which restoration packages are operationally suitable. Someone maintains the recovery-role roster. Someone can declare a corrective need and someone can authorize a particular recovery action. A product can enforce parts of those arrangements, but cannot invent their legitimacy. Allocation should identify both the human decision and the technical representation of that decision, including how the representation remains available and protected during disruption.

NIST distinguishes protection decisions, enforcement and the infrastructure supporting those functions. That distinction is useful here because failure of a supporting contribution can defeat a sound decision rule. [1, Appendix D.4.1] A trustworthy authorization algorithm cannot establish current authority from an untrustworthy status source. A correct result cannot prevent a prohibited change if the component that commits it ignores the result. A protected record is not useful to a maintainer who cannot retrieve it under the required outage conditions.

A functional allocation record should therefore say what each element must contribute, what it relies on and what happens when that reliance is not justified. It should also identify the responsible implementation or service owner. The record can live in an existing architecture repository; its value is the explanation, not a new document category.

9.3 Use several views of the same architecture

Different readers need different views, but the views must describe the same proposed system. A network view can make routing and exposure visible. A functional view can show where decisions and enforcement occur. A behavioural view can show states, ordering and failure responses. A deployment view can expose shared power, hosts, storage and administrators. An operational view can show what people must know and do.

ISO/IEC/IEEE 42010 addresses architecture descriptions organized around stakeholder concerns. [19, public scope] Table 9.1 selects views for this case's decision; the selection is the book's application, not a mandatory set of views from that standard.

Table 9.1. Views needed to challenge the administration architecture

View	What it shows in this case	Question it must help answer
Functional and authority	Admission, permission decision, package approval, commit and outcome responsibilities.	Which contribution prevents an unauthorized effective change?
Behavioural	Transaction states, time relationships, NORMAL and recovery transitions.	What happens to work already in progress when authority or mode changes?
Deployment and dependency	Hosts, storage, connectivity, identity, time, update paths and support services.	Which apparent alternatives share a cause of failure or compromise?
Information and interface	Package identity, target, active generation, authority records and outcome meaning.	Do participants interpret the same exchange consistently?
Operational and support	Personnel, access, approval, restoration task and maintenance arrangements.	Can the intended people perform the task without an unavailable dependency?

A contradiction between views is a finding. Suppose the functional view requires target-side enforcement but the deployment view gives a deployment agent unrestricted access to the active configuration. The design has not explained whether that agent is another enforcement path, a tightly constrained enabling function or an uncontrolled bypass. Labelling it “out of band” does not remove its influence on the protected asset.

Likewise, a recovery workflow can appear independent while its deployment relies on centrally authenticated name resolution, a remote secrets service or an approval portal behind the failed identity provider. The operational task view should follow those dependencies through to the point of use. A diagram is complete enough for a decision when the relevant relationships are exposed, not when every possible component has been drawn.

Maintain correspondence between the views through stable element, interface and requirement identifiers. A changed receiving endpoint should lead the team to its authority rules, transaction behaviour, deployment conditions and evidence. This is practical model-based work even when the representations are modest. A particular modelling tool is not required to keep the reasoning coherent.

In a model-based environment, views draw on controlled engineering information. Represent the receiving coordinator once and expose its allocation, deployment and interactions where needed. When separate models are necessary, maintain correspondence between their elements and revisions. Repeated labels alone do not establish a shared configuration or responsibility.

For the security review, begin with one consequential action: activating dispatch rules. Locate the function that performs it, the elements able to influence it, the authority information used and the interface at which the change becomes effective. Relate that path to SR-02, SR-03 and IC-01. Then ask whether the deployment and support views reveal another writer. A model query can identify gaps among recorded elements, but A-01 also requires evidence that the relevant real-world paths have been represented. Absence from a model is not absence from the system.

Different participants need different views of the same decision: authority exchanges for the identity specialist, commit behaviour for the application engineer, tasks and outcomes for operators, and obligations and evidence for assessors. Chapter 11 connects those views through shared engineering information. The model need not show every feature in every view.

9.4 Prefer reducing loss potential to compensating for unnecessary exposure

The Security Design Order of Precedence provides an order for considering protective design responses: first consider removing the relevant loss potential through selection; then reducing it through alteration; then engineered protection, useful feedback and the supporting human arrangements needed for what remains. [1, Appendix D.3] This is a design preference, not permission to omit mandatory safeguards or postpone operator involvement until the end of development.

Apply that reasoning to dispatch-rule maintenance. Does the capability need a general-purpose command shell, or can the required maintenance be expressed through constrained package operations? A package interface may remove some opportunities to perform unrelated privileged actions. It does not remove every loss pathway: an authorized but unsuitable package, a compromised approval source or incorrect target selection still requires attention. The claim is about the particular opportunities reduced, not the elimination of all risk.

Next examine structure. Separate package preparation from activation when doing so permits independent checks or reduces unnecessary write authority. Limit which component can designate a configuration active. Keep ordinary monitoring functions from gaining configuration-write privilege merely because they run on the same host. These changes may reduce propagation and interference, but introduce coordination, deployment and support costs that must be evaluated.

Then select mechanisms that implement the remaining obligations. Authentication, authorization, integrity checking, transaction coordination and protected outcome records are not inferior substitutes for engineering; their design and integration are engineering work. Their placement, strength, failure behaviour and dependencies determine whether they provide the intended contribution.

Feedback is also a designed function. If an activation remains uncertain after a connection loss, the administrator needs an intelligible status and a safe means of reconciliation. An alert without a defined response can leave the same loss pathway intact. Its recipient must have the information, authority and time needed to act. Procedures and training support that response, but should not be used to excuse a design that unnecessarily makes the dangerous action easy and the appropriate action difficult.

The order of consideration does not always produce the smallest component count. An additional target-side enforcement component may reduce reliance on a large gateway. Conversely, adding another approval service can introduce a dependency that blocks restoration. The question is whether the resulting arrangement better satisfies the complete requirement set, not whether a diagram contains more or fewer security boxes.

9.5 Develop genuinely different alternatives

A trade study needs alternatives that differ in consequential choices. Three suppliers offering nearly identical gateways may provide a useful product competition but not explore the architectural response to identity loss. The following concepts retain normal-mode authentication and target-side enforcement as design concerns while differing in how they address restricted restoration.

ARC-A — Centrally dependent administration. All privileged activation, including restoration, requires a usable enterprise identity service. Gateways may be duplicated to tolerate some local failures. Package and transaction protections still have to be engineered. During the stated identity outage, however, the concept has no independent basis for the required restoration action. Adding gateway redundancy does not alter that dependency.

ARC-B — Governed recovery controller. Normal administration remains centrally authenticated. A separately governed recovery path permits designated staff to select an eligible, previously approved restoration package for the specified target. It has a proposed independent authentication and authority arrangement and a deliberately narrow action set. It does not provide arbitrary rule creation or a general administrative shell. Both normal and recovery actions must encounter a coherent receiving-side commit discipline.

ARC-C — Prepared restoration capsule. While normal supporting services are available, the organization prepares a target-specific restoration instruction containing the package identity, eligible starting conditions, approval and bounded authority information. During disruption, designated staff authenticate through a permitted independent arrangement and request execution of that prepared instruction. They do not select arbitrary replacement content at that point. This can reduce discretionary choices during recovery, but depends strongly on the prepared instruction remaining applicable and available.

The word “capsule” is an author-defined teaching label for a protected package of data and instructions, not a product or standardized security object. Its required protections and interpretation would have to be specified. A capsule containing an approval is not itself proof that the person presenting it is entitled to use it. Neither ARC-B nor ARC-C is exempt from POL-01 merely because its credentials or controller are held locally.

Placing the three concepts beside one another makes the consequential differences visible. The normal path is not the distinguishing feature; the key question is what basis for restricted restoration remains when the central identity dependency is unavailable.

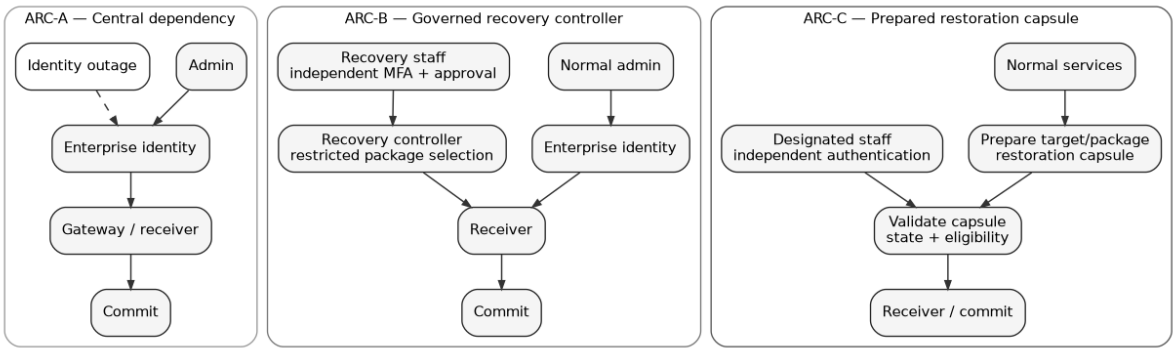


Figure 9.1. Recovery architecture alternatives differ in authority and dependency

Original comparison of ARC-A, ARC-B and ARC-C. The figure is intentionally functional: it does not prescribe products or prove that either recovery alternative is acceptable.

Table 9.2. Architecture alternatives and their principal unresolved contributions

Concept	Intended contribution	Important limitation or decision
ARC-A	Centralized normal authority; possible local gateway redundancy.	Cannot satisfy SR-11 during the stipulated long identity outage if every restoration requires that service.
ARC-B	Restricted package selection and restoration through separately governed recovery authority.	Independent authentication, role control, usable approval, eligibility checks and personnel availability must be established.
ARC-C	Execution of a prepared target/package instruction with limited discretion during disruption.	Preparation age, validity, target state, package suitability, withdrawal and replenishment may prevent required use.

The scope of the case makes one finding possible now: ARC-A, as defined, is not an adequate complete response to SR-11. That is a conditional conclusion from the specified dependency, not an experimental result. It may remain a useful normal-mode subarchitecture. Reconsidering the stakeholder need is an available governance action, but is not part of the present assumption set.

ARC-B and ARC-C remain candidates, not demonstrated solutions. Their distinction creates useful investigations. ARC-B needs evidence that recovery discretion is both governable and usable within the task window. ARC-C needs evidence that eligible preparation remains available across the required outage and target changes. Chapter 11 will show how a decision can progress without inventing favourable answers to either question.

9.6 Place trust where the design can justify it

Trust allocation asks which contribution the system relies on for a particular property and what happens if that contribution is incorrect or hostile. “Trusted network” or “trusted supplier” is too broad to answer that question. For SR-02, the relevant reliance concerns authority at commit. For SR-09, it concerns consistent configuration activation. Those properties can depend on different elements and need different support.

In the proposed common design, the gateway handles session admission and useful interaction with the administrator. A receiving-side commit coordinator owns the final authority, target, package, version and mode checks associated with effective change. A protected transaction record supports repetition and outcome reconciliation. These allocations are proposals, designated AD-01 through AD-03 below, not assertions that an existing product has these features.

The coordinator becomes a significant trusted contribution. Making it smaller may improve the ability to examine it, but size alone does not establish correctness. Its input parser, storage, privileges, update mechanism, supporting time information and ability to exclude other writers remain part of the argument. Moving trust from a gateway into a controller does not make trust disappear.

Apply secure-design principles as questions: what interference does separation prevent; which authority does least privilege retain; where is access mediated; and how much implementation must be trusted? [1, Appendix E.12, E.16, E.19–E.20] The answers inform the architecture; naming the principles does not determine it.

For example, a gateway service account able to write the active configuration directly defeats the intended separation even when a separate coordinator exists. A diagnostic endpoint that can invoke a maintenance hook may carry equivalent authority indirectly. A recovery administrator who can change the controller's approval policy and execute the resulting operation may collapse an intended separation of duties. The design must follow effective influence, not job titles or component labels.

Trust allocation also reaches physical and organizational arrangements. A target-side controller can depend on a shared host administrator. Separate credential stores can still be managed through the same compromised workstation. A tamper-evident enclosure can aid inspection without preventing every unauthorized change. Each measure needs a claim matching its actual contribution and operating conditions.

9.7 Treat protection-function failure as part of the design

The mechanism that enforces a protection requirement can itself be unavailable, incorrect, bypassed or modified. Analyse those conditions as deliberately as ordinary component failures. Otherwise the architecture assigns its most important obligation to a box whose own behaviour under adversity is left unexplained.

NIST's mechanism criteria include being non-bypassable, evaluatable, always invoked and tamper-proof. [1, Appendix D.4.2] These are demanding design criteria to substantiate, not attributes that become true when written in a specification. In explanatory prose, distinguish the intended property from the evidence available for a particular implementation and threat context. A cryptographic signature, for example, does not by itself establish protection of the signing authority, verifier or subsequent execution path.

For the case, examine a stale authorization response, a coordinator restart, loss of transaction storage, an unavailable time reference and an update that changes the path set. Determine which actions remain permissible. A missing fresh status record can prohibit a normal-mode commit under SR-05 while an authorized diagnostic query remains possible under its own rules. That is more precise than saying that the entire system “fails closed.”

Loss of the outcome store creates a different problem. The system may be unable to determine whether a transaction already committed. Allowing an unexamined retry can violate SR-07; reporting that nothing happened can violate SR-10. The design needs a protected relationship between effective change and transaction history, or another defensible reconstruction method. Merely configuring the gateway to reject new requests does not settle the state of requests already accepted.

There are also failure conditions created by protection itself. A strict gate that stops all state changes when a central logging service is unreachable may protect record completeness while preventing necessary restoration. A local protected journal could reduce that dependency, provided its capacity, integrity and later reconciliation are engineered. This is a candidate trade-off, not a universal instruction to bypass logging during an outage.

Include failure detection and response time in the analysis. A protective condition noticed only after a prohibited commit may support investigation but not prevention of that commit. An automatic response can reduce delay while creating a new authority and control function requiring its own evaluation. The correct response follows the affected capability, relevant consequence and evidence, not a blanket preference for either automation or manual intervention.

Table 9.4. From secure-design principle to a reviewable engineering question

Design question	Case application	Evidence needed
What unnecessary authority or loss opportunity can be removed?	Prefer constrained package activation over a general privileged shell when the useful task permits it.	Interface and privilege review showing the restricted action set.
Where must a decision be enforced to remain effective?	Authority is checked at the receiving commit boundary, not only at gateway admission.	Complete writer/path coverage plus commit-path evaluation.
Which contribution becomes highly trusted?	The receiving coordinator and protected history become consequential trusted contributions.	Implementation, privilege, update, storage and failure evidence appropriate to their role.
What happens when the protection mechanism fails?	Loss of fresh status, time confidence or outcome history changes the permitted action and claim.	Failure-response tests and operational handling tied to the affected capability.

9.8 Record decisions with their obligations and reversibility

An architecture decision record should preserve the issue, alternatives, chosen direction for the current development step, rationale, consequences and conditions for reconsideration. It need not imply final selection. A decision to prototype one enforcement arrangement differs from a decision to procure it, and both differ from permission to operate it.

Table 9.3. Proposed TE-03 design directions for further evaluation

Identifier and direction	Reason and trace	Obligation created or retained
AD-01: receiving-side commit control	SR-02, SR-03 and LS-02 require authority at effective change, not only gateway admission.	Establish all writers, final-check semantics and the coordinator's own protection. A-01 remains open.
AD-02: one coordinated activation decision	SR-06–SR-09 and LS-03 concern package, expected version, repetition and consistent change.	Define a realizable transaction boundary and evaluate interruption, concurrency and restart.
AD-03: persistent transaction and mode identity	SR-07, SR-10, SR-12 and LS-05 require history and authority to remain meaningful across transitions.	Resolve retention, namespace retirement, protected state and recovery from old snapshots.

Identifier and direction	Reason and trace	Obligation created or retained
AD-04: investigate ARC-B and ARC-C for restoration	PN-02 and SR-11 cannot be met by ARC-A alone under the assumed outage.	Complete candidate recovery profiles and test feasibility without granting exception or deployment authority.

These directions do not specify a database, cryptographic protocol or hardware platform. They identify what an implementation must explain before those choices can be accepted. Where a commercial product cannot expose a suitable receiving-side interface, the project must assess an alternative product, a different system boundary or an explicitly revised requirement. It must not claim that a gateway feature creates a property outside its control.

Consider reversibility early. Replacing one gateway can be relatively contained if interface semantics are stable; changing the meaning of commit or transaction identity can affect every client and retained record. A decision's cost includes the future cost of correcting it. That does not require speculative analysis of every future technology. It does require identifying commitments that make later evidence or redesign difficult.

9.9 Transfer the reasoning beyond enterprise administration

Consider an illustrative inspection vehicle whose operator can load a new route. The useful function is not “accept a signed file”; it is to operate under an intended route and defined conditions. The engineering questions include when a new route becomes effective, whether a route change is permitted while moving, how incomplete transfer is handled and which physical controller determines the vehicle's state. A valid signature cannot answer those questions alone.

This is a transfer vignette, not a proposed vehicle safety design. A physical plant can continue moving after an information-processing action is cancelled. A command acknowledged by one controller may not yet have changed the actuators. Requirements for a stopped state, controlled transition or continued operation would need the actual dynamics, operating environment and relevant specialist analysis. The administration case's atomic configuration model and numerical tolerances cannot simply be copied.

The common SSE work is to identify the consequential action, allocate protection at the appropriate place, examine supporting dependencies and evaluate both successful operation and adverse behaviour. The concrete mechanisms differ. Hardware packaging, power, sensing, software and human tasks participate in one engineered capability; the security analysis must integrate with the specialties that understand those contributions.

9.10 Exercise: a smaller attack surface and a missing capability

Situation. A reviewer proposes removing every remote activation path. Administrators could still inspect the service, and unauthorized remote changes would become much harder. The reviewer declares that the most secure architecture has therefore been found. How should the team evaluate the proposal?

Worked answer. Treat removal as a genuine design alternative. It may eliminate particular remote loss paths and could be suitable where the useful task can be performed through another acceptable arrangement. In this case, however, SR-00 and SR-11 require activation capabilities. The reviewer must show how the proposed system still provides them under the intended

conditions, including personnel, authority and time. A design that provides no useful state-changing service does not satisfy the current requirement set merely because it rejects prohibited changes.

Compare a restricted package interface with the broader path that prompted the concern. Examine which unnecessary authority can be removed while preserving the required operation. Retain any remaining dependencies and loss scenarios. Do not infer that a constrained interface is immune to misuse, or that a local physical visit is automatically feasible within fifteen minutes. The result should be a defensible comparison, not a contest in how much functionality can be deleted.

Carry forward. The chapter has produced distinct architectural concepts, proposed functional allocations and explicit questions about their trust and failure behaviour. Chapter 10 develops the interfaces and state relationships needed to determine whether those contributions can actually work together.

Cross-domain route. Appendix D develops the inspection-vehicle example through requirements, a stopping-distance calculation and evaluation questions. It shows why an authenticated command and a justified physical response are different claims.

10. Interfaces, dependencies and composition

10.1 An interface is an agreement about behaviour

An interface description that lists fields and addresses can still leave the most consequential behaviour undefined. A gateway may send a valid activation request and receive “accepted,” while the administrator assumes the package is already active. The receiving endpoint may instead have queued the operation for later evaluation. Both components can conform to their local descriptions while the combined interaction misleads the user.

The interface must establish the meaning of the exchange, the responsibilities on each side and the conditions under which a conclusion is justified. For this case, that includes identity, authority, target, package, expected configuration, operating mode, time and outcome. It also includes what happens when an exchange is interrupted, repeated or presented after its assumptions have changed.

This is familiar territory for application and integration teams. API contracts, message schemas and service-level commitments already address parts of it. SSE connects those contributions to the protection needs and challenges the paths by which their assumptions can fail. It does not replace competent interface engineering with a new security-labelled form. NASA's interface-management treatment provides a public account of the need to define, coordinate and control interfaces across participants. [6, Section 6.3]

Composition is the resulting system behaviour when contributions interact. It can create the useful capability that no element provides alone, as well as unwanted authority or dependency paths. Evidence about each element therefore needs an explanation of how the elements interact under the conditions of the receiving system. NIST's compositional trustworthiness principle addresses that system-level reasoning. [1, Appendix E.7]

The TE-03 proposals develop AD-01 through AD-03 into implementation and integration obligations. They also expose gaps in the earlier candidate requirements. Product selection and verification of a realized implementation remain later work.

10.2 Define a contract at the consequential boundary

For the proposed receiving-side activation interface, begin with the result required by SR-02 and SR-06–SR-10. A permitted operation changes the target's authoritative management configuration once, from the expected version to the intended package, with a dependable account of the outcome. A prohibited or conflicting operation does not perform that change. The contract must connect those outcomes to inputs that can be checked.

A precondition describes something that must be true for a specified operation. In a protection interface, it is important to distinguish conditions enforced by the receiver from conditions merely assumed by the designer. “The caller is authorized” cannot remain an unchecked environmental assumption when the interface exists to prevent unauthorized activation. The receiver needs an authenticated, relevant and sufficiently current basis for the authorization decision.

A postcondition describes a result following an operation under its stated conditions. It should not confuse successful receipt with successful commit. An invariant describes a property intended to hold across the relevant states and transitions, such as prohibiting two commits for

the same transaction identity. Stating an invariant helps focus design and evaluation; it does not establish that the implementation preserves it.

Table 10.1 sketches a candidate contract, IC-01. Names such as generation and epoch are introduced below. This is a behavioural contract, not a complete wire protocol. Field encodings, size limits, cryptographic binding, error codes and implementation-specific timing still need development.

Table 10.1. IC-01: candidate receiving-side activation contract

Contract element	TE-03 proposal	Required challenge
Request identity	Bind transaction identifier to package content, target, expected active generation and mode epoch.	Can another caller substitute a field, reuse the identifier or change the interpretation?
Authority	Establish person, action, target and mode authority at the effective-change decision.	Are admission, package approval and current personal authority improperly treated as equivalent?
Version condition	Compare the requested starting generation with authoritative state as part of the coordinated commit decision.	Can another writer change the state after a preflight check?
Effective change	Expose the complete preceding or requested package, with one identified commit outcome.	What happens at every interruption boundary, including storage and process restart?
Repetition	Reuse a known identical transaction without a second commit; reject conflicting reuse.	Does that property survive retention, restore and mode changes?
Response and query	Distinguish receipt, pending work, committed outcome, rejection and temporary uncertainty.	Does the observation establish target state or merely repeat the gateway's belief?

Allocate the contract to both sides. The caller preserves the transaction identifier while resolving a lost acknowledgement; the receiver retains the established outcome instead of assuming that a retry means no commit occurred. The operator interface distinguishes pending from uncertain outcomes. Evaluate the interaction as used, not only the correctness of the interface description.

The receiving dispatch application's contribution remains explicit. Commit means designation of the active management configuration for a target. It does not establish that every operational consumer has loaded it, that in-flight dispatch decisions used it or that the package expresses correct operational policy. Those relationships need separate interfaces and evaluation with the service owner. The SoI has not expanded merely because the contract is more detailed.

In a shared model, IC-01 should have an identified authoritative definition and accountable owners on both sides of the interaction. Its request, commit and outcome concepts can then be referenced by the sequence description, state model, requirements and planned evaluations. A sender's port and a receiver's port are not compatible merely because both are labelled “activation.” They must agree on the meaning of the information, the permitted states and the responsibilities attached to the exchange.

Consider two views that each use the word “complete.” In the gateway sequence it means the request has entered a queue; in the receiving-state view it means the active generation has advanced and its transaction outcome is preserved. That is a semantic mismatch to resolve, not an aesthetic inconsistency to fix by changing a label. Make the distinguishing events explicit and relate the success indication to the right event. A model can record this correction and help locate affected views; it cannot establish the implemented protocol's behaviour without the corresponding analysis and evaluation. Section 11.5 follows these relationships into evidence planning.

10.3 Separate receipt, commit and knowledge of commit

Three events deserve separate names: the receiver obtains a request; the requested state change takes effect at the defined boundary; and the caller learns the authoritative outcome. A connection can fail between any two. A system can therefore have committed an operation while the caller remains uncertain about it.

HTTP offers a familiar illustration: 202 Accepted does not establish that processing is complete. [20, Section 15.3.3] Whatever protocol the case eventually uses, define when an application response establishes receipt, commit or knowledge of the outcome.

Consider transaction TX-41, requesting package P8 for target D1 from generation 27. The receiver commits the package as generation 28, records the transaction outcome and then loses connectivity before the reply reaches the caller. The administrator sees a timeout. Treating the timeout as “activation failed” is wrong; treating it as “activation succeeded” is also unjustified from the caller's observation alone. The correct action is to obtain the authoritative outcome using the same transaction identity.

If the client instead sends TX-42 with the same desired package, the receiver cannot infer merely from matching content that it is a retry of TX-41. It could be a new, separately authorized action. The expected-generation condition may reject it, but that does not eliminate the need for transaction identity and reconciliation. Package content, operation identity and configuration state answer different questions.

The interface can expose a status resource or query operation, provided authorization and state meaning are defined. A history result should distinguish the package and generation committed by the queried transaction from the target's current package and generation. A later legitimate transaction may have changed the target again. Reporting only current state can obscure what TX-41 actually did.

This is a proposed clarification to SR-10's interface profile, not a silent replacement of the Part II wording. IC-01 separates historical outcome from current authoritative state, and requires their relationship to be observable. The requirements owner should decide whether the distinction needs an additional explicit obligation before release. The change record preserves that completion item.

The transaction sequence is easier to challenge when receipt, authority evaluation, commit and knowledge of the outcome are shown as distinct events. The sequence below also shows why a lost reply does not justify a second effect.

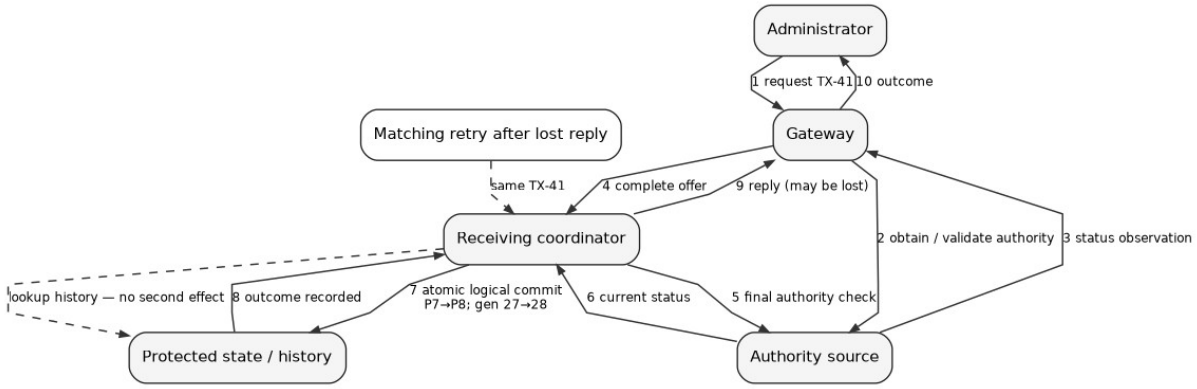


Figure 10.1. Request, authority, commit, lost reply and matching retry

Original sequence view of the recurring activation transaction. Numbered messages identify logical order, not measured network timing. The matching retry consults recorded transaction history and does not create another state change.

10.4 Coordinate checks with the state change

A sequence of individually successful checks can still permit a prohibited operation if the relevant state changes between them. Two administrators can each inspect generation 27, submit different approved packages and receive a favourable preflight response. If both commits then proceed without coordination, one can overwrite the other's change without satisfying the intended expected-version condition.

For the proposed design, the receiving-side decision must coordinate transaction identity, current generation, mode epoch and the authority applicable to effective change. This can be implemented in different ways; calling a procedure “atomic” does not select or verify one. The project must explain which state is controlled together and how external dependencies participate without leaving an unbounded check-to-use interval.

One candidate arrangement prepares and validates package content before entering the final commit operation. At that operation, it checks whether the transaction is already recorded, whether its identity and content agree, whether the expected generation and mode epoch are current, and whether an applicable authority decision remains valid through commit. It then records the effective state change and outcome through a recoverable storage arrangement. The order and exact implementation must be analysed against failures, concurrent callers and the chosen persistence mechanism.

An external identity service generally cannot be assumed to participate in the same local transaction. A status observation can become older while the receiver waits. The design therefore needs a bounded relation between observation, final evaluation and commit, or a mechanism that re-evaluates and rejects when its authority basis is no longer acceptable. Chapter 11 examines that timing explicitly. A local lock on configuration state cannot by itself make a remote authorization record current.

The expected-generation check also requires a suitable version identity. Suppose package P7 is active, P8 is activated, and P7 is later restored. If “version” means only the package name P7, an old request expecting P7 may incorrectly appear to match the restored state. TE-03 proposes a

monotonically advancing activation generation distinct from package content. Returning to old content produces a new generation, not a return to the old state identity.

This proposal develops PKG-P1's expected active version into a specific candidate interpretation, designated PKG-P1/TE-03-draft. It is not an assertion that all systems need a numeric counter. The required property is that an earlier state identity cannot be mistaken for a later, materially different activation context. Counter persistence, rollover, restoration and multi-writer coordination require their own design and evaluation.

HTTP conditional requests offer a related technique for avoiding lost updates through representation validators, but do not establish this case's complete commit property. [20, Sections 13.1.1 and 13.2.1] An interface that compares a validator correctly can still mishandle authority, repetition or persistence. Reuse a protocol mechanism for the property it actually provides.

10.5 Design interruption and repetition together

SR-07 requires no more than one commit for repeated identical transactions. That differs from a promise that a network delivers a message exactly once. The useful end-to-end property concerns the effect of the operation, not the number of packets or invocations observed along the way. HTTP's idempotence definition likewise concerns the intended effect of repeated requests; it does not eliminate the need to examine the application's own side effects and records. [20, Section 9.2.2]

For the case, preparation may be repeated without making the package active. A commit may not be repeated as a new effective change for the same transaction identity. Queries may be repeated under their permitted-use rules. A retry after restart needs the same protection as a retry while the process remains running. These distinctions prevent “idempotent API” from becoming an unexplained system assurance claim.

Table 10.2 gives selected transition cases for a proposed transaction model. It is not an exhaustive state machine or a model-checking result. The table identifies observations and design obligations that VP-03 should challenge as implementation develops.

Table 10.2. Selected transaction transitions and interruption obligations

Situation	Required or proposed response	What must survive interruption
New valid request, no prior transaction	Prepare and evaluate without claiming commit prematurely.	Bound identity and content; retain an intelligible pending state where needed.
Existing committed transaction, identical retry	Return its historical outcome without another commit, subject to query authorization.	Commit identity and sufficient outcome information.
Existing identifier, different activation content	Reject the conflict without performing the conflicting change.	Binding between identifier and accepted content.
Version or mode changes before effective action	Reject or re-evaluate under the current rules; do not use stale approval of execution.	Current generation and epoch; actual outcomes already committed.

Situation	Required or proposed response	What must survive interruption
Restart near effective change	Recover a definite committed or uncommitted state, or report bounded uncertainty while reconciling.	Coherent configuration and history; no false assertion that timeout means no commit.
Reply lost after commit	Allow authoritative reconciliation of the same transaction.	Historical outcome even if current active configuration later changes.

The hardest boundary is between changing configuration and preserving the fact that it changed. If one operation succeeds and the other is lost, the system may be unable to satisfy both consistent activation and dependable outcome. A durable journal, transactional store or equivalent reconstruction arrangement can be a candidate solution, but its failure semantics and scope must be established. A gateway cache containing “completed” is not sufficient if it can disappear while the effective configuration persists elsewhere.

Retention must follow the accepted identity lifetime. A design that forgets TX-41 after twenty-four hours and later accepts TX-41 as new cannot claim the unqualified no-second-commit property solely because it kept a short cache. The project must either retain sufficient suppression and outcome state for the relevant lifetime, or develop a controlled identifier-namespace retirement rule that rejects later execution under retired identities. Neither approach permits silent narrowing of SR-07 or an unjustified reduction of outcome availability.

A proposed TX-P1 profile therefore needs rules for transaction creation, binding, persistence, query access, retirement and storage recovery. Retirement may permit detailed history to move to controlled archival storage while a compact execution-suppression record remains. Whether that satisfies SR-10 depends on required query availability and retention obligations. These are explicit design questions; no retention period is invented here as a universal answer.

Storage capacity and exhaustion belong in the same analysis. Refusing a new activation when protected recording is unavailable may preserve some integrity properties while reducing useful capability. Unbounded logging can itself exhaust the service. The design must establish capacity, backpressure, alarm, retention and operational response together, and evaluate how those choices affect SR-00 and SR-11.

10.6 Treat mode transitions as authority transitions

A mode label is not a security boundary. A queued request admitted in NORMAL mode can remain dangerous after the system enters recovery if it carries old authority into a different set of operating rules. Returning to a label called NORMAL creates another risk: old work may appear to match the label again even though the authority and configuration context has changed twice.

TE-03 proposes a persistent **mode epoch**: an identifier for a particular authority context, not merely the name of the mode. Entry to a new governed context advances that identifier. Requests and grants are associated with the context in which they are valid; the receiving commit decision rejects stale epochs unless a new evaluation has explicitly established destination-mode authority. This is a candidate mechanism for SR-12, not an additional approved recovery policy.

The mode transition and commit operation need a defined order. If a commit completes before the transition takes effect, it remains part of history. If transition takes effect first, pending work cannot use the old context. The implementation must make those cases distinguishable; a race

between two uncoordinated processes is not resolved by a written policy saying that transitions are controlled.

A transition can close admission to new work while existing work is resolved, provided the design bounds that process and prevents old authority from crossing the effective boundary. Cancelling a queue entry cannot undo an operation already committed. Equally, waiting for all queued work to finish may exceed the restoration window or permit work that should now be prohibited. The choice requires a defined linear order at the consequential boundary and an operationally usable transition procedure.

Restoring an old system snapshot must not resurrect old mode epochs, grants or transaction identifiers as newly valid authority. Restoring application package content and restoring security history are different operations. A protected non-rollbackable generation source, durable external record or other justified recovery arrangement may be needed. The design must address loss or compromise of that arrangement rather than assume that persistence alone makes it trustworthy.

Recovery also changes human authority. A person authorized to approve a restoration package is not automatically authorized to declare recovery mode, act as the executing administrator or approve return to normal use. Those roles may be combined where justified, but the combination and its consequences should be deliberate. Separation of duties is a possible design measure, not a substitute for explaining the actual decision chain.

10.7 Follow dependencies beyond the obvious service

ARC-B and ARC-C are intended to address an enterprise identity-service outage. That is a specific independence claim. It requires examining the entire task: authentication material, role information, package approval, package availability, time, communication, target access and the personnel who perform it. A recovery path can avoid the main identity endpoint while still depending on that endpoint indirectly through another service.

Table 10.3. Dependency questions for restricted restoration

Contribution	Possible hidden dependence	Consequence for the candidate design
Recovery authentication	Credential activation or validation calls the unavailable enterprise identity service.	The path is not independent for LS-04, regardless of where credentials are stored.
Approval and eligible package	Approval portal or repository requires normal sign-in.	Staff cannot establish or obtain what SR-11 requires during the outage.
Time and expiry	Validity evaluation requires a remote time source with no justified local fallback.	The design may have to reject the very instruction intended for outage use.
Receiving coordinator	Normal and recovery paths share its storage, update authority and execution privileges.	Identity-outage independence does not establish tolerance of coordinator compromise.

Contribution	Possible hidden dependence	Consequence for the candidate design
Personnel and facilities	Required staff, workstation, power or physical access are unavailable in the task window.	Component readiness does not establish operational restoration readiness.

There is no requirement to make every dependency independent of every other one. Such a goal can be impractical and can itself increase complexity. Identify the cause against which independence is claimed and the remaining common dependencies. Two controllers sharing an approved package store may be adequate for a local controller failure, yet inadequate for a store-integrity compromise. Those are different scenarios and may justify different responses.

Evidence dependencies matter as well. If the same deployment tool configures the receiving writers and generates the only report of which writers exist, a defect can affect both enforcement and its apparent evidence. Independent inspection or another justified source of observation can strengthen the coverage argument. The relevant question is whether the challenge can reveal the error of concern, not whether the report bears a different organizational logo.

10.8 Bound composition claims and repair mismatches

A supplier can demonstrate that its gateway validates an authentication context without demonstrating that every receiving path obeys the resulting decision. Another supplier can demonstrate atomic local storage without establishing that the gateway reports the target's authoritative outcome. The integrator needs a chain of explicit obligations connecting those contributions.

A useful composition argument begins with a system property, identifies the required contributions and checks whether each contribution's assumptions are provided by the surrounding design. It then examines interactions, shared causes and relevant exclusions. A circular argument is not repaired by a longer traceability matrix: if component A assumes that B establishes authority and B assumes that A did so, neither has supplied the needed contribution.

For C-03, controlled state and outcome, the proposed chain is: IC-01 binds the request meaning; the receiving coordinator checks current authority, generation and mode; the persistence arrangement preserves configuration and transaction history; and the client reconciles uncertainty without issuing an unrelated activation. Each step needs evidence appropriate to its maturity. The chain also needs challenge of what happens when the supporting store or coordinator fails.

A mismatch can be resolved through a revised allocation, adapter, interface change, different product or narrower permitted use. An adapter deserves the same scrutiny as the components it connects. It may translate a status field while losing authority binding or transaction identity. A compatibility demonstration that messages pass through does not establish preservation of the protection property.

Some mismatches require returning to the requirement. If the operational application activates rules independently at several consumers, the single management-side commit model may not describe the wider service accurately. Preserve the current scope and specify the additional consumer convergence or operational cutover behaviour separately. Do not rename an asynchronous collection of updates an atomic system change merely to make the diagram agree with the requirement.

10.9 Exercise: rollback restores more than the package

Situation. After recovery, an administrator restores a snapshot containing the earlier package, transaction database and mode state. The service appears healthy. An old activation request is then accepted because its transaction identifier is absent from the restored database and its NORMAL label matches. What has the design failed to distinguish?

Worked answer. It has treated restoration of desired configuration content as restoration of the authority and execution history against which future work is judged. A previously used identity can become apparently new; an old mode can appear current; and a request can execute again despite SR-07 and SR-12. Package content returning to an earlier value does not justify returning the protection context to an earlier state.

A corrected proposal needs controlled recovery of transaction suppression, outcome history, generation and mode epoch. A restored package should normally be represented as a newly authorized activation in the present history, not as erasure of intervening events. Where history cannot be established, the system must not silently infer that no prior commit occurred. The project must define a recovery response, its operational limits and the authority to proceed.

This answer identifies a design obligation, not a verified storage algorithm. Test the proposed arrangement around snapshot restore, interrupted restore and competing requests; inspect the identity lifetime and namespace rules; and evaluate the operator's ability to understand the resulting state. Retain useful earlier component evidence, but do not extend it to these new conditions without justification.

Carry forward. Interfaces have become concrete enough to expose timing, state, persistence and common-dependency obligations. Chapter 11 uses models and trade studies to evaluate those obligations and compare design choices without disguising uncertainty as numerical precision.

11. System analysis, MBSE and trade studies

Reading map. Sections 11.1–11.2 frame modelling; 11.3–11.10 develop the MBSE workflow and exercise; 11.11–11.14 examine quantitative and state models; 11.15–11.18 develop and challenge the trade decision. Table identifiers 11.1–11.5 are unchanged.

11.1 Use a model to answer a stated question

A model is a selective representation of a system or problem. Its value depends on the question it helps answer and on the relationship between the representation and reality. A timing budget can identify an infeasible allocation. A state model can expose a missing transition rule. A dependency model can show why duplicated equipment does not address a shared outage. None is a complete representation of every security concern.

The engineering task is to choose a useful representation, state its assumptions, examine its implications and obtain evidence that the important assumptions apply. This includes deciding when a simple model is sufficient and when greater fidelity is justified. More detailed models can reveal additional relationships, but also require more input information and create more opportunities for hidden error.

The current decision is whether the architecture concepts in Chapter 9 merit further development and what work is required before selecting among them. That decision does not require a production-grade simulation of the entire logistics operation. It does require enough analysis to detect conflicts among SR-00, SR-03, SR-05 and SR-11; expose common dependencies; and determine which unresolved properties could change the architectural choice.

A model record should identify its purpose, represented boundary, variables, assumptions, input sources, limitations and intended decision. Record whether values are requirements, proposed allocations, estimates or observations. A requirement of sixty seconds and a measured delay of sixty seconds are not the same kind of information. In this chapter all numerical inputs are invented for the teaching case; the calculations are real arithmetic over those stated assumptions, not measurements of an operational system.

NASA's decision-analysis process connects alternatives, criteria, analytical methods, uncertainty and recommendations. [6, Section 6.8] The following models and worked decision record apply that general discipline through original examples. They do not reproduce a NASA method or assign its decision authorities to another organization.

11.2 Match the representation to the uncertainty

Use functional analysis to ask which contributions are necessary for the required action. Use behavioural analysis to ask how state and ordering affect it. Use temporal analysis to examine whether those contributions occur soon enough. Use dependency analysis to identify what can fail together. Use a task model to examine whether intended people can perform the work with available resources. These views can share identifiers without pretending that one representation answers all five questions.

For example, a network diagram can show an identity connection, but not whether authorization is rechecked at commit. A sequence description can show a recheck, but not establish the source's freshness semantics. A numerical budget can include a status age, but not demonstrate

that the record is authentic. A complete argument about SR-03 and SR-05 therefore combines several kinds of evidence and reasoning.

Model-based systems engineering can support this coherence through controlled relationships among requirements, functions, states, interfaces and analyses. The important result is consistent, reviewable engineering information. It is not necessary to mandate a notation before deciding what must be represented. Conversely, choosing to work in ordinary documents does not excuse contradictory definitions of target, commit or mode.

A model needs a configuration identity. A timing analysis based on one authorization interface should not silently remain authoritative after the interface changes to a different caching arrangement. Identify the configuration and assumptions that make the result applicable. A change to a model's input can matter as much as a change to executable code when people rely on the result for a consequential decision.

Avoid calibrating a model merely to reproduce a preferred answer. If a measured or analytically justified contribution exceeds its allocation, the result should prompt redesign, reallocation or reconsideration of the requirement with the appropriate authority. Editing the assumed delay until the budget fits does not establish feasibility.

11.3 Practise engineering through models, not merely diagrams

An IT team may already keep a service map, an access-control matrix, an interface specification and a set of tests. Those records contain much of the raw material needed for model-based work. The difficulty is often the relationship among them. Which function does a privilege permit? Which receiving component performs it? Which requirement governs the interaction, and which version of the test examines that requirement? Model-based systems engineering makes such relationships part of the working engineering information rather than leaving them to an individual's recollection.

INCOSE describes MBSE as formalized modelling supporting requirements, design, analysis, verification and validation across the life cycle. [22, Mission] Models help develop, challenge and maintain engineering decisions; they do not replace systems engineering or create a parallel security discipline.

Distinguish five things before selecting software. Systems engineering is the integrating discipline. MBSE is a model-based approach to carrying out that work. A method organizes the questions and activities used to develop a solution. A modelling language supplies concepts and rules for expressing selected aspects of it. A tool provides capabilities for creating, relating, checking, exchanging or analysing the representations. Confusing these levels encourages a project to buy a tool before deciding what engineering problem it must solve.

A diagram is a view, not necessarily the whole model. A model can contain definitions, typed relationships, conditions and values that do not appear in a particular picture. Conversely, a drawing can look technically elaborate while its connections have no defined meaning. The reader should ask what each relationship asserts and which information controls that assertion. A clean drawing is useful communication; it is not, by itself, an adequate model of the relevant behaviour.

Nor must every model be executable. A descriptive architecture model can organize functions and dependencies; a behavioural model can define transitions; a quantitative model can calculate a bound; a simulation can explore responses over time. An assurance argument may

use all of them. The additional semantics needed for execution must be supplied and reviewed. Animation does not become an implementation test simply because the display resembles the intended system.

NASA's Systems Modeling Handbook covers planning, model organization, relationships and generation of engineering work products. [23, Sections 5–7 and 9] The workflow below applies those concerns to the logistics case using the book's own records, without adopting a NASA method or tool requirement.

The immediate objective is modest: create a connected description of one security-relevant interaction that engineers, administrators and assessors can review together. A larger toolchain may improve scale and automation later. Beginning with a manageable slice avoids both extremes—an isolated security diagram that cannot inform design, and an ambitious modelling programme that produces no useful decision.

11.4 Organize a small, controlled model slice

Call the worked slice MS-01. Its subject is NORMAL-mode dispatch-rule activation and the related restoration interfaces in TE-03. Its purpose is to support analysis of authority, effective change and recovery dependencies. It retains the existing SoI: the enterprise identity service and operational dispatch processing are not brought inside the boundary merely to simplify a diagram. The records below describe how a model could be organized; they are not a native model file or exports from an executed tool.

Establish a small information agreement before populating the model. It identifies the kinds of things represented—needs, requirements, functions, elements, interfaces, scenarios, analyses and evaluation records—and the meanings of their relationships. This is the useful purpose of a metamodel in this discussion: a model of the modelling concepts and permitted relationships. It is not necessary to invent a complete language. Adopt the project's conventions where they work, adding only the security information needed for the decision.

Reuse PN-01, SR-03, SR-05, LS-01, AD-01, IC-01, C-02 and VP-02. They already describe different aspects of the same concern. Do not replace them with a second set of "MBSE requirements." The engineering repository can reference an authoritative requirement held in another controlled system, provided the identifier, revision, meaning and synchronization arrangements are maintained. An imported copy needs a declared relationship to its source, not an assumption that the most recently edited copy controls.

For each important element, identify its purpose, owner, revision and status. Distinguish a requirement from a proposed implementation, an assumed parameter from an observed value, and a planned evaluation from a completed result. Store units and scope with quantities. A value of 30 without seconds, a starting event and an authority-status meaning cannot adequately represent SR-05. Distinguish the normal-mode freshness tolerance from a recovery authorization lifetime even if both are time values.

Separate common content from alternatives. ARC-B and ARC-C share candidate receiving-side protections but differ in preparation, authority and permitted recovery actions. A view of ARC-C must not silently borrow ARC-B's flexible package selection. Record which variation of REC-P1 and which eligibility assumptions belong to each concept. Otherwise the model can create a fictitious alternative that combines the advantages of two designs without their restrictions.

Table 11.1 establishes a compact reviewable slice. Its entries reference existing case objects; descriptive function and element names are not new product selections. No row establishes a verified result.

Table 11.1. MS-01: a connected, tool-neutral model slice

Engineering information	MS-01 relationship	What the relationship does not establish
PN-01 and LS-01	The need and stale-authority scenario motivate SR-03 and SR-05; retain their distinct rationale.	That every cause of unauthorized activation has been identified.
SR-03 / SR-05	Relate each requirement to AUTH-P1, TIME-P1 and the event at which authority affects commit.	That withdrawal timing and status freshness are the same property.
Authority and commit functions	Allocate the proposed effective-change contribution to the receiving coordinator under AD-01.	That all real writers are known, or that the implementation follows the allocation.
IC-01 and transaction behaviour	Use the same request, generation, epoch and outcome meanings in sequence and state views.	That matching diagram labels prove compatible or implemented semantics.
Section 11.11 analysis	Bind a, d and u to identified events, units, source assumptions and the selected model revision.	That assumed bounds are measured performance or runtime enforcement.
VP-02 and C-02	Plan evaluation against the same requirements; record actual evidence separately when produced.	That a planned evaluation or satisfaction link is a passing result.

MS-01 can generate a requirements-oriented view for an assessor, an allocation view for a designer and an interaction view for an administrator. A report should identify the selected configuration and alternatives, including filters that hide content. A view omitting RECOVERY may be suitable for a NORMAL-mode decision but cannot establish the whole recovery claim.

Keep the level of modelling proportionate. Not every explanatory sentence needs a formal element, and not every external record needs to be copied into the repository. Capture the identities and relationships that make a consequential decision reproducible. Remove redundant representation where it creates synchronization work without improving analysis. The measure of success is whether the right questions can be answered and challenged, not the number of objects created.

11.5 Follow a requirement into design, analysis and evidence

Begin the walkthrough with PN-01 and the LS-01 stale-authority scenario. They explain why a receiving-side action requires more than a successful login. SR-03 states the withdrawal deadline; SR-05 limits the age of the authority-status record used at commit. Both remain separate requirement objects. A trace relationship records the rationale connecting them to the concern; it does not make them equivalent.

Next represent the functions that admit a request, establish applicable authority, coordinate activation and report an authoritative outcome. Allocate the proposed effective-change responsibility to the receiving coordinator under AD-01. Relate the coordinator to the gateway, enterprise identity interface and protected transaction state. The allocation establishes an intended contribution to examine. It does not establish that a deployed endpoint performs the function or that every writer is covered.

Use a sequence view to distinguish receipt, evaluation, commit and outcome discovery. Use a state view to show when a request can remain pending, be rejected or commit, including the effect of a mode transition. Reference the same IC-01 request and outcome concepts in both. If the sequence assumes that authorization occurs immediately before commit while the state view allows a prolonged queue afterwards, the representations are not yet consistent. Resolve the design meaning before asking a tool to render another view.

Connect a quantitative analysis to those events rather than inventing a detached timing figure. Section 11.11 defines observed record age a, remaining evaluation-to-commit bound d and additional time allowance u. MS-01 relates a to the authority observation, d to the final-evaluation/commit interval, and u to TIME-P1. The constraint $a + d + u \leq 30$ seconds is an analytical obligation at the chosen point. A model annotation containing that expression does not itself enforce the constraint in the running system.

Plan VP-02 against the same requirements and event definitions. The proposed evaluation needs source observation and revocation semantics, receiving-state observations, relevant paths and time relationships. Record the procedure revision, selected configuration and coverage once those exist. A link to VP-02 means that evaluation is planned; its result remains not performed. Attach an actual result only after the work has been executed and reviewed for its stated purpose.

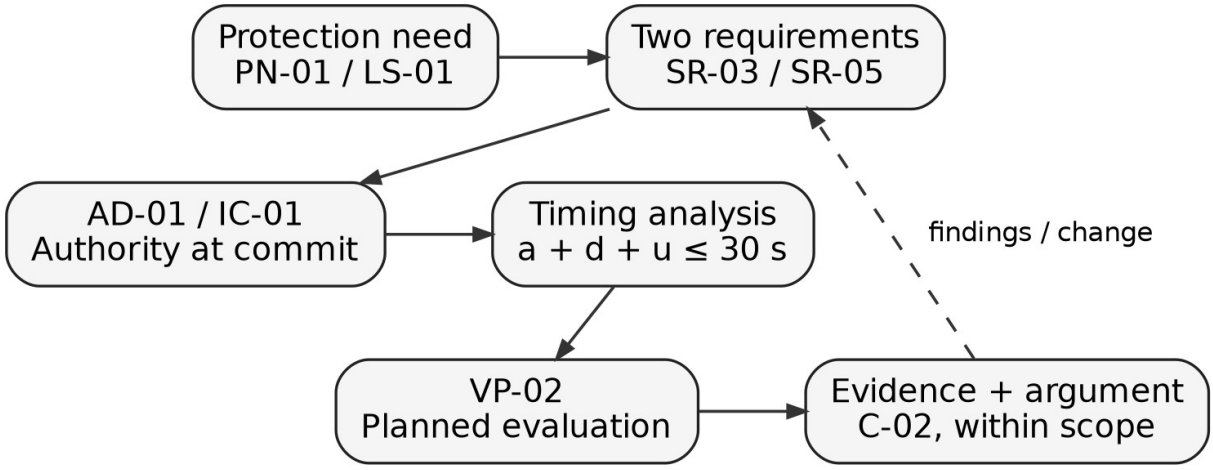
Language-specific relationship names require care. The Object Management Group (OMG) overview of the Systems Modeling Language (SysML) v1 describes satisfy, verify and allocation relationships; SysML v2 has its own requirement-satisfaction and verification-case constructs. [24, SysML Diagram Summary; 25, Sections 7.21 and 7.24] These constructs organize assertions and evaluation meaning within a model. Their presence is not independent evidence that a physical or software implementation has been tested successfully. Keep the design assertion, evaluation definition and actual result distinguishable in both the repository and exported report.

A model-based review can now ask focused questions. Which normal-mode writers have no allocation to effective-change enforcement? Which relevant states have no planned evaluation? Which results refer to an earlier profile revision? Which analyses depend on an unconfirmed external-service bound? A tool can assist with these queries where the information is represented and the query is implemented correctly. No such query discovers an endpoint that the inventory and model both omit.

For the compliance reader, the output is not a replacement for the control assessment. It is a more explicit account of the particular implementation, required behaviour and supporting evidence. POL-01 remains identifiable; authority and source obligations are preserved; missing evaluation is not disguised as a passing trace. The assessor and engineer can examine the same records while retaining their different decision purposes.

The following schematic brings together the relationships developed here.

Figure 11.1. One concern across the model and its evidence



Original MS-01 schematic. Arrows denote recorded engineering relationships, not proof or runtime enforcement. A planned test link is not a passing test result.

11.6 Use the model to assess a change, not authorize it

Suppose a proposed redesign adds work between the final authority evaluation and commit. Its candidate bound d rises from eight to twelve seconds. The change is explored in a separate MS-01 revision; the earlier case input values and conclusions are retained. This is a new fictional what-if branch, not an approved change to TE-03 or to an operational service.

With observed status age a = 18 seconds and additional allowance u = 2 seconds, the earlier age bound was 18 + 8 + 2 = 28 seconds. The proposed branch gives 18 + 12 + 2 = 32 seconds. That exceeds SR-05's thirty-second limit. To justify the same predicted-commit approach with d = 12 and u = 2, the observed age at the final evaluation would have to be no more than sixteen seconds. All quantities retain the definitions and limitations in Section 11.11.

This does not establish that an actual system committed an unauthorized change. It establishes that the proposed bounds no longer justify proceeding with the eighteen-second record under this analytical design. The response could be to obtain fresher status, shorten the remaining interval, move the final evaluation or investigate a different coordinated design. Each has implications for performance and implementation. Relaxing the requirement is a separate decision requiring its source and authority.

If the same new bound replaces the eight-second contribution in the complete withdrawal path from Part II, that illustrative sum changes from 45 to 49 seconds. It remains below SR-03's sixty-second deadline under the original allocation assumptions. This second result does not rescue the failed freshness argument. The model should preserve both conclusions because they concern different requirements.

An impact query identifies candidate consequences through recorded relationships. It does not decide that every linked item requires retesting, or that an unlinked item is unaffected. Review the query results against the actual design change, including missing relationships. Table 11.2 shows a reasoned disposition rather than a universal rule to invalidate every earlier result.

Table 11.2. Model-supported impact review for the proposed delay change

Affected item	Reason to examine it	Proposed disposition
SR-05 and freshness analysis	The candidate d changes from 8 s to 12 s; 18 + 12 + 2 = 32 s.	Do not reuse the 28 s result for this branch. Examine fresher status or another design response.
SR-03 withdrawal allocation	The same replacement would change the illustrative sum from 45 s to 49 s.	Retain its conditional numerical room; re-establish the complete path and bounds. This does not resolve SR-05.
SR-00 and performance analysis	The new work may affect the response path, scheduling or resource contention.	Reconcile interval definitions before recalculating; do not substitute a bound from a different model blindly.
IC-01, VP-02 and C-02	Authority at effective change and the corresponding evaluation depend on the altered interval.	Review contract realization, planned observations and evidence applicability for the proposed configuration.
Recovery branches	The receiving coordinator is shared, but SR-05 is NORMAL-mode specific.	Check actual common implementation and REC-P1 dependencies; do not impose normal-mode timing on recovery without an explicit requirement.

SR-00 also needs attention, but its ten-second response target cannot be assessed by blindly adding four seconds to every number in Section 11.12. That performance illustration has its own critical path, including a 0.7-second coordinated-commit contribution and particular interruption assumptions. Determine whether the new operation lies on that path, whether the bounds overlap, and whether arrivals, retries and resource contention alter the result. A shared unit does not make differently defined intervals interchangeable.

The engineering participants then review the findings. Application engineers address coordination and scheduling; the identity team addresses usable status information; operational staff address task consequences; SSE challenges the protective effect. The responsible authority decides the permitted commitment. A model merge, successful consistency check or change-ticket closure is not the authority to accept a requirements discrepancy.

Once a direction is chosen, update the affected model content, profiles, allocation and evaluation plans together. Preserve the previous analysis inputs and results with their original configuration identity. Mark prior support as applicable, limited, superseded or requiring review as justified. Do not overwrite an old result so that it appears to have evaluated the new design. This is model-supported change control, not automatic change approval.

11.7 Establish confidence in the model and its use

A model can be internally coherent and still represent the wrong problem. Separate checking its syntax and declared modelling rules from checking its suitability for the intended engineering question. A project also needs to establish whether the implementation corresponds sufficiently to the representation before transferring a model conclusion to the delivered system. These are related obligations, not three names for the same tool command.

NASA's Standard for Models and Simulations addresses development, use, acceptance and credibility. [30, public scope] It is a useful public reminder that confidence in modelling results needs explicit treatment. The following review questions are the book's proposed application; they do not assert compliance with NASA requirements or impose NASA technical authorities on the reader.

First examine purpose and abstraction. For SR-07, Section 11.14 assumes an indivisible commit transition and preserved history. Its no-second-commit argument is meaningful within that abstraction. A simulator that includes only those transitions cannot discover that a later snapshot restore erases the history unless restore is represented. Challenge excluded transitions, external actors and resource limits before promoting the conclusion.

Next examine input meaning and provenance. Is an eight-second interval a requirement allocation, an engineering estimate or a justified bound? Does a probability describe local failure, shared-service loss or hostile action? Are the units compatible? Where an assumption substitutes for missing data, retain that status in the result. A plot with many significant digits does not improve the evidence underlying its inputs.

Then examine execution, where execution is used. Record the model revision, selected alternative, parameter set, tool and relevant library versions, solver settings, test inputs and material warnings. Check known small cases independently and investigate sensitivity to settings that can change the conclusion. Re-running a model with an omitted transition produces a reproducible omission; reproducibility is necessary for review but not sufficient for correctness.

Review the connection to implementation separately. A logical action model does not establish the scheduler, storage behaviour, parser correctness or restart semantics of a delivered endpoint. Code generation can preserve specified relationships only within a defined transformation and use context; generated code, configuration and their integration still require appropriate evaluation. Likewise, deriving both test expectations and implementation from one faulty model can allow them to agree about the same error. Use independently reasoned scenarios and observations where they are needed to challenge that common dependency.

Model-based testing can make planned sequences and coverage more systematic. It can help select cases around a mode transition or freshness boundary, provided the model captures the relevant conditions and the test oracle has a justified meaning. It does not turn a model run into a hardware-in-the-loop test or a representative operator exercise. Retain the item or representation actually evaluated and the limits on each result.

A useful model-review conclusion is specific: the representation supports the stated architecture comparison under identified assumptions, while storage recovery and path completeness remain unestablished. That conclusion can justify targeted prototyping without justifying deployment. The quality standard is an intelligible boundary on reliance, not a model-wide badge labelled “validated.”

11.8 Collaborate, exchange and protect the modelling information

An integrated model need not reside in one database or be owned by one team. Requirements, architecture, behavioural analysis, source code and test results may remain in different controlled repositories. The project needs an authoritative source for each item, stable references

and a rule for establishing which combination of revisions was used for a decision. A collection of live links to “latest” is not a reproducible baseline.

For MS-01, identify the controlling requirement revision, IC-01 revision, selected architecture concept, analysis configuration and supporting evidence records. Where a supplier provides a partial model, record what is included, what is omitted and what assumptions are exposed at the receiving boundary. Missing internal detail is a limitation on particular conclusions, not permission to assume a dependency does not exist.

A digital thread, as used here, is the maintained chain of relationships across those engineering records and life-cycle states. The term does not promise one complete or automatically correct source of truth. Design intent, as-built configuration and observed operational state are distinct. Preserve their relationships and discrepancies rather than silently replacing one with another when information arrives from a deployment or monitoring system.

Exchange should be evaluated for meaning, not only file readability. A tool may import names and shapes while omitting a conditional relationship, unit, selected variation or external evidence reference. For an exchange check, choose a small representative slice containing the authority requirement, commit behaviour, timing quantities and a deliberately open result. Compare the receiving representation with the source, including query results and important calculations. Report unsupported content explicitly. Successful import without an error message is not a complete semantic comparison.

The standardized Systems Modeling API and Services provides access, navigation and operations for KerML-based models, including SysML models, and defines specific service and conformance scopes. [26, Clauses 1–2 and 7–8] This supports specifying concrete integration needs. It does not establish that any particular installed pair of tools preserves the project's information or implements all services. Evaluate the relevant versions and integration, including authentication, permissions and unavailable-service behaviour.

Make the review accessible to people who do not author models. Provide readable tables, scenario narratives and selected views with legends, scope and revision. A reviewer should be able to identify an incorrect operational assumption without having to learn an entire modelling language first. Record comments against the underlying elements where possible, then show how their disposition affects the common description. Read-only access is useful only when it exposes enough information for the review being requested.

Protect the environment as an engineering dependency. Model repositories, reusable libraries, scripts, importers and report generators can affect design decisions and their apparent support. Apply appropriate access separation, review of significant changes, version identification, backup and restoration, and controls on executable extensions. Examine a shared script that both changes a requirement mapping and produces the compliance report claiming the mapping is complete. The toolchain has become part of the evidence-dependence problem in LS-06.

Keep sensitive design and vulnerability information within the agreed information-handling arrangements. External collaboration and automated assistance do not justify uploading a complete model to an unapproved service. Where an AI assistant proposes elements, mappings or scripts, treat its output as a proposed contribution requiring review and testing. It has not performed the system evaluation merely by generating an assessment field. The fuller treatment of enabling systems belongs in Chapter 13; its architectural dependencies are already relevant here.

11.9 Choose a toolchain around demonstrated engineering tasks

Before comparing products, ask what the team needs to do with the model. For this case, useful tasks include tracing SR-05 to the effective-change behaviour, exploring the twelve-second delay branch, comparing recovery dependencies, identifying stale evidence and exchanging a reviewable slice with a supplier. These tasks supply a more defensible demonstration script than a long checklist of diagram types.

Distinguish capabilities that may be supplied by different tools or extensions: requirements and configuration management; architecture and behaviour authoring; calculation, simulation or formal analysis; evidence and test management; collaboration and model exchange. One product may support several of these functions, while another may specialize. Establish the responsibility of each part of the toolchain and test the information transferred between them. Purchasing a broad suite does not remove interface engineering.

Include the ordinary constraints of use. Can the relevant people inspect and comment on the information? Can work continue under the organization's connectivity and security restrictions? Can an analysis be repeated with identified library and tool versions? Can the model, its relationships and necessary records be retained and read after a supplier or product changes? Consider training, administration, integration and sustainment as well as licence cost. Those costs and constraints belong in the decision, not in an afterthought labelled implementation.

A proof-of-concept exercise should include an adverse case. Intentionally remove the relationship between the commit action and its authority requirement, change a unit, import an incomplete profile or substitute an old evidence revision. Determine which conditions the tool detects and which still require an engineering review. The result characterizes the configured toolchain's support for a task; it does not certify the system described by the model.

The dated companion note in Appendix A explains SysML v1/v2 and representative tools separately from the core method. The book does not require a commercial modelling licence. A reader can perform the MS-01 walkthrough using the supplied records and calculations; native-tool exercises would require a separately prepared and checked model package. None is implied by this manuscript.

Proportion matters. For a small change, controlled relationships and one targeted model may provide the necessary decision support. A larger or longer-lived system may justify a federated repository, executable analyses and automated checks. The trigger should be the information and evaluation problem, not the desire to declare that the programme has “adopted MBSE.”

11.10 Exercise: a complete-looking model with incomplete support

Situation. A supplier presents MS-01 with a relationship indicating that the receiving coordinator satisfies SR-05 and a planned VP-02 evaluation linked to the same requirement. The tool reports no model-consistency errors. In a proposed branch, the recorded maximum evaluation-to-commit delay has increased to twelve seconds, but the report still displays the earlier 28-second age result. The authority-status age used for that result is eighteen seconds, and TIME-P1 contributes two seconds. An exported view omits the time allowance. The supplier calls the requirement verified.

Worked answer. Separate the findings. The design relationship records an assertion about a proposed contribution, while the VP-02 relationship identifies evaluation work. Neither supplies

a performed result. A tool-consistency pass establishes only what the configured checks examined. It does not establish freshness at commit or correspondence with the deployed endpoint.

The earlier age result belongs to the eight-second-delay configuration. For the proposed branch, the corresponding calculation is 18 + 12 + 2 = 32 seconds. The old result is not applicable unchanged, and the new bound does not support committing on that record under SR-05. The source of the discrepancy is the changed input and its relationship to the consequential event; it is not solved by retaining the old figure in a report.

The exported view introduces another limitation. Omitting the two-second allowance makes a receiver liable to calculate 18 + 12 = 30 and infer apparent satisfaction. The exchange record should preserve the allowance and its meaning, or explicitly state that the receiving view is insufficient for the freshness decision. A diagram that looks the same can carry a materially different analysis.

Correct the affected records and their status, investigate the necessary design response and perform the planned evaluation when its prerequisites exist. Preserve useful evidence for unchanged matters. There is no basis to declare every requirement failed, but no basis to declare SR-05 verified either.

Reader task. Recreate Table 11.1 as a small set of linked records. Add the proposed delay change and identify which source, requirement, function, interface, analysis, claim and evaluation record each link reaches. Write one sentence explaining each relationship. Mark unperformed work and unresolved dependencies explicitly. No modelling software is needed to demonstrate that reasoning. A later tool exercise should reproduce it rather than replace it with unexplained symbols.

The remaining sections connect the shared engineering information to quantitative, state-model and trade studies. Their assumptions and limits matter equally when the calculation is performed in a spreadsheet, a modelling environment or a reviewed script.

11.11 Check freshness at commit, not only revocation propagation

Part II proposed a forty-five-second withdrawal budget: ten seconds for source processing and publication, twenty for distribution and update propagation, five for receiving enforcement, eight between final authority evaluation and commit, and two for timing uncertainty. That leaves fifteen seconds within SR-03's sixty-second deadline if the sequence is complete and each contribution is a justified bound.

SR-05 imposes a separate condition: the conservative upper bound on the age of the authoritative authorization-status record must not exceed thirty seconds at a normal-mode commit. A withdrawal budget below sixty seconds does not automatically satisfy this freshness requirement. One concerns propagation from a revocation event; the other concerns the age of the evidence on which an effective action relies.

Let a be the observed age of a status record at the final evaluation, d the upper bound on the remaining evaluation-to-commit interval, and u the conservative allowance needed for the time relationship. For this simplified model, the record-age bound at commit is a + d + u. The same uncertainty must not be counted twice: a here excludes the uncertainty allowance that u represents. The variables are elapsed-time quantities in seconds, not scores.

With d = 8 and u = 2, satisfying the thirty-second age criterion requires a ≤ 20. At an observed age of 20.4 seconds, the bound is 20.4 + 8 + 2 = 30.4 seconds. The proposed design cannot justify proceeding on that record under SR-05. It needs a fresher record, a shorter justified delay or another valid design response; it cannot label the extra 0.4 seconds harmless merely because SR-03 still has margin.

Table 11.3. Original freshness-budget illustrations

Observed age at final evaluation	Remaining delay bound	Time allowance	Upper age at commit and implication
18.0 s	8.0 s	2.0 s	28.0 s; numerical room remains, conditional on the inputs and their meaning.
20.0 s	8.0 s	2.0 s	30.0 s; at the stated limit, with no additional unallocated allowance.
20.4 s	8.0 s	2.0 s	30.4 s; the record does not support the proposed commit under SR-05.

The calculation assumes that the status observation is genuinely authoritative and correctly timestamped. An old status copied into a newly timestamped message is not made fresh by the new envelope. A compromised source can produce a timely false statement; a freshness budget is not a model of resistance to that compromise. AUTH-P1 and the source-interface agreement must establish what the observation means.

A design can make the freshness decision at the effective-change boundary rather than predict a long remaining interval. That can reduce one uncertainty, but the final guard and commit still need a realizable relationship. If a pause, restart or competing operation can occur between them, account for it. The model identifies the necessary property; the implementation must demonstrate how it provides it.

The budget also exposes the need to coordinate source consistency with status age. A source observation that is recent but does not include a previously recorded revocation needs an explicit consistency bound. SR-03, SR-05 and AUTH-P1 must be satisfied together. Asking only how frequently a cache refreshes is not enough to characterize the full source-to-commit behaviour.

11.12 Test a positive-performance budget for missing assumptions

SR-00 requires a valid, authorized, non-conflicting normal activation and its authoritative outcome within ten seconds under the defined envelope. A protective design that repeatedly refreshes status or serializes state changes must still meet that useful-capability requirement. Security and performance are not separate designs that can be reconciled after implementation.

Consider an illustrative critical path consisting of 0.8 seconds of local validation, a two-second status exchange, one five-second communication interruption, 0.7 seconds for the coordinated commit and 0.9 seconds to make the outcome available. The total is 9.4 seconds, leaving 0.6 seconds within the ten-second target. Each contribution is a proposed bound; none is measured performance.

Traceability note — The 0.7-second coordinated-commit contribution in this illustrative SR-00 critical path is the same proposed allocation carried into the CB-09 final-check/coordinator path in Appendix C.5; it is not an additional 0.7-second interval to be added to that path. The underlying authorization-status freshness calculation remains a separate at-commit constraint (SR-05, Section 11.11) and retains its stated age, delay and uncertainty terms. The shared allocation is a proposal to be validated, not observed performance.

The arithmetic alone conceals an important assumption. NET-P1 describes delivered round trips and interruptions, but Part II does not fully define the number or spacing of interruptions within one operation. The simple 9.4-second model assumes at most one such five-second interruption on the relevant critical path and assumes retry overhead is included in the stated contributions. Repeated interruptions could invalidate its conclusion. The model has found a profile-completion issue, not proven SR-00.

Workload has a similar ambiguity. Ten activation attempts per minute does not, by itself, specify whether those attempts arrive evenly or simultaneously. A design with a single commit resource can experience different queuing delays under those arrival patterns. Five concurrent sessions also does not specify the number of requests each session may have in progress. An average rate is not an upper bound on instantaneous demand.

The correct next step is to define the required arrival envelope and interruption conditions with the service owner and interface providers. A burst allowance, admission rule or maximum outstanding-request condition may be appropriate, but it must not be invented after testing to exclude an inconvenient result. Record the refinement as a change to ADM-P1 or NET-P1 and identify affected requirements and evaluations.

This illustrates why model review is part of requirements validation. A concise, mathematically correct model can expose missing behavioural meaning. Increasing model complexity before resolving that meaning would merely produce more precise calculations for an underspecified problem.

11.13 Quantify shared dependencies without inventing attack probabilities

A small reliability model can demonstrate the effect of a shared dependency. Let qS be the probability that a required shared service is unavailable at a randomly selected observation, and qG the conditional probability that one gateway is unavailable when that service is available. For illustration, assume the two gateways' local unavailability events are independent conditional on the shared service being available, and that these are the only represented causes.

Under those assumptions, the two-gateway capability is unavailable when the shared service is unavailable, or when it is available and both gateways are locally unavailable. The model is:

$$Q(\text{two gateways}) = qS + (1 - qS) \times qG^2.$$

With invented values qS = 0.02 and qG = 0.01, the result is 0.02 + 0.98 × 0.0001 = 0.020098. The corresponding modelled availability is 0.979902, or 97.9902 percent. Treating the entire arrangement as two independent gateways and using qG^2 alone would give 0.0001, omitting the shared dependency.

For comparison, one gateway under the same assumptions has unavailability 0.02 + 0.98 × 0.01 = 0.0298, or availability 97.02 percent. The second gateway improves the result in this model, but does not remove the shared-service contribution. If further local gateways were independent

under the same conditions, increasing their number would reduce the local term while leaving the shared-service floor.

Table 11.4. Shared-dependency calculation for invented snapshot probabilities

Arrangement in the model	Unavailability calculation	Result
One gateway and the shared service	$0.02 + 0.98 \times 0.01$	0.0298
Two conditionally independent gateways and the shared service	$0.02 + 0.98 \times 0.01^2$	0.020098
Incorrect omission of the shared service	0.01^2	0.0001; not the full capability model

These values are not estimates for any real gateway, identity service or cyberattack. The model does not describe outage duration, repair time, hostile coordination, recovery procedures or the probability of meeting SR-11. Snapshot availability cannot determine whether staff can restore a configuration within fifteen minutes of a declared need.

The independence assumption also needs challenge. Common deployment automation, configuration error, power, administrators or software defects can couple gateway failures even when the identity service is available. Adding those causes changes the model. A vendor's component availability figure is not automatically an appropriate input without compatible event definitions, observation conditions and evidence.

For adversarial scenarios, examine capability and causal paths before assigning probabilities. An attacker may deliberately target the shared privileged function that affects both gateways. The arithmetic illustrates why duplication must be assessed against a specified cause; it does not quantify overall security or deliberate attack success.

11.14 Use a small state model to test a precise property

A state model can make an architectural obligation easier to challenge. Consider a simplified state containing an active generation g, a mode epoch e and a history H that maps transaction identifiers to their bound content and outcome. Ignore networking, physical storage failure and hostile modification for the moment. State those exclusions explicitly rather than quietly treating them as solved.

Define an abstract commit transition as one indivisible step. If the identifier is already in H, the transition returns the recorded outcome for matching content or rejects conflicting reuse, without changing g. If the identifier is new, the transition checks the required authority, expected g and current e. On success it advances g and adds the committed identifier and content to H in the same step. Failed guards do not produce a commit.

Within this model, and while H is never erased or rolled back, a transaction identifier cannot produce two commits. Initially no identifier has committed. A successful first commit records its identifier. Every later transition using that identifier takes the existing-history branch and

cannot advance g for another commit. This is an induction argument about the defined transition rules, not a result from an implemented service or an automated proof tool.

Now add a restore transition that resets H to an older snapshot while leaving the external effect of a later commit in place. The earlier argument no longer applies: an identifier can again appear new. The counterexample reveals why snapshot and retention behaviour belong in the transaction design. It does not establish that every restore is unsafe; it identifies the property that a permitted restore must preserve or reconstruct.

A second counterexample appears when expected-generation checking and update are separated without coordination. Two new transactions can both observe g = 27 and proceed. If each later updates state independently, the model no longer guarantees rejection of a conflicting starting generation. The repair is not to add a statement that users should avoid simultaneous work. It is to specify the necessary coordination at the state-changing boundary and evaluate its realization.

This is a useful level of formal reasoning for an introductory practitioner text. It demonstrates a limited property under visible assumptions, then shows how an omitted transition can defeat it. A real assurance argument would need a validated abstraction, complete relevant transitions and evidence that the implementation refines the model, including interruption, storage, clocks and adversarial influence. A proof of the wrong or incomplete model is not proof of the intended system.

11.15 Screen binding constraints before comparing preferences

A trade study should first distinguish conditions an alternative must satisfy from attributes on which stakeholders permit a trade. Applicable obligations, required useful capability and necessary interface conditions belong in the first category. Cost, convenience or a preferred implementation style cannot compensate for failure to meet a binding condition unless the responsible authority explicitly changes that condition or permits a defined exception.

For ARC-A, the present description does not supply restricted restoration during the specified identity outage. That is a failed feasibility condition for the complete SR-11 response, not a low score to average against inexpensive hardware. For ARC-B and ARC-C, recovery authority and evidence of operational capability remain unresolved. An unresolved gate is not a passing gate, but may permit proportionate exploratory work aimed at resolving it.

Table 11.5. Initial architecture screening against the selected obligations

Condition	ARC-A	ARC-B	ARC-C
Preserve applicable MFA and authority obligations	Requires implementation evidence.	Proposed independent arrangement needs authority decision and evidence.	Proposed prepared authority and independent authentication need decision and evidence.
Enable restricted restoration during the assumed identity outage	Not supported by the defined central-only concept.	Conditionally plausible; task and dependency evidence needed.	Conditionally plausible only while an eligible prepared instruction exists.

Condition	ARC-A	ARC-B	ARC-C
Preserve commit, repetition, version and mode properties	Receiving-side design still required.	Common transaction and transition design still required.	Common transaction and transition design still required.
Support a complete production selection now	No, under the unchanged recovery need.	No; material gates remain open.	No; material gates remain open.

The table does not mean that the team is unable to act. It identifies the next justified commitment. Remove ARC-A as the sole response to SR-11, while retaining its useful normal-mode elements. Develop ARC-B's operational task and authority arrangement and ARC-C's eligibility and preparation assumptions. Obtain the information that can distinguish them before spending heavily on low-impact details.

A traceable gate should identify its source, criterion, owner, current evidence and consequence of remaining unresolved. “Security approval required” is too vague to direct work. “The policy authority must determine whether the proposed independent recovery authentication and withdrawal arrangement satisfies POL-01 and the approved authority model” identifies an actionable issue without predetermining the answer.

11.16 Compare preferences transparently and test sensitivity

Once alternatives satisfy the binding conditions, preference analysis can support a choice among them. To illustrate the mechanics, temporarily assume that both ARC-B and ARC-C could satisfy all gates. This is a numerical what-if exercise, not a statement that their outstanding issues have been resolved.

Assume an estimated whole-life cost of 100 arbitrary cost units for ARC-B and 140 for ARC-C, with estimated restoration task times of thirteen and nine minutes respectively. Define an illustrative cost value of 1 at 80 units and 0 at 160 units, decreasing linearly between them. Define a restoration-time value of 1 at five minutes and 0 at fifteen minutes, also decreasing linearly. These are declared stakeholder preference functions for the example, not probabilities or inherent measurements of security.

The resulting cost values are 0.75 for B and 0.25 for C. The time values are 0.20 for B and 0.60 for C. Let w be the weight assigned to the defined cost-value swing, with 1 − w assigned to the time-value swing. The deliberately simplified additive model assumes that these two preference contributions can be combined independently for this illustration.

$$V(B) = 0.75w + 0.20(1 - w) = 0.20 + 0.55w.$$

$$V(C) = 0.25w + 0.60(1 - w) = 0.60 - 0.35w.$$

At w = 0.40, B scores 0.42 and C scores 0.46. At w = 0.60, B scores 0.53 and C scores 0.39. The preference changes at w = 4/9, approximately 0.4444. Those results do not reveal an objectively best architecture; they expose the effect of the declared preferences over the defined ranges.

Now change C’s task-time estimate from nine to 10.5 minutes. Its time value becomes 0.45 and, at w = 0.40, its combined value falls to 0.37, below B’s 0.42. The reversal shows that the operational-task estimate merits investigation before the small original lead informs a choice.

This exercise intentionally excludes other important attributes, such as implementation difficulty, authority exposure, support competence, adaptation cost and evidence maturity. A real decision must consider them explicitly. Excluding an attribute from a small teaching model does not make it neutral or unimportant. Nor should a “security score” be added without defining what it measures and whether its scale supports the arithmetic.

Weights must be understood in relation to the value-function ranges. Changing the range from 80–160 cost units to a different interval changes the meaning of the cost swing. A weight labelled “40 percent cost” is not meaningful independently of that definition. Counting correlated attributes separately can also reward the same benefit twice. The decision record should permit a reviewer to reconstruct both the input values and the preference logic.

The preference crossover can be seen more quickly in a plot than in the equations alone. The chart does not add another decision criterion; it visualizes the value functions and assumptions already defined in Section 11.16.

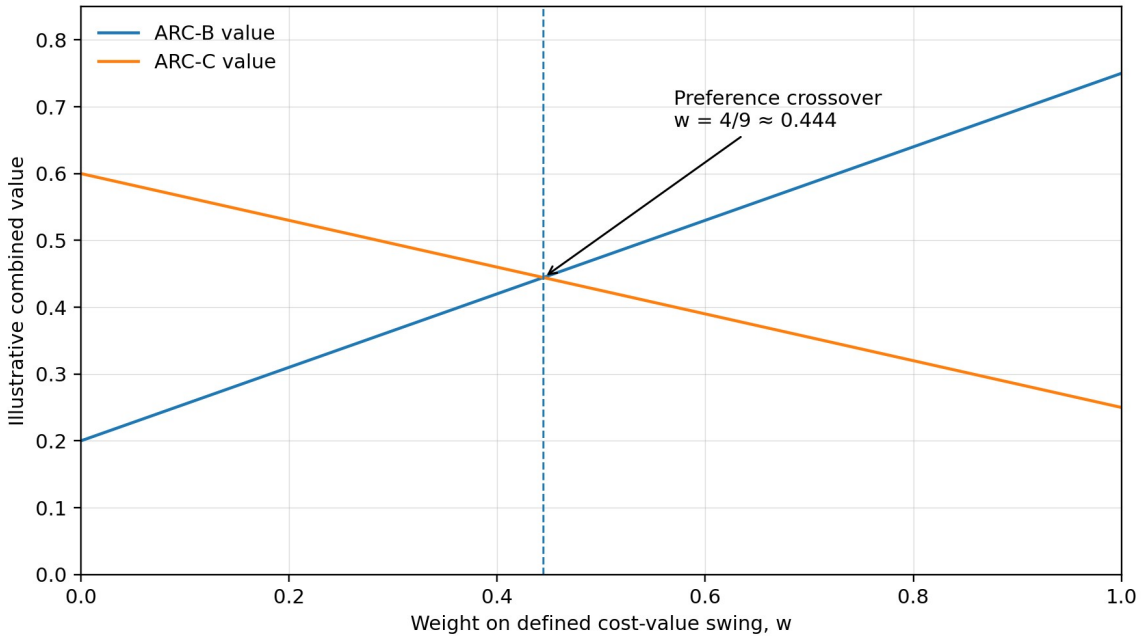


Figure 11.2. Illustrative architecture preference changes with the declared weighting

Original plot of the Section 11.16 teaching calculation. The vertical values are declared preference values, not probabilities, security scores or measures of system trustworthiness.

11.17 Make the next decision proportionate to the evidence

The original teaching decision record TRD-01 is a recommendation for further development, not an operational approval. Its decision question is which concepts and investigations should proceed. Its baseline is TE-02's requirements together with TE-03's proposed architecture and interface interpretations. Its adverse conditions are the selected LS scenarios, not every possible logistics disruption.

The recommendation is to retain a common receiving-side commit design for further analysis, reject ARC-A as the complete recovery response under the unchanged need, and compare ARC-B and ARC-C through targeted authority, eligibility and task investigations. The highest-value early

questions are whether the recovery arrangement can be legitimately independent of the failed identity service; whether C remains eligible across target changes; and whether B's human task fits the fifteen-minute requirement.

This recommendation is stronger than “study everything” because it identifies what can change the choice. It is more honest than selecting C from the two-attribute illustration because the feasibility gates remain unresolved. It also preserves the possibility that neither concept is adequate, in which case the team must develop another alternative or return to stakeholders with the specific conflict.

The record should identify actions and evidence expectations. The policy and identity authorities address recovery authentication and withdrawal semantics. Application engineers establish commit and history behaviour. Operators and maintainers develop representative task information. The systems engineering lead coordinates the trade and its technical consequences. SSE contributes the security analysis and challenge; it does not assume every participant's decision right.

Reconsideration triggers include a changed restoration scenario, a newly discovered writer, inability to preserve history, a changed identity interface or evidence that the required task cannot be completed. A decision record that retains only the chosen product loses this reasoning. Keeping the assumptions and triggers allows later maintainers to know when a once-reasonable decision no longer applies.

11.18 Exercise: the winning score fails the freshness rule

Situation. A supplier offers a low-cost design with a twenty-five-second status record at the final authorization check and an eight-second upper bound to commit. It claims compliance because the withdrawal budget is forty-five seconds and its trade-study score is the highest. TIME-P1 still requires a two-second uncertainty allowance. What should the reviewer conclude?

Worked answer. The proposed age bound at commit is 25 + 8 + 2 = 35 seconds, which does not support SR-05's thirty-second criterion. The withdrawal budget addresses a different property. A favourable preference score cannot compensate for a binding freshness requirement. The design needs a fresher authority basis, a shorter justified delay, a different mechanism or an explicitly authorized requirement change; none is established by the present proposal.

Ask whether the calculation uses compatible definitions. Does twenty-five seconds refer to the authoritative observation rather than receipt of a copy? Does the eight-second contribution bound every relevant path and queue? Is the two-second allowance additional or already included? Clarifying those matters may change the analysis, but cannot be replaced by rounding or renaming a cache setting.

Further challenge. The shared-dependency calculation is also conditional. It demonstrates the effect of the stated probability model, not cyber resilience or the likelihood of meeting the restoration deadline. Each conclusion must remain attached to its model, assumptions and decision purpose.

Carry forward. The team now has concrete feasibility findings, profile-completion issues, a limited state-property argument and a transparent trade recommendation. Chapter 12 develops the recovery and continued-function behaviour that those investigations must ultimately support.

Executable companion. Appendix C.8 and the accompanying Python files exercise a smaller state abstraction. Their recorded model results remain separate from the main case's hypothetical observations and from operational verification.

12. Cyber resilience and controlled degradation

12.1 Specify what must remain possible under adversity

A system can remain powered, connected and responsive while no longer providing the capability stakeholders need. An administration gateway can display its login page throughout an identity-service outage and still be unable to support necessary restoration. Conversely, temporarily restricting normal rule changes can be appropriate while the operational dispatch service continues using an established configuration. Neither a green availability indicator nor a blanket shutdown describes the complete resilience problem.

NIST organizes cyber-resilience goals around anticipating, withstanding, recovering from and adapting to adversity affecting cyber-dependent capability. [2, Section 2.1.1] Apply those goals to the administration task and its mission conditions; they do not prescribe a universal set of technologies.

For PN-02, restricted corrective administration must remain possible. SR-11 requires designated staff to activate a previously approved restoration package and confirm the authoritative active version within fifteen minutes of the service owner's declaration, during an identity-service outage of up to two hours. Authority, package eligibility, personnel, interfaces and support must make that outcome possible.

Security and resilience must be considered together. Continuing to accept arbitrary changes with unverifiable authority may preserve activity while defeating PN-01. Rejecting every possible restoration may preserve a narrow denial property while defeating PN-02. The design must establish which actions are useful and permissible in the stated adverse conditions, and what evidence supports that arrangement.

Resilience is not a substitute for prevention, incident response or disciplined operations. It connects them to the required capability under adversity. A backup, response plan or failover service may already provide an important contribution. The engineering question is whether that contribution works within the specific dependency, authority and recovery conditions on which the mission relies.

12.2 Distinguish outage, degradation and compromise

An unavailable identity service and a compromised identity service are not equivalent conditions. During a simple outage, the receiver may be unable to obtain fresh information. During compromise, it may receive timely but false authority information. A mechanism that rejects stale records addresses part of the first problem; it does not establish resistance to the second.

The selected LS-04 scenario concerns unavailability combined with a need for correction. The current recovery designs are therefore being evaluated first against that condition. A claim that ARC-B or ARC-C works during an identity outage cannot automatically become a claim that it works safely after compromise of identity administration, package approval or the receiving coordinator. Those additional scenarios require their own causal analysis and protection arrangements.

Degradation also has several dimensions. A system may lose normal administration while retaining read-only diagnosis. It may preserve package integrity but lose a dependable account of transaction outcome. It may have an intact recovery mechanism but no available person entitled to operate it. The description should identify the affected function and condition rather than assign one undifferentiated state called “degraded.”

For the teaching case, consider four separate observations: whether normal authority can be established; whether an eligible restoration package and its approval are available; whether the receiving state and transaction history are dependable; and whether the required people and resources are ready. These observations do not form a numerical trust score. They determine which claims and actions are currently supported.

An adversary may also manipulate the conditions used to trigger recovery. Causing an authentication outage must not automatically grant broader privileges. Forging a warning must not make an arbitrary package eligible. Entry to a restricted mode therefore needs a protected decision and a bounded action set, even when the initiating disturbance is easy to observe. The recovery path is part of the attack surface as well as part of the response.

12.3 Develop candidate recovery profiles without inventing authority

REC-P1 was deliberately incomplete in Part II. TE-03 now develops two candidate profiles corresponding to ARC-B and ARC-C. These proposals identify technical and operational content for the policy and system authorities to resolve. Neither is approved, and neither implies that central revocation can be learned through a channel that is unavailable.

For **REC-P1/B-draft**, the proposed capability permits an independently authenticated, designated recovery operator to select one eligible restoration package for an identified target. The service owner's declaration starts the operational timing obligation but does not by itself establish the operator's identity or package authority. An applicable recovery approval binds the action, target, package and permitted context. The receiving coordinator enforces that binding and the current recovery epoch.

“Independently authenticated” means independent of the failed service for the conditions being claimed, not independent of all infrastructure. The identity-policy authority must determine whether the proposed mechanism satisfies POL-01, including factor properties, credential lifecycle, recovery-role control and the permitted treatment of authority withdrawal. Merely storing a normal account password in an offline envelope is not an engineered answer to those questions.

For **REC-P1/C-draft**, the package, target and allowed starting conditions are bound into a prepared restoration capsule before the outage. At use, a designated person authenticates through the permitted independent arrangement and the receiving capability checks eligibility, validity, mode and transaction state. The capsule limits what can be requested, but does not automatically authorize any bearer to execute it.

C requires an explicit preparation and replenishment process. A new normal configuration can invalidate an earlier expected-generation condition. A package can remain intact while its approval is withdrawn or its content becomes operationally unsuitable. The system therefore needs a way to prevent normal changes from silently making required recovery unavailable, or to surface and govern that loss of readiness before accepting the changed operating state.

Table 12.1. Decisions needed to complete the candidate recovery profiles

Profile subject	ARC-B proposal	ARC-C proposal
Permitted action	Select and activate an eligible, already approved restoration package for the target.	Execute the specific prepared target/package instruction under its permitted starting conditions.
Authentication	Independently available MFA arrangement for designated recovery staff, subject to policy approval.	Independently available MFA for the designated initiator; possession of a capsule alone is insufficient.
Authority	Recovery role and action approval with an explicit relationship to normal authority withdrawal.	Bounded prepared authority plus current permitted initiation; expiry and withdrawal limitations must be accepted explicitly.
Content and target	Protected eligible-package set and verified target binding.	Protected capsule binding, target eligibility and preparation-state checks.
Transition and history	Current mode epoch, coherent commit and preserved transaction outcomes.	The same protection; preparation does not bypass receiving-side coordination.
Readiness	Staff, approval, credentials, package and receiving resources available within the task window.	The same, plus a valid, applicable capsule already prepared and accessible.

Both profiles must specify what happens when a person loses recovery authority while central communication is unavailable. Possibilities include a separately governed local roster, another qualified communication path or bounded pre-authorization with explicit residual limitations. These are different authority models with different exposure; none can be selected by the engineering team merely because it improves a timing estimate.

The profile must also define the response when its basis is missing or suspect. It may prohibit the recovery commit and escalate a service limitation rather than silently relax the action, identity or integrity rule. That can mean SR-11 is not achievable in the combined condition. A credible design records that limitation and considers another means of supporting the wider mission instead of claiming unlimited resilience.

12.4 Engineer the whole restoration task

The fifteen-minute requirement starts with the declared corrective need, not with an engineer's first successful login. Model the complete task and identify which activities are sequential, which can overlap and what resources each requires. A controller capable of activating a package in seconds may sit unused while an operator waits for access or authority.

Table 12.2 decomposes Chapter 11's invented thirteen-minute ARC-B and nine-minute ARC-C estimates. The contributions are treated as sequential, not measured. Any proposed overlap requires a task and resource model showing that it is valid. Appendix C.6 later chooses access and authentication before bound approval for the completed CB-09 branch; that branch retains the thirteen-minute total while Table 12.2 remains the earlier concept-development allocation.

Table 12.2. Illustrative end-to-end restoration task allocations

Contribution after declaration	ARC-B	ARC-C	What must be available
Mobilize designated staff	2 min	2 min	On-duty or reachable personnel with the relevant competence and access.
Establish action authority and eligible instruction	4 min	1 min	Applicable approval and package selection for B; valid prepared instruction and initiation authority for C.
Access the recovery capability and authenticate	2 min	2 min	Working endpoint, credentials, factors and receiving path independent of the stipulated outage.
Perform the coordinated restoration	3 min	2 min	Eligible package, correct target, protected state and permitted recovery epoch.
Confirm the authoritative outcome	2 min	2 min	Understandable, dependable target and transaction information.
Total and room within fifteen minutes	13 min; 2 min margin	9 min; 6 min margin	Conditional estimates, not demonstrated capability or a guarantee.

The difference between the concepts partly reflects work moved before the outage. C's short authority-and-instruction step assumes that preparation has been completed and remains applicable. It does not mean the preparation work costs nothing or cannot fail. The whole-life analysis must include routine replenishment, configuration-change coordination, credential administration and exercises.

B's more flexible selection can be valuable when several approved restoration packages are relevant. It also requires sufficient information and judgment during disruption. A user interface that presents similar package names without target, approval and configuration context can consume time or increase the chance of an incorrect choice. Human-system design is therefore part of the architecture, not a training fix added after the controller is built.

Two-person participation can provide useful challenge or separation of authority, but it also creates a personnel dependency. Do not count two roles as two available people without examining staffing, location, workload and concurrent demands. Where one person performs both roles, do not continue claiming the independence of two-person review. The operational arrangement must match the protection argument.

A task rehearsal can reveal these issues before full implementation. Later validation must use representative staff, access, workload and supporting resources, with the stipulated identity outage maintained. An engineer quietly supplying an unavailable credential or interpreting an ambiguous outcome changes the tested arrangement. Record that intervention as a finding, not an invisible convenience.

12.5 Check preparation age, expiry and eligibility across the outage

Prepared authority is attractive because it moves some work out of the disruption window. It also introduces time and state dependencies. A capsule can be cryptographically intact yet

expired, addressed to the wrong target generation or no longer permitted by the governing authority. Readiness therefore concerns applicability as well as possession.

Consider an original timing example for ARC-C. Assume the most recent permissible capsule preparation could be fifteen minutes before the identity outage begins. Assume a corrective need can be declared near the end of the two-hour outage, and the full fifteen-minute task may still be needed after that declaration. A conservative preparation-to-task-completion span is 15 + 120 + 15 = 150 minutes, before any additional clock uncertainty or engineering margin.

This arithmetic is not a recommendation to issue 150-minute credentials. It shows the temporal coverage a particular prepared-authority strategy would need under the added preparation-age assumption, or the point at which another strategy is necessary. A sixty-minute expiry cannot cover every case in that assumed span by itself. Extending validity can increase exposure to withdrawal or eligibility changes and therefore requires an explicit trade and authority decision.

A controlled handover to newly established normal authority may reduce the required offline-authority duration. Its availability, protection and completion time need evidence for the scenario. The handover must preserve the original fifteen-minute clock and prevent unchecked transfer of old recovery work across modes.

State eligibility can be more restrictive than time. Suppose a capsule expects generation 27, but an authorized normal change has already produced generation 28 before the outage. Executing the old capsule against generation 28 would defeat the expected-state protection unless the prepared instruction explicitly permits that state and the permission remains justified. Automatically rewriting the expected generation at use would erase the very condition that was approved.

A practical preparation arrangement therefore needs feedback into normal change management. A change may require a new capsule before the system is considered ready for the specified recovery scenario. Alternatively, the project may select ARC-B's eligible-package approach or another design. This is a substantive architecture trade: narrow prepared authority reduces discretion during use but increases the importance of preparation and eligibility maintenance.

12.6 Define controlled degradation and return to service

Controlled degradation specifies useful reduced capability and its limits. It should explain what remains available, which actions are prohibited, how operators recognize the condition and what permits transition to another state. The objective is not to keep every function running at any cost, nor to shut down every function whenever one dependency fails.

For the case, loss of normal identity support initially prevents new normal privileged sessions under SR-04. Existing normal authority becomes unusable for commits when the SR-05 freshness condition is no longer met. Diagnostic or outcome-query functions can remain available only under their separately established authorization and information-handling rules. The loss of write authority does not automatically authorize unrestricted reading.

Table 12.3 proposes operational states to support the two recovery concepts. These states refine the behavioural model; they do not create new policy permissions. NORMAL and RECOVERY retain their earlier significance, while transition and restricted states make necessary intermediate behaviour visible.

Table 12.3. Proposed controlled-degradation and restoration states

State	Intended capability	Protection and transition condition
NORMAL	Required ordinary administration within ADM-P1 and NET-P1.	Normal authentication, authority, freshness, transaction and outcome rules apply.
RESTRICTED	Authorized diagnosis and outcome reconciliation where their rules permit; normal commits only while their existing authority conditions remain valid.	No automatic privilege expansion. Expose the service limitation and assess the need for governed recovery.
QUIESCING	Establish the ordered transition boundary and resolve or fence pending work.	No old grant or queued action silently becomes destination-mode authority. Preserve completed outcomes.
RECOVERY	Only the action set permitted by the completed recovery profile.	Independent permitted authentication, target/package eligibility, current epoch and receiving-side commit control.
RECONCILING	Establish actual state, history and a valid basis for normal authority before return.	No automatic reuse of stale normal grants; account for recovery effects and any suspected compromise.

An effective transition needs an observable event and responsible participants. A user should not have to infer the current mode from which login page happens to respond. The system must also avoid reporting a return to normal solely because a network connection reopened. Reachability does not establish that authority information, configuration, history and the relevant dependencies are trustworthy enough for their intended use.

Where compromise is suspected, reconstitution may require more than restoring availability. The team may need to establish the integrity of the identity source, revoke affected material, recover protected configuration and determine what unauthorized actions occurred. The specific response depends on the incident and governing arrangements. An outage-only restoration demonstration does not establish that broader recovery capability.

NIST's protective-recovery principle reinforces the need for recovery into a protected condition. [1, Appendix E.24] In the case, that means preserving the requirements that make future actions legitimate and interpretable, not merely recovering a responsive gateway. Configuration rollback does not undo dispatch instructions already acted upon; the wider service owner must address those operational consequences separately.

The state names in Table 12.3 describe more than labels for a dashboard. They mark authority and state transitions that determine which actions can still be justified. The transition view below makes the required intermediate work visible.

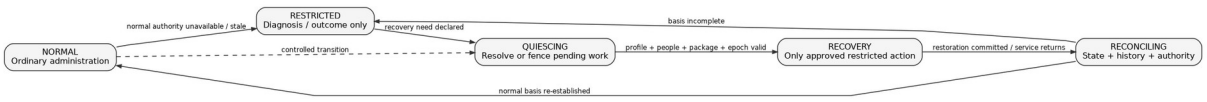


Figure 12.1. Controlled-degradation and recovery-mode transitions

Original state-transition schematic. Transition labels state conditions that must be established; reopening connectivity alone does not justify a return to NORMAL.

12.7 Select techniques for the required effect

Select cyber-resiliency techniques for the effect required. NIST distinguishes goals, objectives and techniques and examines their conflicts, synergies and relationship to existing investments. [2, Sections 2.1.2–2.1.3 and 3.1.4–3.1.5] The following are case-specific options to investigate, not a checklist for every system.

Segmentation can separate normal administrative exposure from recovery preparation and execution. Its value depends on the enforced boundaries, shared administrators and update paths. Redundancy can maintain access to a package or controller after a local failure, provided the copies and paths do not share the relevant cause. Diversity may address a common vulnerability, but can also increase integration and support demands. These effects need analysis rather than assumption.

Substantiated integrity directs attention to whether the package, recovery instruction and critical supporting elements can be shown suitable for reliance. A matching content hash can demonstrate agreement with a reference value while leaving the reference's provenance and approval unresolved. Multiple identical copies of corrupted content do not restore the required integrity merely by agreeing with one another.

Privilege restriction can limit recovery to the necessary action, target and mode. Coordinated protection is relevant when gateway, receiving controller and human procedures must enforce the same transition. Monitoring can reveal loss of readiness, unexpected package changes or inconsistent transaction outcomes. Each contribution needs a response appropriate to the observation; a warning does not restore an expired capsule or supply a missing approver.

Techniques can conflict. Making authority non-persistent can reduce exposure while eliminating the prepared authority needed late in an outage. Aggressive isolation can contain suspected compromise while blocking the only path to confirm a restoration outcome. A secondary platform can add diversity while sharing the same privileged deployment pipeline. Work through these interactions using the scenarios, requirements and task model rather than select techniques independently.

Existing operational capabilities should be reused where they fit. A backup service, privileged-access process or incident-management workflow may supply part of the answer. Its contribution still needs the right scope, availability and evidence. SSE strengthens the connection among those capabilities; it does not require discarding them to demonstrate that engineering has occurred.

The same recovery task should appear in the model used to evaluate the architecture, not only in a separate continuity document. Link each task contribution in Table 12.2 to its performing role, required resource, authority input and predecessor conditions. Link the task to the applicable REC-P1 candidate, the states in Table 12.3 and the planned VA-02 evaluation. This makes a change to staffing, package preparation or the permitted transition visible as a possible change to recovery capability, not just to administrative paperwork.

For ARC-C, a model review should connect normal activation with capsule eligibility: advancing the target generation may make a prepared instruction unusable. For ARC-B, examine whether the approval and selection tasks share a person or supporting service. An executable task model can explore such dependencies if its scheduling and resource assumptions represent them. It cannot demonstrate that the required personnel, credentials or facilities will actually be available. Retain the distinction between a modelled opportunity to recover and an operationally substantiated recovery capability.

12.8 Evaluate resilience as a set of bounded outcomes

A single uptime percentage can conceal whether the required reduced capability exists. For this case, evaluation should separately establish whether designated staff can obtain permitted authority, whether an eligible package can be activated, whether the target and transaction outcome can be confirmed, and whether normal authority is re-established without resurrecting old grants. Each observation supports a different part of C-04 and its relationship to C-02 and C-03.

VP-05 should develop the mode-transition checks identified in Chapter 10. VA-02 should develop the complete restoration task under the approved recovery profile. VP-03 remains relevant because a recovery activation can still encounter repetition, interruption, version conflict and uncertain outcome. A recovery test plan is not a reason to stop evaluating the underlying transaction properties.

Test the boundary conditions that could change the conclusion. Examples include a declaration near the end of the assumed outage, an almost-expired prepared instruction, a normal transaction pending at transition, a package made ineligible by a recent change, lost acknowledgement after recovery commit and return of the identity service during restoration. These are proposed evaluations, not reported results. Their selection follows the specific design and its unresolved questions.

Evaluate failure as well as successful restoration. An intentionally ineligible capsule should be refused without being reported as a successful recovery. Staff need to understand why and what escalation or alternative remains possible. Such a refusal can demonstrate a protective property while revealing that the overall recovery design fails the required operational scenario. Both findings must be preserved.

Measures can include task completion time, retained action set, package eligibility, outcome accuracy, transition correctness and the coverage of relevant dependency conditions. Do not average a critical authority failure into an otherwise favourable resilience score. Similarly, a successful task under one outage duration and staffing arrangement should not be generalized to longer or differently caused disruptions without an applicability argument.

12.9 Integrate physical effects and adaptation

The transfer from digital service restoration to a cyber-physical system requires care. NIST's operational-technology guidance explicitly addresses distinctive performance, reliability and safety concerns in systems interacting with the physical environment. [21, abstract] The relevant protective response can therefore depend on physical dynamics and available alternatives, not just access-control policy.

In the inspection-vehicle vignette, preventing a new route update may be appropriate during uncertain authority, but abruptly stopping all control could create a different hazard. Continuing the previous route may also be inappropriate if the environment has changed. The systems engineering effort needs contributions from control, safety, human factors and operations to define the permitted behaviour. SSE identifies security-related causes and protection needs within that work; it does not prescribe a universally safe state from the administration example.

Adaptation can change operations, allocations or architecture when earlier assumptions fail. Recurring identity limitations might justify replacing a dependency or improving prepared recovery. Such changes need requirements, authority, integration and evidence, not merely a more permissive emergency procedure.

A temporary recovery measure can become permanent through repeated use without the corresponding engineering decision. Track that possibility. If recovery mode is routinely used to bypass inconvenient normal controls, the actual operating concept differs from the one evaluated. Investigate why the legitimate task cannot be performed adequately and revise the design or governing arrangement through controlled work.

Tools can organize evidence and assist analysis, but people must frame the problem, judge alternatives and evaluate the support. A generated design record or automated dashboard neither performs the real-world evaluation nor assumes authority to accept its consequences.

Compromise route. Appendix F applies the resilience method to a different condition: the approval service remains available but is no longer a justified source for consequential decisions. It shows why reconstitution must restore the basis for reliance, not merely service uptime.

12.10 Exercise: the capsule is intact but cannot complete the task

Situation. An ARC-C demonstration uses a capsule prepared immediately before the test. It completes restoration in nine minutes. The proposed operating arrangement permits preparation up to fifteen minutes before an identity outage, outages can last two hours, and the capsule expires sixty minutes after preparation. The supplier declares SR-11 demonstrated for the full envelope. What does the evidence support?

Worked answer. It may support the demonstrated task in its actual configuration, preparation age and authority conditions. It does not support every allowed declaration time across the two-hour outage. Under the additional preparation-age assumption, a conservative preparation-to-completion span can reach 150 minutes before further time allowance. A sixty-minute capsule can expire before a late corrective need is even declared.

Change the prepared-authority strategy, establish an alternative or revise the operational envelope through the relevant authority. Longer validity needs a withdrawal and exposure assessment; normal-mode handover must preserve the deadline and transition protections. Neither is established by this demonstration.

Further challenge. An intact, unexpired capsule may still be ineligible after the target generation changes. Preparation and normal change management must preserve recovery readiness together.

Carry forward. Chapters 13–16 develop implementation and enabling systems, integration, verification and validation, and human-system readiness, carrying forward the unresolved design, authority and evidence obligations.

Worked completion. Appendix C.6 chooses a separately governed ARC-B arrangement for one fictional branch, including an explicit authority premise and full-task conditions. The unselected alternatives here retain their earlier status.

PART IV — REALIZE AND EVALUATE THE SYSTEM

13. Implementation, production and enabling systems

13.1 Make the specified capability real

A design review can establish a promising way to control dispatch-rule changes. It cannot establish that a delivered gateway, receiving endpoint and storage arrangement actually provide that control. The next engineering task is to realize the selected contributions without losing the requirements, interface meaning or assumptions that made the design credible. Implementation is where those relationships must survive contact with code, configuration, components, production processes and the people who perform the work.

For an IT team, this includes familiar activities: configuring a platform, building software, creating deployment automation, provisioning identities and preparing a release. For a hardware team, it includes component selection, fabrication, assembly, programming, inspection and control of production variations. These activities differ in their methods, but share a requirement to identify what was made, how it relates to the intended design and what remains to be established before it can be relied upon. NIST addresses these security contributions within the implementation process, including implementation constraints, enabling systems and traceability. [1, Appendix H.7]

RZ-01 develops receiving-side coordination from TE-03's requirements while preserving its open recovery choice. ARC-B and ARC-C remain alternatives. The twelve-second-delay what-if is a separate analytical branch, not a replacement for the earlier timing inputs.

The worked records in Chapters 13–16 specify proposed work, expected observations and explicitly hypothetical findings. They do not report tests performed on an operational service. This distinction allows the book to develop a complete method without manufacturing evidence that a real system exists. The reader should be able to apply the method to a real project and replace each assumption or planned result with identified information of the appropriate kind.

13.2 Decide what to build, acquire, configure or reuse

A realization strategy is not one decision for the whole system. The project might acquire the gateway, develop the receiving coordinator, configure the identity integration, reuse an approved platform and establish a new operating procedure. Each contribution needs an accountable source, an appropriate specification and an evaluation route. The overall system requirement remains with the integrating effort; it is not divided into disconnected promises simply because procurement uses several suppliers.

Start with the allocated function and the evidence needed about it. For RZ-01, the important question is whether a candidate receiving platform can preserve the relationship among active configuration, transaction history, generation and mode epoch through the specified interruptions. A product described as providing durable storage is a candidate contribution, not an answer to the entire question. The team needs its actual persistence semantics, configuration, failure conditions and integration constraints. A low-cost purchase can become expensive when the missing interface information forces redesign.

Reuse deserves the same relevance test. Evidence from a previous installation may be useful when the configuration, requirements and operating conditions are sufficiently alike. It may be inapplicable to a new restart procedure, different storage controller or altered privilege arrangement. NASA's product-realization guidance explicitly treats purchasing, making and reuse as implementation strategies with associated preparation and information needs. [5, Section 5.1] The proposed records here apply that distinction without importing NASA's contractual or organizational arrangements.

Where internal supplier information is unavailable, identify the specific conclusion that is limited. The project may compensate through a constrained interface, receiving-system observation, an alternative product or a narrower permitted use. It cannot assume source-code access that was not acquired. Equally, a supplier's refusal to disclose internals does not demonstrate that the product is defective. It creates an information and assurance problem to resolve in relation to the required use.

The strategy must include support. Who can rebuild the component, replace a failed unit, rotate its trust material or recover its state? Which tools and information will remain available after the development contract ends? These questions affect selection now, even though detailed sustainment appears later in the book. A design whose only recovery path depends on an unavailable supplier engineer has introduced a human and contractual dependency into the capability.

13.3 Establish enough information to build and to recognize the result

Before implementation, identify the controlled design information that the implementer is expected to follow. This includes the allocated requirements, interface contract, state and timing definitions, relevant platform constraints, security-relevant configuration and expected evidence. The required detail depends on the commitment. An exploratory prototype can proceed with bounded assumptions that would be unacceptable for a production release, provided the prototype's purpose and restrictions remain explicit.

For RZ-01, the implementer needs more than a requirement to make activation atomic. The design must identify the state controlled together, the event that makes a configuration effective, the persistence conditions required before reporting commit, the interpretation of authority and the behaviour after interruption. A reference to IC-01 directs the work, but IC-01 itself still needs its encoding, limits and implementation bindings. A build should not decide those matters through undocumented defaults.

A release or realization manifest identifies the combination of items that constitutes the result. It can reference existing configuration-management records rather than duplicate them. Table 13.1 shows the information needed for the illustrative branch. The entries are information obligations, not fabricated hashes or evidence of an actual build.

Table 13.1. RZ-01 realization information to establish

Controlled item	Information needed	Why the relationship matters
Requirements and profiles	Identified revisions of SR obligations, AUTH-P1, MFA-P1, TIME-P1 and the applicable interface definitions.	A product version alone cannot identify the behaviour against which it was built.

Controlled item	Information needed	Why the relationship matters
Model and design	MS-01 revision, selected variation, AD-01–AD-03 interpretation, IC-01 and relevant analysis inputs.	A generated element must correspond to the model and alternative actually reviewed.
Realized elements	Source or supplier release, build configuration, firmware, platform and security-relevant deployment settings.	Configuration can change enforcement even when application source is unchanged.
State and migration	Transaction-history schema, generation and epoch treatment, initialization and permitted migration procedures.	Rebuilding software is different from restoring authority and execution history.
Toolchain and evidence	Relevant tool and library versions, transformation settings, build records, planned and performed evaluations, and limitations.	A result needs an identifiable production and evaluation context.

Use exact content identities where available, together with their meaning. A digest can identify bytes relative to a trusted reference, but cannot by itself establish that those bytes implement the right requirement. A version label can be readable and still identify more than one build if the release process permits replacement under the same label. The manifest should let another qualified person obtain the intended items and understand the differences between builds.

Keep design intent, as-built information and observed installed configuration distinct. If the installed system differs from the approved deployment model, preserve the discrepancy while investigating it. Automatically changing the model to match deployment can make the record look consistent while concealing an unauthorized or technically unacceptable departure. The digital thread connects different states of knowledge; it should not erase them.

13.4 Realize a consequential state change, not an architectural slogan

The RZ-01 walkthrough explores one possible structure. Package content is prepared and checked before the final state-changing operation. A receiving coordinator then evaluates the relevant transaction identity, target generation, mode epoch and authority. A protected storage arrangement relates the active configuration designation to the transaction outcome. Only after the specified commit conditions hold may the capability report that the change was committed. This is a candidate decomposition of work, not a universal storage algorithm.

One implementation option is to keep immutable package content separate from a small authoritative record that selects the active package. The content must be available, complete and correctly identified before the selecting record changes. The transaction history and selection record then require a coordinated update or an equivalent recovery argument. Moving a pointer is not sufficient when the pointed-to content can disappear, be modified or be interpreted differently by the consumer. The content, reference and interpretation belong in the design together.

Define the storage fault model. A process crash, operating-system restart, power loss and persistent-data corruption are different conditions. An acknowledgement into volatile memory can survive one and fail under another. Establish the chosen platform’s completion semantics and the barriers, replication or recovery actions on which the required result depends.

Authority creates a separate coordination problem. An external identity service does not become part of a local transaction merely because its result is checked inside a software function. The observation can age during scheduling, retries or storage delay. RZ-01 therefore needs an implementation-level explanation of how the final check and commit respect SR-03 and SR-05 together. A last-moment check can reduce an interval, but it does not make that interval zero or establish the freshness of the source's own state.

Consider an implementation that prepares a valid request, checks its authority and then waits indefinitely for a storage lock. The design may preserve configuration consistency while losing the timing basis for authorization. Re-evaluation, bounded reservation or another justified coordination arrangement is needed before committing. Conversely, moving every check into a long critical section can reduce throughput or prevent the positive capability in SR-00. The complete requirement set must remain visible to software, platform and security specialists.

Interruption handling completes the implementation argument. On restart, the coordinator must distinguish a previously committed transaction from an uncommitted request and from a condition requiring reconciliation. It must not manufacture a fresh epoch simply by reusing an old snapshot value, or forget a transaction while continuing to accept its identifier for execution. TX-P1 remains the place to complete lifetime, retention and retirement rules. An incomplete profile should be returned to design rather than resolved by whichever cache duration happens to be convenient.

13.5 Engineer software and configuration together

Secure implementation requires both correct intended behaviour and resistance to ways that behaviour can be defeated. Code review, static analysis, testing and controlled build settings contribute to that work. The SSDF organizes secure-development practices that can be integrated into an organization's software lifecycle, including supporting toolchains, release protection and software evaluation. [14, Table 1, PO.3, PS.2–PS.3 and PW.6–PW.8] These practices support the security specialty; they do not replace the system-level requirements and interface reasoning already developed.

For the coordinator, review the distinction between parsing a request and authorizing its effect. A well-formed message can name an unauthorized target. A correctly authenticated caller can submit a package whose approval is no longer applicable. An integer field can be within its syntactic range while representing an impossible generation transition. Input validation needs the relevant semantics as well as defensive handling of malformed or oversized input.

Review error paths as carefully as successful execution. On failed status lookup, prevent reuse of an earlier authorization result. After a timeout, reconcile the existing transaction before proposing another. During cleanup, preserve the history needed to prevent a repeated commit. These are case-derived implementation questions, not findings against a particular product.

Configuration can defeat correct code. The intended receiving coordinator may coexist with a maintenance account that can write the active configuration directly. A feature flag may disable generation checking. A diagnostic build may expose privileged control hooks. A schema migration can reset a field used to distinguish old authority from current authority. Reviewing the source without the deployment and migration configuration therefore leaves part of the implementation unexplained.

The work should fit the existing delivery workflow. A code change can reference its affected SR and IC-01 clauses; a merge review can examine the scenario that motivated it; automated checks can reject prohibited build settings; a release record can preserve the relevant evaluation and unresolved findings. The purpose is not to add a second security backlog that loses contact with implementation. It is to make security-relevant obligations visible in the work the team already performs.

This chapter uses the published SSDF Version 1.1 practice identifiers. At the source check recorded for Part IV on 8 October 2026, NIST's SP 800-218 Revision 1, describing SSDF Version 1.2, remained labelled an initial public draft. [37, publication record] The draft is not silently substituted for the cited baseline. A real project should identify its applicable version and evaluate changes deliberately rather than treating a framework name as a versionless obligation.

13.6 Include hardware and firmware in the protective design

A software-centred explanation can conceal the hardware and firmware conditions on which it relies. The receiving controller depends on memory, storage, power, timing and boot behaviour. Its management interface may include a service connector, debug facility or recovery mode not visible in the ordinary network view. A firmware setting can affect which image executes or which peripheral can influence memory. These contributions require their own allocated requirements and evidence where they can affect the system's protection needs.

NIST's platform-firmware guidance organizes resilience around protection against unauthorized changes, detection of changes and secure recovery. [31, abstract] That is a useful scope reference, not a promise that enabling one boot feature establishes complete platform trustworthiness. In the case, the project needs to know which firmware and configuration control the intended execution path and how their update and recovery arrangements preserve the receiving-side protection.

A signed firmware image can be authentic to a signing authority yet unsuitable for the intended hardware revision or system configuration. A recovery image can restore functionality while reinstating an obsolete trust store. A factory-reset operation can erase transaction history or recreate default administrative access. The product must specify these effects and the permitted return-to-service procedure. The existence of a reset button is not itself a defect; unexplained consequences for protected state are the engineering issue.

The inspection-vehicle vignette from Chapter 9 makes the physical distinction clear. Restoring a software configuration does not necessarily restore the position, energy or motion of the vehicle. A controller can restart while an actuator remains energized. Physical interlocks, control dynamics, interface timing and operator access require the relevant engineering specialists. The logistics case's management-side atomicity must not be promoted into a claim that physical effects can be rolled back atomically.

Manufacturing and replacement also matter. A different component lot, storage implementation or board revision may change behaviour under power interruption or stress. The engineer should identify the characteristics that are significant to the claim and determine which changes require further evidence. It is neither necessary to declare every substitution unacceptable nor defensible to assume that a matching catalogue description makes it irrelevant.

For IT readers, the practical entry point is the platform and asset record. Extend it beyond product name to include relevant firmware, configuration, support state and the particular recovery assumptions. For hardware readers, connect the existing bill of materials, drawings and production records to the software and authority context that makes a physical interface security-relevant. Both contributions describe the same delivered system.

13.7 Preserve protection through repeated production

Demonstrating a property on one engineering specimen does not establish that every produced or deployed item has that property. Production can introduce differences through substituted materials, programming settings, calibration, assembly, provisioning or release packaging. A deployment pipeline can repeat a configuration error just as consistently as a manufacturing fixture can repeat an incorrect adjustment. Repeatability is useful only in relation to the intended result.

Identify the production characteristics whose variation can affect the security argument. In RZ-01, these might include the absence of an unintended writer, the correct boot and update configuration, the initialized generation and epoch arrangements, and the binding of provisioned credentials to their intended role. In a physical product, the corresponding characteristics could include connector access, protected storage fitting, tamper indication and recovery-interface configuration. The selection follows the actual design and relevant loss scenarios.

Production checks should be linked to those characteristics, with defined sampling or unit-level examination where appropriate. A design qualification addresses the represented design and conditions; an acceptance examination addresses the delivered item or lot within its specified scope. An organization may combine activities, but should not rely on their names to establish equivalence. The inspection and test plan needs to state what is being checked and why the selected coverage is adequate.

A nonconformance requires technical disposition. Rework may return the item to specification; repair may produce an acceptable alternative needing approval; rejection may be necessary; a permitted deviation may authorize a limited use. The applicable organization defines those categories and authorities. The SSE contribution is to identify effects on protection, resilience and supporting evidence, not to override the production or contractual process with an informal security acceptance.

Keep traceability to affected items. If an incorrect provisioning script was used for a group of units, the project needs to identify which units, which credentials or settings, and which evidence may be affected. A general statement that the script has been fixed does not establish that previously produced items were corrected. The same reasoning applies to a container image already deployed in several environments.

13.8 Treat the engineering toolchain as an enabling system

The modelling repository, code generator, build runner, signing service and test environment can influence both the product and the evidence used to evaluate it. They are therefore enabling systems with their own assets, privileges, dependencies and failure conditions. LS-06 already identifies this concern. The security work should examine the specific influence each service has instead of assigning a generic trusted-tool label to the entire delivery chain.

Table 13.2 identifies enabling services, their potential influence and original questions for project-specific evaluation.

Table 13.2. Enabling-system influence and corresponding challenge

Enabling contribution	Influence of concern	Engineering challenge
Model repository and reusable libraries	Alter requirements, units, selected variations or the behaviour used for generation.	Control revisions and permissions; review consequential changes and preserve the evaluated model combination.
Generator and build runner	Produce different code or configuration from the intended inputs.	Identify transformations, dependencies and settings; examine representative outputs and changed behaviour.
Signing and release service	Mark an unintended or unevaluated artefact as an approved release.	Separate content identity, release authority and technical evaluation; protect keys and promotion rules.
Test driver and oracle	Fail to apply the intended stimulus or report a false success.	Establish the actual injection and observation; use independent checks where needed to expose shared errors.
Deployment and inventory automation	Configure an unintended writer while also omitting it from the reported path inventory.	Compare intended and observed configuration through a sufficiently independent source.

Use the same proportional reasoning applied to operational systems. A script that formats a non-authoritative report has different consequences from a generator that produces an enforcement rule. A library update affecting a numerical solver differs from a font change in a review view. Identify which outputs could change, which claims rely on them and what evaluation is necessary before accepting the new tool configuration.

The modelling environment also contains information worth protecting. Architecture, vulnerabilities, supplier details and operational limitations should be handled under the project's information arrangements. Collaborative access should expose enough for competent review without granting unrelated editing or execution authority. Credentials used by automation should not be copied into model examples, test exports or release reports merely because those artefacts are internal.

Signing does not replace technical evaluation. A release signature can support origin and integrity claims within the trust arrangement; it does not establish that required tests were performed or that the signed software satisfies SR-07. Likewise, a reproducible build can help compare outputs from defined inputs while leaving a shared input error intact. A secure process needs both protection of the chain and evaluation of what the chain produces.

13.9 Check transformations and exchanges for preserved meaning

Model-based development adds transformations that deserve explicit contracts. A model can generate configuration, source code, test sequences or a report. An exporter can move selected engineering information to another repository. For each transformation, define the relevant input revision, represented semantics, output identity and limitations. The ability to open the resulting file does not establish preservation of the security meaning.

Use MS-01 as a small acceptance slice. Preserve the distinction between SR-03's revocation deadline and SR-05's freshness limit; preserve units, mode conditions and the fact that VP-02 is planned rather than performed. Include the authority-to-commit relationship and the declared uncertainty allowance. A toolchain that reproduces every requirement label but drops the time allowance has not preserved the analysis needed for the decision.

NASA's systems-modelling handbook addresses organized model information and generation of engineering products. [23, Sections 5–7 and 9] Here, transformation checks address omissions or changes that could alter the protective conclusion. Product capability descriptions do not establish the configured transformation's correctness.

Generation should have a testable scope. If a model generates a state-machine skeleton but developers add persistence and scheduling behaviour by hand, record that boundary. The model's state property does not automatically cover the added code. If the same model generates the implementation and expected test answers, use separately reasoned scenarios to challenge a common omission, such as snapshot restoration of transaction history.

Known small examples can help qualify the use of a tool without pretending to prove every output. A deliberately changed unit, removed authority relationship or stale result reference can test whether an exporter or checker handles the condition as specified. Preserve those fixtures and expected outcomes with the relevant tool revision. Such checks characterize the configured transformation; they do not constitute verification of the entire operational system.

13.10 Prepare a usable handover to integration

The implementation output is an identifiable element with the information needed to integrate and evaluate it. The next team must understand its configuration, interfaces, support needs and limitations, and which claims the evidence supports.

For RZ-01, the handover includes the coordinator configuration, IC-01 bindings, initialization and migration procedures, history treatment, build provenance and evaluation results when available. Path coverage and recovery-profile approval remain visible completion items. The release manifest can reference existing model and configuration records rather than duplicate them.

A software bill of materials can identify included components and support subsequent vulnerability and change analysis. It does not establish their actual runtime use, the completeness of the system boundary or successful enforcement of the integrated requirements. Supply-chain analysis concerns both product/service provenance and the processes through which they are developed and supplied. [32, abstract] The specific conclusions still require receiving-system reasoning.

Distinguish missing evidence from an observed defect. Without evidence about persistent history, SR-07 remains unsubstantiated; it is not necessarily violated. A demonstrated second commit would be a different finding. The handover record should direct integration to the actual unresolved question.

13.11 Exercise: the signed build and the reset controller

Situation. A supplier provides a signed release, a clean source-analysis report and a successful normal-start demonstration. Its factory-reset procedure reinstalls the same signed release but

clears transaction history and resets the mode identifier. The integration team proposes accepting the release as satisfying the repeated-transaction and mode-transition requirements because the executable has not changed.

Worked answer. Retain the signature and source-analysis report for the claims they support. The reset changes security-relevant persistent state even when the executable bytes are identical. The team must examine whether old transaction identifiers or grants can become usable again, whether namespace retirement or another protection prevents that use, and how the actual outcome history is preserved or reconstructed. The present records do not establish those properties.

The reset procedure belongs in the realization baseline and evaluation scope. Link its behaviour to SR-07, SR-10, SR-12, TX-P1 and the generation/epoch design. Define how an item is returned to an acceptable operating condition after reset and which authority must review any remaining limitation. A passing normal-start demonstration cannot be relabelled as evidence about a different reset and history condition.

Carry forward. Implementation has made the required contributions, enabling systems and handover information concrete. Chapter 14 develops how to combine those contributions in controlled increments and learn from their interactions without losing configuration or evidence identity.

14. Integration and emergent behaviour

14.1 Build the interaction deliberately

A gateway can validate an authentication context correctly while a receiving service interprets the resulting authority incorrectly. A storage component can preserve its own records while the client reports an outcome that never occurred. Integration examines these relationships in the realized arrangement. It is not simply the moment when finished components are connected, and it is not the first point at which evidence becomes legitimate. Earlier analyses and component results remain useful; integration establishes additional support about their interaction.

The useful capability also emerges through interaction. Authentication, package preparation, receiving-side coordination and operator feedback together make controlled maintenance possible. Emergence is therefore not a synonym for failure or unpredictability. The engineering question is which collective behaviours are required, which are unacceptable, and how the selected integration work will establish the relevant distinctions. NIST treats integration as planned preparation, assembly and evaluation of the resulting interfaces and interactions. [1, Appendix H.8]

In the logistics case, the team must realize IC-01 across the client, gateway, receiving coordinator and state store. It must then examine the external identity contribution and the receiving operational application's interpretation of active configuration. No component supplier necessarily owns that entire interaction. The integrating organization needs explicit responsibility for the result, access to the relevant information and authority to resolve mismatches through the appropriate design and agreement processes.

RZ-01 provides the common realization branch. Recovery-specific work remains conditional on its authority and operating profiles; normal transaction work must not quietly force the unresolved ARC-B/ARC-C choice.

14.2 Choose an integration sequence that reduces consequential uncertainty

An integration sequence should follow dependencies, consequence and the information needed for the next decision. Integrating the easiest component first can be reasonable for establishing a working environment. It is less useful when repeated demonstrations of easy functions postpone the receiving-side persistence or authority problem that could invalidate the design. The sequence should make such priorities explicit.

For RZ-01, a useful first increment joins the proposed coordinator to its state store and a controlled client. This allows the team to examine transaction identity, expected generation, commit and outcome independently of a complex enterprise identity environment. A simulated authority contribution can support those early questions, but must be labelled as a representation. It does not establish the real identity service's authentication, revocation or consistency behaviour.

A later increment introduces the actual intended identity interface and gateway, then the full relevant receiving-path set and deployment arrangements. Recovery branches add their own authority and mode conditions. Representative people and supporting resources progressively

enter the evaluation; they are not reserved for the final acceptance event. NASA's integration guidance relates the sequence, enabling resources, interface checks and resulting records to the product structure and implementation strategy. [5, Section 5.2]

Table 14.1 proposes integration increments for the selected case. Their identifiers refer to planned combinations and questions, not completed assemblies. Reorder them when a critical dependency or practical constraint justifies the change.

Table 14.1. Proposed integration increments for RZ-01

Increment	Contributions brought together	Decision or uncertainty addressed
INT-01: transaction core	Coordinator, package representation, protected state and controlled client.	Can the proposed implementation preserve identity, generation and outcome through the specified local interruptions?
INT-02: normal authority	Gateway, intended identity interface and receiving enforcement.	Do authentication, source status, revocation and authority at commit have compatible meanings and timing?
INT-03: receiving and enabling paths	Relevant management endpoints, deployment functions, operational configuration interface and instrumentation.	Are writers accounted for, and does management-side state have the agreed receiving-system meaning?
INT-04: recovery branch	A selected candidate recovery profile, mode transitions, preparation and receiving coordination.	Does the proposed branch preserve authority and transaction properties under the stated outage? Profile completion is a prerequisite.
INT-05: representative use	Intended personnel, access, support resources and normal/disrupted task environment.	Can the integrated capability support the intended tasks without unplanned assistance or unavailable dependencies?

These increments are not isolated mini-projects. A finding in INT-03 can require a change to INT-01's state model. A task rehearsal can reveal that the outcome interface needs an additional distinction. Preserve the evidence from each identified configuration, then determine which portions remain applicable after the change. Earlier success is neither worthless nor automatically transferable.

Integration order also has a commercial dimension. A late supplier delivery may prevent the intended sequence. A project can substitute a simulator to continue selected work, but must record which conclusions remain blocked. The schedule should include integration with the actual contribution and time to resolve differences. A simulated service is not transformed into supplier evidence by being used repeatedly.

14.3 Establish readiness before connecting or disturbing the system

A meaningful integration event requires an identified configuration, agreed interface descriptions, expected behaviour, suitable facilities, competent participants and a way to observe the relevant result. For intrusive work, it also needs authority, limits and recovery arrangements. These conditions should be checked before energizing equipment, connecting privileged interfaces or injecting a failure, not reconstructed after an unexpected effect.

For the teaching rig, establish the target identity, starting package, transaction history, mode epoch and authority state. Identify which services are real, simulated or deliberately unavailable. Use controlled test credentials and data appropriate to the environment. Confirm that traffic and actions cannot escape into an operational service unintentionally. A realistic scenario does not require uncontrolled experimentation on a live capability.

Instrumentation must expose the right boundary. A gateway trace can establish that a request was forwarded; it cannot by itself establish the receiving configuration's commit. The rig needs a justified observation of effective change and transaction outcome. An instrumentation account with unrestricted write access may also become an additional writer. Identify that contribution in the integration model and control its use rather than omit it because it belongs to the test team.

Agree stop conditions and recovery responsibilities. These can include unexpected communication with an operational endpoint, inability to identify the active configuration, loss of protected test records or behaviour outside the approved physical envelope. The exact criteria are project-specific. Their purpose is to protect people, assets and evidence while allowing deliberate investigation, not to suppress an inconvenient technical result.

Physical integration adds concerns that a service diagram may not show. Power sequencing, grounding, connector compatibility, environmental conditions and access for inspection can affect both correct operation and protection. Relevant specialists should establish those conditions. A software team should not assume that a controller's restart observation represents power-loss behaviour; a hardware team should not assume that a powered device has initialized its authority and configuration correctly.

A readiness review can authorize a bounded exploratory assembly while withholding permission for a broader evaluation. That distinction is useful when a simulator or provisional profile supports one question but not another. Record the intended commitment and exclusions. Do not turn every open item into an automatic stop for all learning, or turn permission to learn into permission to deploy.

14.4 Reconcile physical, logical and semantic interfaces

An interface has several layers of meaning. Electrical compatibility or successful network delivery establishes one contribution. Valid message encoding establishes another. Agreement about identity, authority, state, units and outcome establishes further contributions. The integrated protection property needs the relevant layers to work together. Passing a connectivity check does not establish that agreement.

For IC-01, compare the request meaning used by the client with the receiving interpretation. Does the target identify the same instance? Does the expected version mean package content, an activation generation or a supplier release? Does the approval authorize this package for this target in this mode? Are timestamps related to source observation or to receipt? These distinctions should be resolved against controlled definitions, not left to interface developers to infer separately.

The gateway and receiving endpoint may also disagree about the scope of revocation. One may withdraw the ability to establish a session while the other retains permissions for queued work. An integration result needs to establish what is actually withdrawn and when. A claim about

identity revocation should not be extended to a different role-change or package-approval mechanism without examining its semantics.

Consider a supplier field named statusTime. It might mean the time at which the authority source observed the status, the time at which the supplier copied it or the time at which the response was transmitted. SR-05 depends on the first relationship. Converting the field into seconds does not resolve the ambiguity. The source and receiver need an agreed meaning, protection of that meaning and evidence appropriate to the chosen interface.

Interface control is shared work. The receiving team should identify the obligations it assumes the supplier provides; the supplier should identify the conditions required for its contribution. Compare them directly. NASA's interface-management guidance provides a public basis for coordinating interface definition and change across technical participants. [6, Section 6.3] The project's actual agreements determine who can change each side and how discrepancies are resolved.

A mismatch may require an adapter, revised allocation, different component or changed requirement. An adapter is another engineered contribution, not a neutral piece of plumbing. It can preserve field names while losing identity binding, timing meaning or an explicit unknown state. Its transformation rules, failure behaviour and evaluation belong in the same integration record.

14.5 Walk the transaction through a lost reply and a later change

A concrete sequence makes the receiving contract easier to inspect. Begin with target D1 at generation 27 using package P7. Transaction TX-41 requests activation of approved package P8 from that expected generation. Assume, for this explanatory sequence, that its authority and other guards are satisfied. The receiver commits P8 as generation 28 and records the outcome, but the reply is lost before the client receives it. These are stipulated events in a teaching sequence, not observations from a test.

The client now has incomplete knowledge, not evidence that the change failed. It should use the same transaction identity to reconcile the outcome under the applicable query rules. A repeated identical TX-41 must not create a second commit. A conflicting use of TX-41 for different content must not be treated as an ordinary retry. Those are separate obligations under SR-07 and SR-07a.

Now introduce a later, independently authorized TX-42 that activates P9 as generation 29. When an operator queries TX-41, the answer must distinguish what TX-41 committed from what is currently active. A report saying only “P9, generation 29” cannot explain TX-41's historical outcome. A report saying only “P8, generation 28” can mislead someone who believes it describes the present target. IC-01's proposed distinction needs both context and clear presentation.

This sequence develops Part III’s proposed SR-10 clarification. The requirements owner still needs to define historical outcome, current state and their availability; the original requirement remains unchanged. Integration exposes why the client and operator must understand both results, not merely receive two fields.

The team should also investigate what happens when the query itself is interrupted, delayed or answered from a replica. A response may describe an authoritative historical commit while its current-state information is older. The design needs to identify the observation basis and acceptable consistency, rather than imply that every displayed field represents one

instantaneous global state. Required timeliness and freshness belong in the applicable interface profile.

This example is useful to an IT team because it resembles familiar retry and support problems. The SSE contribution connects them to protected state, authority, recoverability and evidence. It is useful to a systems engineer because it demonstrates that the system's behaviour cannot be inferred from individually correct components without accounting for ordering and knowledge at their interfaces.

14.6 Integrate startup, shutdown, migration and recovery—not only steady operation

A steady-state demonstration leaves important transitions unexamined. Startup can briefly expose a management interface before policy is loaded. Shutdown can interrupt the relationship between configuration change and outcome recording. An update can change transaction semantics while old requests remain queued. A restored snapshot can make old authority appear current. The integration strategy should identify which of these conditions affect the selected requirements and include them deliberately.

For RZ-01, initialization must distinguish a genuinely new instance from a replacement or restored instance whose history still matters. Treating every empty database as first installation can defeat the no-second-commit property if old requests remain acceptable. The necessary response depends on transaction namespace, retained records and permitted restoration. TX-P1 must specify that relationship before an acceptance claim can be made.

Mixed-version operation deserves explicit treatment. During a staged update, an older client may omit a mode epoch or interpret an unknown outcome as failure. A newer receiver may require fields that the older client does not provide. Compatibility has to include the protection and recovery meaning, not only whether a request parses. A temporary compatibility path that accepts missing authority context can undermine the very property the update intends to strengthen.

Mode transitions require an ordering relationship with effective changes. A commit that occurred before the transition remains part of history. Pending work cannot inherit destination-mode authority merely because it was admitted earlier. Entry to recovery, return to normal and interruption during either transition need to preserve that distinction. The five operational states proposed in Chapter 12 help frame the analysis, but their names do not establish implementation behaviour.

A migration can therefore be a coordinated engineering event involving configuration, persistent state, clients, credentials, models and procedures. The migration plan should state the supported starting combinations, transformations, checkpoints and recovery options. It should also identify what cannot safely be rolled back. Restoring old software may be possible while restoring old authority or transaction history is not.

For a cyber-physical system, additional effects can persist outside the information system. Reverting a route file does not reverse movement already undertaken by an inspection vehicle. The relevant control, safety and operational specialists must establish the permissible transition and recovery. The administration example teaches the need to preserve consequential history; it does not prescribe a physical recovery strategy.

14.7 Keep the receiving operational boundary visible

The selected system of interest includes the receiving management interface, not the complete dispatch operation. A management-side commit identifies the active configuration at that boundary. It does not automatically establish that every dispatch consumer has adopted the configuration, that in-flight work used it or that the approved package expresses the correct operational policy.

Integration with the operational service should make those relationships explicit. A consumer may load configuration asynchronously, reject a package it cannot interpret or continue using a cached version. The service owner and application engineers need to determine whether those behaviours are acceptable, how divergence is detected and which observations support an operational claim. The administration team contributes the controlled change and its evidence without taking over every application requirement.

A useful interface obligation might require a consumer to report which generation it is using, but that is a candidate refinement requiring its own meaning, authority and timeliness. A report can lag actual use. Two consumers reporting the same generation can still interpret its rules differently. Identify the actual question before adding telemetry: configuration convergence, consistent interpretation and correct operational decisions are different properties.

This is also where system-of-systems reasoning begins. An external service remains relevant even when another organization controls it. The integration plan needs an agreed contribution, applicable evidence or an explicit limitation. Expanding a diagram's boundary does not grant design authority over that service. Excluding it from the diagram does not remove the dependence.

14.8 Use MBSE to reconcile the realized interaction

MS-01 should now connect intended architecture to the configuration assembled for an integration event. Record the model revision, requirement and profile revisions, actual or simulated elements, selected alternative and instrumentation. Link the resulting observations to that combination. A model containing a receiving coordinator is not an as-built record until the actual contribution and its relationship have been established.

An integration finding can reveal three different problems: the implementation differs from the model; the model correctly represents an inadequate design; or the observation does not establish what the evaluator thought it did. The response differs in each case. Correcting the deployed configuration, revising the design and repairing the test oracle are not interchangeable actions. Preserve the distinction in both the issue record and the model.

Model exchange remains an integration concern. Suppose a supplier export preserves the timing expression but loses the additional uncertainty allowance or converts “not performed” into a default passing status. The receiving model now supports a different conclusion. Compare units, conditions, variation selection, evidence status and reference revisions, not just element counts. This extends the exchange discipline developed in Section 11.8 into the actual engineering workflow.

For generated implementation or tests, identify the transformation boundary. Which model elements produced source code, configuration, procedures or expected results? Which parts were completed manually? Which tool and library versions were used? The generated outputs need

their own identity. A later model revision does not retroactively change the build that was integrated or the test that was executed.

Provide review views for non-modellers. An administrator should be able to challenge a missing restoration dependency through a task narrative or table. A supplier engineer should be able to identify an incompatible interface assumption without seeing the entire proprietary model. These views must still identify the relevant configuration and exclusions. Accessibility to reviewers is part of making the shared engineering information useful.

14.9 Turn discrepancies into controlled engineering work

A discrepancy record should preserve the expected behaviour, observed or stipulated behaviour, configuration, method, relevant evidence and immediate consequence. It should distinguish the symptom from a proposed cause. A timeout is an observation; a failed commit is a conclusion requiring more information. A missing model relationship is a representation finding; it is not by itself an observed unauthorized path.

Table 14.2 contains explicitly hypothetical findings that illustrate different dispositions. No integration event has been performed for this manuscript. The identifiers are teaching examples for how a real record could direct work.

Table 14.2. Hypothetical integration findings and disciplined responses

Illustrative finding	What can be concluded under the stated facts	Required work before closure
D-04-01: the client reports activation failed after a lost reply, but receiving history records the commit.	The client outcome representation is inconsistent with the stipulated receiving result.	Correct the outcome/retry interaction; evaluate historical and current-state presentation and affected client paths.
D-04-02: a deployment account can change active configuration outside the proposed coordinator.	The asserted writer coverage and allocation are incomplete; the path's acceptability requires analysis.	Identify authority and purpose, constrain or incorporate the path, and update A-01, design and evaluation coverage.
D-04-03: a timing report omits the allowance retained in the source model.	The exported analysis is not equivalent for the affected timing decision.	Correct transformation and affected reports; review conclusions based on them without declaring unrelated system functions failed.
D-04-04: recovery approval can be retrieved only through the stipulated unavailable identity service.	The proposed task does not establish the claimed independence for LS-04.	Rework the permitted authority/access arrangement or the proposed architecture, then evaluate the full task.

Issue closure requires evidence about the affected cause and consequences, not merely a merged change. Preserve the finding, evaluate the correction and identify new limitations. A discrepancy

can also require authorized clarification of a requirement rather than correction of an implementation defect.

The integrating lead should identify which commitments unresolved findings prevent. Unknown writer authority can block a complete-coverage claim while unrelated package-format work continues. The restriction follows the affected claim and activity, not a programme-wide colour assigned to every open issue.

14.10 Exercise: compatibility without agreed meaning

Situation. The gateway and receiving endpoint exchange every field successfully. The gateway uses statusTime as the source's authoritative observation time. The endpoint uses the time at which it received the response. Both teams report successful interface tests, and the integration dashboard declares SR-05 complete. What should the reviewer do?

Worked answer. The exchange works, but its event meaning does not. SR-05 measures age from the authoritative status observation to commit; measuring from receipt can make old information appear fresh. Retain the connectivity result, reconcile the source and receiver definitions, and evaluate the resulting end-to-end freshness relationship.

The reviewer also needs to inspect the tests. If both suppliers used locally convenient timestamps, their separate passes may be accurate reports of inadequate tests. If the interface specification was ambiguous, the correction may include a controlled profile change and an allocation review rather than assigning fault to one implementation prematurely. The updated evidence must identify the configuration and semantics actually evaluated.

Carry forward. Integration produces an identified arrangement, evidence about selected interactions and a controlled account of discrepancies and remaining scope. Chapter 15 develops how verification and validation turn that material into justified, appropriately limited conclusions.

15. Verification and validation

15.1 Ask which proposition the evaluation establishes

A system can conform to a requirement and still be unsuitable for the task that motivated it. It can also perform a useful demonstration while failing a specified protective obligation. Verification and validation address these related but different questions. Verification examines satisfaction of specified requirements or characteristics. Validation examines whether the capability meets stakeholder needs in its intended use and context. Both require appropriate evidence; neither is adequately described by renaming a test report.

Chapter 8 planned the questions and methods. This chapter develops the work of preparing, executing, interpreting and controlling evaluations as implementation becomes available. The distinction matters because a plan can be sound while a particular execution uses the wrong configuration, an inadequate observation or unrepresentative personnel. The result must describe the work actually performed, including departures from the plan.

NIST treats verification, transition and validation as connected lifecycle processes, with identified criteria, methods, enabling resources and results. [1, Appendices H.9–H.11] They are not a single late gate after engineering is finished. A requirements review, prototype evaluation or operator task study can provide early evidence appropriate to its representation. Later work must establish the relevant relationship to the delivered and installed capability.

For RZ-01, a receiving-side test can examine SR-07's no-second-commit property. A task evaluation can examine whether an administrator understands the uncertain outcome after a lost reply. Neither alone establishes usable restoration during an identity outage. The complete argument connects the necessary technical and human contributions while retaining the limits of each result.

Keep three identities visible: the proposition being evaluated, the item or representation evaluated, and the conditions of the evaluation. “The model passed,” “the component passed” and “the operational task passed” are incomplete statements until those identities are supplied. A production decision may rely on several such results, but should not erase their distinctions.

15.2 Select a method and define its evidentiary reach

Inspection, analysis, demonstration and testing can each contribute to verification or validation when appropriate to the question. They differ in how evidence is obtained, not in whether the result deserves scrutiny. An inspection can establish a configuration attribute. Analysis can establish a conditional property of a design. Demonstration can show a capability being exercised. A controlled test can measure behaviour under identified stimuli. NASA's product-realization guidance distinguishes these methods and their application to verification and validation. [5, Sections 5.3–5.4]

The evaluator should be able to explain why the method can reveal the failure of concern. Inspecting a cache parameter is relevant to freshness but cannot establish every delay between source observation and commit. A timing test can examine selected paths while missing an undocumented writer. A review of package provenance does not establish correct handling of a

repeated request. Use complementary methods where the claim requires complementary support.

Table 15.1 applies these distinctions to the case. It is a method-selection aid, not a requirement that every project perform every method for every row.

Table 15.1. Evaluation purpose and limits in the teaching case

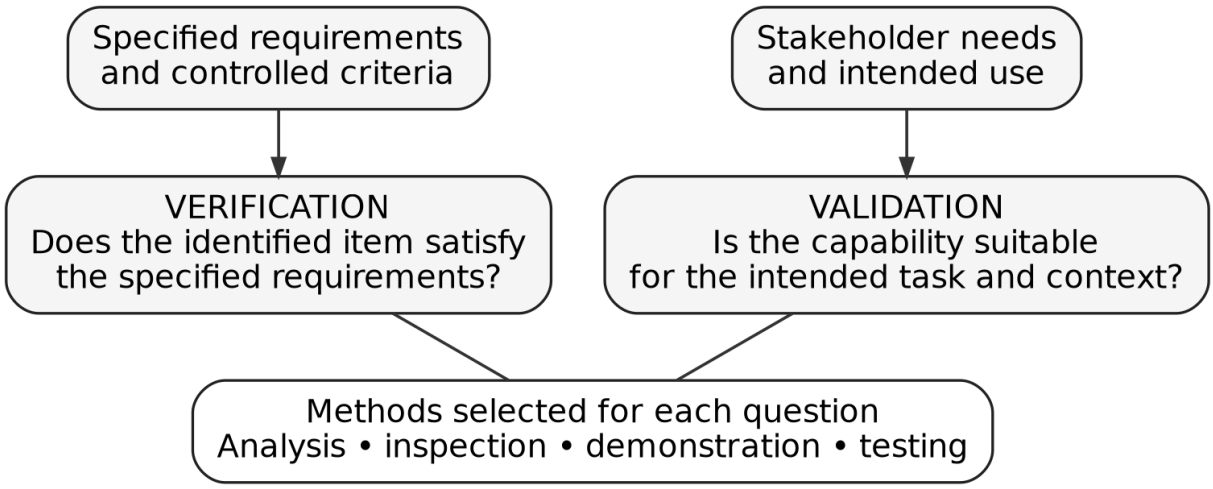
Method and object	Useful conclusion when performed adequately	Conclusion not supplied by that work alone
Inspect the identified deployed configuration and writer inventory.	The examined settings and paths correspond to the recorded state to the established extent.	All possible writers are known or every configured mechanism behaves correctly.
Analyse the authority-to-commit path and state transitions.	The represented design supports a stated conditional property or exposes a counterexample.	A delivered scheduler, store or endpoint realizes every modelling assumption.
Test repeated and interrupted activations on an identified assembly.	The observed effects meet or violate specified criteria under the evaluated conditions.	Every untested timing, platform or recovery condition has the same outcome.
Demonstrate and evaluate a representative restoration task.	The intended participants can or cannot complete the defined task with the provided resources.	An engineer-assisted demonstration represents all intended staffing or outage arrangements.

Distinguish design evaluation from checks on repeated production or installation. Evidence that one representative design meets a requirement may need additional configuration and acceptance checks for each delivered unit or release. Conversely, repeating a simple acceptance check on many units does not necessarily establish the design property that the check does not exercise. The required strategy follows consequence, production variation and the applicable quality arrangements.

The evaluation environment needs its own description. A software-only test may omit hardware timing and reset behaviour. An actual processor connected to a simulated environment can expose platform effects while retaining assumptions about the environment. A complete assembly still may not represent the intended staffing or operational workload. Name what is real and what is represented rather than use a test-level label as a substitute for fidelity.

The following schematic brings together the relationships developed here.

Figure 15.1. Verification and validation answer different questions



Original schematic. Methods can overlap; their purpose and conclusions differ. Both can inform assurance and decisions, but neither alone grants permission to operate.

15.3 Turn a verification objective into an executable procedure

A verification objective such as “test duplicate handling” leaves too much discretion at execution. The procedure must establish the starting state, exact action, relevant conditions, expected observations, evidence to retain and recovery or reset needed before the next case. It also needs a rule for distinguishing a product discrepancy from a failure of the test setup.

DP-01 below is a proposed procedure outline for VP-03, addressing selected relationships among SR-07, SR-07a, SR-08, SR-09 and SR-10. It is not an executed test. Before use on a real implementation, IC-01, PKG-P1 and TX-P1 must be bound to the actual interface, state and retention behaviour. The test must have a justified observation of receiving-side commit and history, not only a client success message.

The stipulated starting state is target D1, package P7, generation 27 and an identified current mode epoch. The evaluator establishes valid authority for the permitted actions, a controlled approved package P8 and a new transaction TX-41. Time and load are kept within the selected normal conditions so that this particular procedure isolates transaction behaviour. Separate procedures must combine those behaviours with authority withdrawal, adverse timing and mode transitions.

Table 15.2. DP-01: proposed transaction-evaluation sequence

Step	Controlled action	Required observation and interpretation
1. Establish the baseline.	Record target, generation, package, epoch, known transaction state, profiles and instrumentation.	The intended starting condition is established; an unexplained difference blocks this execution.
2. Execute the positive case.	Submit TX-41 for P8 from generation 27 under valid authority; deliberately suppress its reply after the receiving commit.	Receiving evidence establishes one complete P8 activation and its historical outcome. Client uncertainty is not reported as proof of failure.

Step	Controlled action	Required observation and interpretation
3. Resolve the lost reply.	Query or repeat TX-41 with the same bound activation content under the applicable rules.	No second commit occurs; the response refers to TX-41's established outcome.
4. Challenge conflicting identity.	Reuse TX-41 with different package or target content.	The conflicting request is rejected without committing that changed content.
5. Examine a later state.	Perform a separate permitted activation, then query TX-41.	Its historical outcome remains distinguishable from the target's later current state.
6. Examine restart.	Restart the specified contribution using the permitted restart procedure; repeat the relevant query and retry.	History and suppression remain adequate for the specified identity lifetime; restart is not treated as permission to execute TX-41 again.
7. Examine a stale starting version.	Submit a new transaction whose expected generation differs from the current target.	The conflict is rejected without the prohibited configuration change.

The sequence must be adapted to the implementation's actual transaction and observation semantics. Step 2 requires precise control over where the reply is suppressed; dropping the request before receipt would evaluate a different condition. Step 6 must identify whether it restarts the client, coordinator, operating system or storage platform. Calling all of those actions “restart” would obscure materially different fault conditions.

Do not reuse state blindly between cases. A test that introduces TX-41 in its setup cannot later treat TX-41 as a new identifier unless the setup establishes a different permitted namespace. Resetting a database between every test can conceal the history-persistence problem that the procedure is intended to reveal. Conversely, failing to document retained state can make a legitimate rejection appear to be a product defect.

The proposed historical/current-state observation in Step 5 depends on the SR-10 clarification identified in Part III. Until its exact requirement is agreed, the work can investigate the interface and inform that decision, but should not be reported as acceptance against an invented final criterion. The procedure makes the gap actionable rather than concealing it in a test label.

15.4 Challenge authority at the point that matters

VP-02 needs a different procedure from a login test. Its starting event is authoritative revocation; the consequential event is receiving-side commit. Include existing sessions, staged requests and queued work as well as new requests. Establish which identity or permission has been withdrawn, which paths are affected and how the source's state becomes available to the receiver.

For SR-03, a late commit using the revoked identity can demonstrate a violation under the defined conditions. Rejection of one request after sixty seconds cannot establish that every path stopped accepting consequential work by that time. The evaluator needs evidence about the relevant enforcement behaviour and path coverage, including intervals between checks and state change. A first rejected probe is not automatically the time at which all unauthorized commits became impossible.

SR-05 needs the age of the authoritative status observation at commit. The evaluator should challenge delayed delivery, replayed status, queued work, time uncertainty and inability to obtain fresh information. A record received just now may still describe an old source observation. An authenticated response can preserve that old observation faithfully; authenticity and freshness are separate characteristics.

Positive cases remain necessary. A capability that refuses all activation can avoid many prohibited commits while failing SR-00. Evaluate valid, authorized, non-conflicting work under the stated service envelope, alongside prohibited and degraded cases. The unresolved burst-arrival and interruption-count meanings in ADM-P1 and NET-P1 must be settled before claiming full-envelope performance. A convenient traffic pattern is not a retrospective definition of the requirement.

Security-specific methods can deepen the challenge. Code review can examine authorization-to-use gaps. Fuzzing can explore malformed input handling. Penetration testing can investigate reachable misuse paths. Fault injection can expose interruption behaviour. None is a universal substitute for the others or for a requirements-based evaluation of the system's useful function. The published SSDF includes both code analysis and executable testing as complementary practices. [14, practices PW.7–PW.8]

These activities need authorized scope and controlled environments. The manuscript supplies evaluation reasoning, not instructions to experiment on an unapproved operational network. The important discipline is to choose stimuli that can challenge a stated property, preserve what was actually done and interpret the result at the correct boundary.

15.5 Make coverage explainable

Coverage begins with a representation of what could matter. For normal administration, relevant dimensions include path, transaction state, authority state, network condition and restart condition. More dimensions can be necessary for particular requirements, including package size, mode transition, resource exhaustion and platform variation. The plan needs a reasoned selection, not a claim that a single percentage captures all of them.

Suppose a teaching catalogue contains two management paths, three transaction states, two authority states, two network conditions and two restart conditions. The unfiltered Cartesian product contains $2 \times 3 \times 2 \times 2 \times 2 = 48$ combinations. This count is an organizational aid, not forty-eight ready-to-run tests. Some combinations may need a particular sequence to be reachable, some may be inconsistent, and each still needs a valid expected result.

Record those constraints explicitly. “Committed transaction with revoked authority” might be a legitimate starting state for querying a historical outcome, but not permission for a new commit. “Restart during pending work” is a temporal condition, not merely a Boolean parameter. A static combination table can miss ordering effects unless the procedure includes them.

Combinatorial methods can select systematic interaction coverage when exhaustive combinations are impractical. NIST discusses t-way testing, generation and limits. [35, abstract] The 48-combination example here is original and has no tool-generated suite. Pair coverage leaves three-way interactions, sequences and unmodelled inputs to separate consideration.

Requirements coverage and scenario coverage should remain separate views. A verification matrix identifies specified requirements with no applicable result. A scenario review asks

whether causal paths such as LS-02 or LS-05 have an adequate response. All specified requirements can have passing results while the specification omits an important scenario. Conversely, a useful exploratory finding can reveal a new requirement even when it was not generated from an existing matrix.

A coverage argument should identify the relevant model, selected equivalence classes, boundary conditions, excluded conditions and remaining uncertainty. A high-risk omitted writer is not compensated for by many tests of a well-understood gateway path. Where results are reused across nominally identical endpoints, establish the relevant identity and environmental similarity rather than infer equivalence from the product name.

15.6 Interpret measurements against a stated decision rule

The measured quantity must correspond to the required property. For timing, identify the starting and ending events, how each is observed, the relationship among clocks, instrumentation delay and the treatment of uncertainty. Many decimal places do not establish accuracy. A log timestamp can be precise as a stored value while providing poor support for the actual event time.

The interval illustrations introduced in Chapter 8 use stipulated conservative bounds. An assumed withdrawal observation of 58.8 seconds with an additional plus-or-minus 1.5-second bound gives a compatible interval of 57.3–60.3 seconds. That interval crosses the sixty-second criterion. It is inconclusive for establishing the measured withdrawal bound. It is not a passing result rounded to the nearest convenient second.

Now consider a different, explicitly hypothetical observation: a receiving-side commit under the revoked identity is established at 61.2 seconds after t0, with a justified conservative uncertainty of plus or minus 0.8 seconds. The compatible commit interval is 60.4–62.0 seconds. Under the requirement's conditions and those stipulated measurement assumptions, the commit is after the permitted deadline throughout the interval and demonstrates a violation. This differs from interpreting the time of a first rejected probe.

Establish the uncertainty’s basis before interpreting a result. A conservative engineering bound, standard uncertainty and statistical coverage interval have different meanings. NIST explains expanded uncertainty and the assumptions required for coverage statements. [34, Section 6] The case uses invented conservative bounds, not calibration results or a complete metrology procedure.

Define the decision rule before examining borderline results. Where the measurement cannot distinguish satisfaction, the team can improve instrumentation, increase design margin or record an inconclusive conclusion. A risk authority may decide how to handle a limitation, but cannot change what the measurement establishes. Similarly, repeated observations from a biased instrument do not remove the bias merely by increasing sample size.

Retain raw observations and processing steps needed for review. If timestamps are corrected, filtered or correlated, preserve the method and relevant versions. Model-derived expectations and measured values should occupy distinguishable fields. An analysis can predict a forty-nine-second bound while a test establishes a different observed result; the discrepancy is information to investigate, not a reason to overwrite one with the other.

15.7 Understand what repeated successful trials can—and cannot—support

A useful statistical illustration concerns a narrowly defined repeated experiment, not a general probability that a system is secure. Assume independent trials with the same fixed probability p of a specified failure, representative sampling from the population of interest and reliable observation of every failure. Those are substantial assumptions. Deliberately selected adversarial tests or repeated executions of one unchanged state need not satisfy them.

Under the assumed binomial model, the probability of observing zero failures in n trials is (1 – p)^n. Setting that probability to 0.05 and solving for p gives the exact one-sided 95-percent upper confidence bound for the zero-failure case:

pU = 1 – 0.05^(1/n).

With n = 100, pU is approximately 0.02951, or 2.95 percent. With n = 1,000, it is approximately 0.002991, or 0.299 percent. The derivation is a simple application of the binomial tail; NIST's statistical handbook describes exact binomial confidence limits. [36, “Exact Intervals for Small Numbers of Failures and/or Small Sample Sizes”] These numbers are calculated examples, not estimates derived from testing the logistics service.

The confidence statement describes the procedure’s coverage over repeated samples under the stated model. It is neither a 95-percent posterior probability for the fixed parameter nor proof of zero failures. A failure condition absent from the sampled population remains unexamined: a thousand successful ordinary activations can still omit snapshot rollback.

Do not attach this formula to an arbitrary count of passing security tests. Test cases can be dependent, selected to investigate particular mechanisms and drawn from different conditions. An adversary can target the condition the test distribution rarely samples. The appropriate assurance claim must therefore combine structural reasoning, requirements and scenario coverage, applicable observations and a candid treatment of what remains unknown. The statistical note teaches a limit on inference, not a universal security score.

15.8 Connect model-supported testing to the actual test article

MBSE can help derive sequences, identify affected requirements and organize results. The evaluator still needs to establish which representation was executed and how it relates to the implemented system. A model run tests properties of that model under its inputs. Software execution against a simulated environment adds implementation behaviour but retains environmental assumptions. Actual hardware can expose platform effects while its surrounding plant or service remains represented.

The familiar labels model-in-the-loop, software-in-the-loop and hardware-in-the-loop are useful only when the evaluated boundary is explained. For RZ-01, a software coordinator linked to simulated persistent storage cannot establish the power-loss behaviour of the intended storage device. A real device driven by a simplified workload may not establish the full traffic envelope. No test-level label automatically supplies missing fidelity.

Record the chain from MS-01 to the generated or manually implemented contribution and to the evaluated article. Identify model revision, selected alternative, transformation version, code or configuration identity, actual platform, procedures, input data and result. Where a model

changes, determine whether the earlier article and result remain applicable; do not let a live link to the latest model imply that an old test evaluated the new design.

A common model can improve consistency between implementation and test expectations while creating a common error source. If both omit snapshot rollback, they may agree on the wrong proposition. Include independently reasoned scenarios and observations capable of revealing the omission. Independence is about the error that can be detected, not simply whether two tools have different names.

Model and simulation credibility should be established for the intended use, not asserted globally. NASA's models-and-simulations standard addresses that credibility and acceptance context. [30, public scope] The practical record here identifies the abstraction, input provenance, execution settings, limits and implementation correspondence on which a conclusion relies. It does not claim that the toolchain has been qualified under any particular regulatory regime.

15.9 Validate the intended task with the intended participants

VA-01 examines normal maintenance as people will perform it. Can the administrator identify the target, select the intended approved package, understand a conflict and determine the outcome after an interruption? Can the dispatch operator distinguish the historical result of an earlier transaction from the configuration currently active? These questions involve information presentation, access, workload and coordination as well as software behaviour.

VA-02 examines the restricted restoration task under the stipulated identity-service outage. The interval begins with the service owner's declaration and includes mobilization, authority, access, action and confirmation. Preparation assumptions for ARC-C and approval/access assumptions for ARC-B must be represented. An engineer who supplies an otherwise unavailable credential changes the evaluated arrangement; the intervention must be recorded rather than hidden in a successful task time.

Operational validation needs a defined scope and suitable participants, not a ceremonial demonstration to a senior audience. Establish the roles, representative scenarios, support resources and criteria. Include conditions in which the appropriate action is refusal or escalation, such as an ineligible restoration capsule. A correctly rejected operation can demonstrate a protective property while revealing that the architecture does not meet the required restoration scenario. Preserve both findings.

REC-P1 remains unapproved in this teaching case. Early prototypes or rehearsals can investigate candidate workflows and expose missing dependencies, but cannot establish acceptance against an unspecified final authority arrangement. Once the profile is settled in a real project, the evaluation must use that arrangement or clearly describe differences. The convenience of the demonstration cannot supply policy authority.

A task failure is not automatically operator incompetence. Examine whether the required information was available, the interface supported the decision, the procedure matched the configuration and the staffing assumptions were realistic. Training can address a knowledge gap; it cannot make an unavailable approval service available. The findings should return to requirements, design, support and operational planning, as developed further in Chapter 16.

15.10 Preserve results, limitations and contrary findings

A useful result record permits another qualified person to reconstruct the claim and its support. Record the requirement or need, configuration, method, relevant procedure and tool revisions, initial conditions, actual actions, observations, measurement interpretation, discrepancies, limitations and review disposition. Where the execution departs from the procedure, retain the departure and its effect on the conclusion.

Use statuses with defined meanings. Passed means the stated criteria were met for the evaluated scope with adequate support. Failed means the evidence establishes a criterion was not met under its applicable conditions. Inconclusive means the available evidence cannot establish the required conclusion. Blocked or not performed describes work that has not produced a result. Not applicable needs a justified scope decision; it is not a convenient label for missing evidence.

Table 15.3 is an original illustration of how different hypothetical results should remain distinguishable. It does not populate VP or VA records with actual evidence.

Table 15.3. Hypothetical result interpretations—not executed evaluations

Stipulated evidence	Appropriate limited interpretation	Improper extension
A valid activation completes and its authoritative outcome is established in 8.6 seconds in one defined normal condition.	Supports that observed positive case against SR-00, subject to its measurement and configuration basis.	Every workload and interruption allowed by an unresolved envelope is verified.
A revoked-identity commit is established at 61.2 ± 0.8 seconds under the stated conservative-bound assumptions.	Demonstrates a violation of SR-03 for the stipulated applicable condition.	Every other requirement or configuration necessarily fails.
A tool reports complete trace links to VP-05, but REC-P1 has not been defined.	Evaluation remains blocked; the links record planned relationships.	The mode-transition requirement has passed.
A restoration rehearsal takes eighteen minutes because mobilization takes seven rather than two.	The stipulated arrangement does not meet the fifteen-minute task criterion.	A faster controller alone will necessarily resolve the staffing and task problem.

NIST's control-assessment guidance supports tailored assessment planning and analysis of results across development. [9, abstract] Engineering evaluations can contribute to those assessments when the evidence fits the objective. Reuse the records without making control-assessment scope, system verification, intended-use validation and authorization identical decisions.

A corrected implementation should produce a new identified result, not overwrite the failed one. Establish the cause, affected requirements and necessary regression scope. A receiving-state

change can affect repetition, version conflict, outcome reporting and recovery even when the original symptom concerned only one of them. Unaffected evidence may remain applicable with a reasoned explanation. Neither blanket retesting nor blanket reuse is a substitute for impact analysis.

The final technical conclusion should say what is supported, what failed, what remains unresolved and which decisions those facts can inform. An assurance case must be able to use contrary evidence; an acceptance meeting must be able to distinguish an unresolved technical claim from an authorized decision about residual risk. The fuller decision treatment belongs in Chapter 20, but the evidence discipline begins with the first result record.

15.11 Exercise: a thousand green tests and an unexamined restore

Situation. A supplier reports one thousand successful activation tests. Every test begins with a new transaction database and uses an always-available identity simulator. The report includes a diagram linking all tests to SR-07 and a conclusion that no duplicate commit is possible. Snapshot recovery is not represented. What does the report establish?

Worked answer. It can provide evidence about the tested new-database sequences, subject to the procedures, observations and actual configuration. It does not establish preservation of execution history through snapshot recovery. Resetting the database between tests prevents the test programme from examining the retained-history condition that the no-second-commit argument requires. A trace link cannot extend the scope of the executions.

Ask for the identity-lifetime and recovery model, the actual persistence arrangement and evaluations across retained history, restart and permitted restoration. Keep the successful results where they remain relevant. Do not attach a general failure probability to the count: the selected tests do not establish the independent representative sampling assumptions of Section 15.7, and a missing transition can remain missing in every repetition.

Further challenge. A new test might reproduce an unsafe second commit, or it might establish that the implementation correctly preserves the relevant history. Neither outcome should be assumed in advance. The engineering task is to obtain discriminating evidence and connect it to the requirement and configuration, not to defend either the supplier's optimism or the reviewer's suspicion.

Carry forward. Verification and validation now produce interpretable, scoped results rather than undifferentiated passes. Chapter 16 connects those results to the people, support resources and transition conditions needed for a capability to be usable and ready for its intended service.

16. Human-system design and transition readiness

16.1 Design the work people must perform

A recovery function is not usable simply because its command exists. Someone must recognize the need, obtain the right authority, identify an eligible package, reach the capability, perform the action and understand the result. Each step depends on information, resources and an interaction with the technical system. When the design leaves those relationships unresolved, the operational team inherits an engineering problem disguised as a training requirement.

Human-system design began in the stakeholder, task and architecture work of earlier chapters. Its position here in the teaching sequence does not place people late in the lifecycle. This chapter develops the allocation, evaluation and readiness consequences as a system approaches intended use. The purpose is not to make users compensate for every limitation, but to establish an arrangement in which people and technology can perform their required contributions together.

NASA connects operational concepts, function allocation and task analysis to engineering. [33, Sections 5.2.2.1, 5.2.2.2 and 5.2.2.4] The logistics example applies those concepts to fictional roles and scenarios; no spaceflight requirements or completed human-factors assessment are implied.

For IT practitioners, much of this work is recognizable in service design, support procedures, incident response and operational acceptance. SSE strengthens its relationship to protection needs and design choices. For systems engineers, the same work makes software interfaces, access arrangements, identity dependencies and support tasks visible as parts of an engineered capability rather than administrative details outside the system.

Transition readiness then asks whether the intended receiving arrangement can support the capability now, with its actual people, configuration, facilities, information and limitations. It differs from proving every possible future condition and from merely shipping a product. A technically acceptable component can arrive at a site that is not ready to use it; an enthusiastic user group cannot make an unsupported technical claim true.

16.2 Allocate functions and authority deliberately

Function allocation asks which contributions should be performed by people, technology or their interaction. Authority allocation asks who may make the associated decisions. The two are related but different. A system can automate a package-eligibility check without acquiring authority to declare an operational emergency. A person can be authorized to approve restoration without being the person who executes it.

In ARC-B, designated staff select among eligible, previously approved packages. The design must support that selection with target, configuration, approval and suitability information. In ARC-C, much of the selection is moved into preparation, while the person at use still needs to establish permitted initiation and recognize whether the prepared instruction is applicable. Reducing discretion during an outage does not eliminate human engineering work; it relocates some of it.

Automation can reduce repetitive checking or delay, but also introduces a function whose decision basis, failure behaviour and authority need evaluation. Automatic entry to recovery solely because an identity endpoint stops responding could turn a deliberately induced outage into a privilege path. Conversely, requiring a human to manually reconstruct every authority

dependency during a fifteen-minute restoration window may be unrealistic. The design must compare feasible combinations rather than declare either automation or manual oversight inherently superior.

Consider the information needed to intervene. An operator told that a transaction is pending needs to know whether cancellation is still possible, whether commit may already have occurred and how to obtain an authoritative outcome. An interface that exposes only a red indicator and an override button does not provide that knowledge. The operator's permitted response should be consistent with actual transaction state and governing authority.

Responsibility must survive handoffs. The service owner declares the corrective need; a defined authority permits the action; the administrator executes it; the receiving capability enforces the conditions; and the operator confirms the relevant operational effect. Roles can be combined where justified, but a single person acting in two roles does not provide the independence claimed for two-person review. Record the actual arrangement and its consequences.

16.3 Analyse the complete task and its resources

A task model should connect each action to its trigger, required information, participant, resource, expected result and important failure condition. Start from the operational scenario, not the sequence that happens to work in the development laboratory. Include waiting, communication, approval, access and confirmation where they contribute to the required outcome.

For SR-11, the starting event remains the service owner's declaration of a corrective need. Earlier detection and decision delay belong to the wider service analysis. The model must not redefine the starting event as the first successful login merely because that is easier to instrument. It must also preserve the applicable outage while evaluating the dependencies intended to tolerate it.

Table 16.1 expands the ARC-B task from Chapter 12. It does not add a new approved operating procedure or claim that the proposed roles are staffed.

Table 16.1. Task and resource questions for the candidate ARC-B restoration

Task contribution	Information and resources required	Question for design and evaluation
Mobilize designated staff.	Duty arrangements, contact route, location, permitted access and relevant competence.	Can the required people begin within the assumed interval during the actual disruption?
Establish the permitted action.	Correct target, eligible package, applicable approval and authority to act.	Are the records available without the failed identity dependency, and are their meanings clear?
Access and authenticate.	Working endpoint, factors, credentials, route and receiving service.	Does the proposed independent arrangement actually remain usable and satisfy the governing obligation?
Perform restoration.	Current epoch, eligible starting generation, protected package and coordinated commit.	Can staff recognize an invalid condition without improvising an unauthorized bypass?

Task contribution	Information and resources required	Question for design and evaluation
Confirm the outcome.	Transaction history, current authoritative configuration and the required operational interface.	Can people distinguish historical success, current state and unresolved operational effects?

The earlier ARC-B allocation totals 2 + 4 + 2 + 3 + 2 = 13 minutes, leaving two minutes only if the contributions and their ordering hold. Increase mobilization from two to seven minutes, with all other assumptions unchanged, and the total becomes eighteen minutes. Improving a short command's execution may not recover the five minutes spent obtaining the people.

Parallel work is justified only by the dependency and resource model. Suppose a separate, available approver can complete the four-minute authority-and-package task while the administrator mobilizes for two minutes, with neither awaiting the other's unfinished output. The critical path becomes max(2, 4) + 2 + 3 + 2 = 11 minutes. This remains a separate hypothetical arrangement, not an amendment to the thirteen-minute allocation.

The calculation exposes a design question, not a staffing guarantee. If the approver needs information available only after the administrator signs in, the assumed overlap is invalid. If the same person performs both tasks, concurrent durations cannot simply be used. If the outage affects several services at once, the supposedly available approver may be occupied elsewhere. These relationships deserve a task and resource analysis before a favourable number becomes a service commitment.

Task estimates should be challenged through representative observations as the design matures. Retain the conditions: personnel familiarity, location, workload, communications, tools, package complexity and assistance provided. An average task duration does not establish an upper bound, and one unusually fast demonstration does not establish readiness across the intended operation.

16.4 Make consequential state understandable

The interface should support the decisions the person is expected to make. In the logistics case, package name alone may not distinguish target, approval, expected starting generation and permitted operating mode. A person selecting P8 needs to know what it will affect and whether it is eligible now. That information should be presented at the point of action, not hidden in an unrelated document.

Outcome presentation needs similar care. After a lost reply, "failed" can encourage an unsafe new activation, while "successful" can conceal an unestablished result. The interface should distinguish request receipt, pending work, committed historical outcome, rejection and unresolved outcome using the meanings defined in IC-01. It should provide the permitted query or reconciliation action rather than leave the user to infer state from a timeout.

Historical and current state should be visibly different. TX-41 may have committed P8 at generation 28 while P9 at generation 29 is now active. Showing both without labels can be as misleading as showing only one. A useful design identifies the transaction result, the current target observation and any limitation on the timeliness or authority of that observation. The operator should not have to understand database internals to avoid confusing them.

Mode information also needs an explicit basis. A recovery page remaining open does not establish that recovery authority is still current. The interface must not allow an old visual context to silently authorize a new operation. The receiving side enforces the actual epoch and authority; the display helps the user understand the context and appropriate action. Presentation and enforcement are complementary, not substitutes.

These principles can be captured as candidate interface requirements. For example, the system could be required to present the target, package identity and applicable approval before requesting activation; preserve transaction identity during outcome reconciliation; and distinguish historical commit from current configuration in the query result. These are proposed refinements for review, not a complete user-interface specification. Timing, information handling, accessibility and the actual interaction need their own criteria.

Use plain explanations appropriate to the user without removing technical precision. A message can state that the expected configuration no longer matches and identify a safe next step, while retaining detailed identifiers for support. Avoid relying solely on colour or a transient alert to convey consequential state. Evaluate the presentation with intended participants and in the conditions in which it will be used, rather than assuming that a developer's familiarity transfers to every operator.

16.5 Treat procedures, training and support as designed contributions

Procedures can be legitimate protective and resilience measures. Their adequacy depends on whether the required action can be performed with available information, authority, time and resources. They should not be dismissed as inherently inferior to technical mechanisms, nor used to excuse avoidable design traps. The right question is which contribution the procedure provides and what evidence supports reliance on it.

For ARC-C, preparation is a continuing task. Someone must ensure that a capsule is available, intact, approved, current for the target and within its permitted validity. A normal configuration change can invalidate the expected starting generation. The operating arrangement needs a controlled replenishment or readiness response; training people to check expiry does not resolve a missing preparation responsibility.

For ARC-B, support includes maintaining the permitted recovery-role arrangement, eligible-package set and independently usable authentication resources. Those resources need protection and lifecycle management as well as availability. An emergency credential that is conveniently available but uncontrolled can undermine PN-01. One protected so thoroughly that authorized personnel cannot reach it during the stipulated outage can undermine PN-02.

Training should be based on the implemented interface and current procedure. Include normal use, uncertainty, refusal, escalation and recovery—not only the ideal success path. Participants should understand the distinction between an accepted request and a committed configuration and know where their authority ends. Competence is demonstrated for the role and task; attendance at a briefing is not evidence that every operational condition has been mastered.

Maintenance support includes compatible replacement parts, firmware, configuration information, diagnostic access, tools and the means to restore protected state. A field replacement can change timing or persistence even when it is sold under the same family name. The support procedure should identify how the replacement is matched to the approved

configuration and what checks are needed before return to service. This connects hardware support to the same information discipline that IT teams apply to releases and configuration baselines.

The support organization also needs a route back into engineering. Repeated difficulty obtaining authority, frequent capsule ineligibility or a confusing outcome message should become design and requirements information, not remain a collection of local workarounds. Operational adaptation can be sensible, but its effects on protection and assurance should be evaluated rather than silently normalized.

16.6 Evaluate human-system performance without hiding assistance

Early evaluation can use scenarios, task walkthroughs, prototypes and representative interfaces before the full technical system exists. Its purpose is to discover missing information, confusing choices or impractical allocations while change is still feasible. Later validation must establish the relevant task in a sufficiently representative realized arrangement. An early prototype finding is valuable, but should not be described as validation of an undelivered operational system.

Plan participant roles and conditions deliberately. The person testing the recovery interface should have the competence and access expected of the intended role, not the development team's unrestricted knowledge. The task should include the information normally available, the stipulated outage and relevant workload. Where specialist assistance is part of the intended operating arrangement, include it explicitly with its availability and response assumptions.

Observers should record interventions. If an evaluator explains which package is correct, supplies an unavailable approval or restarts a hidden service, that assistance becomes part of what was demonstrated. It may reveal a needed interface or support function. Repeating the exercise without documenting the intervention creates an artificial picture of independent task performance.

Measure the required outcome and examine how it was achieved. Completion time matters for SR-11, but so do target correctness, authority, protected state and dependable confirmation. A fast unauthorized action is not a successful restoration. A correct refusal of an ineligible capsule can expose a readiness failure even when the interface behaves exactly as designed. The evaluation should preserve both the technical and operational conclusions.

Treat errors as information about the whole arrangement. Investigate whether similar labels, missing context, workload, interrupted feedback or incompatible procedures contributed. The appropriate remedy may be design, staffing, information or training. The evaluation should be conducted respectfully and avoid treating every problem as a personal failing. This is a practical review principle for the teaching case, not a claim that every error has the same cause.

A limited participant exercise supports a limited conclusion. Identify who took part, the tasks and conditions, the configuration, observations and unresolved differences from intended use. More participants do not automatically repair an unrepresentative scenario. Where the consequences justify specialist human-factors expertise, involve it as part of the integrated engineering effort rather than assume that a security practitioner can supply every discipline's evaluation.

16.7 Plan the transition and the coexistence period

Transition moves a realized product or capability into its next receiving context, which may be a higher-level assembly or operational use. NIST and NASA address preparation of the receiving environment, supporting information, people and relevant readiness conditions. [1, Appendix H.10; 5, Section 5.5] For the book's case, transition includes the administration service, its state, clients, authority arrangements, procedures and support—not merely a successful deployment job.

The coexistence period can create new paths. An old management endpoint may remain reachable after the new coordinator is installed. A deployment agent may retain direct write access for migration. A legacy client may omit the new generation or epoch field. Identify which combinations are permitted, how long they can exist and what prevents them from defeating the receiving-side protection during the transition.

A cutover plan should establish entry conditions, ordered actions, checkpoints, observations, stop conditions and responsibility for each decision. For RZ-01, relevant checks include the active package and generation, transaction-history treatment, current epoch, permitted writers, client compatibility and identity-interface state. The plan must say how new work is controlled while the state is migrated and how already committed outcomes remain discoverable.

Rollback needs a technical meaning. Reverting an application binary can be different from reverting its state schema, package content, transaction history or authority context. A software rollback that restores an old history snapshot may resurrect an earlier transaction or grant. A plan should therefore identify supported rollback combinations and the evidence that they preserve required properties. “Restore the backup” is not an adequate universal instruction.

Where a state change has already affected the operational dispatch service, reverting the management package does not necessarily undo its external effects. The service owner must determine the operational response. The administration team can provide a dependable record of which configuration changed and when. That record supports the wider decision without claiming that an information-system rollback reverses completed real-world actions.

Keep intrusive transition rehearsals controlled. A representative staging environment can reveal ordering and compatibility issues, while its differences from the receiving environment remain explicit. An operational rehearsal needs the applicable authority and safeguards. No such work has been performed for this manuscript; the example establishes what a real transition plan and evaluation must address.

16.8 Make readiness a claim with evidence and an accountable recipient

Readiness should be expressed for a defined capability, environment and commitment. Ready to integrate a component, ready to run a controlled trial and ready for operational use are different conclusions. Each needs appropriate evidence and authority. A project can legitimately be ready for the next bounded investigation while remaining unready for deployment.

The proposed record TR-04 organizes the relevant questions for Part IV. It does not assert that the teaching case has passed them. Its value is to make the expected evidence, responsible contribution and effect of an unresolved item visible before the decision meeting.

Table 16.2. TR-04: proposed transition-readiness record

Readiness subject	Evidence and contribution needed	Current teaching-case limitation
Identified realization	Controlled build, platform, configuration, profiles and state/migration information from the responsible implementers.	RZ-01 remains a walkthrough; no delivered article is represented as established.
Technical satisfaction	Applicable verification results, integration findings and dispositions for the receiving configuration.	VP work remains planned; illustrative findings are not executed results.
Intended use	Representative normal and disrupted task validation, including personnel and support resources.	VA-01 and VA-02 remain planned; no staffing or task performance is demonstrated.
Permitted authority	Approved identity, recovery, action and transition arrangements from the relevant authorities.	REC-P1/B-draft and REC-P1/C-draft remain proposals; no exception or production selection exists.
Receiving environment	Known writers, compatible clients, accessible records, facilities and support arrangements.	A-01 and material interface/profile questions remain open for an actual implementation.
Continuing support	Responsibilities, readable engineering information, controlled updates, recovery preparation and escalation routes.	The chapter defines information and work to provide, not an operational support contract already in place.

A readiness record should not turn every open issue into the same category. An incomplete recovery authority profile can block an operational recovery claim. A minor document correction may not block a bounded integration event if the controlling information remains unambiguous. The responsible participants must judge the effect on the intended commitment, recording their rationale rather than relying on issue counts.

Keep technical findings, contractual acceptance and permission to operate distinct. A supplier component can meet its contract while the customer has not prepared the receiving environment. An authority can decide to proceed under a permitted limitation, but cannot transform an unperformed test into evidence or silently waive an obligation outside its power. NIST's RMF provides a public example of lifecycle risk decisions and accountability; its specific governmental roles are not imposed on this case. [10, abstract]

An operational decision also needs explicit conditions and reconsideration triggers. These can relate to configuration, supported modes, staffing, external-service commitments or unresolved evidence. Conditions should be actionable and observable enough for the receiving organization to know whether they still hold. A limitation hidden in a technical appendix cannot reliably govern a team that does not know it exists.

16.9 Hand over information people can actually use

The handover package should support operation, maintenance, change and future evaluation. It needs the identified configuration, relevant requirements and rationale, interface meanings, permitted modes, known limitations, useful procedures, support arrangements and the evidence

on which the readiness conclusion relies. The exact packaging can use existing service, configuration and engineering repositories; a separate binder for every specialty is not required.

Model-based information should be handed over in forms the receiving organization can access and interpret. Preserve the native model where appropriate, the relevant libraries and tool versions, controlled exports, readable views, assumptions and evidence references. A model that can be opened only by an expired development licence is a support dependency. A static export can improve accessibility while omitting relationships needed for future analysis; identify those limits.

The receiving team should be able to answer practical questions. Which requirement and authority govern this recovery action? Which configuration was evaluated? Which evidence is no longer applicable after a changed endpoint? What happens if the outcome remains uncertain? A readable task view and a versioned dependency record can be more useful to an on-call administrator than an undifferentiated repository containing thousands of objects.

Access during disruption is part of usability. A recovery procedure stored only behind the failed identity service does not support the stipulated scenario. A permitted offline copy needs controlled versioning, protection and a means to determine whether it remains applicable. Printed information can support some tasks, but must not be mistaken for live authority or current configuration when those can change. Select the medium according to the information and decision it must support.

Sensitive material needs appropriate handling. The ordinary handover record should identify where authorized people obtain credentials or protected design information, not scatter secrets through a report intended for broad circulation. The need for continuity does not eliminate access control; it requires an arrangement that satisfies both availability and protection under the specified conditions.

Receiving acceptance should include an actual review of information usability where it matters. Can maintainers locate the relevant procedure, identify the current baseline and explain an important limitation? Can they repeat a material analysis with the retained inputs and supported tools? Those questions test the effectiveness of the handover rather than merely count delivered files.

16.10 Preserve readiness after the handover event

Readiness is conditional on an arrangement that can change. Staff rotate, packages become ineligible, credentials expire, tools are replaced and external services alter behaviour. These changes can affect the support for the original decision even when the gateway still reports healthy status. The receiving organization needs a way to identify and respond to the relevant changes.

For ARC-C, a normal configuration update can require renewed preparation before the prescribed recovery scenario is supported again. For ARC-B, a roster change can remove the only available approver. For both, a receiving-platform update can alter persistence or timing. The change process should identify the affected need, requirement, model and evidence, then select the necessary action. It should not simply renew every assurance statement on a calendar or invalidate every past result automatically.

Operational records can reveal whether the assumed arrangement exists. Actual restoration attempts, incidents, support delays, uncertain outcomes and repeated emergency use provide information for engineering review. They do not all have the same meaning: an outage frequency, an authority failure and a task-time overrun affect different claims. Preserve their context so that improvement addresses the right problem.

SSE contributes the security and resilience reasoning to that review. Operations retains responsibility for running the service; maintainers retain their technical work; systems engineering coordinates changes that affect the integrated capability; and the relevant authorities decide on commitments and residual limitations. Shared information makes those contributions coherent without placing every decision inside a new security approval silo.

The detailed sustainment methods appear in later chapters. The immediate handover obligation is to establish who notices a relevant change, who evaluates it, who can decide, and how the resulting information returns to the controlled baseline. A transition is incomplete when the project has delivered a capability but nobody owns the conditions required to keep using it responsibly.

16.11 Exercise: the demonstration succeeds, but the receiving team is not ready

Situation. Engineers demonstrate ARC-B restoration in thirteen minutes using a locally available approval record. The receiving organization stores that record only in a portal dependent on the enterprise identity service. Its required recovery personnel can take seven minutes to become available. The project proposes operational acceptance because the controller passed its tests and the training session has been delivered. What should the readiness review conclude?

Worked answer. Retain the demonstrated technical and task results for their actual arrangement. They do not establish the receiving organization's performance under the stipulated outage. Its approval-record dependency defeats the claimed independence, and replacing the two-minute mobilization assumption with seven minutes gives an eighteen-minute sequential task under the other unchanged allocations. Training attendance does not resolve either condition.

The team needs a permitted, protected way to obtain applicable approval information during the outage and a feasible staffing/task arrangement. It may investigate genuine parallel work, different allocation or another architecture, but must evaluate the actual dependencies and authority. A component acceptance can remain valid while the wider operational readiness decision is withheld or limited under the proper authority.

Further challenge. A readable model of the recovery task should show the portal dependency and personnel allocation. If those were omitted from the model, a consistency pass would not discover them. The receiving participants' review is therefore part of the engineering evidence, not a courtesy at the end of the project.

Carry forward. Part IV connects implementation, integration, evaluation and usable transition. Part V develops the acquisition, supplier, assurance-case and decision arrangements that obtain the necessary contributions and support accountable reliance without confusing evidence, contractual acceptance and authorization.

PART V — ACQUIRE, ASSURE AND DECIDE

17. Acquisition and contractual engineering

17.1 Acquire the work that makes the capability credible

A customer asks a supplier for a secure administration gateway. The supplier offers multi-factor authentication, a hardened platform and an assessment report. Each may be useful. None yet identifies who will establish that every receiving path enforces authority, that a repeated transaction cannot commit twice, or that designated staff can restore an approved configuration during the stipulated identity outage. The acquisition problem is not simply whether the supplier has impressive security features. It is whether the agreement obtains the contributions needed to deliver and justify the required capability.

Begin acquisition while the problem and alternatives are still developing. Evidence access, supplier work, customer interfaces and support commitments can determine architectural feasibility. Part V's position is pedagogical: a product commitment made before these contributions are resolved can purchase a component while leaving the expected system-level result unassigned.

NIST's acquisition and supply processes connect requirements, agreements, responsibilities, monitoring, delivery and acceptance for both parties. [1, Appendix K] AP-05 applies that relationship to the logistics case. Its engineering content needs the organization's procurement and legal review before contractual use.

The distinction matters to the IT reader. An architecture questionnaire, supplier assessment, statement of work, service commitment and acceptance plan already contain parts of the answer. They need to agree about the subject being acquired and the evidence that supports it. For the engineering reader, familiar specification and deliverable structures need to include software change, identity dependencies, remote support and the ways a supplier's enabling systems can influence the delivered result.

TE-05 adds the acquisition record AP-05 and a hypothetical evidence dossier. It does not select a recovery architecture, approve REC-P1 or populate the earlier planned evaluations with performed results. The dossier is used to practise evidence review and accountable decisions.

17.2 Separate five kinds of obligation

A useful agreement distinguishes what the supplied capability must do from the work needed to develop it, the information needed to evaluate it, the conditions for accepting a delivery and the contributions the customer must provide. These categories interact, but they are not interchangeable. "Provide assurance" is inadequate when one party means a product feature, another means an independent report and a third expects the supplier to accept the customer's operational risk.

The product or system requirement establishes the required characteristic of the identified subject. An engineering-work obligation establishes an activity and its expected outputs, such as an interface analysis or comparison of recovery alternatives. An evidence deliverable defines information the customer must receive or be able to examine. An acceptance criterion identifies the basis for a particular acceptance decision. An acquirer contribution supplies necessary

information, services, facilities, personnel or decisions. Table 17.1 applies these distinctions without inventing new runtime requirements.

Table 17.1. AP-05: distinguish the obligations before drafting the agreement

Obligation class	Case content to specify	What it does not establish
Product or system characteristic	Applicable SR requirements, controlled profiles, permitted configurations and allocated receiving-interface behaviour.	That a gateway alone can satisfy a requirement whose effect occurs at another endpoint.
Engineering work	Analyse writers, authority at commit, persistence and recovery dependencies; compare relevant alternatives and maintain the resulting rationale.	That performing the analysis necessarily produces a favourable result or a deployable design.
Evidence deliverable	Identified models, interface definitions, procedures, results, coverage, assumptions, limitations and unresolved findings.	That receipt of a report establishes technical satisfaction.
Acceptance criterion	Review the stated delivery against agreed content, quality and technical criteria for that commitment.	Permission to operate the complete service or authority to waive unrelated obligations.
Acquirer contribution	Supply governing policy interpretation, operational scenarios, identity-interface information, representative access and responsible decision participants.	That these inputs already exist or that a supplier can make the customer's authority decisions.

This separation permits an important honest outcome. A supplier may complete a contracted feasibility study satisfactorily by showing that an alternative cannot meet SR-11 under the stipulated identity outage. The study can be accepted as an adequate investigation while the alternative is rejected. If the agreement rewards only a favourable assurance conclusion, it encourages advocacy rather than useful engineering evidence.

Conversely, a beautifully structured report may meet its formatting requirements while failing its substantive deliverable criteria. An authority analysis that omits receiving-side commit does not become adequate because it contains every requested heading. Acceptance of engineering information needs criteria for scope, reasoning, traceability and limitations, not merely a file count or a signature block.

An organization can represent these obligations in its existing specification, statement of work, deliverable schedule and acceptance plan. The book does not prescribe a new document for each class. It does require a reader to locate each obligation, identify its controlling revision and determine which decision depends on it. Where documents conflict, the parties must resolve the conflict through the applicable agreement process; a technical cross-reference does not decide contractual precedence by itself.

17.3 Match the commitment to the maturity of the problem

AP-05 cannot responsibly pretend that every recovery detail is ready for a fixed implementation commitment. REC-P1 remains a candidate authority and operating profile. The authentication

binding, transaction lifetime, writer coverage and workload envelope also need completion. The acquisition strategy should distinguish work that can proceed under those uncertainties from commitments that depend on resolving them.

One acquisition strategy could progress from a bounded investigation to a representative integration article and then to the operational contribution and support. The investigation would compare ARC-B and ARC-C, establish external contributions and identify disqualifying conditions. A later implementation commitment could then use resolved requirements and interfaces. This is an option to evaluate, not a mandatory contracting sequence.

The investigation still needs limits and success criteria. It should identify the questions to answer, alternatives to compare, information the supplier may assume, required customer contributions and the form of the resulting recommendation. “Investigate security” is no more useful than “deliver a secure system.” For the recovery question, meaningful outputs include the authority dependency map, complete staff task, package-eligibility rules, proposed mode transitions and a reasoned account of the unresolved decisions.

There is also a risk in using exploratory work to create an irreversible product commitment. A prototype tied to a supplier's proprietary controller can make later alternatives costly even when its original purpose was only to examine task feasibility. Identify which choices are temporary, which information must remain portable and what would be required to change direction. The technical plan and commercial arrangement should make that exposure visible before it becomes an argument that the preferred product is now the only affordable choice.

A mature portion of the design need not wait for every unrelated question. The common transaction contribution in RZ-01 can be investigated against the established repetition, generation and outcome concerns while recovery authority is being resolved. The work must retain the boundary: a normal-mode transaction result is not acceptance of the complete recovery capability. Shared state or interfaces may require revisiting that result once the recovery design develops.

When a project chooses to proceed with unresolved inputs, record the assumptions, the bounded work permitted and the consequences if an assumption fails. Assign a participant to resolve each consequential input before the commitment that needs it. A provision allowing later clarification does not itself make unlimited redesign affordable or technically harmless. Those consequences belong in the acquisition decision alongside schedule and price.

17.4 Write work obligations that expose the engineering

The acquisition package should describe enough of the work to make responsibility and evaluation possible without unnecessarily dictating every method. A statement requiring an analysis is useful only when the supplier and customer understand its subject, inputs, scope and decision use. The following original teaching fragments illustrate engineering content for AP-05. They require adaptation to the actual agreement and authority structure before contractual use.

Illustrative work obligation — authority and path analysis. The supplier is to identify the management and enabling paths within its supplied contribution that can request, approve, perform or bypass a dispatch-rule activation. For each path, the analysis is to identify the source of authority, the enforcement location, the relevant configuration and the assumptions made

about customer or third-party services. The supplier is to relate its findings to the allocated SR obligations and identify receiving-system work that its own evidence cannot establish.

That wording does not make the supplier responsible for discovering every customer-managed writer. It obtains a defined contribution and exposes the interface to A-01. The customer's integrator must combine it with deployment, application and operational information. A supplier should not claim complete system coverage from its own inventory; the customer should not reject a properly scoped contribution merely because it is not the entire system analysis.

Illustrative work obligation — persistence and interruption. The supplier is to explain how its proposed realization preserves the relationship among effective configuration, transaction identity, historical outcome, activation generation and mode epoch during the specified interruption, restart, replacement and recovery conditions. The explanation is to identify supported starting configurations, storage assumptions, permitted migrations and conditions in which the result cannot be established. Proposed evaluations are to distinguish the fault conditions they represent.

This asks for the missing engineering rather than requiring a particular database. It also prevents “durable” and “atomic” from standing in for an account of power loss, mixed-version operation or snapshot restoration. Where a product cannot supply the assumed property, the deliverable should identify the mismatch and proposed disposition. The customer then determines whether to change allocation, select another contribution or seek an authorized requirement change.

Illustrative work obligation — recovery alternatives. The supplier is to compare the agreed candidate recovery arrangements against the unchanged SR-11 task and the applicable protection obligations. The comparison is to include preparation, personnel, authentication, authority withdrawal, package eligibility, receiving-state protection and return to normal operation. Assumptions about independently available customer services are to be identified for confirmation, not treated as supplier-established facts.

The work obligation should also require useful adverse conclusions. An unresolved policy question, missing service commitment or infeasible task allocation is an output requiring action, not a defect to conceal in the final report. The responsible technical participants should review proposed remedies and their effects on the wider requirement set. The supplier's duty is to provide the agreed competent work and evidence, not to invent the customer's permission to operate.

17.5 Define evidence deliverables around their intended use

An evidence request should explain which decision the information supports and what a reviewer must be able to determine from it. “Provide the test report” leaves open the identity of the test article, the relevant procedures, the observation method and whether contrary results are included. A report can be accurate yet insufficient for the customer's intended conclusion because the requested scope was too narrow.

AP-05 uses three evidence packages in Table 17.2. The ED identifiers are proposed deliverable records, not proof that the information has been produced. They can reference controlled source

repositories where permitted, but a decision baseline must identify the actual revisions reviewed rather than a link that will later resolve to whatever is newest.

Table 17.2. Proposed evidence packages and review purposes

Package	Required information	Review purpose and limitation
ED-05-01: definition and allocation	Applicable requirement/profile revisions; provided writer paths; interface assumptions; design alternatives and outstanding decisions.	Determine whether the supplied contribution and its dependencies are sufficiently defined for the next commitment. Not implementation verification.
ED-05-02: realization and evaluation	RZ-01-compatible item identity; build/configuration information; procedures; actual results when available; raw observations needed for review; coverage and discrepancies.	Evaluate what the identified realization demonstrates and which receiving-system work remains. Planned tests remain separate.
ED-05-03: transition and continuing support	Permitted use and modes; support resources; readable model views; recovery and migration information; retained evidence; change and escalation arrangements.	Determine whether the receiving organization can use and sustain the supplied contribution. Receipt alone is not operational readiness.

Specify review access as carefully as file format. A supplier may protect proprietary information through controlled examination rather than unrestricted transfer. That can be workable when qualified reviewers receive enough information to evaluate the relevant claims and preserve a meaningful result. A summary that omits the configuration, method and limitation cannot automatically replace the missing evidence simply because disclosure is difficult.

The customer also needs to identify who will review each package, with what competence and time. Requesting complex evidence without resources to examine it creates a delivery ritual. A technical reviewer may need application and identity specialists, an operational representative and an independent challenge of a critical assumption. The degree of separation should follow the consequence and governing arrangements rather than a universal demand for an external assessor on every record.

Evidence must arrive while it can still affect the decision. As an invented planning example, allow seven working days for an initial review, ten for necessary correction and five for re-review, all sequential. The chain requires twenty-two working days. Delivery two working days before the intended decision leaves twenty working days of the assumed sequence unavailable. This calculation is not a prescribed review duration; it exposes the schedule consequence of expecting meaningful review without allocating time for it.

Some activities can overlap, and an acceptable first submission may not need the full correction allowance. Those are planning assumptions to justify, not reasons to remove all contingency. The acceptance schedule should also account for customer review delays and supplier access to necessary facilities. A date cannot establish that an analysis has been completed or that its findings are favourable. Technical maturity and calendar readiness need a reconciled account.

17.6 Specify the customer's contributions with equal precision

Supplier accountability is weakened, not strengthened, by leaving customer dependencies implicit. For this case, the customer controls the operational need, the policy interpretation, portions of the identity environment, relevant receiving endpoints and the people who would perform restoration. The supplier cannot demonstrate the complete task without those contributions. NIST's acquisition treatment explicitly includes performance of the acquirer's agreement obligations and timely provision of the information and services on which the supplier depends. [1, Appendix K.1]

Identify the required input, its controlling source, the participant responsible and the commitment that needs it. A statement such as “customer will provide access” is inadequate if the planned evaluation needs receiving-side commit observations, controlled identity revocation and records unavailable to an ordinary account. The access arrangement must also protect operational services and sensitive information. More access is not automatically better; sufficient authorized access for the specified evaluation is the objective.

Policy interpretation is especially important for recovery. The customer must identify who can decide whether a proposed independent MFA and authority arrangement satisfies POL-01 and the governing recovery rules. The supplier can describe its mechanism and consequences, but it cannot resolve the customer's authority relationship through an implementation choice. An unanswered policy question should block the dependent acceptance claim, not necessarily every bounded prototype investigation.

Operational input needs similar care. The fifteen-minute interval includes mobilisation and approval, not just the controller's execution. The customer must provide a representative staffing and access arrangement or explicitly restrict the evaluation to a technical subtask. Supplying an engineer with exceptional access for a demonstration is useful for some purposes, but it does not establish designated staff's performance in the receiving organization.

Where customer inputs are late, inconsistent or unsuitable, preserve the effect on the work and use the agreement's change process. Do not assume that the supplier can absorb every consequence, or that customer delay excuses unrelated supplier deficiencies. The engineering record should distinguish which result is affected, what substitute representation is being used and what must be revisited when the real input becomes available.

17.7 Evaluate proposals against meaning, not a column of yes answers

A proposal response should distinguish demonstrated existing capability, a supported configuration, proposed development, reliance on a customer contribution and an exception or unresolved issue. All can be legitimate parts of an offer when represented accurately. Combining them under “compliant” hides the difference between a feature available now and work that still needs design, funding and evidence.

Begin with binding conditions. A central-only recovery proposal does not meet the unchanged identity-outage need merely because its gateway is inexpensive and well assessed. For a conditionally plausible alternative, identify the unresolved conditions and whether the proposed acquisition phase is intended to resolve them. A missing authority decision is not a technical pass; it is also not necessarily a reason to reject an otherwise useful exploratory study.

Ask the proposer to explain one consequential path. For SR-03, have it identify the authoritative revocation event, source consistency, distribution, receiving caches, queued requests and commit. Ask what its existing report covers and what evidence it proposes to obtain. This is more discriminating than asking whether the product “supports revocation.” It also gives a smaller supplier a fair opportunity to demonstrate a well-bounded contribution rather than compete on the volume of its marketing documentation.

Product security deserves attention distinct from the supplier's enterprise security. CISA and the FBI's Secure by Demand guidance makes that distinction in its procurement-oriented framing. [39, p. 1, Overview] In this book's case, an assessment of the supplier's corporate access controls can be relevant to development-environment risk. It does not by itself establish the customer's receiving-side transaction semantics. Preserve both questions and obtain evidence suited to each.

A proposal may honestly identify a gap the customer's draft overlooked. Treat that as engineering information, not automatically as a weaker bid than one that accepts every sentence without analysis. The evaluation should compare the credibility of the response, its assumptions, the work needed to close material gaps and the consequences of remaining uncertainty. The procurement method determines how these matters affect competition and selection; the book's purpose is to make the technical distinctions available to that method.

17.8 Acquire usable model and evidence access

A model deliverable should be defined by the engineering use it must support. Requiring “the MBSE model” does not establish whether the customer receives requirements, selected alternatives, calculations, external references, evidence status or only a set of diagrams. It also says little about whether the receiving organization can read the information after development access ends.

For MS-01, the customer needs the authority-to-commit relationship, the distinct SR-03 and SR-05 meanings, the source and uncertainty parameters, the chosen variation and links to appropriately identified evaluation records. A useful exchange exercise includes an unresolved result and an adverse analytical case. The receiving view should not convert a planned VP-02 into a pass or drop the time allowance that makes a proposed bound unacceptable.

Table 17.3. Model-delivery questions for AP-05

Intended use	Information and access to obtain	Acceptance demonstration
Technical review	Readable requirement, allocation, interface and task views with baseline, exclusions and selected alternative.	A reviewer can follow a specified claim and identify its unresolved assumptions without authoring software.
Re-analysis and change	Identified model content, units, inputs, material libraries and transformation settings, with permitted access to supporting records.	Reproduce a selected calculation and identify effects of the separate twelve-second-delay branch without changing the base case.
Continuing maintenance	Agreed native or exchange representations, explanatory conventions, readable exports, evidence references and support arrangements.	The receiving team can locate the controlling information and explain material exchange or tool dependencies.

Agree the necessary rights and access with the appropriate commercial and legal participants. Possessing a file does not answer every question about permitted modification, disclosure to a maintainer or use by a replacement supplier. Conversely, engineering continuity need not require ownership of every proprietary implementation detail. The agreement should obtain the uses needed for the lifecycle decisions and identify limitations before they become operational surprises.

Data exchange is an interface, not an administrative afterthought. Specify which information and meaning must survive, how unsupported content is reported and how the receiving representation is compared with the source. A standardized API can support defined services, but its existence does not demonstrate compatibility between the installed tools or preserve every project extension. [26, scope and conformance] Use a representative slice and retain the result for the actual versions and configurations evaluated.

17.9 Make change, support and exit part of the acquisition

A delivered contribution can change through supplier updates, platform substitutions, revised hosting arrangements, new subcontractors or retirement of a supporting service. The agreement needs a workable route for identifying changes that could affect required behaviour or evidence. A demand that no supplier ever change anything may be unrealistic; a provision that permits material changes without useful information can leave the customer unable to maintain its assurance basis.

Specify the information needed for impact assessment and the responsibilities for evaluation, coordination and approval where approval is required. For the receiving coordinator, a changed storage component may affect persistent history even if the application version remains the same. For ARC-C, changing the preparation process may affect eligibility or expiry. For MS-01, a library or exporter change may alter calculations or evidence status. Identify the relevant technical effects, not only a commercial product name.

Continuing support should address vulnerabilities, discrepancies, replacement, recovery, knowledge retention and eventual transition away from the supplied contribution. The customer needs enough information to decide whether an update preserves the required properties and what further work is necessary. The supplier needs clear expectations about evidence and response, including what the customer must provide to reproduce a problem. Neither a warranty nor a support subscription automatically supplies the missing engineering analysis.

Exit arrangements deserve early attention because access, knowledge and supplier leverage can change later. Identify how necessary configuration, models, evidence, interfaces and support knowledge will remain available under agreed terms. Plan the conclusion of supplier access without erasing the transaction and authority history needed for safe continuing operation. The detailed retirement process appears in Part VI; the acquisition task is to obtain the contributions that will make that process possible.

The resulting package should connect obligations to real engineering work and accountable decisions. It should not promise that clauses alone change production culture. Clauses can make the work visible, required and reviewable. The customer and suppliers still have to perform it, provide adequate evidence and act on findings that may challenge the intended delivery date.

17.10 Exercise: a complete report arrives too late

Situation. A supplier delivers a signed gateway release, an organizational security certificate and a report two working days before the planned acceptance meeting. The report accurately describes gateway authentication, but excludes the receiving coordinator and restoration task. The draft agreement required only “security assurance evidence before delivery.” The project wants to accept the complete service because the supplier has submitted the named documents.

Worked answer. Separate the evidence and the decision. The release identity, organizational assessment and gateway evaluation may each provide useful support within their scope. They do not establish the receiving-side authority, transaction or full restoration properties. Submission satisfies neither an unstated system-level evidence need nor an operational readiness claim. Whether the supplier has met its actual deliverable obligation is a separate contractual question requiring the governing agreement and authorized participants.

Repair the acquisition package prospectively rather than invent stronger obligations after the event. Define the relevant product allocations, engineering work, evidence scope, review criteria and customer contributions. Identify what can be accepted at component level, what additional agreed work is needed and which wider commitments remain unsupported. Use an honest change or follow-on arrangement where necessary; do not relabel gateway testing as system validation.

The twenty-two-day illustrative review sequence is not a legal deadline or proof that every review takes that long. It shows why two days may be incompatible with the project's own assumed review, correction and re-review activities. Reconcile the schedule with the work and its resources. A compressed calendar cannot enlarge the evidentiary scope of a report.

Carry forward. Acquisition establishes how the required contributions, information and decisions will be obtained. Chapter 18 follows those obligations into the supplier network and examines how to reuse supplier evidence without transferring claims beyond their demonstrated scope.

18. Supplier contributions and supply-chain assurance

18.1 Follow the contribution through its life cycle

A supplier's importance is not determined solely by its contract value or the quantity of sensitive information it stores. A small component can determine whether transaction history survives power loss. A deployment service can alter every receiving endpoint. A model library can change the analysis used to justify a design. A recovery-support provider can become indispensable during the very disruption for which the system was acquired. The relevant question is what the contribution can influence and what the system relies on it to provide.

For the logistics capability, the supplier network can include the gateway producer, receiving-coordinator developer, platform and firmware providers, identity-service operator, deployment integrator, package repository and support organization. Some are contracted directly, some through another supplier, and some are internal organizational services. Their commercial relationships differ; their technical influence still needs to be understood. An internal shared service is not exempt from dependency analysis merely because the customer and provider share an organizational name.

NIST's cybersecurity supply-chain guidance addresses the reduced visibility organizations may have into how products and services are developed, integrated and supplied. Its treatment includes supplier relationships, provenance, assessment and notification arrangements. [32, overview and Appendix A] This chapter applies those concerns to the actual requirements, architecture and evidence of the case rather than treating supply-chain assurance as an independent list of certificates.

The scope remains the selected administration capability. A supply-chain map can reveal dependencies outside that boundary without making the project responsible for assuring every activity of every provider. The project needs enough information to support its own reliance, to identify material gaps and to determine who can act on them. More visibility is useful when it answers those questions; an enormous unmaintained supplier inventory does not by itself improve the argument.

SC-05 extends AP-05 with a supplier-contribution review, not new runtime requirements. Its substitutions and evidence gaps are hypothetical; no actual supplier, country, company or incident is assessed.

18.2 Distinguish organizational assurance from delivered-capability assurance

An organizational assessment examines the entity, activities, scope and criteria covered by that assessment. A product evaluation examines an identified product or realization against its stated criteria. A service evaluation examines the service arrangement and relevant operating conditions. A receiving-system evaluation examines the composition and intended use in the customer's environment. These may share evidence, but one does not automatically establish all the others.

ISO/IEC 27001 concerns an information security management system. [11, public scope] A certificate can support understanding of the identified entity’s assessed activities and security

management. Check that entity, scope and applicable period. Separate evidence is needed for the customer's SR-07 transaction behaviour and receiving-side authority at commit.

This distinction is not a criticism of management-system assurance. The supplier's development and operational practices can materially affect the product and its support. Weak control over a build environment may undermine confidence in a release. A mature management system can provide useful governance and records. The error is promoting evidence about one proposition into evidence about a different proposition without the intervening reasoning.

NIST's due-diligence guide distinguishes supplier and product considerations, including traceability, provenance, resilience and foundational security practices. [40, overview and assessment dimensions] Use that distinction to keep organizational review and product evidence connected without treating one as the other. The two-level case review below is the book's application.

At the organizational level, ask who develops, supplies, operates and supports the contribution; which locations, arrangements and external parties are relevant; how changes and incidents are handled; and what information the customer can obtain. At the contribution level, ask which requirement is allocated, which implementation and configuration are supplied, what the contribution assumes about its environment and what evidence supports its behaviour. Then examine how organizational conditions could affect that technical contribution.

For example, a supplier may demonstrate that its production release is signed. The organizational question concerns who controls release authority and signing material. The technical question concerns what signature validation establishes at the receiving system, whether the correct release is selected and what happens if a signed but unsuitable image is installed. Neither question can be replaced with "the supplier uses signing." The two levels meet at a particular claim and its dependencies.

18.3 Establish a contribution map before choosing the review depth

SC-05 begins with functions and lifecycle influence rather than an undifferentiated vendor list. Identify the contribution, the relevant requirements or claims, the means by which it can affect the system and the information needed to evaluate it. A contribution may be significant because it executes a protection function, maintains its supporting data, changes its implementation or supplies evidence on which an acceptance decision relies.

Table 18.1. SC-05: contribution-centred supply-chain view

Contribution	Influence relevant to the case	Receiving-system question
Gateway and client software	Admission, session context, transaction identity and operator interpretation of outcomes.	Does its behaviour preserve IC-01 and avoid treating a timeout as proof of failed commit?
Receiving coordinator and state platform	Effective authority, generation, epoch, persistent history and configuration change.	Which configuration and fault conditions support SR-02 and SR-06–SR-12?

Contribution	Influence relevant to the case	Receiving-system question
External identity service	Authentication, authoritative status, revocation recording, consistency and availability.	Are source semantics and timing adequate for SR-03/SR-05, and what remains possible when the service fails?
Build, deployment and model services	Modify implementation, configuration, writer paths, analysis inputs and reports.	Could one error affect both the protection and the apparent evidence used to approve it?
Recovery and continuing support	Supply people, credentials, eligible packages, replacement components and usable information.	Are the necessary contributions available and permitted under the stipulated disruption?

Use the map to decide what deserves deeper scrutiny. A commodity item with no credible influence on the protected function may need proportionate identification and ordinary acceptance checks. A small service that can approve a release or restore privileged state may deserve detailed technical and organizational examination. The decision should follow the contribution and consequence rather than assume that every small supplier is low risk or every large supplier is adequately assured.

A contribution can also change significance as the architecture develops. A package store initially used for convenience becomes critical if ARC-C has no other way to obtain an eligible restoration instruction. A diagnostic service becomes a writer if a later feature permits it to activate configuration. Review the map when roles, interfaces or privileged functions change. An inventory collected at contract award cannot indefinitely describe a changing system.

Keep supplier unknowns visible. A future storage component may be unspecified, or lower-tier information may be unavailable. Record the affected claims and the information, design response or limit on reliance needed. Missing information establishes neither absence of a dependency nor supplier misconduct.

18.4 Perform due diligence as a scoped investigation

Due diligence should answer a defined decision question. Is this the legal and operating entity that will provide the support? Does the offered release come from the identified producer? Can the supplier provide evidence needed for the receiving interface? What changes in ownership, control, location or support arrangements could affect a relevant obligation? Those questions require identified sources and corroboration appropriate to the significance of the result, not an inference from a familiar product logo.

NIST SP 1326 provides a public starting point for supplier due-diligence assessment. [40, purpose and assessment dimensions] Its existence does not supply a universal exclusion rule or determine whether a particular supplier is acceptable. The project must relate findings to the applicable obligations, technical influence and actual use. A factual ownership relationship, a suspected exposure and a demonstrated product defect are different findings and should not be presented as equivalents.

A useful record distinguishes source fact, interpretation and resulting concern. A supplier's current declaration can identify its support entity. An independently obtained registry or agreement can corroborate an important identity or relationship where appropriate. The

engineering inference then explains how that relationship affects access, continuity, change authority or evidence. If the source is old, incomplete or inconsistent with another source, preserve that limitation before assigning a conclusion.

Avoid turning unfamiliarity into a technical accusation. A new provider may lack a long assessment history but be able to supply strong evidence for a narrow component contribution. An established provider may have extensive enterprise assurance while offering little relevant information about a changed management interface. The review should be demanding about the evidence needed for reliance, not impressed or alarmed solely by reputation.

Scope determines investigation depth. For a service controlling recovery approvals, the project needs to understand who can change the approval rules, how records remain available during an outage and what support is available if the service becomes untrustworthy. For a replacement storage component, the more immediate questions may concern provenance, controller revision, persistence semantics and affected acceptance checks. Both can require organizational information, but they do not require identical questionnaires.

The result should direct a decision or further work. It might justify proceeding with an evaluated component, obtaining a missing interface commitment, limiting a provider's authority, adding a recovery alternative or declining a proposed contribution. Where a legal or regulatory restriction is relevant, identify the competent authority and source rather than infer the rule from a general supply-chain standard. This book develops the engineering questions; it does not replace jurisdiction-specific legal review.

18.5 Flow obligations down by contribution and bring evidence back up

A prime supplier can only deliver a coherent result if its lower-tier contributions satisfy the assumptions on which that result depends. Flow-down should therefore allocate relevant obligations to the party able to perform them. Copying the customer's entire security specification into every subcontract can create impossible or irrelevant duties while leaving the actual interface contribution undefined.

For SR-03, the external identity provider contributes authoritative revocation meaning and relevant source behaviour. The integration service contributes distribution and mapping. The receiving implementation contributes enforcement, cache and commit behaviour. The integrator remains responsible for the combined argument. A component obligation to refresh within an interval must identify what is refreshed, which event starts the interval and how the result is used. It cannot silently replace the system's authoritative-revocation-to-commit deadline.

The earlier forty-five-second budget is useful here as a proposed allocation, not as measured performance. Each time contribution must be assigned to a party with a defined interface and a way to substantiate the bound. A twenty-second distribution estimate does not become a contractual guarantee merely because it appears in the customer's spreadsheet. Conversely, a supplier agreeing to twenty seconds has not established that the unallocated receiving queue is bounded.

NIST's supply-chain guidance addresses subtier flow-down and supplier assessment, including relevant visibility into dependencies. [32, Appendix A, controls SR-3(3) and SR-6] The case's allocation method makes the technical purpose explicit: pass down the requirement

contribution, context and necessary information; obtain evidence and limitations back through the chain; then evaluate the receiving-system composition.

Evidence flow is as important as obligation flow. A lower-tier evaluation can identify a restriction that affects the prime supplier's claim. If the restriction is removed from the summary sent to the customer, the resulting evidence chain is no longer adequate. Agreements should identify what relevant results, assumptions and material changes must be communicated, while respecting the actual information rights and handling arrangements. Confidentiality is a reason to design controlled access, not a reason to fabricate an unrestricted conclusion.

The customer should also identify retained integration work. A prime supplier may deliver the coordinator while the customer controls deployment accounts and legacy endpoints. The supplier's writer list cannot close A-01 for those customer contributions. The technical interface must identify who reconciles the complete writer set and which evidence supports that reconciliation. Flow-down does not transfer control over a system the recipient cannot inspect or alter.

Open-source dependencies require a related distinction. A community maintainer is not automatically bound by the customer's contract because the product includes that maintainer's code. The integrating supplier or customer must identify the support, evaluation and maintenance contribution it actually obtains and what work it retains. A contractual obligation needs an actual agreement with an accountable party; a dependency declaration alone cannot create one. Where the project needs sustained capability, it may have to fund support, maintain a fork, change the dependency or accept a properly understood limitation.

18.6 Reuse supplier evidence through an applicability argument

Evidence reuse can avoid unnecessary cost and make smaller suppliers' contributions practical. The useful test is not whether the report was produced for this exact contract, but whether its subject, configuration, criteria, methods and conditions support the receiving claim. Reuse needs an explanation of what remains the same and what has changed.

Table 18.2. Questions before reusing supplier evidence

Aspect	Applicability question	Example of a material difference
Identity and configuration	What exact item, build, firmware, settings and dependencies were evaluated?	Same application release with a different storage controller or reset configuration.
Property and criteria	Which requirement or characteristic was actually examined?	Successful new-session revocation offered as evidence of preventing queued commits.
Conditions and methods	Which workloads, modes, faults, paths and observations were represented?	Process restart tested, but power loss and retained transaction history excluded.
Provenance and access	Can the origin, relevant observations, limitations and review status be established?	A summary omits the original report's scope restriction or contrary result.

Aspect	Applicability question	Example of a material difference
Receiving context	Which assumptions does the customer provide, and what integration work remains?	An always-available identity simulator used where the required recovery scenario removes identity service.

A report can be partially applicable. Normal parsing and package-integrity results may remain useful after a deployment change, while writer coverage and timing require further work. Record that disposition explicitly. Universal rejection wastes useful evidence; universal inheritance conceals differences. The responsible engineering participants need to decide which changes can affect the property and what evaluation discriminates between acceptable and unacceptable behaviour.

Review the evidence's independence for the error of concern. A supplier test report and a third-party summary may both derive from the same test execution. Their agreement does not create two independent observations. An external reviewer can nevertheless add value by checking the original method, configuration or inference. Distinguish additional review from additional measurement, and identify what common assumptions remain.

An evidence-access limitation should be attached to the specific conclusion it prevents. A customer may not need proprietary source code to establish a narrow observable interface property, but may need more information to justify internal fault containment or generated-code correspondence. Choose a defensible combination of analysis, observation, controlled review and design constraints. “Black box” is neither a universal disqualification nor a warrant for unlimited confidence.

18.7 Use provenance, bills of materials and vulnerability information carefully

Provenance identifies where a contribution came from and how it reached the evaluated configuration. For software, firmware, hardware and engineering models, that history can include source, transformations, build or manufacturing information, release identity and subsequent changes. Provenance supports investigation and confidence in origin; it does not by itself establish suitability or correct behaviour.

A software bill of materials can identify included components and relationships relevant to vulnerability and change analysis. NIST treats bills of materials as useful supply-chain information that complements broader provenance and risk work rather than substitutes for it. [32, Appendix A, control SR-4] The receiving organization still needs to relate the supplied inventory to the actual installed release and intended use. A bill of materials for one product version does not automatically describe a customized build or a service whose backend changes independently.

Distinguish runtime components, development tools and external services. A library used only in a build process can influence the generated product without executing in production. An external identity service can be essential to the capability without appearing as a bundled software component. A firmware element can affect reset behaviour while the application dependency list remains unchanged. The contribution map and realization manifest together provide a more useful scope than a single inventory format.

Vulnerability information also needs an applicability record. A statement that a component is affected, fixed or not exposed should identify its issuer, product and version, relevant

configuration, technical basis, date and limits. A supplier's assertion can be valuable input, but its conclusion must fit the receiving environment. If the assertion depends on a disabled feature and the customer's deployment enables that feature, the premise has changed. If the available information is insufficient, record the uncertainty rather than equate silence with absence of exposure.

Choose the response from the consequence and available evidence: confirm the configuration, seek clarification, restrict a path, test a change or select another treatment. A fix can affect persistence, timing or interfaces. Focus impact analysis on those effects; urgency does not justify either universal requalification or unexamined release.

A release signature can support integrity and origin within its trust arrangement. It does not establish that all vulnerabilities are absent or that the release satisfies the customer's recovery semantics. Likewise, several identical package copies can preserve the same unsuitable content. Compare the information to the protected need and the authoritative reference, and identify who is entitled to approve the result for use.

18.8 Examine substitutions and service changes at the affected boundary

Consider a hypothetical supplier change to the storage platform supporting RZ-01. The application binary and its signature are unchanged. The replacement component uses a different controller and acknowledges writes under different power-loss conditions. An existing report covered process restart on the former platform. The supplier proposes treating the update as irrelevant to security because no authentication code changed.

The receiving review should begin with the affected contribution, not the label “security update.” Persistent history and configuration selection support SR-07, SR-09 and SR-10, and mode-state preservation supports SR-12. A changed acknowledgement meaning can alter those properties. The project needs to understand whether the new platform still provides the required persistence or whether another measure supplies it. The premise does not establish that a second commit actually occurs; it establishes that the earlier power-loss inference cannot simply be carried forward.

A focused response can preserve useful evidence. Retain applicable parsing, normal authorization and interface-format results where their conditions are unchanged. Examine the platform identity, failure semantics and relevant interruption points. Update the realization record, integration configuration and affected procedures. Re-evaluate the critical state transitions rather than repeat every unrelated test solely to create a new date on a report.

Cloud and managed services introduce a different change boundary. The customer may control tenant configuration while the provider controls hosting, update or support functions. Identify which allocated property each side supplies, what evidence is available and how changes become visible. A provider-level assurance report does not automatically establish the tenant's role mapping, every administrative path or the full restoration task. A well-configured tenant cannot compensate for every provider failure or privilege misuse either.

The agreement and architecture must therefore work together. Where the customer cannot approve each backend change, it can still seek defined interface behaviour, useful notification, evidence of relevant controls, recovery options and an exit strategy. The exact rights and service commitments depend on the agreement. Do not claim the power to freeze or inspect a provider's

entire infrastructure when the project has not obtained it. Instead, identify how the system limits reliance and what remains unresolved.

18.9 Make notification lead to a technical decision

Notification is useful only if it reaches a participant who can interpret it and initiate the right response. NIST includes notification agreements in its supply-chain controls. [32, Appendix A, control SR-8] For the case, the agreement should distinguish a proposed change, an observed incident, an identified defect and a newly suspected exposure. These events may require different urgency, information and technical action.

Define what starts the relevant notification obligation and what information can reasonably be provided at that point. Discovery of a potentially affected release can precede full root-cause analysis. An initial notice should not invent certainty, but should identify known scope, important unknowns, protective actions and the route for updates. A later completed analysis should preserve what changed in the understanding rather than replace the initial record as though uncertainty never existed.

The customer must maintain an intake and triage capability. A notice about altered storage semantics needs the platform and transaction owners, not only the vulnerability-scanning team. A notice about a model-export defect may require review of prior analysis conclusions even if no deployed executable changed. A recovery-support change can affect personnel availability and SR-11 without creating a software vulnerability identifier. Use the contribution map to route the work to the affected claims and decisions.

Notification should also reach through lower tiers where their contribution matters. A prime supplier may need time to identify which releases or delivered units include a changed component. The customer needs a defensible account of those identities and any interim uncertainty. A general notice that “all current products are secure” does not supply that account. Nor does the absence of a known vulnerability identifier establish that a change is irrelevant to resilience or assurance.

Reconsider the supplier and architecture relationship when repeated events expose a structural limitation. If adequate evidence cannot be obtained, support disappears or the receiving organization cannot act on important changes, another supplier, a reduced dependence or a different operating arrangement may be necessary. Such decisions should reflect the actual contribution, alternatives and consequences, not a ritual renewal of the same questionnaire.

18.10 Sustain supplier diversity, support and retained knowledge

Diversity is meaningful in relation to a common cause. Two suppliers can still use the same upstream identity service, firmware family, signing infrastructure or specialist support organization. Separate contracts do not establish technical independence. Conversely, shared contributions are not automatically unacceptable; they require analysis against the loss and continuity claim being made.

For ARC-B and ARC-C, independence from the stipulated identity outage is a narrower claim than independence from coordinator compromise, loss of power or corruption of the approved package source. The project should identify which causes each alternative addresses and which common dependencies remain. A supplier map that distinguishes these causes can support a

useful choice without pretending that complete independence is achievable or necessary everywhere.

Continuing support includes retaining enough knowledge to manage change after personnel and contracts change. Preserve the applicable interface definitions, configuration identities, evidence, assumptions and unresolved limitations. Obtain the access needed to use those records under the receiving organization's permitted environment. A dependency is not sustainably managed when its meaning exists only in one supplier engineer's memory or a tool licence that expires at handover.

Scale the burden to the contribution. A supplier of a narrowly bounded component may provide focused evidence reused across customers, supplemented by receiving-system evaluation. A supplier that controls privileged deployment or recovery may need deeper information and continuing obligations. The customer should avoid demanding a complete enterprise architecture from a small component producer while neglecting the one service that can alter all protected state.

The same proportionality applies to the customer's own work. A well-resourced prime does not remove the need for integrator competence; a small supplier does not excuse missing system-level reasoning. The acquisition and assurance arrangements must assign work to parties capable of performing it and fund the retained responsibilities. Supplier oversight becomes useful when it produces a technically coherent chain of contributions, evidence and action.

18.11 Exercise: a changed platform and an unchanged certificate

Situation. The coordinator supplier changes the storage platform but retains the application release identifier. It provides an unchanged organizational certificate, a signed binary and the previous process-restart report. Its new platform documentation describes different write-acknowledgement behaviour during power interruption. The prime supplier states that no receiving-system review is needed because the software is identical.

Worked answer. The unchanged binary is only one configuration attribute. The supplied contribution to persistence has changed, and that can affect the requirements for repetition, configuration consistency, outcome history and mode recovery. Identify the exact platform and relevant settings, determine the new failure semantics and review the applicability of the earlier evidence. The organizational certificate remains evidence about its own assessed scope; it does not decide the persistence question.

The appropriate next work is focused. Update SC-05 and the realization identity, examine the affected state transitions and obtain evidence for the required interruption conditions. Retain unrelated results where their continuing applicability is justified. Do not report an actual duplicate commit unless one is established, but do not declare the power-loss property demonstrated by a report that never examined it.

Also examine the supplier change process. Was the platform identity available early enough for review? Did the lower-tier change reach the prime's impact assessment? Does the customer need a revised notification or evidence arrangement? Those questions connect the immediate technical discrepancy to the supply process without assuming that a single mistake invalidates every supplier claim.

Carry forward. Supplier information becomes useful assurance evidence only when its identity, scope, assumptions and receiving-system contribution are understood. Chapter 19 assembles those relationships into an argument that can support, limit or reject a claim rather than merely collect favourable reports.

19. Evidence, assurance cases and technical challenge

19.1 Build an argument for reliance, not a dossier of reassurance

A project has a requirements baseline, architectural models, supplier reports, configuration records and test results. The volume is substantial. A decision authority still needs to know what those records establish about the particular system, which conclusions depend on assumptions, and what contrary findings prevent reliance. An assurance case supplies the argument connecting that information. It does not add protection merely by existing, and it should not serve as a polished defence of a decision already made.

NIST develops assurance through claims, reasoning and evidence concerning requirements, realization and use. [1, Appendix F.2] ISO/IEC/IEEE 15026-2 addresses assurance-case structure and terminology. [38, public scope] CASE-05 is an original teaching application, independent of any required notation.

For the logistics capability, the intended subject is bounded reliance on the administration contribution. The case cannot establish that every dispatch decision is correct, that all operational consumers have adopted a configuration or that the complete mission will succeed. Those conclusions involve additional systems, people and conditions. A careful boundary does not diminish the administration argument. It identifies what that argument must actually establish and where another contribution is required.

An IT practitioner can begin with familiar assessment and authorization records. A system description defines scope, a control statement identifies obligations, a test report supplies observations and a risk record explains remaining concern. The assurance task makes the inferential relationships explicit. It asks why these particular results support this particular claim, not whether every template contains text. An engineer can use the same argument to expose a missing allocation, unsupported model assumption or unexamined operating mode.

The chapter develops CASE-05, an original argument and review arrangement for TE-05. Its demonstration dossier contains stipulated examples, not reports produced by an operational test programme. The earlier C-01–C-05 remain candidate claims, VP and VA work remains planned, and REC-P1 remains unresolved. The hypothetical items allow the reader to practise interpreting positive, negative and missing support without treating invented evidence as a real project accomplishment.

19.2 Distinguish a requirement, a claim and an argument

A requirement states what a specified subject must provide. A claim states a proposition whose truth or adequacy is being asserted. An argument explains why evidence and subordinate propositions justify that assertion within a defined context. A claim can concern satisfaction of a requirement, but can also concern whether the requirements adequately address the needs or whether the evaluation method is suitable. These are different questions and need different support.

SR-03 requires a specified revocation effect within a defined interval. A claim that a particular configuration satisfies SR-03 requires evidence about source semantics, paths, timing, queued work and commit. A separate claim that SR-03 adequately addresses the relevant loss scenario

requires reasoning about stakeholder needs and what authority withdrawal actually means. Successful implementation of an inadequate requirement cannot establish an adequate protection response.

Similarly, “the product has MFA” is narrower and less precise than C-02's concern about covered normal-mode authority. Authentication is one contribution. Receiving-side authorization, current status, alternative writers and the mode in which authority is used remain relevant. The argument must avoid allowing a convenient subclaim to stand in for the complete proposition.

An assurance argument can legitimately use several forms of reasoning. Analysis may establish a property under a model's assumptions. Test evidence may establish behaviour under selected conditions. Inspection may establish configuration or path information. Representative task evaluation may establish whether intended people can perform the required work. NIST's assurance discussion recognizes different analytical and evidentiary approaches rather than treating one technique as universally sufficient. [1, Appendix F.2.2–F.2.3]

Write the inference in ordinary language before drawing a diagram. For example: the receiving coordinator is intended to prevent unauthorized activation because all relevant effective-change paths are constrained to use it, it evaluates applicable authority at the consequential boundary, and its supporting state and configuration preserve that enforcement under the claimed conditions. Each part of that sentence creates an evidence obligation. If another writer exists or the source status has a different meaning, the inference is incomplete even when the coordinator's local tests pass.

An argument should expose those obligations rather than conceal them in an unexplained arrow. The reader must be able to ask which assumption makes the next inference valid. Where the answer refers back to the claim being established, the argument is circular. Where it refers to a different team's unspecified responsibility, the argument has a missing contribution. Both are useful findings before the case is used in a consequential decision.

19.3 Establish scope, context and claim structure

Begin CASE-05 by identifying the decision, system boundary, configuration maturity, permitted use and relevant conditions. Its immediate decision is what further engineering and evaluation work the available information supports. It is not operational authorization. The case references the TE-04 realization walkthrough and TE-03 architecture proposals, with AP-05 and SC-05 describing the contributions to obtain. No approved operational baseline is implied by collecting those identifiers.

C-01 through C-05 provide a practical structure. They are not five independent boxes to average into one assurance score. C-02 and C-03 depend on an adequate problem definition and evidence appropriate to the implementation. C-04 depends on technical protection, operational resources and a permitted recovery arrangement. C-05 concerns the applicability of the evidence used across those claims. A weakness in one can affect the support for several others.

Table 19.1. CASE-05: candidate claims and their necessary support

Existing claim	Argument that needs to be developed	Important dependency or limit
C-01: adequate problem definition	The selected needs, losses, scenarios and obligations are an adequate basis for the identified administration decision.	Requires stakeholder and boundary review; a complete requirement list does not prove all relevant concerns were found.
C-02: covered normal-mode authority	Relevant effective-change paths enforce the required authentication, authority, withdrawal and freshness behaviour.	Depends on A-01, source meaning, configuration, timing and protection of enforcement itself.
C-03: controlled state and outcome	The receiving interaction preserves package, version, repetition, configuration and outcome properties.	Depends on IC-01, persistent-state realization, interruption and historical/current-state meaning.
C-04: usable restricted restoration	Designated staff can perform the required restoration under the stipulated outage while retaining applicable protection.	Requires completed REC-P1, eligible preparation, independent resources and representative task evidence.
C-05: applicable evidence	The evidence used supports the identified propositions for the relevant configuration and conditions.	Requires provenance, method, coverage, limitations and evaluation of changes and common dependencies.

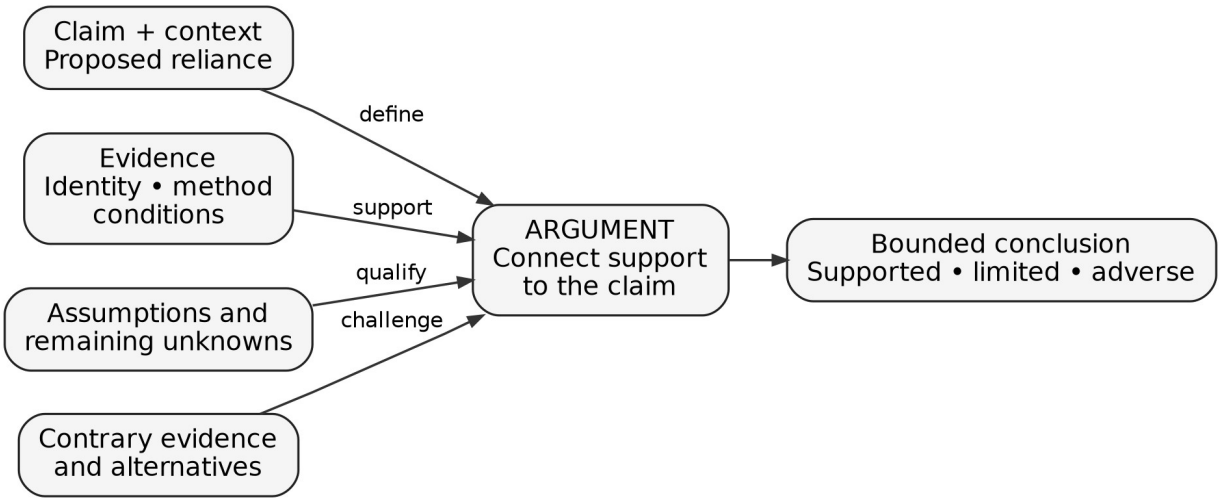
A top-level claim should reflect the decision and the support actually available. During development, the defensible proposition may be that a particular concept is sufficiently understood to justify a bounded prototype investigation. That is a real engineering claim when supported by coherent requirements, feasible interfaces and an adequate investigation plan. It is not a weaker wording intended to smuggle an unready product into operation.

For a later operational decision, the claim would need the realized configuration, supported modes, intended users, relevant adversity and remaining conditions. The current case cannot supply those facts by assuming that the project will eventually resolve them. Future evidence belongs in a plan, with owners and dependencies. It does not occupy the same role as a reviewed result.

Keep scope exclusions honest. Excluding operational dispatch-policy correctness from the administration claim is appropriate because the selected system does not establish it. Excluding an unexamined administration writer while asserting that every administration path is controlled is not an adequate boundary choice. An exclusion is legitimate only when the remaining claim and decision can be supported without the omitted contribution, and the relevant authority understands the resulting limit.

The following schematic brings together the relationships developed here.

Figure 19.1. Evidence supports an argument, not a preferred answer



Original schematic. Evidence, assumptions and contrary findings converge on the argument. An attractive document or a desired decision cannot supply a missing inferential step.

19.4 Construct an evidence record that can survive challenge

An evidence item needs a stable identity and a description sufficient to establish what it can support. Record the source, evaluated item or representation, relevant configuration, method, conditions, observations, processing, result and limitations. Identify the requirement or claim for which it is being considered and the reason for that use. A link to a file does not explain applicability, and a recent date does not establish quality.

Distinguish the item from its interpretation. A record may contain timestamps and receiving-state observations. An analysis relates them to the required event and applies a decision rule. The assurance argument then determines what that result means for the claim. Keeping these layers identifiable permits a reviewer to correct an erroneous inference without discarding valid observations or pretending that the underlying system changed.

The E-05 identifiers in Table 19.2 belong to a fictional review dossier. Each item is explicitly stipulated for teaching. Some echo the hypothetical findings in Part IV; none represents a performed VP or VA activity. The dossier is deliberately mixed so that a reviewer must preserve useful evidence while refusing unsupported conclusions.

Table 19.2. Hypothetical dossier items for evidence interpretation

Item and stipulated content	Useful support within the stated facts	Limit or contrary implication
E-05-01: organizational assessment covering the supplier's identified development activities	Relevant context for management of the assessed activities and their controls.	Does not demonstrate receiving-system transaction or recovery behaviour.

Item and stipulated content	Useful support within the stated facts	Limit or contrary implication
E-05-02: one valid activation and authoritative outcome completed in 8.6 seconds in a defined normal condition	Supports the observed positive case against SR-00, subject to adequate observation and configuration identity.	Does not resolve the incomplete burst and interruption envelope or establish every path.
E-05-03: revoked-identity commit at 61.2 seconds with a stipulated conservative ±0.8-second bound	The compatible interval is 60.4–62.0 seconds; under applicable conditions, the commit violates SR-03.	Contrary evidence must remain visible; it is not converted into a pass by a plan to fix it.
E-05-04: deployment writer outside the proposed coordinator, as in D-04-02	Reveals an incomplete coverage/allocation argument requiring analysis of that path.	Does not by itself establish that the path is malicious or violates every authority rule.
E-05-05: exported timing report omits a source-model allowance, as in D-04-03	Supports a finding that the export is not equivalent for the affected analysis.	Conclusions relying on the omitted allowance require review; unrelated behaviour is not automatically disproved.
E-05-06: candidate recovery task relies on an approval portal behind the failed identity service, as in D-04-04	Refutes the proposed arrangement's claimed independence for the stipulated outage.	A separately available controller does not establish the whole restoration task.
E-05-07: proposed corrective plan and linked future evaluations	Establishes intended work, responsibilities and information needs if adequately defined.	Supplies no performed evidence that the correction works.

The dossier does not support a single binary answer about whether “the system is secure.” It supports several different conclusions about particular propositions. E-05-02 is a favourable but narrow observation. E-05-03 is a stipulated counterexample under the requirement's conditions. E-05-04 identifies an unresolved coverage issue. E-05-07 is a plan. A credible case must preserve those distinctions even when the summary needs to be brief.

An actual evidence record would also identify access and preservation requirements. Reviewers may need the raw receiving observations, relevant procedure revision and measurement treatment rather than only the supplier's summary. Retain the information necessary to reconstruct the conclusion under the project's agreed handling and retention arrangements. A cryptographic digest can identify a file, but does not establish its origin, completeness or correct interpretation without the corresponding provenance and review.

19.5 Work one argument from requirement to conclusion

Take C-02's authority concern and focus on withdrawal under SR-03. The claim is not that the gateway rejects a new login after revocation. It is that the specified configuration prevents every relevant dispatch-rule commit using the revoked identity at or after the permitted deadline under the defined conditions. That proposition requires several connected contributions.

First establish the authoritative source event and the meaning of the identity being revoked. Then identify the relevant receiving paths and the way status reaches them. Examine established sessions, staged requests and queued work, including the interval between authority evaluation and effective change. Establish the clock and measurement relationship for the observation. Finally, relate the evaluated configuration and scenarios to the scope of the claim. A test can address selected parts of this chain, but the case needs the chain itself.

The earlier allocation of forty-five seconds provides a conditional analytical argument only if its intervals are complete, compatible and justified. It does not establish the actual implementation. Likewise, SR-05's thirty-second freshness condition is related but distinct. A short cache setting cannot establish source consistency, and an authenticated record can still be too old. The assurance argument should preserve those different propositions instead of merging them into a generic “identity controls effective” conclusion.

Now introduce E-05-03. Under the stipulated facts, a revoked-identity commit occurs within the interval 60.4–62.0 seconds after t0, entirely beyond the sixty-second deadline. This is direct contrary evidence for SR-03 in that evaluated condition. The correct conclusion is not that the design is nearly compliant, nor that every part of C-02 is necessarily false. It is that the unqualified withdrawal-satisfaction claim is not supported and the specified requirement is violated in the stipulated evaluation.

The case should then identify the scope of the discrepancy and the work needed to resolve it. Was the source status late, the queue unbounded, the check-to-commit interval excessive or the observation misinterpreted? Those are hypotheses until investigated. A confirmed test-setup error could change the interpretation; it must be supported, not invoked merely because the result is inconvenient. A confirmed implementation correction needs a new identified evaluation and appropriate regression work.

E-05-04 creates a different problem. The deployment writer is outside the proposed coordinator. That finding challenges the completeness of the current argument. The path might enforce equivalent authority under another controlled mechanism, or it might be an unacceptable bypass. The evidence given does not decide which. The next work is to establish its purpose, privileges, operation and relevant conditions, then revise the allocation and evaluation accordingly. The reviewer should neither ignore the path nor report an unsupported security incident.

The resulting review conclusion can be precise: selected gateway and positive-operation evidence remains useful, the stipulated receiving-side withdrawal result is adverse, and complete writer coverage remains unresolved. That conclusion supports focused corrective and investigative work. It does not support an assertion that every normal-mode authority obligation has been satisfied. The distinction is exactly what a decision authority needs before accepting residual limitations or committing to wider use.

19.5.1 Instantiate the assurance structure with a real case claim

The generic assurance structure becomes more useful when an adverse item is shown affecting a specific subclaim. For C-02, path coverage, current authority and timing applicability are separate propositions. Evidence can support some of them while contrary evidence defeats an unqualified conclusion about another.

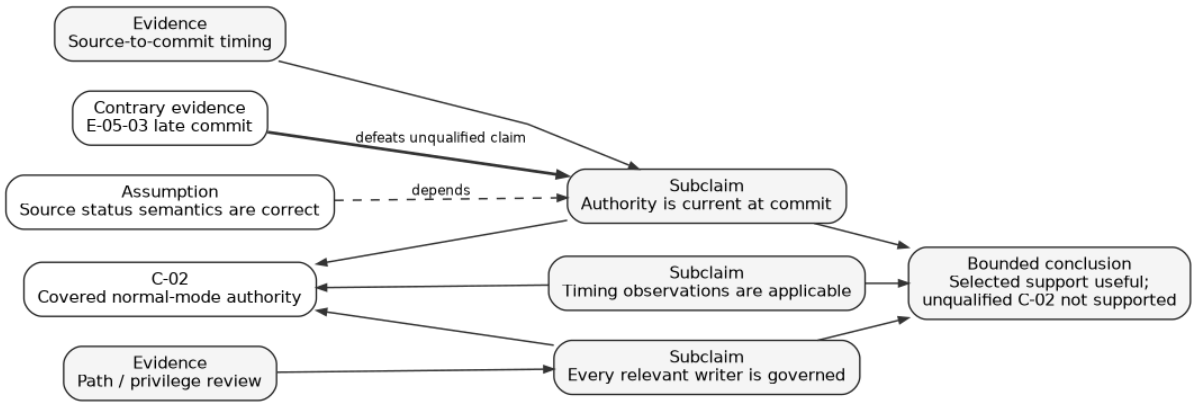


Figure 19.2. Instantiated authority claim with supporting and contrary evidence

Original assurance-argument example using the book's C-02 and E-05 identifiers. The adverse item defeats the unqualified authority-at-commit subclaim; it does not erase unrelated evidence or prove every other subclaim false.

The practical review question is therefore not whether the case has enough documents. It is whether each inferential step has support appropriate to the proposition and whether a contrary result changes the conclusion. A later corrected configuration needs new evidence; it does not rewrite the historical adverse observation.

19.6 Examine evidence quality and dependence

Evidence quality follows its intended use. Accuracy concerns whether the observation or information corresponds to the represented event. Relevance concerns its relationship to the claim. Credibility concerns whether the source and method merit reliance. Coverage concerns the represented requirements, paths, states and conditions. Completeness and sufficient detail concern whether a reviewer can reconstruct the necessary reasoning. NIST's assurance treatment discusses evidence attributes and the sufficiency needed for the claim. [1, Appendix F.2.3]

Those attributes should not become an unexplained numerical score. A report with excellent formatting and provenance can still be irrelevant to the critical property. A narrow but carefully observed counterexample can defeat a broad claim more decisively than a large collection of routine successes supports it. The case needs an argument about the meaning of the evidence, not an average of document-quality ratings.

Dependence matters when combining items. Suppose an architecture report, a compliance dashboard and a supplier assurance summary all obtain writer coverage from the same deployment export. If that export omits the direct writer in D-04-02, all three can repeat the omission. Their agreement is not independent support for complete coverage. An independent

configuration observation or qualified examination of deployment privileges may address the shared error, provided it actually reaches the relevant paths.

The same issue appears in model-based development. A model can generate implementation structure and expected test responses from the same incomplete transition set. If snapshot restoration is absent from both, they can agree perfectly while leaving SR-07's history obligation unsupported. Separately reasoned scenarios and observations can expose that omission. Independence should be chosen in relation to the fault or bias that needs challenge, not inferred from different product names or organizational letterheads.

Evidence age should also be interpreted in context. A reviewed design analysis may remain applicable for years if its relevant assumptions and configuration remain unchanged. A newly issued report can already be unsuitable if it uses the wrong deployment settings. An older certificate may have defined validity and scope conditions that need examination; its age alone does not establish the technical state of every product. Record the change or condition affecting applicability rather than equate recent with trustworthy and old with worthless.

When combining evidence, identify which items add new information and which add review of existing information. A second expert's critique can improve confidence in interpretation without constituting a second test. A second test using the same biased instrumentation may add repetition without addressing the bias. The argument should explain what each contribution changes in the basis for reliance.

19.7 Invite defeaters and preserve contrary evidence

A defeater can undermine an inference or its support by challenging an assumption, introducing contrary evidence or offering another causal explanation. Use this practical review concept to make the assurance argument challengeable; no particular notation is required.

A possible defeater is not automatically a demonstrated defect. “Another writer might exist” is a coverage question. D-04-02 stipulates that a writer does exist outside the proposed coordinator, strengthening the challenge to the present allocation. Whether that writer is unacceptable still needs analysis. E-05-03, by contrast, stipulates a commit that violates a criterion under its applicable conditions. These findings deserve different labels and responses.

Table 19.3. CASE-05 challenge ledger

Challenge	What must be examined	Evidence or disposition needed
The authority argument omits an effective-change path.	Actual writer set, privileges, activation mechanisms and their relationship to A-01.	Reconciled coverage and evidence of the path's required protection, or an explicit unresolved limitation.
The status timestamp represents receipt rather than authoritative observation.	Source and receiving interface meaning, translation and timing analysis.	Agreed semantics and relevant end-to-end evaluation; a field name is insufficient.
The duplicate-commit argument excludes history rollback.	Permitted restore transitions, namespace lifetime, generation and epoch persistence.	A recovery design and evidence that preserves or reconstructs the necessary history.

Challenge	What must be examined	Evidence or disposition needed
The recovery task assumes unavailable authority information.	Complete task and dependency path, including people and approval access.	A permitted alternative arrangement and representative evaluation, not merely a faster controller.
Multiple favourable reports depend on one incomplete model or export.	Evidence lineage, selected variants, omitted fields and independent observations.	Correction and review of affected inferences; preserve useful unaffected evidence.

Assign challenges to participants able to investigate them and identify the decisions they can affect. Do not close a challenge merely because someone has accepted an action to investigate it. A recorded action is useful project control; the corresponding claim remains limited until the action produces relevant support. Where a challenge is dismissed, preserve the reason and the evidence that justifies dismissal.

Challenge must remain proportionate to the decision. Identify the considered adversity, coverage, exclusions and remaining uncertainty rather than pursue an unbounded search for unknowns. An exploratory commitment may need less support than operational use of a mission-critical capability; neither permits false statements about what was demonstrated.

A challenge can reveal that the claim itself is poorly framed. “No duplicate can ever occur” is ambiguous about messages, calls and effective changes. SR-07 concerns the number of commits for the same transaction identity and content under its specified lifetime. Refine the claim to the actual obligation. Do not narrow it so far that an inconvenient requirement disappears while the project continues to present the original capability as delivered.

19.8 Keep model assertions, performed results and decisions separate

MS-01 can organize CASE-05 through controlled links among requirements, functions, interfaces, analyses and evaluations. That improves navigation and change impact when the relationships have clear meaning. It does not convert an assertion that an element satisfies a requirement into independent evidence of satisfaction. The case must retain which links represent design intent, planned evaluation, actual observations and reviewed conclusions.

Establish the combination of revisions used for a review. The requirement, profile, architecture variant, model library, analysis inputs, realized article and result can each have a different configuration identity. A live link to the latest version is useful for ongoing work, but not a reproducible account of an earlier decision. Retain a review manifest identifying the material actually considered and any later applicability decision.

The twelve-second-delay branch provides a concrete example. Its freshness calculation is 18 + 12 + 2 = 32 seconds, while the earlier eight-second branch gave 28 seconds. A report that preserves the old result but points to the new model has broken the evidence relationship. An export that removes the two-second allowance can falsely suggest a value at the thirty-second limit. E-05-05 challenges the reporting and exchange contribution; it does not establish that a real endpoint committed under those conditions.

Review generated reports against representative underlying records, including a known open item and a contrary result. A report generator should preserve status, configuration, scope and

limitations. A dashboard that excludes blocked work from its denominator may be mathematically consistent while misleading a reader who interprets “100 percent passed” as complete requirement satisfaction. State what the percentage describes, and do not use it as a replacement for the critical unresolved claims.

The evidence repository also needs protection, access and retention arrangements. A person able to replace failed results or alter their configuration references can affect a decision without changing the product. Limit consequential changes, preserve history and ensure qualified reviewers can obtain the necessary records. Cryptographic integrity mechanisms can support those controls; they do not determine whether an argument is logically sufficient or an observation was meaningful.

19.9 Review competence, independence and the limits of access

Technical review requires competence in the proposition being evaluated. A security practitioner may understand identity threats while needing application expertise to assess transaction persistence. A software engineer may need operational input to understand restoration timing. A human-factors specialist may identify a task failure that a component test cannot reveal. Systems engineering coordinates these contributions; SSE supplies its specialist reasoning rather than claiming every form of expertise.

Independence should support candid challenge of consequential errors and incentives. An implementer can provide essential explanations while a separate qualified reviewer evaluates the evidence or conclusion. A nominally independent organization that receives only a marketing summary may be unable to perform the claimed review. A reviewer with detailed access but no permitted route to report adverse findings is similarly constrained. Identify the actual review scope, access and authority.

Reviewers should record what they examined, the questions they addressed, the evidence they considered and the limits of their conclusion. “Reviewed” is not a universal endorsement. A review of requirement clarity does not establish implementation satisfaction; a review of test calculations does not establish that the tests were performed as reported. The summary should identify the type of contribution so that it can be reused appropriately.

Confidentiality arrangements can support rather than prevent competent review when they are designed around the necessary information. Controlled examination, scoped evidence access and appropriately sanitized summaries may be sufficient for particular purposes. Where access prevents a reviewer from establishing a material claim, that limitation should remain visible to the decision authority. It must not be concealed behind the independence or reputation of the reviewing organization.

When reviewers disagree, compare their boundaries, configurations, failure conditions and criteria. They may be analysing different operating arrangements. Reconcile the decision context, retain substantive dissent and identify evidence that could distinguish the positions. Removing the difficult condition from a summary creates apparent agreement, not stronger support.

19.10 Produce a conclusion that directs the next action

An assurance conclusion should identify the claim supported, the configuration and conditions, the material limitations and the decisions for which the conclusion is suitable. It should preserve adverse evidence and explain the work needed for a stronger claim. A concise conclusion can be useful without compressing every distinction into a red, amber or green status.

For CASE-05's fictional dossier, an appropriate review is that the information supports continued targeted investigation and correction but not an unqualified operational claim for the complete administration capability. The stipulated positive operation remains relevant within its narrow conditions. The stipulated late commit is contrary evidence against SR-03. Writer coverage and export correctness require resolution. Recovery authority and the full independently usable task remain unestablished. No amount of planned future evaluation changes those present conclusions.

Table 19.4. Preserve the meaning of disposition status

Status in the review	Meaning for the evidence or claim	Next action
Supported within scope	Applicable evidence and reasoning justify the specified proposition under identified conditions.	Preserve the scope and identify changes that require reconsideration.
Contrary evidence established	The stipulated or actual evidence, as applicable, contradicts a criterion or material proposition.	Investigate cause, correct or seek the appropriate disposition, and obtain new evidence without overwriting the earlier result.
Unresolved or inconclusive	Available information does not establish the required conclusion.	Identify the missing information, method or decision and its effect on the next commitment.
Planned or blocked	The relevant evaluation has not produced a result, or a prerequisite prevents meaningful execution.	Resource and complete the prerequisite/work; do not represent the plan as performed support.

The label supported should never be broader than the argument. It may apply to one interface property and not to the complete system. The label unresolved should not erase a definite favourable result on another proposition. Conversely, a narrowly supported property cannot be used to conceal a requirement the system has not met. The receiving authority needs both the progress and the limitations.

The case should remain maintainable. Identify which requirements, assumptions, interfaces and configurations each conclusion depends on. When they change, review the affected evidence and inference rather than automatically discarding everything or renewing every claim without analysis. An assurance case becomes valuable institutional memory when it explains why a conclusion was once reasonable and what would make it unreasonable now.

19.11 Exercise: a complete trace is not a complete case

Situation. A supplier submits a model with every requirement linked to a design element and a planned evaluation. Its dashboard reports complete trace coverage. The accompanying evidence

includes the stipulated E-05-03 late commit, but the supplier excludes that item from the summary because a corrective plan has been approved. The recovery profile remains unresolved. The proposed top-level conclusion is that the capability is ready for unrestricted operation.

Worked answer. The trace demonstrates recorded relationships to the extent those relationships are correct. It does not demonstrate that evaluations were performed or that their results support the claims. The late-commit item is contrary evidence for the stated condition and must remain visible. An approved plan identifies intended treatment; it does not establish corrected behaviour. The unresolved recovery profile also prevents an implemented recovery claim from being substantiated.

The summary should separate supported subclaims, adverse results and unperformed work. Connect each to the relevant configuration and decision. Review the corrective plan for adequacy and obtain the resulting evidence when the work is performed. Complete the recovery authority and task definition before treating recovery evaluations as executable acceptance work. Preserve useful existing results rather than declaring the whole dossier worthless.

The appropriate conclusion is not unrestricted operational readiness. It is a bounded account of what further work the evidence supports and which commitments remain unjustified. Chapter 20 then asks who can make those commitments, how remaining risk and obligations are represented, and why an authorized decision does not change the truth of the underlying technical findings.

Completed argument. Appendix C.7–C.9 carries a separate branch through a stipulated evaluation dossier and a bounded fictional acceptance decision. The original E-05 findings remain part of the development history.

Carry forward. An assurance case makes confidence reviewable by connecting claims, evidence, assumptions and contrary findings. It informs accountable decisions; it neither supplies missing protection nor grants the authority to accept a limitation.

20. Risk decisions, acceptance and authorization

20.1 A decision to proceed is not a new technical fact

A project reaches a difficult meeting. Some component evidence is useful, one receiving-side result is adverse, the recovery authority is unresolved and the delivery date is close. Someone proposes accepting the risk and closing the outstanding requirements. That proposal combines several different decisions. An authority may be entitled to permit a defined activity despite a known limitation. The decision does not make the failed requirement pass, establish missing evidence or satisfy an obligation that remains applicable.

The distinction is central to responsible engineering. A technical finding describes what the evidence supports. An assurance conclusion explains the resulting grounds and limits for confidence. A compliance conclusion addresses applicable assessed obligations. A contractual acceptance addresses a defined agreement and delivery. A risk decision concerns the consequences and treatment of uncertainty or exposure. An operational authorization permits a defined use under the relevant authority. The same person may perform more than one role, but the decision purposes should remain identifiable.

NIST's Risk Management Framework provides a public example of accountable authorization informed by assessment, risk response and conditions for continuing use. [10, Section 3.6 and Appendix F] Its governmental role names and powers do not automatically apply to another organization or jurisdiction. The general lesson used here is that evidence, decision authority and operating conditions must be explicit. The reader's governing arrangements determine who may decide and which obligations can be changed or excepted.

For the logistics case, no operational authorization is sought or implied by this chapter. DEC-05 is an original decision-support example based on the synthetic CASE-05 dossier. It preserves the absence of an approved recovery profile, actual production realization and performed VP/VA programme. Its purpose is to show how a decision can be useful and specific without manufacturing readiness.

Engineering is not the practice of refusing every uncertain commitment. Early work necessarily proceeds with incomplete information. The discipline is to match the commitment to the evidence, expose what remains uncertain and ensure that the people entitled to decide understand the consequences. A controlled investigation, component acceptance and unrestricted operational release can require materially different support even when they concern the same eventual system.

20.2 Name the decision and identify its subject

Before debating whether the project is ready, identify what is being decided. Accepting an adequate feasibility study is not accepting the feasibility of every design it examined. Receiving a software package is not accepting its conformity. Accepting a component within its contracted scope is not demonstrating the receiving organization's operational task. Payment, delivery, technical concurrence and permission to operate can be connected by an agreement, but they should not be treated as synonyms without examining that arrangement.

Table 20.1. Different decisions that may use the same evidence

Decision	Main question	Boundary to preserve
Technical finding or concurrence	What characteristics and limitations are supported by the identified evidence and engineering analysis?	Does not itself alter the governing requirement or grant permission to operate.
Assurance conclusion	What specified claims merit confidence, under which assumptions and conditions?	Does not create protection or erase contrary evidence.
Compliance determination	Were the applicable requirements or criteria met within the assessed scope?	Does not automatically establish all system properties or intended uses.
Contractual acceptance	Has the defined delivery met the acceptance provisions of the governing agreement?	Requires the actual agreement and authorized participants; may concern an artefact or component, not the complete mission.
Risk disposition	Which exposure or uncertainty will be avoided, reduced, retained or otherwise addressed, by whom and under what conditions?	A decision to retain exposure does not turn an unperformed or failed evaluation into a pass.
Authorization for a defined activity	May this configuration be used for this purpose in this environment under specified conditions?	Its scope and authority must be established; permission for a trial is not unrestricted production permission.

The subject should include the relevant configuration and use. “Approve the gateway” leaves unclear whether the decision covers a supplier product, its configured deployment, the receiving coordinator, the recovery branch or the complete administration service. Identify the boundary, supported modes, users, environment and interfaces necessary to understand the commitment. A named product can exist in several configurations with different authority and persistence behaviour.

The timing of the decision also matters. A design review can support proceeding to a prototype while still requiring evidence of implementation later. A test-readiness decision can permit a controlled evaluation without making the evaluated property true in advance. A transition-readiness decision needs the receiving arrangement, including people and support. These commitments should be distinguished in project controls so that a permission granted for one purpose is not silently reused for another.

When the meeting contains several decisions, record them separately. The customer may accept ED-05-01 as a satisfactory analysis while declining to select the complete recovery architecture. It may authorize further laboratory work after the necessary safety, access and test-environment conditions are established, while withholding operational use. Such a result is not contradictory. It is an appropriately differentiated decision based on what has and has not been obtained.

20.3 Distinguish nonconformance, exposure and missing confidence

A requirements discrepancy is not identical to a risk statement. E-05-03 stipulates evidence of a late revoked-identity commit that violates SR-03 under the stated conditions. The technical

finding concerns that criterion. The associated risk statement explains how continued use could allow unauthorized configuration changes, what consequences may follow and under which conditions. Correcting the implementation can address the discrepancy; evaluating the risk requires understanding its operational significance and remaining uncertainty.

Missing evidence creates a different situation. If persistence has not been evaluated, the no-second-commit property remains unsubstantiated. That does not establish a duplicate-commit defect. It does create uncertainty relevant to relying on the capability, especially where a failure would have serious consequences. The response can be to obtain discriminating evidence, constrain the use, change the design or defer a commitment. It should not misreport unknown behaviour as either demonstrated safety or demonstrated failure.

An unresolved authority profile is different again. REC-P1 is not complete enough to define the permitted recovery arrangement. A controller may be technically capable of executing a restoration without the normal identity service, but that capability does not determine which identity, approval and withdrawal rules are legitimate. The project needs the responsible authority's decision and an implementable profile before it can evaluate conformance to that arrangement.

Late or incomplete evidence can create a contractual discrepancy and leave a technical claim unsupported at the decision date. Identify the missing contribution and its effect. The actual agreement and authorized participants determine the contractual response; the engineering record should not invent a remedy or assign blame before the facts are established.

These distinctions prevent inappropriate closure. A risk register cannot serve as a disposal bin for requirements failures, missing information, supplier obligations and policy questions. It can reference all of them, but should preserve the different owners, evidence and decisions needed. A concise issue taxonomy is useful when it directs the right work; it is harmful when it makes unlike problems appear resolved through the same administrative status.

20.4 Describe current and intended residual risk separately

A risk record identifies the capability, causal scenario, affected outcomes, current measures and limitations, and accountable owner. Distinguish established facts from assumptions. Apply the organization’s risk method with a rationale another qualified person can follow; the logistics example supplies no universal numerical scale.

In particular, separate the current residual position from the intended position after proposed treatment. A funded correction, approved plan or scheduled test can increase confidence that work will occur. It does not demonstrate the correction's effectiveness. NIST's RMF keeps assessment findings visible where risk is accepted and distinguishes planned remediation from completed, assessed response. [10, Section 3.6, Task R-3; Section 3.7, Task M-3]

RR-05 connects a risk statement to the stipulated late-commit finding. It illustrates the record’s contents without assigning an operational rating, invented probability, monetary loss or executive acceptance.

Table 20.2. RR-05: a scoped risk record for the hypothetical late-commit finding

Field	Teaching-record content
Capability and scenario	NORMAL-mode dispatch-rule activation; LS-01 concerns stale authority remaining effective after revocation.
Technical basis	E-05-03 stipulates a revoked-identity commit in the compatible interval 60.4–62.0 seconds after t0 under applicable conditions. SR-03 is not met in that evaluation.
Consequence of concern	Unauthorized change can affect the administration contribution and potentially the operational instructions that depend on the changed rules. Wider dispatch consequences require the service owner's analysis.
Current position	The stipulated adverse finding remains open. Scope across paths/configurations and causal mechanism require investigation; no actual operational risk rating is assigned in the book.
Proposed treatment	Investigate source-to-commit behaviour; correct the affected mechanism or allocation; evaluate SR-03/SR-05 together and perform justified regression work.
Intended position after treatment	A target to be assessed after applicable implementation and evaluation evidence exists, not a current residual-risk conclusion.
Accountable and action roles	A designated owner of the affected operational consequence; technical owners for the affected implementation and evidence; the relevant authority for any permitted use or requirement disposition.
Reconsideration basis	New receiving path, changed source semantics, corrected realization, contrary test result, altered mission use or expiry/failure of an imposed condition.

The technical owner and risk owner can be different. A software team can correct a queueing defect without being entitled to accept the service owner's operational consequence. A service owner can understand the mission impact without being qualified to determine whether a storage change preserves history. Identify the contributions and decision rights rather than expect one role to supply every competence.

Risk aggregation also requires care. Several findings may share one cause, such as an unbounded receiving queue. Counting them as independent risks can overstate diversity of exposure, while collapsing them into one vague item can hide different consequences. Parent and related records can preserve the common cause and distinct affected claims. Do not average a critical authority weakness into an acceptable portfolio position merely because many unrelated requirements have passed.

Uncertainty should be actionable. Record which unknown could change the decision and what information would reduce it. If the writer set is incomplete, obtaining the actual privileges and deployment paths may have greater decision value than refining a low-impact performance estimate. This connects risk work to engineering priorities rather than treating risk reporting as a parallel activity after the design has been selected.

20.5 Compare courses of action against the complete consequence

A decision needs alternatives, not only a recommendation to proceed or stop. Depending on the maturity and governing arrangements, alternatives can include continued investigation, correction before use, a different architecture, restricted use, a supported interim capability or retention of the existing service. Each has its own risks, resources, delays and evidence needs. There is no universal rule that doing nothing is safe or that deploying the new design is necessary.

For CASE-05, unrestricted operation is not supported by the current synthetic dossier. That does not imply that all engineering work must halt. The project can pursue the receiving-side correction, reconcile writer coverage, repair the model export and resolve the recovery authority arrangement. Those activities can be scoped and resourced before any operational decision. A controlled prototype investigation should have its own permitted environment, access, stop conditions and information boundaries.

A proposed restricted deployment needs more than a narrower label. If recovery is excluded, the project must examine whether the intended use still requires SR-11 and what other arrangement provides that capability. Saying “NORMAL mode only” does not satisfy the unchanged restoration need. An alternative operational arrangement might be acceptable after proper needs, requirements, technical and authority decisions, but none is assumed in the base case.

Compare the consequences of delay honestly. The existing system may have known weaknesses or support limitations, and an extended transition may preserve an additional writer or incompatible client. Those are facts to establish, not reasons to invent an emergency. Where the current system's risks are unknown, record that uncertainty rather than assume the new system must be better because it has more recent technology or more documentation.

NASA's decision-analysis treatment relates alternatives, criteria, assumptions and uncertainty to an explicit recommendation. [6, Section 6.8] The case's application should preserve binding obligations before comparing preferences. A cost or schedule advantage cannot silently compensate for a mandatory condition that the relevant authority has not changed or excepted. Where an exception is permitted, record who can grant it and how its consequences alter the decision basis.

A limited study may be the highest-value immediate action. The Part III trade already showed that ARC-B and ARC-C cannot be selected responsibly from a small preference score while authority and operational feasibility remain unresolved. Part V adds the acquisition and evidence work that can resolve those uncertainties. The decision should state what information the next investment is expected to obtain and what outcome would change the direction.

20.6 Define conditions that people can enforce and review

Conditional permission is useful only when its conditions are intelligible, achievable and connected to the risk or uncertainty being managed. “Proceed provided security is maintained” supplies no practical instruction. The receiving organization needs to know which configuration and activities are permitted, who checks the condition, how deviations are detected and what response is required.

For a hypothetical controlled investigation of RZ-01, the proposed conditions might require an isolated, authorized test environment; controlled credentials and data; no connection capable of

changing operational dispatch configuration; an identified model and test-article baseline; and a plan for handling unexpected effects. Those are proposed prerequisites for a separate trial decision. The book does not assert that the conditions have been implemented or that a trial has been authorized.

For an eventual operational decision, conditions would need the actual realized system, supported modes, defined recovery authority, known writer set and continuing support. A restriction tied to staffing must be reflected in a feasible staffing arrangement. A condition requiring approved packages to remain available must identify who maintains eligibility and how loss of readiness is reported. A condition nobody can observe or satisfy is not a reliable mitigation.

Table 20.3. Turn vague conditions into reviewable obligations

Vague statement	Information a usable condition needs	Why it matters
Use only in a safe environment.	Identified environment, permitted connections/actions, responsible custodian and stop conditions.	Prevents a bounded investigation from silently becoming operational use.
Maintain the approved configuration.	Controlled realization identity, permitted changes, verification of relevant installed state and discrepancy route.	A product version does not identify every security-relevant setting or dependency.
Recovery must remain available.	Completed authority profile, eligible resources, staffing, readiness checks and response when a prerequisite fails.	A controller's uptime does not establish the complete fifteen-minute task.
Fix the finding promptly.	Defined correction, responsible action owner, resources, agreed milestone and evidence needed for closure.	A promise without a method or receiving decision does not establish treatment effectiveness.
Review when circumstances change.	Specific triggers, monitoring responsibility, notification route and the commitment requiring reconsideration.	Makes conditions operational rather than hidden prose in an approval memorandum.

Specify how long the decision remains applicable or what ongoing review arrangement governs it. An expiry date can force reconsideration, but it does not protect the system before that date if a material assumption fails. Conversely, a calendar review should not imply that every unchanged evidence item is worthless. Use event and condition triggers alongside the governing review cycle.

Do not let a condition create a new unexamined failure. Automatically disabling all administration when a report is late may reduce some exposure while preventing required restoration. A monitored restriction needs a designed response that considers the useful capability and relevant consequences. Where a protective refusal prevents the required task, record the service limitation and escalation arrangement rather than call every refusal a successful resilience outcome.

20.7 Accept a limitation without falsifying the record

An authorized decision may retain a known exposure where the governing framework permits it. The record should identify the exposure, evidence, rationale, scope, conditions and accountable authority. It should also state which requirement or claim remains unmet or unsupported. Risk acceptance is not a method for changing historical test results or making a contradictory model correct.

For E-05-03, a decision to permit some defined activity despite the stipulated late-commit behaviour would not change the finding that SR-03 was violated in that evaluation. A new requirement baseline or an authorized deviation might change the criterion for a future commitment where permitted, but the original criterion and result remain part of the history. The technical and contractual consequences of such a change require their proper participants.

Similarly, accepting an uncertainty about an unexamined path does not establish that the path is secure. It records a decision to rely despite the stated uncertainty under specified conditions. The quality of that decision depends on an intelligible explanation of the consequences and alternatives. A phrase such as “low probability” is insufficient when no basis for that characterization has been established.

Not every obligation is within a local risk owner's power to alter. Identify the source and relevant authority before proposing a waiver, deviation or exception. The actual law, contract or policy determines the available mechanism. This book does not supply jurisdiction-specific permission. An engineering reviewer can explain the technical effect of a proposed departure without asserting legal authority to approve it.

Risk transfer also needs a bounded meaning. Contractual remedies, insurance or supplier obligations may allocate certain costs or responsibilities, subject to their terms. They do not restore a missed operational outcome or make a failed protection mechanism function. The technical consequence still needs an owner and response. A supplier's commercial commitment is relevant information, not a substitute for the receiving system's capability and evidence.

The value of an acceptance record is its precision. It allows later personnel to distinguish a known limitation consciously retained from a claim believed to have been demonstrated. That distinction matters when conditions change, an incident occurs or a new project seeks to reuse the evidence. Concealing the limitation to simplify approval removes the information needed for responsible continuing use.

20.8 Keep technical dissent and decision accountability visible

A technically difficult decision can involve legitimate disagreement about consequences, evidence sufficiency or feasible alternatives. The record should distinguish those disagreements. A service owner may accept a disruption that a specialist initially assumed intolerable; an engineer may identify a failure mode the owner did not understand; a reviewer may question whether a simulated result transfers to the intended environment. None is resolved merely by seniority or by counting votes in a review meeting.

Ask what information would change each position. If the disagreement concerns the meaning of commit, resolve the interface definition. If it concerns staff availability, obtain representative task and resource evidence. If it concerns an authority's permitted exception power, obtain the

relevant governance determination. Some disagreements may remain matters of judgement under uncertainty, but the basis of the judgement should be explicit.

A decision authority should receive material contrary evidence in a form it can understand. Technical detail can be attached or referenced, but the summary must not say that all requirements passed when a critical result is adverse or work remains unperformed. The executive explanation should identify the useful capability, the consequence at issue, the evidence limit, the available options and the recommended commitment. Precision does not require reproducing every test trace in the decision memorandum.

Technical participants also have a duty to keep their own conclusions within evidence. A possible weakness should not be presented as a proven compromise to force a preferred architecture. An unexamined supplier should not be accused of misconduct because its report is insufficient. Accurate negative and uncertain conclusions are stronger than exaggerated warnings because they allow the decision authority to identify what action is actually necessary.

Where a decision departs from a technical recommendation, retain the recommendation and the reasoned decision, with the authority and conditions. The record should not be rewritten to imply unanimous technical concurrence. This preserves accountability without requiring every specialist to own the final operational judgement. It also provides a basis for reconsideration when an accepted assumption changes or a predicted consequence becomes more significant.

20.9 Develop a decision brief from the same controlled information

DEC-05 illustrates how the case can be summarized without abandoning technical distinctions. It is an original, unsigned teaching brief for the synthetic dossier, not an authorization instrument or a record of an actual review board. It uses the existing claims and findings so that the executive reader can trace each material statement to its underlying basis.

Decision requested. Determine the next bounded development commitment for the remote-administration capability. The present material does not support unrestricted operational use or selection of a complete recovery architecture. Consider authorizing the defined investigations and correction work only after their separate environment, access and resource prerequisites are established.

Required capability. The capability must support authorized dispatch-rule maintenance and a restricted restoration task within fifteen minutes of a declared corrective need during an identity outage of up to two hours. It must preserve the applicable authentication, authority, transaction, state and outcome obligations. The selected system boundary does not establish complete dispatch-policy correctness or mission success.

Present support. The conceptual work identifies receiving-side coordination, explicit transaction and mode history, and two conditionally plausible recovery alternatives. The hypothetical dossier contains a favourable observation for one normal activation, but also a stipulated late revoked-identity commit, unresolved writer coverage, a non-equivalent model export and an approval dependency incompatible with the stated outage. Proposed corrective work is not evidence of corrected behaviour.

Recommended commitment. Obtain the agreed supplier and customer contributions needed to resolve source-to-commit timing, reconcile every relevant writer, establish adequate persistent-state behaviour, correct the affected exchange and complete the permitted recovery

authority and task arrangements. Retain ARC-B and ARC-C as alternatives for the targeted investigations. Do not select an operational architecture from the illustrative preference scores alone.

Decision boundaries. Accept a completed analysis deliverable only against its actual agreed content and quality criteria; do not treat that acceptance as acceptance of the resulting product. Permit laboratory or prototype work only under separately established limits. Any future operational proposal must identify the realized configuration, applicable results, remaining limitations, receiving resources and the authority for the intended use. A narrower trial scope does not waive the unchanged production requirements.

Accountability and reconsideration. Assign the technical owners of timing, state, interface and exchange work; the customer's policy and operational participants for recovery authority and staffing; and the integrating lead for the combined evidence. Reconsider the direction when critical source semantics change, another writer is found, a candidate recovery task proves infeasible or new evidence contradicts a relied-upon property. Preserve adverse results and their correction history.

A real brief would add the actual options, costs, dates, resources, named roles and governing authorities. Those facts are not supplied for this fictional project, so the book does not fabricate them. The example demonstrates the information structure and disciplined conclusion that the actual decision would need. Its brevity is useful because the detailed technical and evidence records remain accessible and consistent with the summary.

20.10 Sustain the decision after the meeting

The decision record becomes an operational input. The receiving organization must know which configuration and modes are permitted, which limitations remain, who monitors the conditions and what changes require reconsideration. A decision hidden in an inaccessible project repository cannot reliably govern an on-call administrator's recovery action. Make the relevant conditions available in the procedures and information used to perform the work, without scattering sensitive details beyond their permitted audience.

Link the decision to the controlled realization, models, evidence and risk records. A change to a receiving endpoint can affect C-02 and C-03. A changed recovery roster can affect C-04 without altering code. A model-tool update can affect C-05 if an important calculation or status transformation changes. The change process should identify the affected conclusion and the scope of re-evaluation needed, not merely ask whether the product name remains the same.

Conditions also require an effective response. If an eligible capsule is no longer available, the organization needs a way to report lost readiness, restore it or impose the appropriate service limitation. If a newly discovered writer challenges coverage, the technical and operational participants need to determine its influence and any interim action. An alert is useful information; it is not itself risk treatment unless someone or something can take the required action.

NIST's authorization and monitoring treatment connects ongoing risk decisions to changes, assessment results and the continuing applicability of the authorization basis. [10, Section 3.7 and Appendix F] The case applies that relationship through the existing configuration and

service-management workflows. It does not require every change to receive the same review depth, nor permit every new configuration to inherit an earlier approval without analysis.

The organization should learn from the decision process as well as the technical system. Repeated late evidence, recurring exceptions, unstaffed recovery assumptions or contradictory authority roles indicate problems in acquisition and governance. Addressing those patterns can improve future engineering work. The detailed organizational treatment appears later; the immediate obligation is to preserve enough rationale and outcome information to make that learning possible.

20.11 Exercise: accept the risk and close the failed requirement

Situation. A senior manager reviews the hypothetical E-05-03 late commit and says, “We accept the risk. Mark SR-03 as passed and close the finding so the dashboard reflects the decision.” A correction is scheduled, but no new evaluation has occurred. The same decision note refers to RECOVERY as approved even though REC-P1 remains undefined.

Worked answer. Separate the proposed decisions from the technical status. Under the stipulated evidence, SR-03 was not met in the evaluated condition. An authorized decision to retain the associated exposure would need a defined activity, scope, rationale, conditions and authority. It would not convert the result into a pass. Preserve the finding, record the risk decision separately where permitted, track the proposed correction and obtain applicable evidence before claiming corrected behaviour.

The recovery statement has another defect: an undefined profile does not identify the authority, action set and conditions being approved. Resolve those matters with the relevant policy, technical and operational participants. A manager's intention to support recovery cannot establish that the chosen mechanism satisfies POL-01 or that designated staff can complete the task. Neither the correction schedule nor the decision note supplies the missing implementation and validation evidence.

A clearer dashboard can distinguish technical satisfaction, evidence maturity, risk disposition and permission for the next activity. The fields should use shared identifiers so that the relationship remains traceable, but should not overwrite one another. The point is not to make reporting more bureaucratic. It is to prevent a decision about consequences from being misremembered as a fact about system behaviour.

Carry forward. Part V connects acquisition obligations, supplier contributions, assurance arguments and accountable decisions. Part VI turns to operation, response, configuration and change, legacy and cloud-dependent systems, and eventual retirement. The system must retain not only the capability delivered, but the conditions, information and responsibilities that make continued reliance defensible.

PART VI — SUSTAIN TRUSTWORTHY CAPABILITY

21. Operations, response and continuing support

21.1 Continue the capability, not only the equipment

At handover, a project transfers more than an executable release. It transfers a way of operating: who may perform which actions, what the system depends on, how an uncertain outcome is resolved, which conditions permit continued use and what support is available when those conditions fail. A functioning gateway is only one contribution to that arrangement. The service can remain reachable while an expired recovery credential, an unavailable approver or an unrecognized configuration change makes the required operational task impossible.

Operation is the performance of the intended service using the people, procedures, technology and resources provided for it. SSE contributes by keeping the security-relevant assumptions, limits and findings connected to that work. It does not turn every operator into a design authority or reclassify every routine service activity as an engineering project. An administrator following a valid, evaluated maintenance procedure is performing an operational or support function. A discovery that the procedure cannot preserve required authority or state calls for engineering attention as well as an immediate operational response.

NIST's operation process connects preparation, qualified resources, secure modes of operation, monitoring and feedback to the system's continuing use. Its maintenance process addresses the related work of sustaining or restoring capability. [1, Appendices H.12–H.13] The distinction helps preserve responsibilities without creating a handover boundary beyond which engineering no longer listens. Operational findings can reveal an inadequate requirement or design assumption; engineering findings can change the limits within which operation remains justified.

TE-06 examines what an operating arrangement would require. It does not deploy the unresolved case: DEC-05 remains unsigned, the late-commit finding remains adverse and recovery authority remains open. The in-service situations are separate conditional examples.

That boundary also preserves the wider mission. The administration capability can report which package was committed at its management interface. It cannot alone determine whether every operational consumer uses that package or whether the dispatch policy is correct. During operation, those distinctions become practical coordination duties. A support team may restore the administrative function while the dispatch owner still has to reconcile instructions issued during the disturbance. Restoring a technical contribution and restoring the mission outcome are connected but different tasks.

21.2 Turn decision conditions into usable operating information

A lengthy acceptance package is not an operating instruction. The on-call team needs a concise account of the permitted configuration, available modes, dependencies, support arrangements and actions required when a condition is no longer met. It also needs a route to the underlying engineering information when an unexpected situation requires deeper analysis. The aim is a usable interface between the decision record and daily work, not a second, independently maintained copy of the whole assurance case.

OP-06 connects decision conditions to everyday work through a service record, handbook, runbook or linked configuration information. Identify each condition's source and maintainer. Where a source such as REC-P1 is unresolved, mark the dependent capability as unestablished; publishing its runbook cannot supply the missing definition.

Table 21.1. OP-06: information an operating team must be able to use

Operating question	Information to carry forward	Responsible contribution and limit
What arrangement may be used?	Identified realization, applicable decision, permitted modes and user/task scope; link to RZ-01 and the controlling requirements.	The operating custodian maintains the installed account; the designated authority determines permitted use. A product name alone is insufficient.
What must remain available?	Identity and time contributions, protected state, eligible packages, people, facilities and support needed for the stated task.	Service and support owners maintain the contributions. Reachability of one endpoint is not evidence of complete task readiness.
What does an abnormal indication mean?	Defined observation, relevant condition, urgency, response, escalation and the limits of the measurement.	Monitoring and technical owners establish interpretation; operators act within their assigned authority.
How is an uncertain action resolved?	IC-01 outcome-query meaning, transaction identity, current versus historical configuration, and escalation when history cannot be established.	Application and operations teams preserve the shared semantics. A timeout is not proof that nothing changed.
What requires reconsideration?	Changes or findings affecting requirements, evidence, support or decision conditions, with a route to the responsible review.	Engineering and decision owners assess impact; a change ticket does not itself amend an operating authorization.

Consider the condition that an eligible restoration capsule remains available. The useful operating entry does not merely repeat that phrase. It identifies the target, preparation source, eligibility check, person or service responsible for replenishment, evidence of the most recent check and response when readiness is lost. The entry must also distinguish a damaged package from one that is intact but no longer approved for the current generation. Those conditions can require different technical work and different operational restrictions.

The information needs a controlled update path. If a project changes the permitted target set, the service catalogue, access arrangements and runbook must reflect that decision before users rely on the new scope. Conversely, discovering that operators use an unlisted target should not cause an inventory synchronization job to expand the approved scope automatically. Observed use is evidence of actual activity, not proof of permission. Preserve the discrepancy and obtain the appropriate technical and authority decision.

Make operating information available under the conditions in which it is needed. A recovery instruction stored only behind the unavailable identity service has the same dependency problem as the approval portal in E-05-06. A protected local copy can be useful, but needs version, access and withdrawal arrangements. Unlimited uncontrolled copies create a different

problem: a maintainer may follow a superseded instruction whose assumptions no longer hold. The design should reconcile availability with the need to identify which instructions control.

21.3 Specify monitoring around a decision and a response

Monitoring becomes useful when an observation can be interpreted and acted upon. Begin with the condition or claim being maintained, identify what can reveal a change, determine how the observation is obtained, and specify the action it can support. This sequence is more discriminating than collecting every available log and hoping that a dashboard will explain the system later.

NIST relates monitoring frequency to decision needs, including event-driven assessment; continuous monitoring need not measure every property uninterrupted in real time. [41, Executive Summary and Section 3.2.2] Distinguish runtime enforcement, operational detection, periodic inspection and engineering review according to the required response and how the relevant condition changes.

For the administration case, an observation of authority-status age may support investigation of SR-05, but only if the timestamp refers to the authoritative observation and the time relationship is understood. A record saying that the gateway refreshed its cache does not establish the age of the status used by the receiving coordinator at commit. Similarly, a successful connection to the identity service does not establish that its response is valid, current or suitable for the requested action.

MON-06 describes selected monitoring questions rather than an implemented monitoring system. Each signal must be designed and evaluated for its intended use. A detector cannot be credited with coverage simply because the condition it is meant to detect appears in a requirement record.

Table 21.2. MON-06: relate signals to claims without enlarging their meaning

Condition of concern	Candidate observation	Interpretation and response boundary
Normal authority is no longer demonstrably fresh.	Source-observation time, receiving evaluation and commit records with a justified time relationship.	Can challenge SR-05 in the represented path; missing or unreliable time produces uncertainty, not an automatic pass. Runtime enforcement remains necessary.
Recovery preparation is no longer eligible.	Target generation, permitted package reference, approval state and preparation/expiry information.	Initiate readiness correction or an appropriate service limitation. A notification cannot make the instruction eligible.
A new effective-change path appears.	Reconciled deployment privileges, interface inventory and relevant receiving observations.	Reopen the affected A-01 coverage argument. The discovery is not by itself proof of malicious activity.
Transaction outcome cannot be reconciled.	Identified request, historical result, current target state and persistence-service condition.	Prevent unsupported assumptions about retry or rollback; follow the defined reconciliation and escalation procedure.

Condition of concern	Candidate observation	Interpretation and response boundary
The monitoring channel is itself impaired.	Expected source activity, collector health, queue condition and evidence of successful end-to-end delivery.	Mark affected visibility as limited. Silence from a failed collector is not evidence of healthy operation.

The last row is essential. Monitoring depends on sources, collection, time, transport, processing and presentation. An expired collector credential can remove visibility while the application continues to serve users. A queue can deliver correct records too late for the intended response. A parser change can preserve a message while dropping the field that distinguishes receipt from commit. These are enabling-system failures with consequences for the assurance argument, even where they do not directly cause a prohibited activation.

A monitoring record should therefore preserve event time, observation or collection time, the relevant configuration and the transformation used to interpret the data. The system may need to tolerate a bounded interruption in collection, but that tolerance must be derived from the decision and consequence. Delaying a report and losing the underlying evidence are different conditions. A buffered record can support later investigation without supporting a response that had to occur before the buffer was drained.

21.4 Examine detection and response delay explicitly

A detective measure needs a timing argument when the consequence depends on how long a condition persists. Suppose an original teaching design polls a persistent condition every ninety seconds. Allow twenty seconds to deliver the observation, fifty seconds for a designated responder to interpret it and twenty seconds for the selected action to become effective. If the condition begins just after a successful poll, the proposed maximum interval is ninety plus twenty plus fifty plus twenty, or 180 seconds.

That is a conditional calculation, not a measured service level. It assumes the condition persists until observed, the polling interval is bounded, the monitoring chain remains available, the responder is ready and the action has the stated effect. It excludes a transient event that occurs and disappears between polls unless another mechanism records it. It also excludes repeated transport failure or a responder queue longer than the allocation. Each excluded condition can matter to the actual protection claim.

The calculation immediately answers one question: this arrangement does not, by itself, establish SR-03's sixty-second restriction on commits under revoked authority. Even under its own favourable assumptions, it may act much later. It can be a valuable supplementary detector or investigation aid. It cannot replace the receiving-side preventive property merely because the organization calls the dashboard continuous monitoring.

Do not confuse the requirement's interval with an incident-response target. SR-03 starts at the authoritative revocation event and concerns a commit. A security team's target may start when a relevant alert is received and concern acknowledgement or containment. Meeting the second target does not demonstrate the first. Both measures can be useful when their starting events, end events and scope are preserved.

For the same reason, an observation that response took less than one minute after an analyst opened the alert does not establish that the exposure lasted less than one minute. Event

occurrence, source recording, collection, detection, triage and effective action can be separated by substantial unmeasured intervals. The engineering task is to identify the intervals necessary for the claim, not add every timestamp to a report without explaining what it represents.

The response also needs a consequential boundary. Disabling a gateway account may prevent new admissions while a previously accepted request remains capable of committing at the receiver. For the LS-01 scenario, the response must reach whatever still holds effective authority. If a protective action changes only one path, record the other paths and the limits of containment. A fast action at the wrong boundary is not a timely system-level result.

21.5 Separate an anomaly, an incident and an engineering problem

An anomaly is an observation that differs from the expected account. It may arise from an attack, an authorized change, a defect, a misunderstanding of state or a fault in observation. An incident determination applies the organization's response criteria to the available facts. An engineering problem identifies a condition requiring technical investigation or correction. The same event can generate all three records, but their conclusions should not be copied from one another without the necessary evidence.

Suppose a hypothetical monitoring extract shows an activation attributed to a person whose authority was withdrawn. The first questions concern the identity, authoritative withdrawal event, actual commit event, paths, configuration and timing. A record timestamped after revocation might describe delayed collection of an earlier commit. Conversely, a record showing timely login rejection might omit a later queued commit. Treat each explanation as a hypothesis until the observation and its meaning are established.

Operational response need not wait for perfect causal certainty. The responsible team can take a permitted, proportionate containment action where the consequence justifies it, while recording the facts and uncertainty that support the action. The engineering investigation can continue in parallel. Avoid the opposite errors of postponing every response until a complete root cause is known, or presenting a cautious response as proof that a compromise occurred.

The Chapter 19 dossier provides a useful contrast. E-05-03 explicitly stipulated a late commit under applicable conditions; it was not merely an ambiguous alert. Its interpretation remains adverse within that hypothetical exercise. TE-06 does not reopen that result as harmless without new support. What remains unknown is the cause, broader scope and effectiveness of a future correction. Keeping those questions separate prevents both premature closure and unsupported expansion of the finding.

An operational problem record should link the observed condition to the affected requirement, scenario and configuration where those links are known. Where they are not, record the discovery work. “Investigate whether the deployment writer participates in the authority model” is more useful than assigning every unusual observation to generic configuration drift. The investigation may reveal a new scenario that needs requirements or architecture work, rather than a departure from an otherwise adequate baseline.

21.6 Connect incident response to the engineering baseline

Incident response and engineering have complementary purposes. The response team coordinates urgent detection, analysis, containment and recovery under its authority.

Engineering specialists explain relevant dependencies, state behaviour and the consequences of proposed actions. The service owner evaluates operational impacts. Legal, privacy and other organizational functions address their own obligations where applicable. No single team should acquire all those responsibilities because it owns the ticketing system.

The 2025 NIST incident-response publication places response within wider cybersecurity risk management and treats preparation and improvement as continuing organizational contributions. [42, Sections 2.1–2.2] Here the practical extension is to make the engineering baseline usable during response: the relevant writers, authority sources, transaction states, recovery assumptions and support contacts should be accessible without requiring responders to reconstruct the design during a crisis.

Containment can alter the system that remains in operation. Isolating the identity service may protect against its suspected misuse while preventing normal maintenance. Revoking a deployment credential may remove an unexamined writer while also disabling a recovery action. Stopping the receiving coordinator may preserve evidence but leave the operational service using an uncertain configuration. Those consequences do not make containment wrong; they require an explicit account of the chosen action, the capability retained and the conditions for the next step.

Preserve evidence where feasible without claiming that forensic completeness always outranks an urgent protection need. A volatile state capture, configuration manifest or transaction record can help explain what happened. The permitted method and priority depend on the environment and the consequences of delay. Intrusive collection on a cyber-physical system may itself affect timing or safe operation. The responsible specialists must determine an appropriate approach rather than importing a workstation response procedure unchanged. [21, abstract, for the distinctive performance, reliability and safety context of OT]

After immediate control is established, feed findings back to the design. A repeated need to disable a protection mechanism to restore service may reveal an inadequate recovery architecture. A recurring inability to identify authoritative state may expose a missing observation requirement. An incident that exploited a permitted interface can challenge the original threat or loss analysis even where the implementation met its documented requirements. The response record should preserve these distinctions so that corrective work addresses the actual weakness.

21.7 Restore a basis for reliance, not merely a responsive process

Recovery has at least three questions in this case. Can the required administrative function operate? Is its configuration and supporting state suitable for the intended use? Can the relevant people establish and exercise permitted authority? A restarted process answers none of those questions completely. It may be responsive while using an obsolete generation, uncertain transaction history or a credential arrangement no longer justified after compromise.

The recovery plan should identify what must be established before new state-changing work resumes. For the receiving coordinator, that includes the active package, transaction-suppression state, historical outcomes and mode epoch. A restored package can be an acceptable content choice while a restored security-history snapshot is not. Reintroducing an old snapshot without reconciling later commits can make a previously used transaction appear new. The storage and migration treatment in Chapters 10 and 14 remains relevant during operational recovery.

Distinguish outage recovery from compromise recovery. If identity service was simply unavailable, restoring its agreed operation may make normal authentication possible again. If its authority source is suspected of issuing false information, restored connectivity does not resolve that concern. The investigation may need a different source of confidence, credential replacement or constrained use while integrity is established. The exact response belongs to the actual incident and governing arrangements; the outage-only ARC-B and ARC-C proposals are not general compromise-recovery guarantees.

An operator also needs an intelligible account of what happened during the interval. A transaction committed in recovery remains part of history after NORMAL mode returns. It must not disappear because the normal client uses a different record store. An action still pending at transition must either be resolved or evaluated under the destination context before it can have an effect. The user interface and support procedure should make these states understandable, not leave the operator to infer them from a successful login.

Return to use is a decision with a stated scope. Technical staff establish the relevant findings; operations confirms the receiving resources and task; the designated authority determines the permitted commitment. A restoration that supports read-only diagnosis may be useful while state-changing administration remains restricted. That restriction does not satisfy the unchanged SR-11 requirement unless the complete required restoration contribution is provided through an acceptable arrangement. Preserve any remaining service limitation explicitly.

21.8 Maintain support as part of readiness

Support capability includes people, tools, information, facilities, replacement items and access to external assistance. A component can remain supportable in a catalogue while the organization lacks the credential, compatible diagnostic tool or specialist needed to use the support. This is why support readiness should be checked against a task rather than inferred from an active maintenance subscription.

In the case, the earlier thirteen-minute ARC-B allocation assumed two minutes to mobilize designated personnel. A roster change, concurrent incident or inaccessible facility can invalidate that assumption without changing one line of code. The appropriate response is to assess the task and support arrangement, not to declare a software defect. Conversely, retaining the original task estimate in the assurance case after the assumption changes misrepresents operational readiness.

Supplier support arrangements must preserve the boundary between advice and customer decisions. A supplier may identify an affected release, recommend a setting or propose a restoration method. The receiving organization still needs to determine whether the advice applies to its configuration, interfaces and operational conditions. Supply-chain notification and support information should identify the relevant contribution and change, as developed in Chapter 18. [32, overview and Appendix A, SR-8; conceptual basis carried forward]

A temporary workaround requires an owner, permitted scope, reason, observation method and an end or reconsideration condition. It can be the least harmful available response while a proper correction is developed. It should not become invisible simply because it succeeds operationally. Repeated reliance may reveal that the nominal operating concept is no longer realistic. The maintenance and change processes then need to resolve the discrepancy between the official design and the way the service is actually sustained.

At handover, ask the receiving maintainer to locate the governing interface, distinguish historical outcome from current state and identify what blocks recovery. This tests usable information, not memory. When those answers depend on the original designer, capture the missing explanation and provide appropriately protected access under the intended support conditions.

21.9 Keep observation, models and assurance in a controlled relationship

Operational information can update the understanding of a system without automatically changing its approved definition. Maintain three distinguishable accounts: the intended or authorized arrangement, the identified installed configuration, and observations of its behaviour. MBSE can connect these accounts through stable identities, but should not merge them into an undifferentiated “live truth” that erases discrepancies.

For MS-01, a new observed writer should be linked to the allocation and coverage concern. It should not silently receive a satisfy relationship to the authority requirements. A timing observation should retain the configuration, conditions and uncertainty under which it was obtained; it should not overwrite the model's allocated bound merely because it is more recent. When a model changes, retain the earlier analysis and identify whether it still supports the decision for which it was used.

Table 21.3. Route operational learning without rewriting the evidence

Information received	Immediate engineering use	Record that must remain distinct
Installed configuration differs from the authorized account.	Identify the difference, permitted change history and effect on requirements or use.	Observed state does not automatically become the approved baseline.
A timing observation exceeds a modelled allocation.	Check measurement meaning, actual conditions and the affected analysis; investigate cause.	A prediction, allocation and observation are not interchangeable values.
A support task cannot be completed with the rostered resources.	Reassess task assumptions, required staffing and operational restrictions.	Technical component evidence can remain useful while task readiness is unestablished.
A model export misstates a result or its status.	Review the transformation and affected decisions, preserving source and output versions.	Correcting the report is not evidence that a runtime defect was corrected.

Measures should support those decisions rather than reward administrative activity alone. Ticket closure time can describe response workflow but not necessarily restored protection. Patch coverage can describe installation progress but not whether a critical, unpatchable interface remains exposed. Counts of valid capsules can describe preparation inventory but not whether the right target is covered at the time of need. Define the population, observation period and missing data before using a measure to support a claim.

A balanced operating review asks what remains useful, which conditions have changed, what evidence is now limited and which actions are required. NIST's RMF provides an example of connecting monitoring and changed conditions to continuing authorization. [10, Section 3.7;

relationship carried forward from Chapter 20] The book does not prescribe an identical review cycle for every service. A rapidly changing dependency may require event-driven attention; an unchanged analytical record may remain applicable after a justified review. The important result is a traceable decision about continued reliance.

21.10 Exercise: the dashboard is green but the task is unavailable

Situation. In a hypothetical operational review, the gateway is reachable, all scheduled polling jobs completed and the incident queue contains no open critical alert. The recovery package is intact, but the portal needed to establish its approval is unavailable with the enterprise identity service. One collector has not delivered receiving-side records since its credential expired. The team proposes reporting that the administration capability is resilient and that SR-03 is continuously verified.

Worked answer. Separate the observations from the proposed conclusions. Gateway reachability and completed polling provide evidence about those checks. They do not establish the full restoration task when required approval information cannot be obtained. The intact package does not resolve its current eligibility or the operator's authority. The impaired collector limits visibility into the receiving behaviour, so the absence of an alert cannot be treated as evidence that the relevant commits remain controlled.

Use OP-06 to identify the service limitation, the responsible approval and monitoring contributions and the response available to operators. Use MON-06 to establish which conditions remain observable and what can be concluded from retained records. Preserve useful component observations, but do not promote them into an unsupported system-level claim. A monitoring arrangement with the hypothetical 180-second observation-to-action bound cannot establish the sixty-second preventive requirement by itself.

The case does not authorize operation of the earlier unready design. It demonstrates the information and reasoning an eventual operating review would need. Its outputs are a usable account of permitted operation, claim-specific monitoring, an incident-to-engineering feedback route and a maintained support basis. Chapter 22 develops the controlled changes that those observations can initiate.

22. Configuration, change, maintenance and drift

22.1 Distinguish evolution from loss of alignment

Change is necessary to keep a system useful. Components wear out, vulnerabilities are discovered, operational needs develop and suppliers alter their contributions. Some changes improve both security and service. Others expose a conflict that the earlier design did not resolve. Treating every change as harmful drift discourages necessary maintenance; treating every successful deployment as an improvement conceals effects that ordinary functional checks may miss.

Drift, as used here, describes a developing mismatch in the system definition, actual arrangement, operating context or basis for reliance. Investigate its cause, scope and consequence; it is not a physical law or a universal failure explanation. An authorized change with adequate evaluation and records is controlled evolution. An unchanged product used for a materially different task can instead need renewed engineering.

A maintenance action sustains or restores capability. It can include inspection, servicing, repair, updates, replacement and support. Some actions follow established, evaluated procedures with limited discretion. Others introduce a changed configuration or use that requires new analysis. The required depth follows the effect on the system and the governing obligations, not whether the activity is labelled maintenance, enhancement or security remediation.

For IT practitioners, existing change, release, configuration, incident and problem processes provide useful structures. SSE strengthens the technical reasoning inside those workflows: what could change, which requirements and claims depend on it, what evidence remains applicable and what permits return to use. It should not establish a separate approval queue for every ordinary change while leaving the actual system implications unexamined.

This chapter develops CR-06, an original change-review example for a proposed transaction-history migration. The migration is not an implemented correction to E-05-03, and the earlier findings remain open. The example is chosen because it connects the realization, interface, assurance and operational records already developed. It shows how a change can affect protection without changing the visible login experience.

22.2 Maintain baselines that identify meaningful configurations

A baseline is an identified reference under change control. It provides a basis for comparison, evaluation and decisions; it is not a claim that the system will never change or that every aspect of its behaviour has been demonstrated. An organization can maintain different baselines for requirements, design, realization and operational use, provided their relationships and differences are understood.

The installed system needs more than an application version. In the case, relevant identity can include receiving-end code, firmware, storage behaviour, authority configuration, model and profile revisions, migration state, transaction namespace and support procedures. Not every detail has equal assurance significance. The configuration model should identify the attributes that can affect a required property and the source of that judgement.

NIST's security-focused configuration-management guidance connects baseline configuration, change control and monitoring to protection while keeping security within the wider configuration effort. [43, Sections 2 and 3.2–3.4] The practical lesson is to avoid two disconnected accounts: an engineering configuration that omits security-relevant settings and a security baseline that does not correspond to the realized system. RZ-01 and MS-01 can link those accounts without copying every item into one repository.

A desired-state template is not automatically the installed state. A configuration-management database can identify recorded relationships without proving that every deployment matches them. An infrastructure definition can describe what automation was asked to create while a later manual change alters effective permissions. These are not reasons to abandon such tools. They are reasons to define how configuration records are reconciled with observations and how discrepancies are handled.

Version the meaning as well as the object. Changing PKG-P1's interpretation of active version, a model library's time unit or the query behaviour of a transaction store can affect requirements without changing an SR identifier. Record those revisions in the evaluation manifest. A test associated only with the product version cannot reliably support a claim whose essential profile or platform has changed.

NASA's configuration-management treatment likewise connects identified configuration, change control and status information. [6, Section 6.5] In the case, configuration status accounting should answer what is proposed, approved, built, installed, observed, evaluated and retired. Those states may differ legitimately during a rollout. The record becomes misleading when it presents the final desired configuration as if every instance already uses it. For a distributed deployment, identify the permitted combinations and the period during which mixed versions can interact. An average rollout percentage does not establish that a critical receiving path has been updated.

22.3 Classify mismatches by the engineering question they raise

A single drift label can hide different kinds of work. Table 22.1 supplies a small author-defined classification for this book. The categories are investigative lenses, not mutually exclusive failure modes or an externally standardized taxonomy. One event can affect several lenses; the purpose is to route the analysis rather than count the same concern several times.

Table 22.1. Working drift lenses and their different responses

Mismatch being examined	Example in the case	Question that directs the response
Configuration or realization differs from its reference.	A receiving endpoint uses another storage setting or an unlisted deployment credential.	What actually changed, was it permitted, and which required behaviour can it affect?
Behaviour or interface meaning differs from the relied-upon account.	An update changes the meaning of accepted, commit or historical outcome without corresponding client changes.	Which interaction and requirement no longer have a consistent interpretation?

Mismatch being examined	Example in the case	Question that directs the response
Operational context or support assumptions no longer fit.	Restoration requires a person or facility no longer available within the task window.	Is the existing design still suitable, or do needs, support or permitted use require revision?
Evidence or model applicability has changed.	An analysis uses a former profile or an export omits a relevant allowance.	Which conclusions require review, and which observations remain valid for their original scope?
An external contribution has changed.	Identity status, package access or vendor support behaves differently from the agreed assumption.	What can the receiving system establish or constrain, and what must be resolved with the provider?

Do not infer severity from the category. A minor documentation discrepancy can have little effect on a decision; an incorrect authority label in a generated report can materially mislead it. A small implementation change can be significant; a large refactoring can preserve an externally relevant property when adequate evidence supports that conclusion. Scope and consequence need their own analysis.

Also distinguish harmful divergence from acceptable variation. A deployed system may permit a range of operating loads, approved configurations or interchangeable items. Observing a permitted variation within that range is not automatically drift requiring correction. The question is whether the variation is covered by the requirements, design and evidence that support the intended use. A configuration list should not be so narrow that every valid variation becomes an exception, or so broad that unrelated implementations inherit one result.

Sometimes the reference is inadequate. A runbook that omits the actual approval task needs correction, and the omitted work's timing and resources need review. An intentionally added writer belongs in both the model and the coverage argument. Correcting the representation identifies the contribution; evidence must still establish its adequacy.

22.4 Frame a change against the actual decision

A useful change request identifies the reason for the change, affected items, intended result, relevant dependencies, alternatives and the commitment being sought. It should explain the consequences of applying, deferring or replacing the proposed action. Urgency is an input to that decision, not a technical property that makes the change harmless.

For CR-06, assume a supplier proposes a new receiving-coordinator release that migrates transaction history to a more compact schema. The stated benefit is reduced storage demand and simpler support. The proposed schema retains the current active package and a short recent transaction list, but the submitted description does not explain older execution-suppression identities, historical committed generations or mode-epoch recovery. These are invented proposal facts, not findings about a real product.

The change review should immediately identify a possible conflict with SR-07 and SR-10. Forgetting an older transaction while allowing its identity to be accepted again can defeat the no-second-commit property. Retaining only current configuration does not answer a query about what an older transaction committed. The review must also examine SR-12 because recovering

old mode state can make obsolete authority appear current. None of those questions is resolved by a successful login test on the new release.

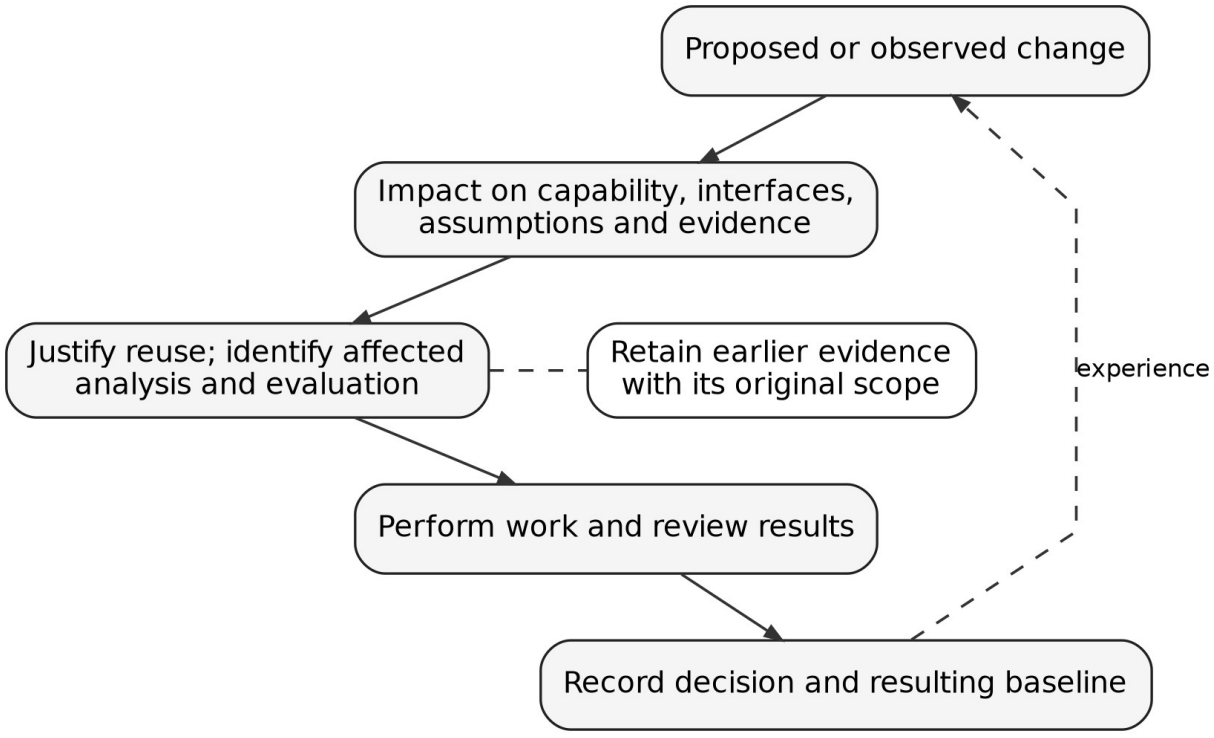
The next decision can be permission to investigate the migration in a controlled environment rather than permission to install it operationally. The supplier should provide the old-to-new state mapping, retention and namespace rules, interruption behaviour and supported rollback or forward-recovery arrangements. The customer should provide the actual query and retention needs, installed platform, relevant interfaces and operating constraints. Both should identify which information remains unavailable.

Compare alternatives. The team might retain a compact permanent execution-suppression record while archiving detailed history, change the migration to preserve the required data, use a different supported store, or defer installation with an explicit capacity-management plan. A reduction in retained detail may be acceptable only if the governing query and retention obligations remain met or are legitimately revised. No supplier default can silently narrow those obligations.

CR-06 should not become a universal claim that compaction is unsafe. It identifies the properties the particular compaction must preserve. A well-designed migration may reduce support risk while maintaining those properties. The work is to obtain and evaluate that design, not to reject it because it changes a security-relevant component or approve it because the change is sold as maintenance.

The following schematic brings together the relationships developed here.

Figure 22.1. A change loop that preserves the evidence history



Original schematic. Follow the arrows and return to the affected definition when findings require it. Time available for fallback and technical reversibility of state are separate conditions.

22.5 Determine the affected evidence and the necessary evaluation

Change-impact analysis follows the changed contribution into requirements, interfaces, assumptions, scenarios and claims. Begin with the direct effect, then examine shared state and secondary effects. A transaction-store change can affect timing and storage pressure as well as history. A new recovery path can affect normal authority through shared configuration. The analysis should be complete enough for the decision without claiming that every connected record requires identical re-evaluation.

Table 22.2 contains proposed work, not completed evaluations.

Table 22.2. CR-06: focused impact and evidence-applicability review

Affected contribution	Requirement or claim relationship	Proposed evaluation and reuse decision
Transaction suppression and namespace history	SR-07/SR-07a; LS-03; C-03	Examine migration and retirement rules; challenge repeated and conflicting identities before and after migration and restart. Earlier pre-migration results do not establish the new history behaviour.
Historical and current outcomes	SR-09/SR-10; IC-01	Compare historical commit records, current generation and query interpretation. Retain useful observation methods if their semantics and instrumentation remain valid.
Mode and authority state	SR-02/SR-12; LS-05	Examine epoch preservation, in-progress work and migration transitions. A normal authentication test may remain relevant but cannot close transition coverage.
Performance and storage demand	SR-00; ADM-P1 and platform assumptions	Evaluate the changed critical path, supported workload and capacity behaviour. Do not carry a former timing total forward solely because the API fields are unchanged.
Recovery preparation and support	SR-11; C-04; candidate REC-P1	Check compatibility of recovery packages, tools and instructions. The incomplete recovery authority is not resolved by successful data migration.
Model and evidence records	MS-01; RZ-01; C-05	Identify the changed schema/profile and result scope; preserve old evidence with its baseline and update only the justified applicability decisions.

Explain consequential decisions to reuse evidence. An unchanged parser operating on identical encoding before the modified persistence boundary may retain an applicable result. Record that scope and evaluate the affected downstream properties separately. Neither universal reuse nor full requalification follows merely from calling the change small or security-relevant.

Regression selection should include scenarios that can reveal the changed failure. Repeating thousands of ordinary activations may never examine migration interrupted between history conversion and active-state selection. A small, carefully chosen interruption test can be more discriminating for that concern. It still needs a reviewed model of the relevant states and evidence that the test arrangement represents the implemented mechanism.

The result can require a requirements clarification. If SR-10's expected history availability is not precise enough to judge the compacted store, return to the requirements owner and affected stakeholders. Preserve the existing obligation and identify the needed decision; do not assume that the new implementation defines the requirement simply because it is available. The change is exposing unfinished engineering, not granting permission to ignore it.

22.6 Treat emergency change as controlled work under different constraints

Security updates can be urgent because delay preserves a serious exposure. They can also affect timing, state, compatibility or recovery. A defensible emergency process considers both the consequence of acting and the consequence of waiting. It identifies the minimum information needed for the immediate decision, which evaluation can be performed safely and what uncertainty must remain visible.

NIST's patch-management guidance treats patching as preventive maintenance and discusses routine, emergency, mitigation and unpatchable-system scenarios. [44, Sections 3.3–3.5] That framing supports a practical alternative to both extremes: waiting for a complete requalification before every fix, or treating urgency as an exemption from understanding the receiving system. The particular update still needs identification, applicability and an appropriate route to return to use.

For an emergency update to the coordinator, establish the actual affected release and exposure, the source and integrity of the update, the critical state or interface changes and the protection that must remain possible during installation. A representative subset of checks may support a bounded decision where broader evaluation cannot be completed first. The authority must understand what those checks establish and what they omit. A reduced test scope is not a claim that the omitted properties passed.

A temporary mitigation can be preferable while a correction is evaluated. Restricting an exposed interface, reducing a role's authority or isolating a contribution can reduce risk where its effect is understood. The same restriction may prevent required maintenance or remove the only means of collecting evidence. Specify the useful capability retained, the loss path addressed, the monitoring and the reconsideration condition. Do not call the restriction a complete substitute for the original requirement without the appropriate evaluation and authority.

Emergency execution should preserve a minimum coherent record: triggering facts, affected configuration, decision authority, action taken, known limits, resulting state and follow-up work. The record can be completed in stages when circumstances demand, but should not be reconstructed later as though all evidence had been available before the decision. Post-change review addresses the outstanding effects and process learning; it does not retroactively create an earlier approval or erase an observed discrepancy.

An emergency path itself requires design. Credentials, people and tools needed to apply a fix must remain available during the conditions that make it urgent. If an emergency procedure

depends on bypassing controls informally, the organization should examine whether its ordinary maintenance architecture is inadequate. Repeated emergency use can be an operational signal that the official process no longer matches the required work.

22.7 Plan migration, fallback and the point beyond which reversal is not simple

A release rollback restores executable content; a system recovery must also account for changed state, external effects and authority. If a new release has converted transaction history or admitted new identities, reinstalling the former binary may not restore a configuration it can interpret correctly. If operational dispatch instructions have already been acted upon, no local rollback erases those consequences.

For CR-06, identify the migration boundary before installation. Can writes continue during conversion? Which process owns the authoritative history? How are arrivals buffered or rejected? What distinguishes completed conversion from a partially converted state? What occurs if either process restarts? These questions need answers appropriate to the selected design. A script that exits without an error does not establish that every required relationship survived.

One candidate arrangement pauses state-changing admission while preserving permitted diagnosis, establishes a recoverable checkpoint, converts the history, verifies the relevant mappings and resumes under an identified configuration. Another may support online migration with a controlled synchronization mechanism. The first can create a service interruption; the second introduces coordination and mixed-version obligations. Neither is chosen here. Their suitability depends on the available window, workload, preservation requirements and implementation evidence.

A maintenance window also needs a latest decision point. As a separate, invented scheduling example, assume a forty-minute interruption window and an applicable fallback sequence requiring twelve minutes to restore the prior arrangement plus six minutes to check it before return. If these are justified sequential upper bounds and fallback remains technically valid, the latest start is minute twenty-two: forty minus eighteen. A decision at minute twenty-six leaves fourteen minutes, four minutes short of the assumed sequence.

This is not the fifteen-minute SR-11 restoration clock, and it does not amend that requirement. It is a distinct maintenance-planning illustration. Its arithmetic is conditional on the fallback remaining possible. If the migration passes an irreversible state-conversion point at minute ten, the time available at minute twenty-two does not restore reversibility. The plan must identify both temporal limits and state-dependent limits, with a different permitted response where ordinary fallback is no longer valid.

Fallback evidence needs the complete state relationship. A snapshot may preserve the earlier binaries while omitting subsequent transaction suppression or mode history. A valid recovery approach may require retaining or reconciling later history instead of simply restoring the old image. In some designs, completing a corrected forward migration is safer than returning to incompatible earlier state. That conclusion must be supported for the actual mechanism; forward correction is not a general excuse to proceed without a recoverable plan.

22.8 Execute maintenance with positive item and state identification

The person performing maintenance needs to know which item and configuration are affected, what procedure applies, what prerequisites have been checked and which conditions require stopping or escalation. The receiving system needs a distinguishable resulting state. “Patch applied” is too narrow when the protective property depends on firmware, storage, settings, credential mappings and history as well as the application binary.

For the proposed migration, preserve the source-state identity, transformation version, selected target, relevant backup or checkpoint identity, execution record and post-change checks. Do not place secrets or unnecessary personal information in a general change ticket. Reference protected records through the appropriate access arrangement. The record should permit qualified reconstruction of the work without creating a new uncontrolled disclosure path.

Hardware and firmware maintenance can require physical as well as digital checks. A replacement item may have a different hardware revision, reset default, interface timing or power-loss behaviour. A part with the same marketed name may not be interchangeable for the relevant property. The responsible platform and support specialists should establish suitability; SSE identifies the security and resilience consequences that must be preserved. An origin check does not replace a compatibility or behavioural evaluation.

Temporary maintenance access needs closure as part of the work. A diagnostic credential, enabled port, test route or support account can remain an effective-change path after the repair appears complete. Identify what is temporary, who controls it, what evidence shows it was removed or constrained, and which continuing access remains legitimate. Deleting every maintenance path indiscriminately can make future support impossible; leaving every temporary path active creates unnecessary authority. The disposition must match the support design.

Completion should distinguish the intervention performed, the resulting configuration established, the affected properties evaluated and the decision to return to use. Those steps may be performed by different people. A maintainer's confirmation that a component was replaced is valuable evidence of the action, not automatically a system-level assurance conclusion. Equally, a separate review does not make the maintainer's record irrelevant; it depends on an accurate account of the actual work.

22.9 Close changes without erasing their residual obligations

A change can be closed for execution while an associated evaluation or operational restriction remains open. The information system should permit that distinction. Otherwise a single green status can imply that the update was installed, the defect corrected, the requirements satisfied and the risk accepted, even when only the first event occurred.

Table 22.3. CR-06 completion record: different questions need different answers

Completion question	Evidence or decision needed	What must not be inferred
Was the intended intervention performed?	Actual target, procedure, material, execution and resulting version/state identity.	Completion of installation does not establish every affected property.

Completion question	Evidence or decision needed	What must not be inferred
Were the relevant properties evaluated?	Applicable observations, analysis, criteria, coverage, discrepancies and review.	A scheduled regression run is not a completed result.
Is previous evidence still applicable?	Reasoned disposition for changed and unchanged contributions, with baseline references.	A recent report is not automatically stronger than an older relevant one.
May the resulting arrangement return to use?	Technical findings, receiving readiness and the appropriate scoped decision.	Closing a change ticket does not grant powers absent from the governing arrangement.
What continues after closure?	Monitoring, temporary restrictions, supplier actions, retained records and reconsideration triggers.	A successful immediate check does not retire future support or evidence duties.

Maintain MS-01's model revision and the actual installed account separately. An accepted model change defines a proposed or approved engineering description. Deployment establishes the realized instance, and observation may reveal a discrepancy. Use the relationships to identify affected claims and evaluations, not to replace their status with the model repository's merge status. A restored model database also needs its own history and baseline checks; restoring an old report must not make it appear to describe the new implementation.

Recurring changes can expose a structural problem. Repeated capacity emergencies may indicate inadequate retention or sizing decisions. Frequent manual corrections to model exports may indicate an unmaintainable exchange arrangement. Recurrent exceptions for one operational task may indicate that the design or authorized use needs reconsideration. Address the pattern through the appropriate technical and organizational work rather than renewing individual exceptions indefinitely.

The change record should retain the reason a decision was reasonable at the time. Later maintainers need to know whether a limitation was accepted, deferred, corrected or never evaluated. The current residual position and intended position after treatment remain distinct, as in RR-05. A record that preserves those distinctions helps an organization learn without rewriting its history into an uninterrupted sequence of successes.

22.10 Exercise: the migration completed, so the old tests still apply

Situation. A proposed transaction-store migration completes within twenty-six minutes of the separate forty-minute maintenance window. The new client can authenticate, inspect the current package and perform one valid activation. The supplier did not demonstrate preservation of old transaction identities or mode history. The earlier release is available, and the team assumes that reinstalling it provides immediate rollback. The documented fallback plus checks require eighteen minutes under their stated conditions.

Worked answer. The observed positive functions would support only their actual evaluation scope. They do not establish old-identity suppression, historical outcome queries or epoch preservation after migration. Those are affected properties requiring appropriate analysis and

evidence. The earlier tests remain valid descriptions of their original configuration, but their conclusions cannot automatically be transferred to the new storage and migration arrangement.

The proposed rollback also has two unresolved problems. Fourteen minutes remain, less than the assumed eighteen-minute fallback sequence. More importantly, the team has not established whether the former release can interpret the changed state without losing necessary history. A binary package and remaining calendar time do not create a technically valid reversal. The permitted response must follow the actual state, operational consequence and authority; the example does not prescribe blindly reverting or continuing.

Use CR-06 to identify the missing state mapping, affected requirements and controlled next action, preserving the useful positive result. Future execution needs defined prerequisites, state-dependent limits and a latest fallback decision. Chapter 23 applies this reasoning to inherited systems with limited design information or control.

23. Legacy, COTS and cloud-dependent systems

23.1 Begin where control and knowledge actually exist

Many practitioners inherit systems rather than design them from a clean sheet. The service is already needed, its supplier may restrict internal information, and a complete architecture description may never have been maintained. A cloud provider can control changes the customer cannot approve individually. A commercial component can expose only a narrow management interface. Those conditions constrain engineering; they do not make engineering irrelevant.

The useful starting question is not whether the organization can rebuild everything. It is which decisions must be made, which functions and losses matter, what can be established about the actual arrangement and where remaining uncertainty limits reliance. A targeted reconstruction can support a better decision about one critical interface without claiming that the entire inherited system has been recovered into a complete model.

Treat legacy as an inherited context, not a verdict based on age. An older supported component may have well-understood behaviour; a newer service may have uncertain authority, dependencies or support. Evaluate capability, operating conditions, change options and evidence for the actual decision.

COTS is used descriptively in this chapter for commercially supplied off-the-shelf products. It is not a procurement-law definition or an exemption criterion. The actual acquisition rules and agreement determine any legal classification. Engineering still needs to distinguish the supplied product, its configuration, integration, extensions and intended use. A product obtained off the shelf becomes a contribution to a particular system; its commercial availability does not establish that system's security or resilience.

LEG-06 is a separate inherited-service vignette. Assume an organization already relies on a commercial dispatch-rule maintenance service and must improve or replace it. This is not deployment of the unresolved TE-05 candidate. The earlier requirements guide the assessment questions; they do not establish the inherited service’s contract or authorization basis.

23.2 Reconstruct the decision-relevant system

Begin with the useful task and its consequential actions. Who can inspect, stage, approve, activate or restore configuration? Which components can make an effective change? Which interfaces provide current and historical outcomes? What happens when supporting identity or management services are unavailable? These questions bound the information to recover before choosing a protective overlay or a replacement product.

Collect several kinds of evidence. Existing diagrams can identify intended connections. Deployment records can identify installed releases and accounts. Authorized configuration inspection can reveal effective privileges. Operational records can show which tasks people actually perform. Supplier documentation can describe supported behaviour and its limitations. None is automatically the complete truth. Reconcile disagreements against the decision being made and preserve the provenance of each source.

For LEG-06, stipulate three discovered candidate paths: an interactive gateway, an automated deployment job and a local maintenance interface. The gateway uses enterprise MFA. The

deployment job uses a service identity, and the local interface's current role in activation is not yet established. These are teaching facts about the vignette, not findings about a real supplier. Their immediate significance is that the gateway cannot be assumed to represent all effective-change paths.

Distinguish necessary discovery from intrusive experimentation. Passive records, controlled inspection and supplier explanation may establish enough to plan a safe test. Active probing or fault injection can disrupt the service or alter evidence. Use an authorized representative environment where possible and identify what it does not reproduce. The need to understand a system does not grant permission to experiment on a live mission function without the relevant operational and technical arrangements.

Table 23.1 maps LEG-06's decision-relevant evidence. Express confidence through the supported proposition and its limitation, so that the next investigation follows an identifiable gap rather than an unexplained percentage.

Table 23.1. LEG-06: reconstruct the inherited contribution

Question	Available teaching information	Next discriminating work
Which paths can activate rules?	Gateway and deployment paths are identified; local maintenance influence is unresolved.	Reconcile privileges, supported interfaces and authorized observations. Do not declare complete coverage from the gateway diagram.
What does success mean?	Client reports accepted, but the supplied description does not distinguish queueing from effective activation.	Obtain interface semantics and a justified observation of authoritative state; test an interrupted exchange in a permitted environment.
What survives restart or migration?	Supplier states that configuration is backed up; transaction-identity and epoch treatment is unspecified.	Identify exactly which state is preserved and evaluate the relevant interruption/restore conditions.
Can staff restore during identity loss?	A support procedure exists; approval and credential dependencies are not yet mapped.	Follow the complete human and technical task before accepting an independence or timing claim.
What can be changed or examined?	Customer controls some settings; internal code and provider operations are not disclosed.	Establish supported configuration, interface commitments, evidence access and realistic options for reducing reliance.

Preserve the historical basis for earlier decisions while establishing a current, identified account for the new one. Record differences between description and observed behaviour instead of silently revising history. Outdated diagrams need not remain the working model merely because they are part of that history.

23.3 Distinguish a visibility gap from a demonstrated defect

Missing internal source information does not establish that a product is defective. It limits the methods available to substantiate particular claims. A narrow interface property may be

supported through documentation, controlled observation and independent analysis without access to every internal design detail. A stronger claim about all internal paths, failure containment or resistance to a privileged provider may require information the customer cannot obtain.

Conversely, the absence of incidents is not evidence that all required properties hold. It may indicate that the relevant scenario has not occurred, was not detected, was not reported or did not produce an obvious consequence. Use operational history for the proposition it actually supports. A stable service record is useful context; it is not a substitute for the receiving-side authority and persistence questions that remain unexamined.

For LEG-06, a sequence of successful normal activations can establish that the task worked in the observed configuration and conditions. It does not establish whether a request timed out before or after commit, whether a local maintenance path uses equivalent authority or whether a restored backup preserves transaction suppression. Those are different questions requiring different observations. Expanding the number of ordinary successful trials cannot compensate for omitting the relevant scenario.

Supplier evidence should be reused where its subject and assumptions fit. A test of the same receiving interface on the supported platform may provide substantial value. A report about an unrelated hosted edition or a different storage setting may not. The applicability method in Chapter 18 remains the basis: identify the property, configuration, conditions, method and inference. “Commercial product” does not justify universal inheritance, and “black box” does not justify universal rejection.

The outcome of the assessment may be a limited claim rather than an unqualified verdict. For example, the customer might establish that an enumerated gateway path enforces the required admission rule while writer completeness and interrupted-activation semantics remain unresolved. That is useful progress for deciding the next investigation or design restriction. It is not an adequate basis to assert that all administrative changes are governed simply because the known path is well controlled.

23.4 Design compensating protection at the boundary it can influence

Where internal modification is unavailable, an external measure can provide a legitimate protective contribution. A gateway, broker, constrained operating procedure, network separation or controlled service interface may reduce exposure, limit authority or improve observation. The key is to identify the path and property the measure can actually influence and the dependencies on which its effectiveness relies.

Suppose LEG-06 introduces a package broker that admits only identified, approved activation requests. It can reduce discretionary actions entering through that broker. If the deployment job and local interface can still write directly, the broker cannot establish complete mediation of effective changes. Those paths must be constrained, separately governed or retained as explicit gaps in the claim. Adding another well-protected front door does not close an unexamined side entrance.

The broker may also alter the interaction. It could retry after a timeout, translate accepted into completed, or assign a new transaction identity on every submission. These behaviours can undermine the very integrity property it is meant to support. The adapter must preserve the

relevant request and outcome meaning, not merely convert fields successfully. IC-01 supplies useful questions even where the inherited product cannot implement that contract exactly.

A restriction can be valuable without restoring every requirement. Disabling a rarely used write interface may reduce an exposure while leaving recovery unavailable. Restricting administrator access to an isolated workstation may reduce some credential risks without establishing correct authority at commit. State what loss path is addressed, what capability remains and what evidence supports the contribution. Do not claim equivalence to an unmet control or requirement unless the governing framework and technical analysis justify that conclusion.

Evaluate the restriction's support consequences. If only one workstation can now perform restoration, its power, availability, credentials and maintainability become important dependencies. If a supplier must attend physically to recover the system, the response time and access conditions matter. A more restrictive architecture can improve one protective property while making the required task infeasible. The complete engineering decision includes both effects.

23.5 Choose among repair, containment, replacement and changed use

An inherited-system assessment should produce alternatives tied to actual constraints. Retain and maintain the current contribution where it can be supported. Reconfigure or constrain it where the relevant property can be established externally. Replace an element or interface where its limitation prevents the required result. Redesign the operating arrangement where another capability can meet the need. Retire the function when it is no longer justified. None is automatically the least risky option.

Support status belongs in that comparison. An unsupported component may lack timely corrections, compatible spares or a party able to explain a critical failure. A supported component can still fail the intended integration. The decision should identify what support is needed, what is actually available and the consequence of losing it. A support end date is a planning trigger; it is not itself a measurement of present exploitability or a universal instruction to shut down immediately.

For systems interacting with physical processes, immediate replacement or patching may need significant preparation. Timing, safety, reliability and process continuity can constrain intervention. NIST's OT guidance explicitly recognizes these distinctive considerations. [21, abstract] That does not excuse indefinite inaction. It requires a defensible interim arrangement, targeted evaluation and a supported transition plan with the responsible operational and engineering specialties.

In LEG-06, the most useful first investment may be resolving the local writer and outcome semantics rather than procuring a second gateway. The second gateway can improve a particular availability contribution while preserving the same unknown receiving behaviour. A replacement product can offer better evidence but introduce new integration and data-migration work. Compare those consequences before assuming that the newest option is the most trustworthy or that the existing service is the safest default.

A decision to tolerate a limitation should identify the useful service preserved, the exposure or uncertainty retained, the permitted scope and reconsideration conditions. It should retain the unmet requirement or unsupported claim rather than reclassify it as satisfied. If the

organization changes its need or requirement, record the actual decision and rationale. Do not use a rewritten specification to conceal the fact that the former capability was not established.

23.6 Allocate cloud responsibilities by service and property

Cloud-dependent systems add an organizational and technical boundary to the engineering problem. The customer may configure identities, data, networks or applications while the provider controls other infrastructure and service behaviour. The allocation changes with the service being used. NIST's cloud definition distinguishes infrastructure, platform and software service models partly through the capabilities provided and the control retained by the consumer. [45, Section 2, Service Models] Those categories orient the inquiry; they do not define every obligation of a particular offering.

Provider guidance is useful when treated within its scope. Microsoft's shared-responsibility explanation, for example, distinguishes responsibilities across its cloud service models and explicitly describes itself as guidance rather than the legal agreement. [46, introductory scope and division of responsibility] The receiving project still needs the actual service terms, enabled features, configuration, interfaces and evidence. A general responsibility diagram cannot establish the effectiveness of a tenant's authority mapping or the behaviour of a specific API during interruption.

Ask three responsibility questions: what the provider supplies, what the customer configures and operates, and what the integrator must establish about their combined task. A provider may meet its service description while the receiving application retains old permissions too long. Neither an infrastructure certificate nor a tenant setting alone resolves that interaction.

Table 23.2. A contribution-based cloud responsibility record

Required property or task	Contributions to establish	Receiving-system evidence question
Authority at effective change	Provider identity semantics; customer role and application configuration; receiving enforcement and queue behaviour.	Do these contributions satisfy the actual source-to-commit and freshness requirements together?
Protected, recoverable configuration	Provider storage/service behaviour; customer package and key arrangements; application history and restore semantics.	Can the required state be recovered without losing transaction suppression, outcome meaning or permitted authority?
Usable recovery during disruption	Provider service availability and management access; customer credentials, approvals and people; alternate paths.	Does the whole task remain possible under the specified cause, rather than merely in another provider region?
Continuing evidence and change visibility	Provider notices and scoped reports; tenant audit/configuration records; integrator impact analysis.	What can be concluded for the current arrangement, and what information remains unavailable?

Required property or task	Contributions to establish	Receiving-system evidence question
Exit and information disposition	Provider export/retention mechanisms; customer receiving capability, rights and key decisions; successor validation.	Can the organization actually transition and account for remaining copies and obligations?

The entries need actual owners and interfaces in a real project. “Shared responsibility” should not mean that no participant is accountable for a necessary contribution. Nor should it mean that the customer can control every provider action. Where a contribution is outside direct control, establish what commitment, evidence, limiting design or alternative supports reliance. If none is adequate, preserve the resulting limitation in the decision.

23.7 Examine cloud behaviour beyond nominal availability

A regional failover can preserve one service while leaving a common management or identity dependency unavailable. Two accounts can share a privileged administrator, an approval source or a deployment pipeline. Separately billed resources are not necessarily independent against the cause that matters. Define the disruption being addressed before accepting a redundancy claim.

For the administration example, follow both the operational interface and the management path. The data used by the application may remain accessible while a control-plane action needed for restoration is unavailable. An emergency credential may exist while its activation depends on a remote approval workflow. A local backup may be intact while its decryption or restoration requires a service that is part of the same outage. These are specific dependencies to test or constrain, not arguments that all cloud services are unsuitable.

The meaning of a provider operation can also differ from the meaning assumed by a client. An accepted asynchronous update may not yet be effective. A returned version identifier may identify a stored object but not every application consumer's current configuration. A retry may be safe for one API operation while a surrounding workflow performs an additional side effect. The receiving design must establish the actual semantics of the selected interface; the generic promise of an API does not settle the transaction model.

Changes outside the customer's approval authority require a workable observation and decision arrangement. Useful contributions can include stable interface commitments, deprecation information, versioned configuration, representative regression checks and an escalation route. The customer should identify which provider changes become visible only indirectly and how that limitation affects the claims. A continuous deployment feed or recent assurance report does not necessarily describe every backend configuration at the moment of use.

Evidence scope remains important. A provider report may address controls over a named service during a stated period. The customer's tenant, workload, administrators and integration can introduce other relevant conditions. Conversely, a tenant test may observe the interface without establishing the provider's internal isolation or support process. Combine these contributions through a bounded argument rather than asking one kind of evidence to answer every question.

23.8 Make portability and exit an engineering capability

An export feature does not by itself establish a usable exit. The organization needs the necessary data, configuration, identity mappings, transaction history, interfaces, tools, rights and receiving support. It must also know what remains with the former provider and how continuing retention or deletion obligations will be handled. The acquisition work in Chapter 17 should obtain the information and access needed for that transition before the current arrangement becomes difficult to replace.

An original scheduling example makes the distinction concrete. Assume a particular exit requires transferring sixty gigabytes through a path capable of no more than one hundred megabits per second. Use decimal units: sixty gigabytes equals 480 gigabits. At that maximum rate, transferring the payload alone takes at least 4,800 seconds, or eighty minutes, ignoring protocol overhead, retransmission and throttling. This is a lower-bound calculation, not a measured transfer estimate.

Suppose the proposed plan then requires twenty-five minutes for a dependent mapping step, thirty-five minutes for validation and ten minutes for cutover, all sequential after the complete transfer. Their sum adds seventy minutes. The resulting plan cannot finish in less than 150 minutes under those assumptions. It therefore cannot fit a separate two-hour exit window; even its idealized schedule is thirty minutes too long. This is not SR-11's restoration task and does not alter the earlier fifteen-minute requirement.

Table 23.3. Separate exit-window illustration, using invented planning inputs

Contribution	Stated basis	Time implication
Transfer sixty decimal gigabytes	Payload is 480 gigabits; path maximum is 100 megabits per second.	At least eighty minutes before overhead or interruptions.
Map the complete received information	Assumed twenty-five-minute dependent step.	Starts after transfer in this particular plan.
Validate the receiving arrangement	Assumed thirty-five-minute dependent step.	Demonstrates selected receiving properties, not merely file readability.
Perform controlled cutover	Assumed ten-minute final step.	Does not omit authority, state or old-path disposition.
Compare with the separate 120-minute window	Minimum combined duration is 80 + 25 + 35 + 10 = 150 minutes.	At least thirty minutes beyond the proposed window.

The team can reconsider the design rather than hide the gap. Preparation, staged transfer or genuinely parallel work may reduce interruption, provided the consistency and dependency assumptions are satisfied. A higher supported transfer rate may help, but the end-to-end bottleneck and remaining tasks need evidence. Reducing the validation scope requires a justified decision about what remains unestablished; it is not a free scheduling adjustment.

For changing data, a staged export needs a coherent cutover model. Identify the checkpoint, subsequent changes, authority for the final writer and the way the receiver determines it has the required state. Two active services with independent transaction histories can each appear

healthy while violating the combined no-second-commit or outcome expectations. A transfer test should therefore include relevant state and role transitions, not only a comparison of file counts.

23.9 Preserve a maintainable model of the inherited arrangement

MBSE can support a targeted model of an inherited system without pretending that every internal detail is known. The model-planning and readable work-product approach introduced in Chapter 11 draws on NASA's systems-modelling handbook; its use here is an original application to incomplete inherited information. [23, Sections 5–7 and 9; conceptual basis carried forward] Represent the functions, interfaces, dependencies and decisions relevant to the current question. Label a supplier-declared behaviour, an inferred relationship and an observed condition differently where that distinction affects reliance. Uncertainty belongs in the model and its readable views, not only in a separate risk spreadsheet that reviewers never see.

For LEG-06, a useful initial slice contains the three candidate writer paths, their known authority sources, the unresolved local interface, the transaction and outcome questions and the cloud-service dependencies. Add links to actual evidence when it exists and to planned discovery where it does not. A missing internal model can be represented through an explicit interface contract and assumptions. It should not be replaced with an invented decomposition simply to make the diagram look complete.

Configuration identity must include the provider or product contribution at the level the customer can actually establish. The customer may not know a cloud provider's internal build at every instant. It can still identify the subscribed service, relevant API/version, region or arrangement, tenant settings, observation interval, known changes and limits of provider visibility. State those limits in the evidence argument rather than fabricating a finer-grained baseline.

Maintain the ability to read and use the model when suppliers, tools or licences change. A native file is useful where the receiving organization can interpret it with the necessary libraries and permissions. Readable views and agreed exchange representations preserve additional access, but their semantic fidelity needs checking. The baseline should identify what each representation preserves, particularly units, alternatives, unresolved relationships and evidence status, as developed in Chapter 11.

The outcome is a decision-relevant account, not a claim of complete reconstruction. It may justify a narrow protective improvement, a more discriminating evaluation, a supported interim restriction or a replacement investigation. Keep the work proportionate while retaining the consequential gaps. A small model that honestly identifies an unknown writer is more useful than an elaborate one that silently assumes it away.

23.10 Exercise: certified service, constrained broker and an unexamined writer

Situation. LEG-06's proposed broker permits only approved package requests, and its supplier provides a well-scoped organizational assessment. The inherited deployment job can still activate configuration directly. The local maintenance interface has not been evaluated. The provider offers a data-export feature but has not demonstrated transfer of transaction history or the receiving application's ability to interpret it. The project reports that the broker assures all changes and that exit risk is closed.

Worked answer. The broker can provide a useful contribution on the path it controls. The organizational assessment can support relevant statements about the assessed activities. Neither establishes complete mediation of effective changes while another writer exists and the local interface remains unknown. Reconcile those paths, determine the actual authority and effect, and constrain or evaluate them as necessary. Do not assume that each is malicious, but do not exclude them from a claim covering all changes.

Exit remains a separate capability question. Establish what information the export includes, its semantics, usable access and the receiving-system work. Transaction history, configuration generation and permitted authority may be essential to preserving the required behaviour. A collection of exported files does not establish a completed migration, and a bandwidth figure does not establish task duration without the remaining dependencies and validation work.

The next decision should identify what can be supported now and which investigation can change the choice. The method accommodates inherited, commercial and cloud-dependent contributions without pretending that their internal design is fully controllable. Chapter 24 addresses the final transition: ending a specified use while preserving required capability, evidence and obligations that do not end with the service subscription.

24. Retirement, transfer and disposal

24.1 Design the ending before the final shutdown

A system can stop providing its service while retaining consequential information, privileges and dependencies. An endpoint removed from the catalogue may still receive deployment requests. A retired account's token may remain accepted elsewhere. A backup may preserve an old authority context. Ending useful service does not automatically end these relationships.

Retirement therefore needs an engineering definition. What use is ending? Which capability must continue elsewhere? Which functions, data, authority and records must be removed, transferred or retained? What evidence will establish that the intended withdrawal has occurred? Those questions belong in acquisition, architecture and support planning, not only in a final asset-disposal ticket.

NIST's disposal process addresses ending a specified use and managing the affected information and assets. [1, Appendix H.14] The outcome can include controlled transfer, storage or reuse, not only destruction; ending the former role does not erase every historical consequence.

The security contribution is to identify residual access, information, functionality and dependencies that can affect the withdrawal or continuing systems. Safety, environmental, records-management, commercial and other specialist responsibilities remain with their appropriate participants. A security team should not direct destruction of a record subject to a retention obligation or physically dispose of equipment without the required safety and environmental arrangements. The actual requirements and authority determine the permissible disposition.

RET-06 plans withdrawal of a future administration contribution or the separate LEG-06 service. It records proposed work, not completed deployment, transfer or deletion. The earlier findings and recovery choices retain their stated status.

24.2 Define the retirement boundary and completion criteria

Retiring one gateway is different from ending remote administration. Ending one tenant is different from terminating the provider service used by other systems. Removing a product from active use is different from ending access to its archived evidence. Begin by identifying the subject, related continuing functions and the intended final state. A retirement instruction that simply says “remove the old system” leaves too much of that meaning to local interpretation.

For the administration case, a successor might take over authorized package activation while the former capability continues only to provide controlled historical queries. Alternatively, the required history may be transferred to an archive and the former runtime removed. Each arrangement has different access, support and evidence needs. Keeping the old runtime indefinitely to answer occasional queries can preserve unnecessary privilege; copying only current configuration can lose the historical outcome required for investigation or continuity.

Completion criteria should address both what must cease and what must remain. The old receiving path must no longer be able to commit new activations under retired authority. The successor must provide the required useful task within its own defined and evaluated scope. Required historical records must remain accessible through the agreed arrangement. Items

whose location or disposition is unknown need explicit treatment rather than being counted as removed because the main service is offline.

Table 24.1 defines the principal decision boundaries in RET-06. It is a planning aid, not an instruction to apply the same final disposition to every category.

Table 24.1. RET-06: identify which use and obligation are ending

Subject of withdrawal	Intended ending to define	Continuing contribution to examine
Operational administration path	No new effective changes through the retired path under its former authority.	Successor task, pending work, supported interfaces and permitted historical queries.
Accounts, roles and credentials	Former execution and support privileges no longer usable in the withdrawn scope.	Other legitimate uses of shared identities and the evidence needed to interpret past actions.
Packages, state and records	Disposition of active configuration, transaction history, generations, epochs and copies.	Recovery, investigation, retention, successor interpretation and protected archival access.
Hardware, media and hosted resources	Identified transfer, reuse, release, sanitization, storage or destruction.	Shared infrastructure, provider obligations, custody and physical specialist requirements.
Models and enabling systems	End unnecessary write, build, deployment and integration authority.	Readable evidence, reproducible analyses where needed, permitted information rights and maintenance knowledge.

The completion record should distinguish an action finished from a commitment still open. An endpoint can be disconnected while provider-held backups remain under an agreed retention process. A disk can be treated while another unlocated copy remains unresolved. Those facts do not necessarily prevent every aspect of retirement, but they must limit the claim of completion. A single closed asset record cannot erase outstanding responsibilities held elsewhere.

24.3 Inventory relationships and copies, not only assets

An equipment inventory identifies objects to dispose of. A withdrawal inventory also needs the relationships through which the system acts or is relied upon. Include incoming requests, scheduled jobs, service discovery, identity federation, deployment pipelines, package repositories, outcome queries, support connections and shared services. An item that is inexpensive or no longer directly visible to users can still supply a critical dependency.

For RET-06, trace an activation from its caller through approval, admission, receiving commit and historical outcome. Then trace the records and privileges left behind at each point. A client may retain a queued request. A broker may retry after the old server appears unavailable. A deployment agent may possess a credential accepted by the successor. An archived model may contain connection details or sensitive vulnerability information. Their dispositions need not be identical, but each needs an accountable explanation.

Information can exist in several forms and locations. Active data, replicas, exports, logs, backups, snapshots, caches, support bundles, removable media and test copies may all need attention. Identify which copies are controlled directly, which are held under provider or supplier arrangements and which cannot yet be accounted for. A database entry describing a copy is not proof of its existence or deletion; its role is to direct the necessary confirmation.

Include replaced items and prototypes. A maintenance return can carry configuration, credentials or data even though the system as a whole remains in service. A test environment can retain production-like access after the development contract ends. A failed storage component may be difficult to treat through its normal interface. These are disposal questions during the lifecycle, not exceptions that can wait for final retirement.

The inventory should be proportionate but not falsely complete. Use available configuration, data-flow and support records to identify the likely disposition set, then investigate critical unknowns. A missing low-consequence historical drawing can have a different effect from an unlocated credential store. The consequence and obligation determine the required follow-up. Do not report every uncertainty as equally severe or close it merely because its investigation is inconvenient.

A shared resource requires a dependency review before removal. Deleting the old service's identity application might disrupt a continuing integration that uses the same registration. Destroying a shared encryption key might render required records from another service unreadable. Removing a network route might disable the successor's management path. Withdrawal must distinguish the obsolete use from legitimate continuing uses rather than assume that every shared object belongs exclusively to the retiring system.

24.4 Transfer capability without leaving two uncontrolled authorities

Transition and retirement overlap when a successor assumes the useful function. The plan needs a coherent sequence for preparation, state transfer, authority cutover, evaluation and removal of the former path. A period of coexistence may be necessary, but it introduces a state and authority problem that must be designed rather than left to parallel project schedules.

Consider a future transfer of administration from receiver R-old to R-new. If both can independently commit requests using the same transaction namespace but different histories, an identical request may appear new to each. If each maintains its own generation without a defined relationship, clients can make decisions against incompatible state identities. Merely directing normal traffic to R-new does not prevent a delayed request or retained credential from reaching R-old.

Define the cutover boundary: the receiver permitted to commit, the disposition of pending work, the history needed by the successor, and the authoritative location and state shown to clients. Fencing, history transfer and credential control can supply parts of the solution. Evaluate their combined effect; this record does not specify a distributed transaction algorithm.

Treat historical query continuity separately from execution continuity. A retained archive can allow a reviewer to determine what TX-41 committed without permitting TX-41 to execute again. A new system may use a new namespace while preserving an explicit mapping for old outcomes. Such arrangements must satisfy the actual transaction-lifetime and query requirements; a convenient migration design cannot silently narrow SR-07 or SR-10.

A cutover rehearsal should examine failures as well as the intended sequence. What if a client retries after its acknowledgement is lost? What if the new receiver starts with an incomplete history? What if a role mapping is transferred but its withdrawal state is not? What if the old endpoint becomes reachable after restoration of a network component? These are proposed evaluation questions, not observed defects. Their purpose is to identify the evidence needed before the former capability can be safely removed.

A permitted fallback also needs an end. If restoring R-old would reactivate obsolete grants or ignore commits made at R-new, it is not an ordinary reversal. The transition plan must define the state that makes fallback valid, how later effects are reconciled and which point requires a different recovery action. Chapter 22's distinction between available time and technical reversibility applies directly. An old virtual-machine image is not an adequate fallback plan by itself.

24.5 Preserve required information without preserving unnecessary authority

Retention and destruction serve different obligations. Some records must remain available for investigation, contractual accountability, continuing engineering or other governing purposes. Some sensitive working copies should no longer remain accessible after their use ends. Establish the applicable information requirements and decision authority before selecting a technical treatment. This book does not prescribe a universal retention period or jurisdiction-specific legal disposition.

For the administration capability, retained records may need to explain the package, target, historical outcome, configuration, relevant authority context and evidence used for a decision. They need not retain every credential that once made execution possible. Distinguish records of authority from usable authority. An audit record identifying that a role approved an action is not the same as a still-valid token that allows another action to be committed.

Encryption and key management make the relationship concrete. Suppose one hypothetical key protects both disposable staging copies and transaction records that must remain readable. Destroying that key to render the staging copies inaccessible also prevents legitimate access to the retained records unless another valid, protected arrangement exists. The required outcome is not achieved by choosing between keeping everything forever and destroying everything immediately. The information architecture and disposition plan need to separate the uses appropriately.

Archival access needs its own governance. Identify who may retrieve records, for what purpose, with which tools and protections, and how integrity and meaning remain available. Retaining an encrypted file without the required key or format information may preserve bytes while losing usable evidence. Keeping unrestricted administrative credentials to make archive retrieval easy preserves unnecessary operational authority. The design should provide the minimum capability needed for the legitimate archival task.

Signature verification and decryption also require different material. Retaining public verification information and relevant provenance can support interpretation of a signed historical record without retaining the private signing authority that could create new records. Retained encrypted content may require protected decryption capability. The actual algorithms, validation context and long-term evidence needs require specialist review; do not generalize one key-retention decision to every cryptographic role.

Use an explicit disposition matrix to prevent contradictions. A rule to retain outcome evidence should not be contradicted elsewhere by a blanket instruction to delete all logs and backups. A rule to remove obsolete execution authority should not be defeated by restoring a retained operational snapshot into an uncontrolled environment. Records-management and security participants should review the same matrix, each bringing the obligations and consequences within their competence.

24.6 Select information treatment for the actual technology and claim

Logical deletion, access revocation and media sanitization are different actions. Removing a file reference or closing an account may end an ordinary access path without establishing that residual information is infeasible to recover. Revoking a user's permission may protect retained content while leaving it intentionally recoverable by an authorized custodian. State the desired outcome before choosing a method or interpreting a completion certificate.

NIST SP 800-88 Revision 2 distinguishes clear, purge and destroy as sanitization approaches and treats cryptographic erase subject to specific conditions. It also distinguishes checking the sanitization outcome from deciding whether that outcome is acceptable for the required disposition. [47, Sections 3.1–3.2 and 4.5] These sanitization-specific uses of verification and validation must not be confused with every system-level V&V activity discussed in Chapter 15.

Select information treatment for the actual medium or service, sensitivity, intended reuse and applicable obligations. Obtain the technical information and competent review needed to justify it. A result on one technology or configuration cannot establish sanitization of another without an applicability argument; no generic erasure command is prescribed here.

Cryptographic erase requires more than disabling access to a key. The relevant information must have been protected by suitable encryption, and the key material that permits its recovery must be addressed, including applicable copies and dependencies. Information previously left in plaintext is not covered merely because a later copy was encrypted. [47, Sections 3.2.2–3.2.3] The receiving organization must establish these conditions for the intended claim rather than infer them from a product's encryption label.

Apply that reasoning to RET-06. A provider's deletion of one customer-visible key reference may not establish the disposition of an exported key, a recovery copy or previously decrypted support material. Conversely, continuing to retain a separately controlled key for required archival records may be legitimate when the disposable and retained information are appropriately separated. The claim should identify which information becomes unrecoverable, which remains protected for authorized use and which copies are unresolved.

Physical treatment also needs custody and evidence. Identify the actual item, authorized disposition, responsible party, treatment record, results and any exception. A certificate referring only to a batch label may be insufficient where the organization cannot link the retired device to that batch. This is an identity and evidence problem, not proof that the contractor failed to perform the work. Obtain the necessary correspondence before making a stronger claim.

24.7 Address provider-held data and virtual resources explicitly

A cloud customer often controls a logical resource without controlling the physical medium on which it resides. Terminating a virtual machine, deleting a tenant or revoking a role therefore

needs a provider-specific interpretation. The applicable service arrangements determine what is removed, what remains in backups or records, which actions the customer can perform and what evidence the provider supplies.

NIST's July 2026 sanitization FAQ discusses the difficulty of applying physical-media methods directly to virtual resources and directs attention to provider practices, agreements and key-management arrangements. [48, Questions 12–13] The practical implication is to avoid claiming customer-performed physical sanitization where the customer has not controlled or observed the relevant media. Establish a bounded claim from the actions and evidence actually available.

RET-06 should record the service and logical resource identity, the requested action, its effective status, the provider's stated retention or deletion treatment, relevant key decisions and remaining limitations. A general ticket marked closed can establish that a request was processed; it may not establish the scope of data disposition. The customer needs enough detail to determine whether the action meets the actual information requirement.

Backups and retained service records deserve separate treatment. A provider may retain certain information under stated service or governing arrangements after the active resource is removed. Identify that treatment rather than assume immediate universal erasure or describe all retention as a failure. The relevant question is whether the retained information, access and eventual disposition satisfy the applicable obligations and whether the customer can substantiate the required conclusion.

The same discipline applies to customer-held exports and support copies. A provider cannot delete an export already transferred to the customer's workstation. A customer cannot establish deletion from an independent supplier's support system merely by closing its own tenant. Follow the copy and authority relationships to the party able to perform the action, and preserve confirmation or the specific limit on confirmation.

Information location is one attribute of the arrangement, not a substitute for access and disposition analysis. A storage region, organizational certificate or contractual statement can provide relevant context but does not itself establish that every copy or credential has been removed. Where legal or regulatory requirements affect the decision, obtain the actual requirements and competent interpretation. Do not infer a jurisdictional conclusion from the cloud service model alone.

24.8 Verify intended absence and required continuing presence

Retirement evaluation needs both positive and negative evidence. Required successor functions and retained records should be available and usable. Retired actions and authority should no longer be effective within the specified scope. Neither objective is established by observing only that the old login page no longer responds.

A useful withdrawal test could examine whether old credentials, scheduled jobs or delayed requests can still cause an effective change; whether clients reach the intended successor; whether historical outcomes remain interpretable; and whether required data has received its assigned treatment. Select methods appropriate to the system and authority. A test that attempts to use retired privileges must be controlled so that an unexpected success does not create an uncontained operational effect.

Absence claims need bounded interpretation. No traffic observed during a short period does not establish that no dormant integration exists. An endpoint unreachable from one location may remain accessible from another. A deletion confirmation for one resource does not establish the disposition of its exports. Combine inventory, configuration, agreement records and appropriately scoped observations, retaining material unknowns where the evidence is incomplete.

Table 24.2. RET-06 evaluation: distinguish the action from the conclusion

Proposed closure claim	Relevant evidence to obtain	Limitation to preserve
The former writer can no longer change operational state.	Disabled or constrained effective-change paths, credential and route disposition, controlled checks at the consequential boundary.	A missing login page or removed DNS record does not establish every path.
Required historical outcomes remain usable.	Identified archive or successor records, authorized retrieval and interpretation checks, protection and format information.	Retained bytes alone do not establish readable, meaningful evidence.
Required information treatment was completed.	Item/copy identity, selected method, responsible execution, outcome and acceptance of disposition.	A generic completion certificate does not cover unidentified copies or media.
The successor provides the intended capability.	Applicable integration, task, protection and support evidence for its own configuration.	Retirement of the predecessor does not establish the successor's readiness.
Residual obligations are controlled.	Named continuing custodian, permitted access, retention/disposition trigger and unresolved-item route.	Administrative project closure does not automatically end those obligations.

A qualified completion statement can establish removal of identified paths, transfer of specified records and treatment of listed media while provider-held copies remain under a continuing obligation. Identify the remaining custodian and trigger. An unresolved resource should not disappear merely because it no longer fits the active-asset inventory.

If a required observation cannot be made, determine what other evidence or action can support the decision. A contractual assurance may contribute, but its scope and limitations remain relevant. In some cases the lack of adequate evidence can prevent a planned release or reuse. In others, controlled retention or a more conservative physical disposition may provide an acceptable alternative. The decision requires the actual facts and authorized participants rather than a universal rule that every item must be destroyed.

24.9 Retire the enabling environment and preserve usable engineering knowledge

The runtime is not the only system that can retain influence. Build agents, deployment credentials, model integrations, test fixtures, artifact repositories, signing arrangements and support accounts may continue after the operational service ends. Some should be removed;

others may need to support a successor or retained evidence. The disposition should follow their actual role rather than preserve an entire privileged environment because one historical report might be needed.

For MS-01, retain the requirement and profile revisions, selected architecture interpretation, significant analysis inputs, result status and the means of understanding relevant relationships. A frozen model can provide institutional memory without remaining connected to live deployment. Revoke or restrict obsolete integration authority separately from preserving the model's readable content. A report generator does not need indefinite write access to the retired service merely to reproduce a historical decision view.

Long-term use can require agreed native files, readable exports, libraries, format descriptions and rights to access the information. Test the retained representation against an actual review task: can a qualified successor locate SR-05's governing assumptions, reproduce the appropriate calculation and distinguish the separate twelve-second branch from the earlier case? If not, retention may preserve documents while losing their decision meaning. The answer need not be indefinite purchase of every original tool; it must be an adequate, permitted preservation arrangement.

Table 24.3. Separate retained knowledge from obsolete execution capability

Enabling contribution	Knowledge or evidence potentially retained	Authority or resource needing separate disposition
Model repository and exports	Baselines, relationships, assumptions, selected alternatives, calculations and evidence status.	Live integration tokens, unrestricted edit access and obsolete automated publication routes.
Build and artifact environment	Release identity, provenance and information needed for agreed support or investigation.	Credentials capable of releasing, deploying or altering operational content.
Test environment	Relevant procedures, configuration, observations and limitations.	Production credentials, copied sensitive data and unneeded privileged connections.
Support knowledge and tools	Instructions, diagnostic meaning and rights needed for the successor or archive.	Former supplier access, temporary accounts and unsupported executable environments.

Check whether a retained executable environment creates a new support obligation. Keeping a virtual machine solely to read a proprietary format can preserve vulnerabilities and privileges if it is left connected as though it were a supported service. A suitably isolated, controlled preservation arrangement may be appropriate, but requires its own access, integrity and use rules. An archive should not become an unmanaged production dependency by accident.

Retirement learning should inform future design and acquisition. Difficulty identifying copies, separating keys, exporting usable history or withdrawing a provider's authority can reveal a requirement that should have been addressed earlier. Preserve that finding with enough context to improve the next programme. The aim is not to blame the retirement team for architecture decisions it inherited, but to make future endings less dependent on improvised reconstruction.

24.10 Exercise: shutdown complete, obligations unresolved

Situation. A hypothetical withdrawal removes the old gateway and deletes its tenant. The project retains an encrypted backup because historical outcomes must remain available, but plans to destroy the only key that can read it. A deployment token from the retired service is still accepted by the successor. A provider confirms deletion of the active resource but states that a backup follows a separate retention arrangement. The project proposes closing RET-06 as “all access and data removed.”

Worked answer. The proposed conclusion contradicts the stated facts and the intended retained function. The gateway and active tenant actions can be recorded as completed within their scope. They do not establish removal of the deployment token's effective authority at the successor. That relationship requires specific disposition and confirmation without disrupting any legitimate continuing use.

Destroying the only archival decryption capability would defeat the stated need to retrieve historical outcomes unless another valid preservation arrangement is established. Resolve the information requirement and key architecture before taking the irreversible action. The objective is controlled retention of required information and removal of unnecessary authority, not universal destruction of every key or indefinite preservation of every credential.

The provider-held backup also remains a distinct obligation. Record its identity to the extent available, the governing treatment, permitted access, continuing custodian, expected trigger and limits of confirmation. Closing the active tenant does not establish that the retained copy has already been erased. The accurate completion statement distinguishes completed withdrawal actions from continuing retention and unresolved authority.

Part VI has connected operations, maintenance, inherited-system decisions and withdrawal to the same requirements, evidence and accountable contributions used during development. The next part examines how organizations make that discipline durable: technical management, competence and integration, independently managed systems, and AI-enabled or AI-assisted engineering. The knowledge needed to sustain a trustworthy capability must survive the project that first assembled it.

PART VII — LEAD AND SCALE THE CAPABILITY

25. Technical management and engineering control

25.1 Manage the technical work, not merely its paperwork

The logistics project has reached a point at which its next difficulty is organizational as much as technical. The receiving-side authority question requires an identity specialist and an application engineer. Persistence depends on a platform contribution. Recovery depends on a permitted authority arrangement and the people who must use it. The model export has its own discrepancy. Each issue has a plausible next action, but a list of actions is not yet a coordinated engineering programme.

Technical management connects that work to the commitments the project intends to make. It establishes who will produce the necessary results, which inputs they need, how the contributions will be integrated and reviewed, and what happens when the results do not support the intended decision. A report delivered on time is one project fact. A requirement demonstrated under the applicable conditions is another. The programme needs both facts without allowing one to impersonate the other.

For an IT organization, much of the infrastructure already exists: release planning, service ownership, architecture review, change control, risk management and operational reporting. SSE should strengthen the technical content exchanged through those arrangements. It need not introduce a parallel approval chain with a second version of every record. For an engineering organization, the corresponding challenge is to make security-relevant software, information, identity and operational dependencies sufficiently explicit within its established technical planning and control.

NIST's systems security engineering treatment includes eight technical-management processes, covering planning, assessment and control, decisions, risk, configuration, information, measurement and quality assurance. Their purpose is to support and govern the engineering effort, not to replace the technical work that produces the system. [1, Appendix I] NASA similarly connects technical planning, assessment and decision analysis to the information needed for programme commitments. [6, Sections 6.1, 6.7 and 6.8]

TE-07 organizes the remaining concerns through technical-management record TM-07. The base case is still developmental: its adverse result, unsigned decision and conditional recovery alternatives remain visible. A work package identifies work to be resourced and completed, not evidence that it has occurred.

25.2 Integrate the security contribution into the technical plan

A systems engineering management plan, or the organization's equivalent, should explain how the technical effort will be conducted. The security contribution belongs in that account. It identifies the protection questions that affect requirements, alternatives, implementation, evaluation and continuing use, together with the specialist work and resources needed to answer them. Its detail should fit the project; a small bounded change does not require the same planning volume as a new, long-lived capability.

Begin with the decisions rather than a catalogue of deliverables. For TM-07, the next commitment concerns targeted development and evaluation of the receiving-side design and recovery alternatives. The plan must obtain enough information to discriminate among those alternatives and correct the known problems. It must not quietly substitute a production-release objective simply because a supplier has offered a delivery date.

Identify the controlling requirement and profile revisions, the alternatives being investigated, and the important unresolved inputs. An activity that depends on REC-P1 cannot be planned as though its authority rules are already settled. An activity examining transaction persistence may proceed using an explicitly limited normal-mode configuration, provided its conclusions are not extended to unexamined recovery transitions. The planning distinction is between a useful partial investigation and a falsely complete result.

Table 25.1 applies the eight technical-management processes to the same case. Keep their information in the existing repository, backlog or engineering plan when it preserves the necessary meaning and control. Eight process contributions do not require eight new documents.

Table 25.1. Technical-management contributions to the continuing case

Process	Case question and work	Information needed for control
Project planning	What work will resolve timing, writer coverage, persistence, recovery authority and exchange meaning?	Defined scope, inputs, outputs, participants, resources and dependent commitments.
Project assessment and control	Do achieved results support the next commitment, and what departure needs action?	Technical status against criteria, current forecasts, discrepancies and corrective direction.
Decision management	Which investigation, design or restriction should proceed?	Alternatives, binding conditions, evidence, uncertainty, rationale and decision authority.
Risk management	What consequence or uncertainty does each unresolved issue create?	Current position, proposed response, owner, effectiveness evidence and reconsideration triggers.
Configuration management	Which requirements, design, article and conditions were actually considered?	Identified baselines, controlled changes, status and relationships among configurations.
Information management	Can the appropriate participants obtain, interpret and retain the necessary records?	Authoritative sources, access, exchange, protection, retention and usable handover.
Measurement	What observation helps distinguish adequate progress or behaviour from an unsupported claim?	Defined quantity, collection method, scope, denominator, uncertainty and response rule.
Quality assurance	Are the agreed work and resulting information being produced with sufficient integrity and discipline?	Applicable criteria, process and product checks, independence where needed, findings and closure evidence.

The processes interact. A measurement may reveal that a proposed delay exceeds its allocation. Assessment establishes the effect on the commitment. Decision management compares responses. Planning and configuration management implement the chosen direction. Information management preserves the basis, while quality assurance can challenge whether the activities and records meet their agreed criteria. Risk management retains the consequence and remaining uncertainty throughout.

The security lead contributes to this interaction but does not become the owner of every process. The project manager controls resources within the assigned authority. The systems engineering lead coordinates the technical result. Application, identity, platform and operational specialists supply their expertise. The relevant policy and risk authorities make decisions within their actual powers. TM-07 must represent those relationships accurately enough that an unresolved question has a real route to resolution.

25.3 Define work packages around results that can be reviewed

A work package should identify a result whose adequacy can be judged. “Complete security engineering” is too broad. “Produce an authority analysis” remains incomplete unless the subject, inputs and expected use of that analysis are clear. The package needs a responsible performer, an integrating recipient and a criterion for deciding whether the output is adequate for its purpose.

For the case, the timing package should examine the source event, source consistency, distribution, receiving caches and the final authority-to-commit relationship. Its output is not necessarily a passing result. It may establish a design conflict, expose an unavailable service commitment or recommend a different allocation. A satisfactory investigation can produce an adverse engineering conclusion.

Combine supplier, deployment and application information to address writer coverage; a gateway inventory alone cannot establish A-01. For persistence, obtain platform and interruption semantics. For recovery, obtain authority and complete-task information. These inputs make the work packages evaluable where labels such as durable storage or fast recovery would not.

Table 25.2 organizes four work packages. Their identifiers are teaching aids within TM-07 and do not replace AP-05's agreement obligations or the existing VP/VA evaluation records.

Table 25.2. TM-07: proposed work packages for the next development commitment

Package	Inputs and participants	Reviewable result and boundary
WP-07-01: source-to-commit authority	SR-02, SR-03, SR-05; AUTH-P1, NET-P1, TIME-P1; identity and receiving-application specialists.	Coherent event and timing account, candidate correction, affected evaluations and unresolved external commitments. Not a performed verification result.
WP-07-02: writers and protected state	A-01; IC-01, TX-P1 and RZ-01 proposals; deployment, application and platform owners.	Reconciled writer model and defined persistence, interruption and migration obligations. Coverage remains limited where actual configuration is unknown.

Package	Inputs and participants	Reviewable result and boundary
WP-07-03: usable recovery alternatives	SR-11/SR-12; ARC-B/ARC-C; policy, service, identity, operations and support participants.	Permitted candidate authority basis, complete task and dependencies, feasibility findings and selection conditions. Not operational authorization.
WP-07-04: model and evidence exchange	MS-01, relevant exporter and report definitions, CASE-05 evidence statuses.	Reviewed preservation of units, revisions, alternatives and open/adverse findings in a representative exchange. Not validation of the implemented system.

Each package needs a completion rule suited to its maturity. A definition package can be complete when its required questions are answered or its unresolved decisions are explicitly identified with enough information for the next authority. An implementation package cannot be complete merely because the design explanation is convincing. An evaluation package requires the identified work to have occurred and the resulting observations and limitations to be recorded.

Dependencies should be represented explicitly. WP-07-03 can develop a task model while the policy interpretation is pending, but cannot close recovery-conformance work without a defined criterion. WP-07-04 can repair the treatment of an uncertainty allowance without waiting for a real endpoint test, but its repaired report remains an analysis report. These distinctions allow useful concurrency without creating fictional progress.

A work breakdown also needs integration effort. Four individually adequate outputs can still disagree about what “current authority,” “commit” or “recovery” means. Assign time and responsibility for reconciling those meanings. Where a disagreement changes a parent requirement, route it through requirements and decision management rather than asking each team to choose the interpretation that makes its own package easiest to close.

25.4 Resource the actual dependency chain

An engineering plan is not feasible merely because every task has a name beside it. Required competence, available time, access, facilities and decision participation must coincide when the work needs them. A specialist listed on several projects cannot provide the same hour to all of them. A booked test environment is not useful when the interface owner cannot supply the conditions required for the test.

Consider a separate, invented capacity illustration. One qualified specialist has twenty-four hours available for the relevant work during a planning week. Three packages request eighteen hours for timing analysis, ten for persistence review and eight for a model-migration review. Demand is thirty-six hours before any allowance for unplanned clarification. If the plan also reserves four hours for that clarification, total demand is forty hours against twenty-four available: a sixteen-hour gap.

The arithmetic establishes a conflict in this stated allocation; it does not estimate the duration of a real project. The activities may be divisible or may admit other qualified contributors, and some effort estimates may change after examination. Those possibilities need an actual revised

plan. Relabelling the specialist as “consulted” does not create capacity if the package still depends on their substantive analysis.

Resolve a capacity conflict by changing sequence, adding qualified support, narrowing the next commitment or investigating the uncertainty that matters most. A deferred review leaves its claim unestablished. Additional staffing helps only to the extent that it supplies the required competence as well as available hours.

Account for the customer as well as the supplier. An identity provider's engineer, a service owner's policy decision and access to operational representatives may be outside the implementing team's direct control. The plan should identify the request, the needed result, the commitment that depends on it and the escalation route. A schedule assumption is not an agreement from the external participant.

Reserve is useful when its purpose and control are visible. It can accommodate clarification, discrepancy investigation or targeted repeat work. It is not unused capacity that every package may independently consume. Management should know when reserve is being used and why. Repeated use for the same unresolved interface may indicate a structural planning problem rather than unusually bad luck.

The objective is not to protect engineering from all time pressure. It is to make the consequences of that pressure explicit. Accelerated work may justify parallel activities, focused evidence or a narrower release. It cannot make an unperformed evaluation true. A responsible plan preserves the distinction between changing how work is done and changing the technical claim that the work can support.

25.5 Measure technical progress without manufacturing confidence

Measurement begins with a decision question. Does the candidate design have enough freshness margin? Is recovery preparation keeping pace with normal changes? Is the writer inventory adequate for the claimed scope? Are critical evaluations blocked by unavailable inputs? Different questions require different measures. No single completion percentage answers them all.

Distinguish useful operational outcomes, technical performance and work-process measures. A service owner's outcome concerns timely, correct administrative support for dispatch. A technical measure might concern the age of authority at commit. A process measure might concern the age of an unresolved interface action. Their relationships matter, but none should be renamed as another. Closing an old action is useful only if the underlying issue has an adequate disposition.

NASA's technical-assessment treatment connects measures and review results to technical plans and decisions. [6, Section 6.7] A usable measure identifies its quantity, population or conditions, method, source, interval, interpretation and response. Keep estimates distinct from observations: an eight-second allocation is not measured implementation performance.

A separate hypothetical dashboard illustrates the denominator problem. Suppose twenty-six evaluation obligations comprise nineteen recorded passes, two failures, three blocked activities and two not performed. Nineteen of the twenty-one executed obligations passed, approximately 90.5 percent. Nineteen of all twenty-six obligations passed, approximately 73.1 percent. Both ratios are arithmetically correct; neither justifies the statement that the system is 90.5 percent secure or ready.

The dashboard should identify what each ratio describes, preserve the failures and show the blocked work. A critical authority failure can prevent a particular commitment despite many unrelated passes. An unperformed recovery validation cannot disappear because the denominator excludes blocked activities. Conversely, a known failure in one criterion does not erase the useful evidence produced for other properties.

Technical trends should retain comparable meaning. A reduction in reported discrepancies can reflect improved design, reduced testing, changed classification or incomplete reporting. An increase can reflect more thorough examination rather than worsening implementation. Compare scope, configuration, methods and exposure before interpreting direction as improvement. The point is not that indicators are useless; it is that their causal interpretation needs engineering judgement.

Use a small number of decision-relevant measures rather than a large catalogue that nobody acts upon. For TM-07, suitable candidates include completion of the relevant writer reconciliation, remaining source-to-commit uncertainty, readiness of the recovery authority inputs, and whether exchanged evidence preserves its actual status. Each should have a defined owner and action when its state threatens a commitment. An indicator without a response is information, not control.

25.6 Preserve the relationship among configuration, information and decisions

A decision baseline identifies the combination of requirements, profiles, architecture variation, analysis configuration, realized article and evidence actually considered. These records may change independently. Links to the latest versions support current work but do not preserve an earlier decision basis.

TM-07 should identify which combinations are under consideration. The eight-second and twelve-second model branches remain different analytical configurations. A repaired exporter changes the representation used for review, not necessarily the implemented endpoint. A new storage controller can alter the realized system while the application binary remains unchanged. Technical control depends on recognizing those different kinds of change.

Configuration management maintains identity, relationships and controlled evolution. Information management makes the relevant material available, intelligible, protected and retainable for its users. One without the other is inadequate. An impeccably versioned model that a maintainer cannot open may not support continuing engineering. A readable report with no identifiable source configuration cannot reliably support a particular release. [1, Appendices I.5–I.6; 6, Sections 6.5–6.6]

The model-based environment can support impact queries and consistent views, but the team still needs to review the meaning of the relationships. A query can identify that SR-05 depends on a changed interval. It cannot establish that every missing link represents an unaffected property. A report generator can preserve an adverse status, but cannot decide the authority's response to it.

Information protection should also account for the value and sensitivity of the engineering material. A dependency map, privileged interface description or evaluation weakness may require controlled distribution. The aim is sufficient access for competent work without

indiscriminate disclosure. Restrictions that prevent a necessary review must be visible as a limitation on that review, not concealed beneath an unqualified approval label.

Preserve decisions with their rationale and conditions. A choice to continue a bounded investigation is not a production design selection. An approved deviation does not rewrite the historical result against the earlier requirement. The record should let a future reader determine what was believed, what was known, what remained open and which change would require reconsideration. This is practical engineering memory, not archival paperwork for its own sake.

25.7 Use quality assurance to strengthen the work and its products

Quality assurance and an assurance case are related but distinct. Quality assurance examines whether the agreed processes and work products meet the applicable expectations and are being managed adequately. An assurance case argues for specified claims about the system using evidence and reasoning. A sound quality process can support confidence in evidence, but does not automatically establish every technical property of the product. NIST retains quality assurance as a technical-management process with its own preparation, evaluation and improvement work. [1, Appendix I.8]

For TM-07, useful quality checks include whether requirements retain their controlling definitions, whether evaluations identify the actual configuration and method, whether contrary results survive reporting, and whether corrective work is closed on an appropriate basis. These checks are substantive. A document containing the right headings can still fail them.

Examine the method as well as the output. A timing report may be numerically correct but use the first rejected probe instead of the event required by the criterion. A supplier review may be independent in organizational terms but lack access to the necessary observations. A generated dashboard may correctly compute a ratio whose label misrepresents its denominator. Quality review should be capable of identifying those problems rather than certify that the document looks complete.

Independence should be proportionate to the consequence and the error being challenged. A designer can explain the mechanism; another competent participant can challenge its assumptions or evidence. Using a separate organization does not compensate for insufficient expertise or information. Nor is duplicating every activity necessarily the best use of scarce resources. The plan should state what separation is required, why it matters and how material disagreement reaches the decision authority.

Corrective action needs a closure criterion. Assigning an owner and approving a plan establishes that work is intended. A repaired implementation requires evidence relevant to the affected behaviour. A corrected report requires confirmation that the intended information and status are preserved. Where a finding concerns a broader process weakness, closure may also need evidence that the revised process is actually being used. These conclusions should not be collapsed into one undifferentiated “done.”

The quality function must itself remain open to challenge. An inappropriate template, inaccessible repository or excessive review burden can impede useful engineering. Improvement should examine whether the control reduces relevant error, supports decisions and can be sustained. The goal is not maximal formality. It is work whose integrity and limitations can be understood well enough for justified reliance.

25.8 Tailor controls and escalate with precision

Tailoring selects the extent, timing and form of work needed for the actual system and commitment. It should consider consequence, novelty, coupling, available evidence, supplier influence and the cost of correcting an error later. A small configuration change and a new recovery authority arrangement can justify different review depths even within the same service.

In incremental delivery, some requirements span releases or several work items. A story about adding a status display does not remove the obligation to preserve historical outcome meaning. A platform migration may affect evidence across several teams. The technical plan should retain those cross-cutting relationships while allowing implementation work to be divided into useful increments. A backlog can hold them, provided its closure conventions do not make a completed story equivalent to a completed system argument.

Assessment should trigger action when the next commitment lacks support. The action may be further analysis, correction, a narrower investigation, changed sequencing or an authority decision about a remaining limitation. State the issue in terms that the recipient can act upon. “Security objects” is less useful than “the proposed recovery claim depends on an approval service unavailable in the specified outage; the complete task remains unestablished.”

The right to raise a technical concern is not the same as unrestricted authority to stop every activity. The organization's governing arrangements should identify who can restrict work, approve a trial, accept a deliverable or authorize use. SSE supplies a clear technical recommendation within those arrangements. Where a recommendation is not followed, preserve the rationale, evidence and responsible decision rather than rewrite the technical position to imply agreement.

Good escalation also identifies what can continue. Repairing the model export may be independent of completing a recovery role definition. A safe, authorized laboratory investigation may proceed without an operational authorization. Distinguishing these activities avoids both unjustified release and unnecessary paralysis. The decision should match the evidence and the actual consequences of the proposed work.

25.9 Exercise: a green schedule with an unresolved technical commitment

Situation. A project marks all four TM-07 work packages complete because their reports arrived on time. The timing package contains the stipulated E-05-03 adverse observation, the recovery report still assumes access through the unavailable approval portal, and the exchange report excludes the unresolved result field. A dashboard displays the 90.5 percent executed-test pass ratio. The manager proposes a complete operational-release recommendation.

Worked answer. Submission and technical completion have been conflated. Review each package against its agreed purpose and maturity. An adequate investigation may be complete while its design remains unacceptable or incomplete. The late-commit finding must remain adverse under its stipulated conditions. The recovery dependency prevents the claimed complete restoration outcome from being established. The exchange result cannot support preservation of evidence status when that status was excluded from the review.

The ratio describes only the specified executed evaluations. It neither removes the failures and blocked work nor establishes readiness of the complete capability. Retain the useful reports and observations, identify the corrective and decision work, and revise the forecast for the commitment they actually support. No requirement passes because the report containing its failure arrived on time.

Carry forward. Technical management makes engineering work, evidence, resources and decisions coherent within a project. Chapter 26 asks how the organization sustains the people, methods, infrastructure and incentives needed to perform that work across projects and through change.

26. Organizational capability, integration and competence

26.1 Make good engineering repeatable without making it mechanical

A capable project team can solve a difficult authority or recovery problem and still leave the next project to solve it again. The explanation may remain in one person's memory. The model may depend on an unavailable tool configuration. The organization may reward delivery dates while assigning no one to maintain the evidence or support arrangements. The weakness is not necessarily a lack of technical insight. It is a failure to turn that insight into a durable organizational capability.

Organizational capability means being able to provide the necessary people, methods, infrastructure, information and decision support repeatedly under the conditions in which projects actually work. It is more than creating an SSE office. An office can coordinate expertise and own shared resources, but the security contribution still has to reach requirements, design, implementation, evaluation and continuing use. Conversely, distributing security responsibility among every project without supplying expertise or common support can leave important work unowned.

Two organizational mistakes are worth distinguishing. In the first, security becomes an isolated specialty that reviews work after consequential design choices have been made. In the second, the organization declares security everybody's responsibility and loses the specialist depth needed to examine it. The remedy is neither a stronger silo nor the disappearance of specialists. It is defined specialist contribution within the common systems engineering effort.

NIST's organizational project-enabling processes cover lifecycle models, infrastructure, portfolios, human resources, quality and knowledge. [1, Appendix J] These processes provide conditions in which projects can perform their work. Their existence does not demonstrate that a particular delivered system is trustworthy. The organization must examine both the capability it provides and the results obtained when projects use it.

CAP-07 proposes organizational support and COMP-07 a task-based competence record. These teaching structures illustrate how to make SSE practical without assigning every IT practitioner design authority or expecting every systems engineer to master every security technology. They are not a credentialing scheme or an assessment of an employer.

26.2 Establish a capability charter with a useful service boundary

A capability charter should explain what support the organization provides and what responsibilities remain with the projects and service owners. “Own cybersecurity engineering” is too broad to guide either side. Identify the functions: specialist consultation, method stewardship, model and evidence infrastructure, independent challenge where needed, reusable patterns, competence development and escalation of cross-project concerns.

CAP-07 should name the intended users of those functions. A small acquisition team may need help writing reviewable supplier work obligations. An application team may need support analysing authority at commit. An operational team may need engineering help determining whether a changed recovery dependency invalidates its readiness assumption. The service

should be organized around these decisions, not only around the specialist team's preferred deliverables.

The charter should also state its limits. The capability team may advise on technical adequacy without owning the operational consequence. It may maintain a requirements-writing method without approving every requirement. It may support an assurance review without possessing the authority to accept the customer's risk. Clear limits protect both the provider and recipient from assuming that an informal consultation constituted a complete assessment.

A centralized arrangement can concentrate scarce expertise and maintain common methods. It can also become a bottleneck or lose knowledge of local operational context. Embedded specialists can develop close understanding of a project but may become isolated from their professional peers or pressured to suppress inconvenient findings. A federated arrangement can combine shared method and specialist support with embedded contributions, but needs explicit ownership of the interfaces between them.

There is no universally correct organizational chart. Evaluate the arrangement against demand, consequence, available competence and the ability to sustain work. For the logistics case, the practical question is whether the project can obtain the application, identity, platform, operational and assurance contributions required by TM-07. A centralized security team that can only review a control matrix does not supply all those contributions. An embedded specialist without access to the relevant platform expertise does not supply them either.

The charter needs an intake and prioritization method. Requests should identify the decision, scope, required date, available information and consequence of delay. The provider should return an agreed scope and a realistic contribution, including what it cannot establish. “Reviewed by SSE” should never conceal whether the work was a one-hour consultation, a requirements assessment or an independent examination of implementation evidence.

26.3 Provide the organizational foundations projects depend on

The enabling processes should be recognizable in ordinary organizational work. They need not appear as six new committees. Their usefulness lies in supplying stable, maintainable conditions for projects while allowing justified tailoring. Table 26.1 applies their purposes to the security specialty through concrete organizational contributions.

Table 26.1. Organizational support that makes the project work possible

Enabling concern	Useful organizational contribution	Evidence that the contribution is usable
Lifecycle model management	Shared process expectations, tailoring rules and connections to acquisition, delivery, operation and retirement.	A project can identify the work needed for its next commitment without inventing a parallel security lifecycle.
Infrastructure management	Protected modelling, analysis, build, test and evidence facilities with appropriate access and support.	Qualified participants can use the required configuration and recover necessary work when a service is unavailable.
Portfolio management	Visibility of shared dependencies, specialist demand and cross-project priorities.	Competing commitments are reconciled rather than assigning the same scarce resource to incompatible dates.

Enabling concern	Useful organizational contribution	Evidence that the contribution is usable
Human resource management	Defined competencies, development routes, succession and access to specialist review.	People can demonstrate the relevant task at the required level of responsibility, with known limits.
Quality management	Organization-wide expectations for methods, work-product quality, findings and improvement.	Projects use the expectations, report material departures and obtain support to correct recurring weaknesses.
Knowledge management	Maintained rationale, patterns, lessons, source records and usable handover.	A new participant can reconstruct a consequential decision and identify when its assumptions no longer apply.

Lifecycle expectations should describe purposes and outputs, not only document names. A project using incremental delivery may hold its requirements and evaluation relationships in linked repositories. Another may use a controlled specification and review package. Both need coherent meaning and adequate evidence. Standardization should make those obligations easier to understand, not force identical storage formats where they add no value.

Maintain the enabling infrastructure as a capability. MBSE repositories depend on libraries, scripts, access, backups and administration; test environments may require specialized hardware or agreements. Preserve the contributions needed to repeat material analyses, not merely the original files. Plan both availability and integrity.

Portfolio management should make common dependencies visible. Several projects may use the same identity service, model library or recovery-support organization. A change to that shared contribution can affect multiple technical arguments. The organization needs a way to communicate the change, identify affected uses and coordinate evaluation without assuming that one project's result applies everywhere. Shared support can reduce duplication while also concentrating risk; both effects belong in the portfolio view.

Quality and knowledge management should form a feedback loop. If several projects misunderstand transaction outcome semantics, the organization may need a better pattern, an improved review question or targeted training. If the pattern is correct but unavailable in the permitted environment, the issue is access and support. A generic instruction to “increase awareness” will not resolve every kind of recurring problem.

26.4 Define competence through the work a person must perform

Competence concerns the ability to perform a task appropriately in its context, not merely familiarity with its vocabulary. An individual may understand access-control principles while needing support to evaluate distributed persistence. Another may be skilled in modelling but unfamiliar with adversarial manipulation of an input. Both can contribute effectively when their strengths and limits are explicit.

INCOSE’s public competency overview addresses tailorable knowledge, skills, abilities and behaviours. [54, public overview] COMP-07 applies that orientation by identifying the task, expected contribution and supporting evidence. Competence for a particular responsibility still needs task-specific evaluation.

For each task, identify the required knowledge, practical reasoning, context, support and authority. Distinguish performing under supervision from independent performance and from reviewing another person's work. The reviewer needs competence in the proposition being assessed, not necessarily every implementation detail. Independence of judgement requires enough knowledge and access to challenge the important assumptions.

Table 26.2 illustrates bridge competencies. The entries describe possible development targets rather than deficiencies presumed to exist in everyone holding the named role. Many practitioners already possess several of these abilities through their experience.

Table 26.2. COMP-07: task-based routes into integrated SSE work

Familiar contribution	Engineering extension to practise	Evidence of the particular capability
IT administration and service management	Relate installed paths, changes and recovery tasks to the system boundary, requirements and dependencies.	Explain one real or representative task, identify an overlooked dependency and propose a controlled impact assessment.
Cybersecurity assessment	Distinguish obligation, mechanism, configuration, evaluation result and scope of the supported claim.	Review a mixed evidence dossier without turning a certificate, trace link or future test into system-level proof.
Systems or architecture engineering	Incorporate adversarial influence, authority state and security-relevant software behaviour into alternatives and allocation.	Develop and challenge an authority-to-commit model with the appropriate security and application specialists.
Software and platform engineering	Explain how implementation and enabling-system behaviour preserve the allocated protective properties.	Trace a persistence or timing property through design, configuration, interruption conditions and evaluation.
Acquisition and programme management	Obtain defined engineering work, evidence, customer contributions and decision criteria.	Distinguish acceptance of an adequate investigation from acceptance of the resulting product or authority to use it.
Technical leadership and review	Integrate specialist findings, resolve interfaces and communicate a bounded recommendation.	Preserve contrary evidence, identify decision rights and explain which next commitment is supported.

A course can establish concepts and provide practice. Completion alone does not demonstrate every capability in the table. A professional certification or degree can contribute meaningful evidence of education or assessed knowledge, but should be interpreted within its actual scope. The organization still needs to establish that the person can perform the assigned task in the relevant environment. This book does not confer a professional designation or an unrestricted right to make technical or operational decisions.

The expected depth depends on the role. An executive should be able to distinguish a technical failure from an accepted risk and ask what evidence supports the proposed commitment. That does not require the executive to implement a persistence algorithm. An assessor should understand the relevance of receiving-side commit without being presumed to possess design authority over the application. Accessibility means enabling informed participation, not erasing the differences between contributions.

Record competence with context and limits. A successful exercise on a simplified service may justify supervised participation in a more complex project, not independent approval of a safety-critical control system. Conversely, unfamiliarity with one tool's syntax does not prove lack of systems reasoning. Separate tool proficiency, domain knowledge, engineering method and decision authority so that development addresses the actual need.

26.5 Develop people through supervised, reviewable practice

A practical development route combines explanation, guided application, challenge and increasing responsibility. Begin with a bounded task whose inputs and expected reasoning are visible. The logistics case provides several: writing a requirement that distinguishes login from commit, identifying a recovery dependency, interpreting the E-05-03 interval or tracing a changed model parameter to affected evaluations.

Pair complementary expertise. An experienced administrator can explain what actually happens during a failed change. A systems engineer can help formalize the boundary and dependencies. A security practitioner can examine authority and misuse. A platform specialist can identify the persistence assumptions. The learning is not a one-way transfer from engineering to IT; the engineering model improves when operational knowledge corrects an unrealistic abstraction.

Use worked products as evidence of development. Ask the participant to explain their model, defend the allocation, identify uncertainty and respond to a contrary observation. A polished diagram is less informative than a clear explanation of why one result does not support the proposed conclusion. Assess the reasoning, not whether the participant reproduces a preferred phrase from the training material.

Progression should be conditional on demonstrated capability. A participant who can produce a requirements draft may next perform the task under supervision on an actual project. Independent work needs evidence of consistent performance and judgement within a defined scope. Reviewing others adds another responsibility: the reviewer must recognize omissions, understand method limits and communicate findings constructively. The organization should provide access to deeper specialists when the task exceeds the reviewer's competence.

Make learning time real in the resource plan. Pairing, review and correction require experienced people. Assigning mentoring as invisible work can overload the same specialists whom the organization is trying to make less indispensable. A useful development programme identifies the tasks, available reviewers and expected contribution to future capacity. It does not assume that attendance automatically solves a staffing shortage.

Maintain competence through relevant practice and feedback. Changes in technology, role, operating context or analytical method can create new learning needs. Not every change requires retraining the entire workforce. A targeted exercise on a revised authority interface may be more useful than another generic security-awareness session. The depth should follow the decision and its consequences, with appropriate supervision where knowledge is still developing.

26.6 Integrate specialties through shared decisions and records

Integration becomes tangible when specialists work on the same consequential question. For recovery, the question is not whether each department has approved its own document. It is whether designated people can perform the permitted task within the required conditions while

the relevant protections remain effective. The answer requires compatible operational, identity, application, platform, human and assurance contributions.

A joint review should identify the issue, the represented configuration and the decision being supported. Participants should know what contribution is expected from them. Asking a safety specialist to attend without a defined physical or operational question can produce ceremonial involvement. Asking that specialist to assess a security-induced shutdown in a particular operating scenario can produce design-relevant information. Where no safety consequence is established, do not label every mission concern a safety requirement merely to strengthen its rhetoric.

Shared vocabulary helps, but words alone do not create agreement. The participants may all say “recovery complete” while referring to different events: the gateway responds, authority is re-established, the management configuration is active, or field operations have returned to an acceptable condition. Record the distinctions and their interfaces. A glossary should support this explanation rather than suppress legitimate domain meanings.

The model-based environment can provide role-appropriate views of shared information. The operator needs a readable task and mode account; the designer needs allocation and state detail; the assessor needs obligations and evidence status. They should be able to raise a concern without first mastering the authoring tool. Comments should connect back to the underlying element or relationship and have a traceable disposition.

Agreement is not the same as unanimous comfort. A specialist may accept that an option is feasible while retaining a concern about the evidence needed before use. An authority may choose a different course from the technical recommendation. Preserve those distinctions. Removing an adverse finding to obtain a clean meeting record does not strengthen either the design or the organization's ability to learn.

A recurring integration problem deserves a structural response. If the identity team is routinely consulted after gateway selection, change the planning and acquisition interface. If recovery staff never receive usable model views, change the information product. If disputes recur because no one owns the receiving-side interface, assign that responsibility. Integration is improved by altering how the work connects, not merely by adding another meeting to the calendar.

26.7 Reuse knowledge without propagating unexamined assumptions

Reusable patterns can help projects avoid rediscovering common problems. A requirement pattern can remind a team to define the consequential event. An interface pattern can distinguish receipt, commit and outcome knowledge. A review aid can expose shared recovery dependencies. These are valuable contributions when they retain the conditions under which they are useful.

A pattern should identify its purpose, context, applicability, assumptions, variation points, limitations and evidence basis. It should also identify a maintainer and a way to report problems. A template with a field labelled “trusted supplier” may spread ambiguity more efficiently than an informal discussion. Reuse is beneficial when it transfers sound reasoning, not simply consistent wording.

For the case, a transaction pattern may include expected-generation checking and persistent identity suppression. A receiving project still needs to establish the storage semantics, identifier

lifetime, authority context and operational consequences. Copying the pattern into a model does not transfer the earlier implementation evidence automatically. A pattern can be correct while its application is inappropriate.

Libraries also need change control. A corrected definition, updated analysis script or retired extension can affect several projects. Maintain the version each project used and identify which changes require review. Automatically updating every model to the newest library may improve consistency in one respect while destroying the reproducibility of earlier analyses. Keeping an obsolete version forever can preserve a known error. The organization needs both controlled migration and retained historical identity.

Capture lessons at the level that another team can use. “Improve communication” is rarely enough. A useful lesson identifies the omitted interface, the assumption it concealed, the consequence for the decision and the change made to the method or information product. The lesson should state whether the change's effectiveness has been evaluated. An agreed improvement action is not yet evidence of improved outcomes.

Knowledge continuity also requires practical access. Retaining a native model file without its libraries, conventions or permitted reader environment may preserve bytes but lose meaning. A successor should be able to reconstruct at least the consequential requirements, rationale, evidence and outstanding limitations. More detailed reproducibility may require additional tool and data arrangements. Agree those requirements while the original team and suppliers remain available.

26.8 Evaluate capability without confusing activity with maturity

An organization can improve its processes while still operating systems with known limitations. It can also obtain a good outcome through circumstances that conceal weak processes. Capability assessment and system assurance therefore answer related but different questions. Neither should be inferred entirely from the other.

CAP-07 should use evidence of how work is performed and whether the organizational contribution is useful. Are specialist requests scoped and fulfilled? Can projects obtain the needed facilities? Do requirements retain their sources? Are adverse findings preserved and acted upon? Can a successor reconstruct a decision? These questions examine capability more directly than counting meetings, templates or licences.

Outcome measures still matter, but need context. A lower rate of late interface discovery could indicate better early analysis if project type, evaluation depth and reporting remain comparable. An increase in findings after a new review method may mean that previously hidden problems are now visible. A quiet period without incidents may be welcome but does not establish that the capability can handle a disturbance it has never encountered.

A useful pilot compares the current workflow with a proposed improvement on a defined class of work. For example, introduce a shared authority-at-commit review into a small number of representative changes. Record the effort, information gaps discovered, decisions affected, correction burden and usefulness to the receiving team. Do not promise a universal return on investment from one successful case or assume that an unfavourable pilot result proves the discipline unnecessary.

Evaluate the cost of the support as well as its benefit. An approval bottleneck, redundant model entry or inaccessible evidence requirement can undermine adoption. Simplify the method where it does not contribute to the decision. Retain rigor where the consequence warrants it. A capability that depends on heroic overtime or a single irreplaceable reviewer is not sustainably resourced even when its current outputs are excellent.

Improvement should have ownership and follow-through. A recurring problem may require changing a policy, funding an enabling system, developing competence or renegotiating an interface with another organization. Training alone cannot resolve every authority or capacity problem. The organization should identify the cause it is addressing and the evidence that would show the response working.

26.9 Preserve professional challenge through organizational change

Leadership turnover, restructuring, supplier change and staff departure can alter the conditions on which engineering work relies. Treat those events as possible changes to capability and accountability, not as proof that every technical conclusion is invalid. Identify which responsibilities, access arrangements, resources and assumptions are affected.

A new manager should receive an intelligible account of the capability's purpose, current commitments, material limitations and decision boundaries. An incoming specialist should be able to find the governing requirements and understand why the present design was chosen. A departing supplier should not take the only usable explanation of a recovery dependency. These are handover outcomes to establish, not merely files to list.

Technical challenge should be both candid and precise. People need a route to report an unsupported claim without being required to exaggerate it into an incident. They also need to distinguish a serious requirement failure from a personal preference for a different tool. Constructive disagreement identifies the proposition, evidence, consequence and possible resolution. It is not weakened by acknowledging uncertainty; that acknowledgement often makes the concern more actionable.

Leaders reinforce the capability through ordinary decisions: providing time for review, preserving adverse findings in summaries, funding support and refusing to equate an approved plan with completed work. They also need to challenge excessive process that does not improve the decision. The desired culture is not unconditional deference to a security function. It is disciplined cooperation in which technical claims can be tested and responsibility remains visible.

26.10 Exercise: more findings after the new review

Situation. An organization introduces a joint engineering and operational review. The next set of projects reports twice as many security-relevant interface findings as the previous set. One manager concludes that the new method has made quality worse. Another concludes that the increase proves the method is successful. Both want to make the result an enterprise maturity score.

Worked answer. Neither conclusion follows from the count alone. Examine project comparability, review depth, reporting rules, when findings were discovered, their consequence and whether the new review changed a decision or prevented later rework. More findings can

reflect improved discovery, more complex projects, worse design or a changed classification rule. The causal explanation requires evidence.

Retain the count as one observation and connect it to useful measures: resolution quality, late discovery, effort, affected commitments and the receiving teams' ability to act. A capability review should also examine whether qualified participants, information and resources were available. Do not use an enterprise score to erase a particular project's unresolved authority or recovery limitation.

Carry forward. A durable organizational capability connects specialist depth, common engineering work and accountable decisions. Chapter 27 extends that reasoning to situations in which the participating systems and organizations do not share one management authority or one development schedule.

27. Systems-of-systems and critical infrastructure

27.1 Recognize when the management boundary changes

The administration capability has a defined system boundary and identified external dependencies. Its engineering becomes a different kind of coordination problem when the required outcome depends on systems that remain independently managed, independently useful and subject to different priorities. The challenge is no longer only to integrate components within one delivery. It is to obtain and sustain compatible contributions from participants who can make their own decisions.

A system-of-systems perspective is useful in that setting. It does not follow merely from connecting many devices or drawing a large network diagram. Ask whether the participating systems retain their own operational purposes, management, resources and development paths while contributing to a wider capability. The historical US Department of Defense summary of its systems-of-systems engineering guide emphasizes these distinctions and the resulting limits on a coordinating team's authority. [53, pp. 1–4] The summary is used here as an explanatory source, not as a current policy governing the reader's organization.

A regional service, industrial estate, transport network or defence capability may contain such relationships. An enterprise can also contain them where divisions control their own systems and budgets. The label matters less than the consequence: a requirement written by one coordinating team does not automatically become a binding commitment from every constituent-system owner.

SSE contributes to understanding the security and resilience implications of those relationships. It examines authority, information meaning, dependency, failure propagation, recovery and the evidence needed for the combined use. Systems engineering integrates that contribution with the wider technical and operational work. Neither discipline acquires political, regulatory or operational powers merely by drawing the wider system boundary.

SOS-07 is a separate regional infrastructure-repair vignette examining shared outcomes, independent management and resource conflicts. Its invented timing values belong to that example, not to SR-11’s fifteen-minute administration task. It neither deploys the administration case nor extends its requirements to national infrastructure.

27.2 Define the shared outcome before mapping the network

In SOS-07, several infrastructure operators need to coordinate repair work after a regional disruption. A coordination service receives requests, helps establish priorities, communicates work assignments and tracks their status. Infrastructure owners retain responsibility for their assets. A maintenance contractor supplies qualified crews and equipment. Communications and identity services support the exchanges. Local operators and field personnel act on the resulting information.

The useful outcome is not simply that the coordination application remains available. Authorized work must reach the right people, at the right location, with enough information and resources to perform it, and the result must be understood by the participants who depend on it. Incorrect or duplicate assignments can waste scarce capacity. A false completion message can cause

another operator to restore a dependent function prematurely. Loss of coordination can leave local systems functioning while the wider recovery remains ineffective.

Begin with an operational thread: a coherent account of the activity and its necessary contributions. Identify the initiating need, the decision points, the information exchanged, the physical actions and the confirmation that the intended result has occurred. The thread should include relevant alternatives and adverse conditions rather than describe only the successful path.

The participants need to agree what the shared outcome means and which parts each can provide. The infrastructure owner may define an asset condition but not control contractor availability. The contractor may confirm arrival without being authorized to declare the asset returned to service. The coordination service may record a completed assignment without directly observing the physical repair. These are boundaries to specify, not reasons to distrust every participant indiscriminately.

Table 27.1 develops one selected thread. It is deliberately modest: the purpose is to expose consequential interfaces, not to model every activity in a region.

Table 27.1. SOS-07: a repair-coordination thread and its protection questions

Step and contribution	Meaning needed across the interface	Security or resilience question
Infrastructure owner reports a condition	Identified asset, location, observed condition, time and uncertainty.	Can a stale or incorrect report be mistaken for a current, authoritative request?
Responsible participants prioritize work	Identified authority, operational consequence and available alternatives.	Who may change priority, and how is conflicting authority resolved?
Coordination service issues an assignment	Unique work identity, target, scope, required competence and permitted action.	Can repetition, substitution or an obsolete instruction create unintended work?
Contractor assigns people and resources	A real commitment of suitable crew, equipment, access and time.	Is the same resource being promised to several simultaneous tasks?
Field team performs and reports the work	Distinguish receipt, arrival, action completed and condition established.	Does a status message justify the downstream conclusion being drawn from it?
Asset owner permits return to service	Evidence and authority appropriate to the asset and its dependencies.	Does local restoration create an unacceptable condition for another participant?

The thread reveals positive capability and protection needs together. Authentication of a work message is necessary where required, but it does not establish that the correct crew is available or that the physical condition has been verified. Conversely, a well-resourced maintenance task can be misdirected by an incorrect target or obsolete instruction. The engineering must connect information, authority, resources and physical effects.

Do not infer that one central application owns the entire outcome. It may be the best place to coordinate some information while local owners retain critical decisions and actions. The shared model should show the actual authority and observation boundaries, including what the

coordinator cannot know directly. A wider view is valuable because it makes those limits visible rather than removing them by assumption.

27.3 Establish contribution agreements at consequential interfaces

A contribution agreement describes what one participant supplies to another, under which conditions and with what supporting information. In this chapter, CA-07 is an engineering record for that purpose. It is not a contract template or a grant of authority. Binding commitments depend on the actual agreements and governing arrangements of the participants.

The record should identify the function, interface, source of authority, information meaning, performance expectations and failure response. It also needs to state how a material change becomes known and what evidence supports the contribution. A generic promise to “maintain interoperability” does not tell the receiver whether an old work assignment remains valid after a priority change or how to interpret an absent acknowledgement.

Distinguish the guarantee offered from the conditions assumed. A provider may promise to deliver messages within a defined service envelope but assume that the receiver maintains a valid endpoint and current credentials. The receiver may assume that message receipt means operational acceptance, while the provider promises only transport. Comparing those assumptions can reveal a gap even when neither party has violated its stated local commitment.

Table 27.2 describes the information CA-07 needs for one work-assignment exchange. It can be held in existing interface and service records rather than a new standalone form.

Table 27.2. CA-07: a reviewable constituent contribution

Subject	Information to establish	Why a local success can be insufficient
Authority and scope	Who may request, change, cancel, execute and close the identified work.	A valid sender identity does not establish authority over every asset or action.
Data and state meaning	Work identity, asset identity, location convention, status, timestamps and uncertainty.	“Complete” can mean transmitted, accepted or physically accomplished.
Performance and resources	Starting event, required result, time conditions, capacity and concurrent-demand assumptions.	A local response-time test can omit mobilization, travel or a shared crew queue.
Adverse behaviour	Loss, delay, repetition, inconsistent state, unavailable authority and suspected compromise.	A normal-mode demonstration does not establish a coordinated response under disruption.
Evidence and change	Evaluated configuration, scope, material limitations, notification and receiving review.	An earlier constituent report may not apply after a changed interface or operating context.
Continuing responsibility	Support, escalation, retained records, transfer and ending the contribution.	Ending one agreement can leave another system dependent on its data or access.

Information can have different authoritative sources. The asset owner controls the asset identifier and permitted work; the contractor controls its actual resource commitment; the coordinator controls a shared assignment record within its role. The architecture must reconcile

those sources without inventing one universal source of truth. A locally cached record can be useful, but its age, update meaning and permitted use need to be defined.

The exchange should also identify how disagreement is handled. If two systems report different work states, the recipient should not automatically select the newest timestamp without examining clock relationships and status semantics. A later report of receipt is not necessarily stronger evidence of completion than an earlier field observation. The agreed reconciliation process needs people or mechanisms able to establish the relevant fact.

Where the participants cannot obtain a desired commitment, the system design must account for the limitation. It may use another contribution, reduce dependence, retain a local capability or narrow the supported operating scenario through the appropriate decision. Repeating an unaccepted requirement in the coordination plan does not make the external contribution available.

27.4 Analyse common causes and recovery dependencies

Constituent independence is not the same as technical independence. Two organizations can share communications, power, identity administration, update infrastructure or specialist support. Their systems may have different owners while still being affected by one cause. Conversely, shared resources are not automatically unacceptable; their significance depends on the capability and adverse condition being considered.

For SOS-07, ask what is required to initiate repair coordination when normal services are disrupted. Staff may need access to the assignment system, but the assignment system may require an identity service reached over the communications network being repaired. A backup console may be locally powered while its approval information remains in a remote repository. The recovery dependency can therefore survive the apparent removal of the main application dependency.

A circular dependency deserves a concrete starting-state analysis. Suppose communications restoration requires a dispatched field crew, dispatch requires access to a coordination service, and access requires the unavailable communications path. The diagram identifies a cycle, but does not yet establish that recovery is impossible. A permitted local dispatch procedure, separately available communication or pre-positioned crew could supply a starting contribution. Determine which exists, under what authority and with what evidence. Do not invent one merely because the cycle is inconvenient.

Model the transition as well as the initial outage. A local fallback may support a limited assignment, but later reconnection can introduce duplicate work, conflicting priorities or obsolete instructions. The architecture needs a way to reconcile assignments already acted upon. Deleting the local record on return to central service does not undo the physical work or establish that no obligation remains.

Common cause can also be organizational. A shared expert may be required to approve several repairs; a support contract may provide one specialist team for many customers; a common release process may install the same defective configuration. The dependency analysis should follow effective contribution rather than assume that separate contracts or product names establish independence.

For security, examine deliberate exploitation of those relationships as well as ordinary failure. An actor who can alter a shared assignment or approval source can influence several otherwise protected constituents. This does not make every participant untrustworthy, but it changes the protection and evidence needed at the shared interface. Containment, constrained authority and independent confirmation may reduce the consequence where their implementation and operational effects are justified.

27.5 Check resource contention against the joint scenario

A local recovery plan can be feasible in isolation and infeasible when several systems need it together. Resource contention is therefore part of resilience analysis, not merely a scheduling detail. Identify the people, equipment, access, energy, communications and decision capacity shared by the relevant tasks.

Consider a separate invented SOS-07 example. Two sites declare a repair need at the same time. Each requires eighteen minutes of work by a particular qualified crew, and each has a thirty-minute completion criterion measured from that declaration. One crew is initially at the first site. Travel from the first to the second takes six minutes. The task definitions require the full work at each site and do not permit overlap with travel.

Under these assumptions, the first site completes at minute eighteen. The second begins at minute twenty-four and completes at minute forty-two. Its criterion is missed by twelve minutes. Reversing the order does not establish both results; it only changes which site waits, and any additional initial travel would make the relevant sequence longer. A local eighteen-minute demonstration for each site does not establish the simultaneous two-site capability.

The calculation is conditional planning arithmetic, not measured crew performance or a probability of failure. Its value is to expose the allocation conflict. The project can investigate a second qualified crew, a different task allocation, pre-positioned resources, an alternative technical response or a revised operational priority under the appropriate authority. It cannot solve the conflict by averaging the two completion times or declaring each local plan independently adequate.

Table 27.3 records the distinction between local and joint evidence. The same principle applies to shared approval personnel, recovery appliances and provider support capacity.

Table 27.3. Resource-aware interpretation of the two-site example

Evidence or assumption	Supported interpretation	Missing joint condition
Eighteen-minute task at one site	The stipulated local task fits a thirty-minute window when its resources are immediately available.	Availability of those resources during simultaneous demand.
One crew, six-minute inter-site travel	The stated sequence completes at minutes eighteen and forty-two.	Another arrangement is needed to meet both unchanged criteria.
Two contracts naming the same crew pool	Commercial access to a resource may exist under each agreement.	Separate contracts do not create two simultaneously available crews.

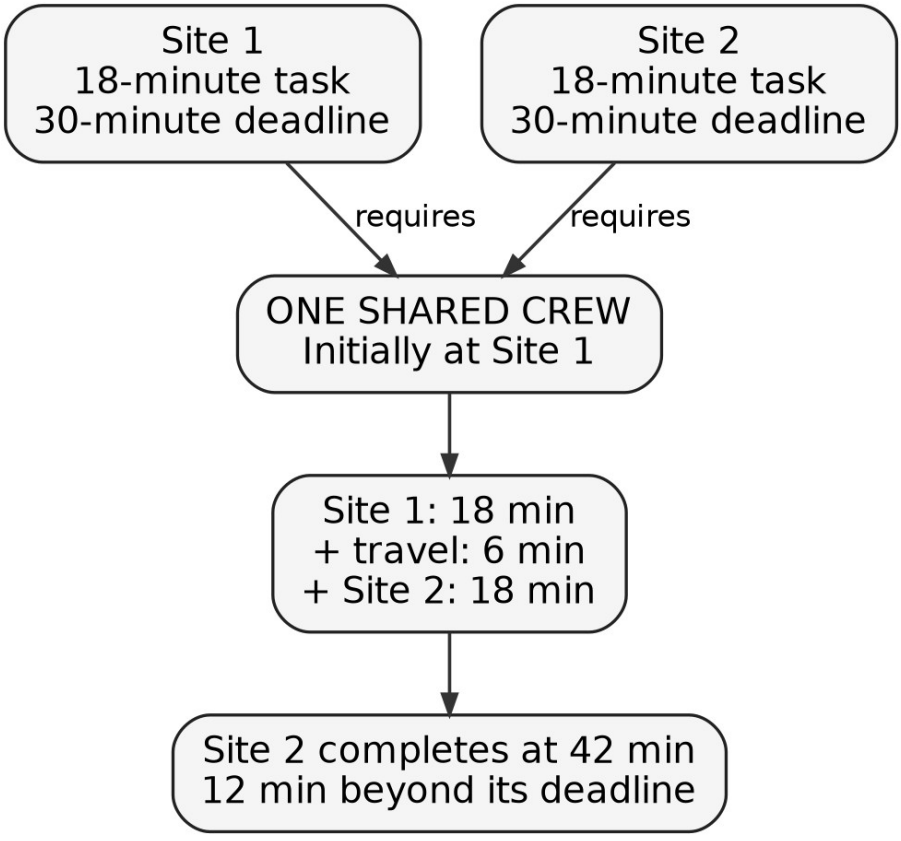
Evidence or assumption	Supported interpretation	Missing joint condition
Proposed second crew	A possible response to the identified contention.	Competence, access, equipment, authority and actual availability still require establishment.

The shared scenario should reflect the consequence that motivates resilience investment. Testing only independent local failures may be reasonable for a narrowly stated claim but not for a region-wide event affecting several participants. The team should identify the demand combinations represented and the rationale for exclusions. A complete list of every imaginable disruption is neither practical nor necessary; a claim that omits the motivating simultaneous demand is inadequate.

Prioritization may be unavoidable when resources cannot satisfy all demands. The resulting decision involves operational and public consequences, not only technical convenience. Engineers should explain the feasible sequences, constraints and effects. The participants entitled to decide priorities must consider the affected people and services. An optimization result does not create authority to assign social value or waive a constituent's obligations.

The following schematic brings together the relationships developed here.

Figure 27.1. Two locally feasible tasks can share one limiting resource



Original SOS-07 schematic using the chapter’s fictional values. It illustrates resource contention, not measured infrastructure performance or an unavoidable national failure.

27.6 Manage asynchronous change and federated engineering information

Constituent systems rarely share one release schedule. An identity service, coordination application, contractor portal and field device can change under different management processes. A successful integration at one point does not establish compatibility of every later combination. The coordination arrangement needs a way to identify relevant versions, interface assumptions and supported combinations.

A federated engineering model can organize those relationships without requiring every participant to disclose its complete internal design. Each provides the information needed at the agreed boundary: offered functions, assumptions, authority, performance, supported modes, change notifications and evidence limits. The receiving-system argument then examines the combination. Missing detail must remain a limitation on particular conclusions rather than an assertion that the omitted dependencies do not exist.

Maintain a decision manifest for the combination actually reviewed. References to each participant's latest interface are insufficient to reconstruct a past result. Where a provider changes dynamically, the agreement and architecture should identify the observable interface commitments and change controls that support reliance. Do not claim a frozen internal configuration if that is neither known nor contractually obtained.

Model exchange should preserve the meaning that matters for the joint claim. Asset identifiers, time units, status states, selected operating modes and evidence status can be more consequential than the visual similarity of two diagrams. A successful import or a matching field count does not establish semantic compatibility. Use representative exchanges that include an adverse condition, an unavailable contribution and an unresolved result.

Plan transition overlap deliberately. During migration, two coordination paths may both issue work assignments. The design needs to identify the authoritative writer for the relevant action and how outstanding work is reconciled. A temporary bridge can translate formats while losing cancellation semantics or authority context. It therefore needs the same interface and evaluation discipline as other system contributions.

The shared change process should be proportionate. A cosmetic display change need not require every participant to repeat an entire integration programme. A changed meaning of completion or withdrawal can affect the whole thread. Use the dependency and claim relationships to select receiving analysis and evaluation, while recognizing that the model itself may omit a relevant effect. Technical review completes the impact decision; a repository notification alone does not.

27.7 Establish assurance for the composition without transferring trust automatically

Define the joint capability and conditions before combining evidence. Constituent assessments support their respective contributions; the complete operational thread and simultaneous resource demands need an additional argument. NIST’s compositional trustworthiness principle supports examining the integrated result explicitly. [1, Appendix E.7]

For SOS-07, a candidate claim might concern delivery and acknowledgement of an authorized assignment to the correct contractor under an identified service envelope. A wider claim about timely completion of repair also needs resource, access, physical-work and return-to-service

evidence. Keep those claims separate so that useful transport evidence is neither discarded nor promoted into proof of the physical outcome.

Test the assumptions at the interfaces. If the coordinator relies on the contractor's acknowledgement as a resource commitment, determine whether the contractor actually makes that commitment at acknowledgement. If the contractor relies on the owner's “ready” status as permission to work, determine which checks and authority establish it. A circular assumption between the parties does not supply the missing contribution.

A compromised or failed constituent need not make every joint outcome impossible. A design may isolate its authority, use another source, retain a limited local function or suspend only the affected action. The relevant containment and recovery properties require evidence. This is why trust should not be treated as an automatically transitive status flowing through every dependency. Describe what is relied upon and how its failure affects the particular claim.

Joint evaluation can combine interface analysis, controlled integration tests, simulations, exercises and operational evidence. Each supports different conclusions. A tabletop exercise can reveal an absent decision-maker or contradictory procedure without demonstrating platform timing. A laboratory test can establish message behaviour while omitting travel, access or public-safety coordination. A field exercise can include more of the task but still represent only selected conditions.

Where one participant cannot join an evaluation, state the substitute representation and its limits. A simulator can be useful if it reproduces the relevant interface, but cannot establish the absent organization's staffing or decision behaviour. The remaining uncertainty can direct a later exercise or a narrower claim. The purpose is a truthful argument about the shared capability, not a uniform badge declaring every constituent approved.

27.8 Connect critical-infrastructure engineering to community consequences

Critical infrastructure supports activities valued by people and institutions. Technical continuity targets should therefore be related to the services and consequences they support, rather than selected only from equipment capability or provider convenience. NIST's community-resilience planning overview connects social and economic functions with buildings, infrastructure, dependencies and cascading consequences. [55, public planning-guide overview] That broader framing complements the security engineering contribution; it is not replaced by it.

For the vignette, restoring one communication service may enable repair coordination, while another restoration supports a different urgent community function. The technical team can identify dependencies, task timing and feasible alternatives. Decisions about priority, acceptable disruption, equity, public safety or legal obligations require the relevant stakeholders and authorities. SSE does not make those judgements politically neutral or automatically solve them through an architecture model.

Participants may use different risk and assurance methods. Identify useful correspondences and material differences. A provider’s certificate can contribute facts within its assessed scope, but cannot grant another jurisdiction’s authorization or establish a regional task it never evaluated.

Information sharing should obtain enough detail to support the required decision while respecting applicable handling and disclosure arrangements. A dependency report can be useful without publishing sensitive internal access paths to everyone. Conversely, a high-level

assurance statement may be insufficient for the integrating participant to assess a critical assumption. The parties need a workable arrangement for controlled access, review and preservation of the relevant conclusion.

Scale should increase attention to shared resources and decision interfaces, not justify universal claims. A coordinated SSE contribution can improve understanding of exposure and resilience choices. It cannot by itself guarantee national mission success, sovereignty or the absence of cascading failures. Those outcomes depend on wider technical, operational, economic, institutional and societal conditions. A bounded contribution remains valuable precisely because others can understand what it establishes.

27.9 Exercise: two assessed providers, one unavailable capability

Situation. Two infrastructure operators each hold an assessment showing that its local repair task can be completed in eighteen minutes. Each contract permits access to a maintenance provider. The regional plan assumes both sites can recover within thirty minutes after simultaneous declarations. The providers draw from the same single crew, with six minutes of travel between the two sites. The joint dashboard labels the capability assured because both local assessments are current.

Worked answer. The local evidence does not establish simultaneous resource availability. Under the stipulated sequential conditions, completion occurs at minutes eighteen and forty-two. The second task exceeds its criterion by twelve minutes. The assessments can remain useful for their local task scope, but the composition claim needs a different resource or operating arrangement.

Identify the actual crew commitments, competence, equipment, access and authority. Examine an alternative allocation or an explicit operational priority decision. Do not infer independent resources from separate provider names or contracts. If the region accepts a restricted recovery objective through its proper authorities, preserve the unchanged local evidence and record the revised joint criterion separately.

Further challenge. A second crew solves only the stated contention if it can actually reach and perform the work. Shared identity, communications or approval dependencies may still block both crews. The joint scenario and contribution agreements must examine those conditions rather than assume that adding a resource closes every resilience concern.

Carry forward. Systems-of-systems engineering makes shared outcomes, independent authorities and consequential interfaces explicit. Chapter 28 applies the same discipline to AI-enabled capabilities and to the human–AI work systems used to produce engineering information and decisions.

28. AI, autonomy and human–AI engineering

28.1 Distinguish the two engineering problems

A team can use AI to help develop a system without putting AI into the delivered capability. It can also acquire an AI-enabled capability without using AI in its own engineering workflow. Those are different system boundaries, with different functions, users, dependencies and evidence needs. A useful security treatment begins by identifying which problem is being addressed rather than treating every use of AI as one category.

In an AI-enabled system, the learned model may classify an input, generate a recommendation, select an action or contribute to a control function. The engineering question is how that contribution helps provide the required capability and how incorrect, unavailable or manipulated behaviour affects the system. The model is an element within an arrangement of software, data, interfaces, people and authority; it is not the whole arrangement.

In AI-assisted engineering, the model may help draft requirements, compare source material, suggest scenarios, write code or assemble an evidence summary. The immediate product is engineering information or an action within an engineering toolchain. Errors can affect design and decisions even when the AI never connects to an operational system. A fluent assurance summary can misrepresent a failed requirement; a generated script can alter the model used to justify a release.

NIST's AI Risk Management Framework treats AI systems as socio-technical and connects their risks to context, participants and the lifecycle. [49, Executive Summary and Section 1.2.4] That perspective is compatible with the book's system-level approach. It does not make every AI concern a security-owned issue. Privacy, human factors, safety, fairness, domain validity and operational suitability require their relevant expertise and governing arrangements alongside security engineering.

HAI-07 proposes an assistant for reviewing CASE-05, with no operational administration role. AI-07 separately explores an AI contribution to preparing a restoration recommendation. Both are unevaluated teaching arrangements; neither changes the earlier requirements, recovery alternatives or adverse findings.

The objective is not maximum AI use, nor maximum human intervention. It is to allocate functions so that the complete work system provides useful results with adequate protection, evidence and accountability. A non-AI method remains a legitimate alternative. An AI-assisted method deserves consideration when its actual contribution, cost and limits can be established rather than assumed from a demonstration of fluent output.

28.2 Identify what changes and what remains controlled

AI, machine learning, generative capability and autonomy are not interchangeable descriptions. For this discussion, a learned model produces outputs using parameters developed through training. A generative model produces content, such as text or another representation, rather than only assigning a predefined label. Autonomy concerns the discretion and action that a system can exercise within its assigned role. A system can be highly automated without using a learned model, and a generative model can be used only for a human-reviewed draft.

The engineering record should describe the actual mechanism and function instead of relying on the broad label. A fixed model used for inference does not necessarily update its trained parameters when it receives a new input. Its outputs can nevertheless change with the input, surrounding instructions, retrieved material or operating conditions. Retraining, fine-tuning, online learning and changes to persistent memory are different mechanisms that need distinct configuration and evaluation treatment.

A retrieval-based assistant adds another dependency. It may obtain documents or records at the time of use and incorporate them into its response. That can improve access to relevant information, but the retrieved material can be incomplete, outdated, incorrectly selected or hostile. Updating the retrieval collection can change the work system even when the model version remains unchanged. The engineering baseline therefore needs more than a model name.

For AI-07, first decide whether the intended function is to explain an already selected restoration option, suggest candidate options, prepare a structured request or execute an action. Each step can increase the consequences of error and the scope of required authority. A system authorized to summarize the recovery procedure is not thereby authorized to choose a package or invoke the receiving coordinator. The interface must preserve those distinctions.

Table 28.1 is an original classification aid for the engineering discussion. It is not a universal taxonomy or a prediction about how every future model will operate.

Table 28.1. Distinguish changes in an AI-supported arrangement

Change or behaviour	What has changed	Engineering implication
New input to a fixed model	The case presented for inference.	Evaluate input scope and response behaviour; do not infer that training has changed.
Revised instructions or retrieval collection	The context and information presented to the model.	Reassess source selection, authority, omissions and relevant task performance.
New trained or fine-tuned model	Parameters or other learned configuration.	Identify provenance, intended use, evaluation differences and affected integration claims.
Online adaptation or persistent memory update	State or learning changes during use under the system's configured mechanism.	Define update authority, persistence, contamination risks, limits and recovery.
New tool access or workflow automation	The actions the surrounding system permits.	Revisit authorization, target binding, human allocation and consequences even if output quality is unchanged.
Provider-side change not fully observable to the customer	A dependency may change beyond the receiving team's direct control.	Establish available identifiers, commitments, monitoring and alternatives; retain uncertainty where configuration cannot be fixed.

The word model also needs context. An MBSE model describes engineering information such as requirements, functions, states and allocations. A learned AI model is a computational contribution used for inference or generation. The former can describe the latter and its

interfaces, but their validation questions are not identical. A well-formed architecture model does not establish the learned model's task performance. A good benchmark result does not establish that the architecture allocates authority appropriately.

Configuration control should identify the combination relevant to the decision: model and service identity where available, instructions, retrieval sources, preprocessing, tools, permissions, parameters, memory, output handling and evaluation records. Where an element cannot be observed or preserved, record that limit and its effect on the claim. Calling an environment “the same AI” does not establish that its decision-relevant configuration is unchanged.

28.3 Engineer the human–AI work system

HAI-07 is intended to help a technical team prepare an evidence review, not to manufacture the evidence. Its inputs are controlled source records from the continuing case. Its useful output is a reviewable draft that identifies claims, supporting material, contrary findings and unresolved work. Its users are engineers, assessors and decision-support staff who retain responsibility for the interpretation and resulting recommendation.

The author-proposed Human–AI Engineering arrangement connects five functions: Frame, Ground, Allocate, Challenge and Assure, with learning feeding all five. It is a teaching synthesis, not a standard or a validated superiority claim. The functions interact: challenge can expose a framing error, and assurance can reveal missing grounding.

Frame the actual need and the consequence of error. For HAI-07, the need is not “produce a positive assurance case.” It is to determine what the available material supports for a defined commitment. A beautifully worded recommendation that omits E-05-03 is not a successful output because it defeats the purpose of the review. The frame also specifies scope, users, information handling and what the assistant is not permitted to decide.

Ground the work in the information needed for that purpose. The team needs governing requirements, relevant configurations, actual or explicitly stipulated evidence, decision context and operational knowledge. The documented process can differ from how work is performed. An operator's explanation that the approval portal uses the failed identity service can change the engineering conclusion even when the written procedure looks complete. That contribution should be recorded and evaluated, not silently subordinated to a more polished document.

Allocate functions to suitable performers. AI may propose a summary or identify candidate relationships. A deterministic calculation can evaluate a stated arithmetic expression. An application specialist may be needed to interpret commit and persistence semantics. An operational participant establishes whether the task reflects actual work. The decision authority determines the permitted commitment. Merely placing a human at the end of the workflow does not establish that these functions have been provided.

Challenge the result deliberately. Search for missing assumptions, contrary evidence, alternative interpretations, scope changes and weak inferences. For CASE-05, check that the failed revocation result survives summarization and that a future corrective test has not become a completed pass. Check whether the model export still contains the allowance that changes the freshness result. These are targeted challenges to the work's purpose, not adversarial conversation for its own sake.

Assure the resulting work product and its use. Confirm that the required functions were actually performed, that the draft addresses the decision, that sources support the claims attributed to them, and that remaining limitations are visible. The reviewer should be able to distinguish the AI's proposed reasoning from actual observations, executed calculations and accountable decisions. The level of review should follow consequence and uncertainty rather than a universal demand for maximal formality.

Table 28.2 connects the five functions to usable outputs. It preserves the option of a simpler workflow where the consequence is low and ordinary competent review is adequate.

Table 28.2. HAI-07: a proposed work-system arrangement for evidence review

Function	Application to the case	Reviewable output
Frame	Identify the bounded development decision and preserve the absence of operational authorization.	Task purpose, scope, users, obligations and unacceptable interpretation errors.
Ground	Obtain controlled requirements, configuration and the mixed E-05 dossier, including operational context.	Source manifest, relevant locators, information gaps and provenance limits.
Allocate	Assign drafting, arithmetic, domain interpretation, challenge and decision responsibilities.	Defined human, AI and deterministic-tool contributions, with access and authority boundaries.
Challenge	Examine omitted adverse evidence, false equivalence, stale results and unperformed work.	Recorded questions, corrections, contrary findings and unresolved interpretations.
Assure	Evaluate the work product against the task and preserve what each result actually establishes.	Reviewed conclusion, evidence-status record and limits on the permitted use.

Learning should improve the arrangement rather than merely refine the prompt. If the assistant repeatedly confuses planned and performed work, the response may include better source structure, a status-preserving extraction method, an explicit review check or a different allocation. If the critical information is absent, no prompt can supply an actual observation. The team must obtain the information or limit the conclusion.

The engineering effort remains human work even when AI contributes substantially. People establish the problem, negotiate conflicting needs, investigate the actual system, judge alternatives and assume assigned responsibilities. Tools can assist those activities and sometimes perform bounded functions within them. A generated artefact does not demonstrate that the corresponding real-world work occurred.

28.4 Grounding must preserve meaning, not just attach references

A source reference is useful only when the source is appropriate to the claim and actually supports it. HAI-07 should identify the requirement or proposition being evaluated, the relevant passage or observation, its configuration and status, and the inference made from it. A citation to a long report is not enough when the report's scope excludes the property claimed.

The assistant should distinguish authoritative requirements, implementation descriptions, plans, observations and interpretations. They can all be legitimate inputs, but they perform different roles. The sentence “a corrective evaluation is scheduled” supports the existence of intended work, not corrected behaviour. A supplier's claim of platform durability needs its own scope and supporting basis; it does not become measured persistence merely by appearing in a controlled document.

Retrieval needs evaluation as part of the work system. Did the selected source set include the adverse result and relevant appendix? Did a heading filter omit blocked evaluations? Did a summary remove the configuration restriction? A response can be faithful to the excerpts it received and still be unsuitable because the retrieval omitted material needed for the decision. The task should therefore evaluate source coverage, not just the fluency of the final paragraph.

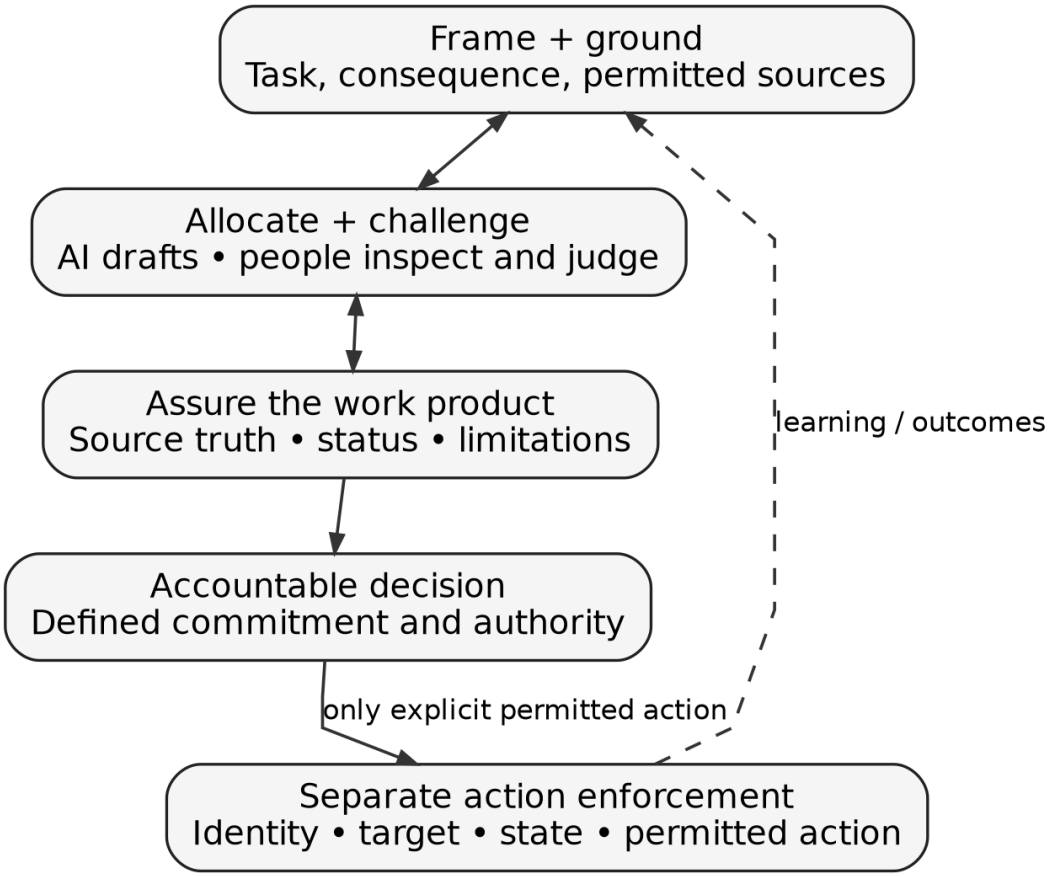
Source content and source authority should not be confused. A document can contain instructions addressed to a human reader, examples, obsolete policy or text introduced by an untrusted contributor. Its presence in a retrieved collection does not grant it permission to control the assistant's tool actions or override the task's governing instructions. The system needs explicit boundaries around how retrieved content is used.

NIST's generative-AI profile identifies confabulation and information-integrity concerns and recommends evaluation tied to actual capabilities and use context. [50, Sections 2 and 3, MEASURE 2.5] For this book's application, an engineering summary should permit a reviewer to trace a material statement to its basis and identify where that basis is insufficient. The requirement is stronger than “include citations”: the cited information must retain its actual meaning.

An AI-generated explanation of how it reached a result is not an execution record. When arithmetic, queries or code are used, preserve the actual inputs, execution identity and outputs needed for review. When a test is performed elsewhere, preserve the test record rather than a generated account of what such a test would probably show. An honest statement that the necessary evidence is unavailable is a useful result when it prevents an unsupported conclusion.

The following schematic brings together the relationships developed here.

Figure 28.1. Human–AI work with a separate action boundary



Original schematic of the author-proposed Human–AI Engineering arrangement. It does not claim empirical superiority or that a human can supply an unachievable real-time protective response.

28.5 Design action authority outside the generated recommendation

AI-07 explores a different function from HAI-07: assisting preparation of a restoration recommendation. The proposal might retrieve the relevant recovery procedure, identify eligible package information and draft a structured request for review. Even this apparently modest function needs a defined boundary because an incorrect recommendation can influence a consequential action.

Allocate only the authority the function needs. A drafting assistant has no need for package-activation credentials, and its read access must respect information-handling limits. When a workflow permits action, the receiving interface establishes identity, permission, target, package, mode and state; the generated recommendation does not establish those conditions.

A structured output format can help a receiver parse the proposed action. It does not establish authorization. A request can be syntactically valid while naming the wrong target, an obsolete package or an unauthorized operation. A schema check and a policy check answer different questions. The receiver should preserve that distinction even when the user interface presents the recommendation as a convenient single button.

The existing IC-01 concepts provide a useful basis for the candidate branch: bind the proposed request to its content, target, expected generation and authority context; distinguish receipt from

effective change; preserve transaction identity across repetition; and return an authoritative outcome. The proposal does not amend the base case or establish that an AI component can satisfy the still-incomplete REC-P1 rules. It identifies the additional work required before granting any action capability.

Human approval also needs a precise object. A person should know what action, target and content they are approving. If the system permits the payload to change after the approval, the approval may no longer apply to what executes. A later change to target state can also invalidate an earlier decision. The receiving design needs to bind and re-evaluate the relevant conditions at the consequential boundary rather than treat an old conversational “yes” as unrestricted permission.

Do not place the only protective decision in the same generative component whose output is being constrained. A separate enforcement mechanism can restrict actions to an explicitly permitted set, but its own implementation, supporting data and failure modes still need evaluation. Deterministic software is not automatically correct simply because it is not AI. The value of separation is that the required rule and its enforcement can be specified and challenged independently of the generated recommendation.

Tool access introduces interruption and repetition concerns already familiar from Chapter 10. The assistant may request an action and then lose the response. It must not infer that nothing happened or issue a new unrelated transaction as a blind retry. The work system needs status discovery and controlled reconciliation. Delegating the request's wording to AI does not remove the distributed-system problem of learning whether the action actually took effect.

28.6 Examine AI-specific attack paths and familiar system weaknesses together

The system still depends on ordinary software, identities, storage, networks, update mechanisms and support arrangements. Protecting the learned model alone does not protect the complete capability. Conversely, ordinary platform hardening does not establish that the model uses retrieved information appropriately or that a generated action remains within its intended task.

NIST's adversarial-machine-learning taxonomy distinguishes attack approaches involving training or data manipulation, inference-time inputs, privacy and generative-AI interactions. Its treatment of indirect prompt injection and agent-related risks is particularly relevant where retrieved content can influence tool actions. [51, Chapters 2–3, particularly Sections 3.4–3.5] The engineering task is to select the pathways relevant to the actual system and examine their consequences, not to assume that every named attack applies identically.

For HAI-07, consider an untrusted document containing text that attempts to suppress an adverse finding or redefine a failed result as passed. The protective concern is the integrity of the evidence review. For AI-07, similar content could attempt to redirect a proposed action or induce disclosure through an available tool. These are hypothetical threat scenarios. They do not assert that a particular product is vulnerable or that a specific filter has been defeated.

Potential responses include controlling source access, distinguishing data from governing instructions, limiting tools and destinations, validating proposed actions against explicit rules, retaining provenance and reviewing high-consequence outputs. Each response needs an applicability argument and evaluation. A prompt telling the model to ignore hostile instructions

is not, by itself, evidence of an enforceable authorization boundary. An input filter can be useful without demonstrating complete coverage of manipulated content.

Training and adaptation create other questions. Who can change training data, fine-tuning examples, labels or persistent memory? How are contributions reviewed and versioned? Can evaluation data become part of development feedback in a way that makes later results misleading? Can a poisoned or inappropriate memory persist after the immediate interaction ends? The answers depend on the configured learning and storage mechanisms; a fixed inference-only model does not have every one of these update paths.

NIST's AI secure-development profile extends the SSDF to generative AI and dual-use foundation-model development. Its scope includes development activities rather than serving as a complete deployment and operations framework. [52, Section 1.2] Use that development contribution alongside the lifecycle, operational and authority work in this book. A development profile does not resolve the receiving organization's complete use case by itself.

Threat analysis should include the people and tools that produce the evidence. A compromised evaluation script can conceal a model's failures; a biased test selection can create an exaggerated performance claim without any adversary. The same dependency principle used for LS-06 applies: examine whether one source or mechanism can affect both the system and the apparent support for trusting it.

28.7 Make human oversight a feasible function

Human involvement is valuable when the person has a defined function, sufficient information, relevant competence, time and actual means to act. A label such as “human in the loop” does not establish any of those conditions. The person may be reviewing a proposal before action, monitoring a process, deciding an exception, investigating an anomaly or maintaining the operating rules. Those functions require different interfaces and resources.

NIST's AI RMF discusses human–AI interaction as a distinct risk-management concern, including the possibility of inappropriate reliance and the need to define human roles. It does not assume that adding a human always improves the result. [49, Appendix C] The practical application here is to evaluate the complete work arrangement rather than treat nominal human presence as a universal final safeguard.

A separate invented timing example makes the issue concrete. Suppose a proposed protective response must take effect within two seconds of a specified event. The human-response design allocates 0.4 seconds for notification, five seconds to obtain and interpret the information, two seconds to decide, and 0.6 seconds for the action to take effect. The serial allocation totals eight seconds. It cannot support the two-second requirement as stated.

These values are fictional design allocations, not a study of human reaction or a general claim about human capability. The finding is that the proposed allocation does not fit its own requirement. The team may redesign the protection, use an evaluated automatic response for a bounded condition, change the operational arrangement or return the requirement to its source where revision is permitted. It cannot close the problem by asserting that an alert has a human recipient.

Time is only one constraint. A reviewer may need access to the underlying source, the proposed target, current configuration and consequences of the action. A concise explanation can help, but

a persuasive explanation is not necessarily a faithful account of the model's operation or a complete basis for the decision. The interface should expose the information needed for the person's assigned task, including uncertainty and the ability to request specialist help.

Workload and staffing matter. If every routine request demands a lengthy expert review, the operating arrangement must supply that capacity or reduce demand through a different allocation. If rare critical requests are hidden among many routine approvals, the evaluation should examine whether the intended distinction remains usable. The solution need not be more human review everywhere; it should be the right human and technical functions at the points where they contribute.

A stop or fallback control also requires engineering. Stopping a recommendation service is different from safely stopping an actuator or an operation already committed. The system must define what halting changes, what continues, what state is retained and how the resulting condition is communicated. The correct response depends on the actual domain and specialist analysis, not a universal assertion that turning everything off is safe.

28.8 Evaluate the complete task and preserve the denominator

Evaluation should begin with the function, intended users, operating conditions and consequence of error. A benchmark can provide useful evidence for a model under its defined conditions. It does not automatically establish the receiving workflow, tool permissions, retrieval quality or human performance. The evaluation programme should distinguish model behaviour, integration properties and the complete operational or engineering task.

For HAI-07, useful cases include accurate summaries, missing source material, conflicting records, obsolete profiles, an adverse result accompanied by a corrective plan, and a model export that loses a material allowance. The evaluator should establish whether the assistant preserves status and scope, asks for necessary information and supports a competent review. A generic writing-quality score is insufficient for that purpose.

For AI-07, additional evaluations would concern proposed actions, wrong targets, stale state, source manipulation, unavailable tools, repetition after lost responses and the limits of the permitted recovery arrangement. The plan must identify which cases are executable with the available design and which remain blocked by unresolved definitions. Test cases cannot demonstrate conformance to an undefined authority profile.

A simple fictional scorecard illustrates why denominators and abstention matter. Suppose a set of one hundred requests is evaluated against a defined answer criterion. The assistant answers ninety, of which eighty-one are judged correct and nine incorrect. It declines to answer ten. Accuracy among answered requests is 81/90, or 90 percent. The fraction of all requests receiving a correct answer is 81/100, or 81 percent. The unanswered fraction is 10 percent.

None of those figures alone determines whether the workflow is useful or acceptable. An abstention can be appropriate when information is insufficient, but the task still needs a route for resolution. The nine errors may include a critical omitted adverse finding or minor wording differences, which have different consequences. A system that answers only trivial requests can achieve high answered-request accuracy while failing the useful workload. Conversely, counting every justified abstention as an erroneous factual answer can misrepresent a protective behaviour.

The numbers are stipulated arithmetic, not measured AI performance, population estimates or a prediction about adversarial success. A real evaluation needs a defined case population, ground-truth or judgement method, uncertainty treatment and analysis of meaningful subsets. It should preserve both the quality of accepted outputs and the coverage and burden of the complete workflow.

Table 28.3 organizes proposed evaluation work without inventing performed results. AE-07 identifies an evaluation plan for the two teaching arrangements, not a test execution record.

Table 28.3. AE-07: distinct evaluation questions for AI contributions

Evaluation layer	Question and proposed evidence	Limit on the conclusion
Source and task adequacy	Do selected records and scenarios represent the decision, including material adverse and missing information?	A complete retrieval log does not establish that the chosen source set is sufficient.
Model output	Does the configured model preserve meaning, identify gaps and meet the defined task criteria on selected cases?	Results apply to that configuration, case set and judgement method, not every future request.
Tool and interface enforcement	Are permitted actions, content, targets, state and repetition handled under the specified rules?	Correct schema parsing does not establish authority or the complete human task.
Human–AI workflow	Can intended people detect consequential errors, obtain required information and complete the task with available resources?	A developer demonstration does not establish representative workload or competence.
Change and resilience	What remains useful after a source, model, tool or dependency changes or becomes unavailable?	Successful restoration of access is not automatically restoration of trustworthy content, memory or authority.

28.9 Compare against a credible alternative and challenge generalization

Compare an AI-supported arrangement with a credible alternative: competent people using clear requirements, appropriate search, deterministic automation and ordinary review. An unstructured unassisted task can make improvements in problem definition look like benefits attributable to AI alone.

A planned comparative study should identify the task, participants, input information, available time, output criteria and consequence categories. Use independent assessment where justified, and account for learning or order effects when participants encounter similar cases more than once. Where feasible, separate those developing the method from those judging its outputs. The design needs enough detail to support the intended conclusion, not the appearance of a scientific experiment.

Measure the total work, not only generation time. Review, correction, specialist escalation, source preparation and maintenance can change the value of the arrangement. An assistant may save drafting time while increasing the effort needed to verify material claims. It may also create value by helping specialists notice a consequential issue earlier. Those are different possible outcomes to investigate rather than assume.

Performance should be examined where it matters. An average can conceal poor results on ambiguous authority, unfamiliar configurations or incomplete evidence. A system that preserves normal summaries but converts plans into passing results fails an important part of HAI-07's task. Evaluate selected high-consequence conditions explicitly instead of expecting a broad aggregate score to reveal them.

A model's expressed confidence should not be accepted as the probability that a particular engineering claim is correct. Any numerical confidence used for a decision needs a defined meaning and evidence of its relationship to outcomes in the relevant population and conditions. A fluent assertion of certainty, a token-level quantity and a calibrated task-level estimate are different things. The case should not invent a general trust score from any of them.

NIST's generative-AI profile cautions against extrapolating system capability from narrow or anecdotal evaluations and connects measurement to use context. [50, MEASURE 2.5] For the book's purposes, the conclusion should identify which function improved or failed, for which cases and configuration, at what total effort and with what remaining uncertainty. That is more useful than declaring AI either universally reliable or inherently unsuitable for professional work.

28.10 Maintain the learned model, surrounding system and evidence together

A change to the model is not the only event that can affect performance or protection. The retrieval collection, parser, prompt template, tool interface, permissions, runtime settings and operator workflow can each change the result. A provider can alter an external service without giving the receiving team complete visibility into internal implementation. The configuration and assurance arrangements should identify what is known, controlled and observable.

For HAI-07, retain the source manifest, task instructions, relevant service identity, generated draft, material tool results and review disposition needed to reconstruct the conclusion. The record should distinguish original generation from human correction. An edited final paragraph is the team's reviewed work product; it should not be presented as evidence that the model independently produced the same accurate result.

For AI-07, changes to action permissions deserve particular attention. Granting a new write-capable tool can materially change consequences even if every earlier text-generation test still passes. Revisit the authority boundary, target binding, failure response and human allocation. A model update may require different evaluation from a change to the interface that executes its requests, but neither should disappear behind the same product name.

Select re-evaluation by impact. A new document in the retrieval collection may require targeted source-selection and interpretation tests. A changed model can affect a broader range of tasks. A new memory policy can introduce persistence and recovery concerns. Existing evidence can remain useful where its applicability is justified, while affected conclusions need new support. Universal invalidation and universal inheritance are both poor substitutes for this analysis.

Keep monitoring connected to an actionable response. Detecting an unusual output or a source mismatch is useful only when the work system can pause, obtain review, correct the input or use an alternative. Monitoring should not be claimed as proof that every harmful output will be detected before consequence. Its coverage and delay need the same scrutiny as other protective and observational mechanisms in the book.

Evidence repositories and evaluation environments are themselves enabling systems. A script that relabels uncertain outputs as correct or excludes failed cases from a report can affect the decision without changing the AI model. Protect the relevant records, transformations and review history. Automated assistance may help organize this information, but it must not be allowed to rewrite the historical evidence merely to produce a more favourable summary.

28.11 Engineer useful fallback and controlled adaptation

Fallback should preserve the required useful capability to the extent claimed, not merely remove the AI from the display. If HAI-07 is unavailable, can the team still obtain the controlled sources and perform the evidence review? If AI-07 cannot supply a recommendation, can designated staff identify the permitted recovery procedure and act under the appropriate authority? The alternative requires people, information, access and time.

A manual fallback that depends on the same unavailable retrieval or identity service may not be independent of the initiating failure. An operator who has not practised the task may not be able to perform it within the assumed window. A non-AI tool can provide a valuable bounded function, but it also has configuration, input and support dependencies. Compare the complete arrangements rather than treat manual or deterministic as synonyms for safe.

Recovery from compromised content can differ from recovery from an outage. Reopening an inference endpoint does not establish that retrieved records, persistent memory, tool credentials or evaluation baselines are trustworthy. The response needs to identify what was affected, restore or reconstruct the necessary state and decide which earlier outputs or actions require review. A confident new response does not erase an unauthorized action already taken.

Adaptation should be a controlled change to the relevant work system. The organization can improve instructions, revise retrieval, retrain a model, change function allocation or replace a dependency when evidence supports doing so. The appropriate authority and evaluation follow the effect of the change. A mechanism that adjusts its behaviour within a specified operating envelope still needs evidence for that envelope and the means of handling departure from it.

There are circumstances in which automated action is necessary because the required response cannot depend on timely human intervention. There are others in which a qualified human decision is essential because the problem includes ambiguity, consequences or authority that the automated arrangement does not resolve. The engineering task is to establish which applies to the actual function and to evaluate the chosen allocation. Neither maximal autonomy nor maximal human involvement is a universal design principle.

The same discipline preserves opportunities for innovation. A limited prototype can explore a promising allocation without asserting readiness for operational use. An adverse result can improve the design rather than end the investigation. A useful AI contribution can be expanded when the corresponding requirements, protections, resources and evidence support the next commitment. Progress is not weakened by keeping those commitments distinguishable.

28.12 Exercise: the assistant has written a convincing case

Situation. HAI-07 produces a polished CASE-05 summary with references. It describes the planned correction in E-05-07 as completed, omits the late-commit result in E-05-03 and treats a link from the model to VP-02 as a passing test. The draft also recommends using the unapproved

recovery profile. A reviewer praises the clarity and proposes attaching it to DEC-05 without examining the underlying records.

Worked answer. The output has failed the assigned evidence-review function, regardless of its readability. The adverse result remains part of the stipulated dossier. The correction is planned, the evaluation link does not establish execution, and REC-P1 remains unresolved. The source references do not rescue an interpretation that contradicts their content or scope.

Correct the summary against the controlled records and preserve the changes made. Investigate whether the error arose from missing retrieval, an ambiguous status representation, generation or inadequate review. Use the finding to improve the work system: source coverage, allocation, output checks and escalation may matter as much as prompt wording. Do not claim that the improvement is effective until it has been evaluated for its intended purpose.

If the assistant also has a tool capable of updating the official evidence status, restrict that path to its defined authority and require the appropriate controlled review. A drafted conclusion should not automatically overwrite the source result. For a future action-capable branch, the receiver must establish the actual permitted action, target, state and authority rather than rely on the confidence of the recommendation.

Further challenge. A second assistant agreeing with the first is not automatically independent corroboration. They may use the same incomplete source set or inherit the same framing error. A useful challenge must be capable of exposing the error of concern, whether performed by another tool, a deterministic check or a qualified person with the relevant evidence.

28.13 Carry the engineering discipline forward

The book began with a connection that is easy to endorse and difficult to sustain: security must influence the engineering of the useful system, and the engineering must remain understandable to the people who acquire, operate, assess and govern it. The later chapters have not replaced that task with a new tool, a stronger adjective or an assurance label. They have made the work more explicit.

The same questions recur across an administration service, a supplier contribution, a regional coordination capability and a human–AI workflow. What outcome is needed? Which losses and obligations matter? Where does a consequential action occur? What does the design rely upon? Can the necessary people and mechanisms provide their contributions under the relevant conditions? What evidence supports the resulting claim, and who may decide to rely on it?

Those questions do not eliminate uncertainty. They organize the work needed to reduce, contain and communicate it. Requirements preserve intent; architecture and implementation create the capability; evaluation challenges what has been provided; assurance explains the grounds and limits for confidence; accountable decisions determine the permitted commitment. Operations, change and eventual withdrawal continue the same responsibility rather than start a disconnected security programme.

AI and model-based tools can make important contributions to that work. They can also create new dependencies and new ways for an apparently complete explanation to diverge from the system it describes. Their value must therefore be evaluated through the same engineering discipline, with people performing the professional work that framing, judgement, collaboration and responsibility require.

The durable objective is not perfect prediction or a guarantee that no system will fail. It is a capability whose purpose, protections, dependencies, evidence and limits are understood well enough to support responsible use and change. SSE contributes the security specialty to that common endeavour. Its success is measured in the quality of the engineering and the justified reliance it enables—not in how successfully it separates itself from the people and practices it needs to work with.

Acknowledgements

Books are never written alone, and this one is certainly no exception. This work exists because of decades spent with teams who lived the realities of mission assurance, systems engineering, operations, and cybersecurity under conditions where failure was not an option.

To the Canadian defence and security community — colleagues, commanders, analysts, engineers, operators, procurement specialists, SOC teams, frontline responders, and policy leaders — thank you for shaping the professional foundation that made this book possible.

To the Systems Security Engineering pioneers at NIST, especially Ron Ross, whose clarity on engineered trust provided the intellectual grounding for much of this manuscript — your work continues to guide entire professions.

To the global engineering community — software, hardware, avionics, space, maritime, AI/ML, and everything in between — thank you for advancing the disciplines we all depend on.

To friends and collaborators who encouraged, challenged, and refined these ideas along the way: your conversations live throughout these pages.

And to my family — your patience and support were the true enabling factors behind this book.

— P.J.H.

About the author

Peter J. Hillier, CD, CISSP, is a Canadian cybersecurity consultant, Systems Security Engineering practitioner, and founder of Hillier Information Protection Solutions Inc.

With more than four decades of experience spanning military intelligence, national defence, enterprise cybersecurity, and security governance, Peter brings an operational and systems-level perspective to the engineering of trustworthy capabilities.

A 20-year veteran of the Canadian Armed Forces, his subsequent career has included enterprise Chief Information Security Officer roles, cybersecurity incident-response leadership, security architecture, and advisory work supporting complex defence systems and major naval acquisition programmes.

His more recent work focuses on Systems Security Engineering, cyber resilience, supply-chain assurance, secure procurement, and integrating security considerations into the engineering and sustainment of mission-critical capabilities.

A published practitioner, educator, and contributor to international security-engineering initiatives, Peter advocates moving beyond compliance as the primary measure of security toward disciplined engineering, meaningful evidence, and accountable decisions.

Through his writing and professional work, he seeks to make engineering-led security more accessible to the engineers, cybersecurity practitioners, acquisition professionals, and leaders responsible for the systems upon which modern institutions depend.

Appendix A — MBSE languages, tools and exchange

Purpose and currency

This is a dated orientation, checked on 8 October 2026, not a product ranking or an implementation guide. The main chapter teaches the engineering relationships independently of a vendor. The examples below identify different ways a toolchain can support that work. Product configurations, extensions, availability and licensing need verification for an actual acquisition. No product has been installed or benchmarked for this book.

Languages, methods and execution are different choices

SysML v1 represents requirements, structure, behaviour and parametric relationships, including allocations. Its graphical views can help readers locate functions, interfaces, states and quantitative constraints; learning the symbols is useful only alongside their engineering meaning. OMG's overview describes the UML-based language and its principal diagram families. [24, What is SysML? and SysML Diagram Summary]

SysML v2 uses a foundation based on the Kernel Modeling Language (KerML) with textual and graphical notation. Its language specification includes requirements, calculations, constraints, analysis cases and verification cases. These are language capabilities, not evidence that a particular tool executes every analysis or that a model accurately represents the target system. [25, Sections 6.1–6.2 and 7.19–7.24] A textual representation can be convenient for review and comparison, while a graphical view can make interactions clearer. Neither form removes the need to inspect the underlying semantics.

OMG's specification record lists September 2025 as the formal adoption date for SysML 2.0. The language document consulted for this update is formal/2026-03-02, dated March 2026. [25, specification record and cover] Distinguish a language version, the exact document issue and a tool's supported implementation. A reference to “SysML v2” alone does not identify the configuration used to produce an analysis.

MBSE is not synonymous with SysML. Arcadia is a modelling method implemented by Capella with its own modelling environment. [27, Arcadia and Capella overview] A project may also connect system architecture models to specialized simulation, requirements or analysis environments. Choose representations that answer the question and maintain an explicit mapping where concepts cross a tool or language boundary.

For MS-01, a SysML v1 implementation might use requirement, internal-block, sequence, state and parametric views. These are possible representations of the existing records, not mandatory diagrams or a native model supplied with the book. A v2 or Arcadia implementation would organize the corresponding information using its own constructs. The acceptance question is whether requirement meaning, allocation, time, modes and evidence status survive that representation—not whether every environment draws the same picture.

Representative environments

Table M.1 records narrowly scoped descriptions from official product or project pages. The review questions in the last column are original selection questions, not reported product

deficiencies. These examples are not exhaustive, and inclusion is not endorsement. The public descriptions do not demonstrate the installed version, configuration or integration needed for a particular project.

Table M.1. Representative modelling environments — dated orientation

Environment	Illustrative capability described by its official source	Question for a project demonstration
Cameo Systems Modeler	SysML modelling, requirements relationships, model-consistency checks and engineering views; analysis and collaboration can involve additional products. [28]	Demonstrate the selected release and licensed extensions using the freshness-change and evidence-status case.
Eclipse Capella / Arcadia	Open-source architecture-modelling environment implementing Arcadia; the ecosystem includes extensions for additional concerns and collaboration. [27]	Show how security scenarios, allocated obligations and external evidence are represented and exchanged under the chosen method.
System Composer with the MathWorks modelling environment	Architecture specification and analysis, requirements allocation and links to behavioural models and simulation in Simulink and related tools. [29]	Show preserved units, timing, interfaces and configuration identities between the architecture and executable analyses.

Architecture modelling, model-based design and simulation can participate in the same engineering workflow without being identical. A plant or controller simulation may explore detailed dynamics, while the architecture model records allocations, dependencies and requirements. Where the tools are connected, establish how time, units, parameter values and configuration identities cross that connection. An automatically transferred number still needs the right interpretation.

A digital twin is not simply another name for any executable model. For this book's purposes, a proposed twin must explain which actual system or configuration it represents, how observations update it, and what decisions that relationship supports. A simulation of the fictional logistics case has no operational counterpart merely because it runs. Claims about a twin's predictive or assurance value require evidence for the intended use, including the quality and timeliness of the data feeding it.

Specify exchange and migration by the information that must survive

The Systems Modeling API and Services specification addresses model access, navigation and operations through defined services and mappings. The specification record identifies version 1.0 and the formal/2026-03-04 document issue used here. [26, specification record; Clauses 1–2] It is distinct from the SysML language specification. Model export, API access and analysis execution are also distinct capabilities; a procurement should state which are required.

An exchange acceptance exercise for MS-01 should preserve element identities, requirement revisions, selected alternatives, units, conditions, allocation meaning, analysis inputs and the difference between planned and performed evaluation. Include content expected to remain

unresolved. A receiver that converts “not performed” into a passing default has changed the engineering conclusion even if it imports every diagram.

For migration, preserve the original controlled model and test a representative slice before committing the whole programme. Compare meaning, not only object counts. Identify unsupported custom extensions and any manual mapping. Recheck important queries, calculations and generated reports, then document which earlier results remain applicable. A successful conversion is not, by itself, revalidation of the system described.

Long-term usability requires more than an editable vendor file. Agree the model scope, reader access, machine-readable exports where needed, referenced libraries, explanatory conventions, evidence references and the means of reproducing material analyses. The operational organization should not discover at handover that the only person able to interpret a recovery model has left, or that essential source information cannot be accessed under its permitted environment.

The modest objective of this book is therefore durable model literacy: readers can identify what a model says, question its omissions, understand the work its tools actually performed, and use its results within their demonstrated scope. Selecting and qualifying a production toolchain remains a project-specific engineering and governance task.

Appendix B — Worked-case and record guide

This guide locates the chapter identifiers and their status. The main development history remains in Tables B.1–B.4. Section B.5 distinguishes the new completion branch and physical example from that history. The guide is not an operational baseline or an independent acceptance record.

B.1 Read the case as a developing engineering explanation

The recurring case concerns remote administration of a logistics dispatch service. The selected system includes managed administration endpoints, the gateway, receiving management endpoints, enforcement configuration and the human/procedural contributions included in the capability. Enterprise identity is an external dependency. Operational dispatch processing remains outside the selected administration boundary. A management-side commit does not, by itself, establish downstream consumer convergence, correctness of dispatch policy or mission success. See Sections 2.2, 5.1 and 10.2–10.3.

Teaching Edition identifiers TE-02 through TE-07 mark the stages through which the explanation develops. They are not a series of released operational products. Later chapters make some definitions more precise while preserving earlier wording so that the reader can follow the engineering. Architectural proposals and separate what-if branches do not amend the requirement set merely by being analysed.

Table B.1. Development of the recurring case

Location	Added teaching content	Status retained
Part I; Chapters 1–4	Purpose, boundary, initial obligations, requirements and assumptions.	Starting explanation; no evaluated deployment.
Part II; TE-02; Chapters 5–8	Needs, losses, scenarios, profiles, candidate requirements and evaluation planning.	Definitions and recovery authority still need completion.
Part III; TE-03; Chapters 9–12	ARC alternatives, interface and state proposals, analyses, MBSE slice MS-01 and recovery profiles.	No architecture selection; the twelve-second delay branch remains separate.
Part IV; TE-04; Chapters 13–16	Realization walkthrough, integration increments, procedure outline and human-system readiness.	Planned work and hypothetical findings; not performed builds or tests.
Part V; TE-05; Chapters 17–20	Acquisition, supplier review, mixed evidence dossier, risk and decision records.	No agreement execution, supplier clearance or operating authorization.
Part VI; TE-06; Chapters 21–24	Operating, monitoring, change and retirement records; separate inherited-service vignette.	No implied deployment or disposal of the development case.
Part VII; TE-07; Chapters 25–28	Technical management, competence and human–AI arrangements; separate regional vignette.	No organizational assessment, AI experiment or regional exercise.

B.2 Locate the principal records

Identifiers name different kinds of information. A need, requirement, design direction, planned evaluation, stipulated observation and decision are not interchangeable. The source numbering is preserved, including SR-07a and the distinction between the book's SR requirements and identically prefixed control identifiers in cited publications.

Table B.2. Principal record families and where to find them

Identifier family	Meaning in the book	Primary locator and limit
POL-01; STK-01/02	Illustrative MFA obligation and stakeholder capability statements.	Sections 3.4 and 7.2; not universal policy.
L-01–L-04; PN-01–PN-04	Loss concerns and protection needs.	Sections 5.5–5.6; selected scope, not every loss.
LS-01–LS-06; A-01–A-03	Scenario hypotheses and material assumptions.	Sections 2.5, 5.8 and 6.6; A-03 is not retained under the later recovery need.
SR-00–SR-12; SR-07a	Candidate system requirements developed from the initial set.	Sections 7.4–7.6; neither approved design nor implementation evidence.
ADM/AUTH/MFA/NET/TIME/PKG/REC-P1; TX-P1	Case definitions for actions, authority, authentication, communications, time, packages, recovery and transaction lifetime.	Sections 7.3, 10.3–10.6 and 12.3; several bindings remain open.
ARC-A/B/C; AD-01–AD-04; IC-01; TRD-01	Architecture concepts, proposed design directions, activation contract and development recommendation.	Sections 9.5–9.8, 10.2 and 11.17; no production selection.
MS-01; RZ-01	Connected model slice and realization walkthrough.	Sections 11.4–11.10 and Chapter 13; no native model or production build.
C-01–C-05; VP-01–VP-05; VA-01/02	Candidate claims and planned verification/validation work.	Sections 8.2, 8.5–8.6; a link to a plan is not a passing result.
INT-01–INT-05; DP-01; D-04-01–04; TR-04	Integration increments, procedure, hypothetical discrepancies and readiness record.	Chapters 14–16; hypothetical teaching status is retained.
AP-05; ED-05-01–03; SC-05	Acquisition package, evidence-delivery packages and contribution-centred supplier review.	Chapters 17–18; not executed agreements or assessments.
CASE-05; E-05-01–07; RR-05; DEC-05	Assurance review, stipulated dossier items, risk record and unsigned decision brief.	Chapters 19–20; E-05-03 remains adverse; E-05-07 remains planned treatment.
OP-06; MON-06; CR-06; RET-06	Operating, monitoring, change-impact and withdrawal teaching records.	Chapters 21–24; no completed service activity implied.
LEG-06	Separate inherited-service reconstruction vignette.	Chapter 23; not a deployed version of the administration case.

Identifier family	Meaning in the book	Primary locator and limit
TM-07; WP-07-01–04; CAP-07; COMP-07	Technical work, organizational support and competence-development records.	Chapters 25–26; not funded plans or actual competence assessments.
SOS-07; CA-07	Separate regional repair-coordination vignette and contribution record.	Chapter 27; not an expansion of the administration boundary.
HAI-07; AI-07; AE-07	Human–AI evidence-review arrangement, separate AI-enabled branch and planned evaluation.	Chapter 28; no autonomous operating authority or AI experiment granted.

B.3 Preserve what has—and has not—been resolved

The following distinctions prevent the case's development from being mistaken for silent correction or approval. They summarize existing chapter content. Completing a table in this guide does not discharge any missing source, design, authority or evidence obligation.

Table B.3. Requirement and profile continuity

Subject	Continuity to preserve	Remaining question / locator
SR-00; useful maintenance	Part II gives the positive ten-second activation-and-outcome target within the defined envelope.	Arrival/burst and interruption conditions need further definition; Sections 7.4 and 11.12.
SR-03; identity withdrawal	Part I's request-prevention wording is refined to prevention of commits at or after sixty seconds from authoritative revocation.	The source-to-commit path and applicable evidence remain necessary; Sections 3.4 and 7.4.
SR-05; freshness	Introduced in Part II: a separate thirty-second conservative authority-status age limit at NORMAL-mode commit.	Not the earlier blueprint's five-minute session proposal and not interchangeable with SR-03; Sections 7.4 and 11.11.
SR-07/SR-07a; transaction repetition	No second commit for the same bound transaction; conflicting identifier reuse is rejected.	Lifetime, persistence, retirement and restore semantics remain design work; Sections 7.5 and 10.5.
SR-10; outcome meaning	Part III distinguishes historical transaction outcome from current active configuration.	IC-01 is a proposed clarification; the requirements owner must resolve any added wording; Section 10.3.
PKG-P1; generation; mode epoch	Candidate generation and epoch identities prevent old context from being treated as current after restoration.	Not selected implementation mechanisms; persistence and recovery require justification; Sections 10.4–10.6.
SR-11/SR-12; REC-P1	The fifteen-minute full task and controlled authority transitions remain required within the stated case.	REC-P1/B-draft and REC-P1/C-draft are unapproved alternatives; Sections 7.6 and 12.3.
A-01/A-02; writer coverage and withdrawal	Unresolved for an actual implementation; a model or local gateway result does not establish the full receiving-system property.	Reconcile actual paths and external-service contributions; Sections 6.7, 8.10 and 19.5.

Subject	Continuity to preserve	Remaining question / locator
E-05-03; E-05-07; DEC-05	The stipulated late commit remains adverse; the correction remains planned and the decision brief unsigned.	No source update in this consolidation changes those statuses; Sections 19.4–19.5 and 20.9.

B.4 Keep separate clocks and branches separate

The numerical illustrations answer different questions. Sharing a unit does not make the intervals interchangeable. They are assumptions or stipulated observations for teaching, not recommended production values or measured performance of an operational service.

Table B.4. Numerical context and branch guide

Illustration	Defined context	Locator and non-equivalence
60 s withdrawal; 30 s freshness	Revoked-identity restriction and age of authority evidence at commit.	Sections 7.4 and 11.11; distinct criteria.
15 min restoration; up to 2 h outage	Full staff task after the service owner declares a corrective need.	Sections 5.7 and 7.6; earlier detection/decision time is outside this clock.
18 + 12 + 2 = 32 s	Separate model-change branch with a twelve-second check-to-commit bound.	Section 11.6; does not overwrite the eight-second case input.
15 + 120 + 15 = 150 min	Prepared-authority coverage with an additional fifteen-minute preparation-age assumption.	Section 12.5; not a new recommended credential lifetime or changed outage requirement.
61.2 ± 0.8 s	Stipulated revoked-identity commit, wholly after the sixty-second criterion under the stated conditions.	Section 19.5; remains hypothetical contrary evidence, not a first-rejected-probe measurement.
180 s monitoring/response	Separate polling, collection, interpretation and effective-action chain.	Section 21.4; does not demonstrate the SR-03 runtime property.
40 min maintenance window	Separate change window; eighteen-minute fallback/check allowance gives a latest start at minute 22.	Section 22.7; not SR-11 and not proof of state reversibility.
150 min exit lower bound	Separate service-exit payload and sequential receiving work compared with 120 min available.	Section 23.8; not a measured throughput or restoration requirement.
Regional crew and human–AI examples	Separate joint-resource, human-response and answer-denominator illustrations.	Sections 27.5 and 28.7–28.8; no inherited administration timing limit or empirical benchmark.

B.5 Distinguish the completion branch and executed model work

CB-09 in Appendix C adds explicit fictional decisions and a stipulated system dossier. SE-C9-01–SE-C9-07 are not new observations of the original case. They demonstrate how a completed

engineering and acceptance argument could be represented. The earlier adverse E-05-03 and planned E-05-07 remain unchanged.

The companion is a separate executed contribution. Revision 9 recorded thirty-five passing unit tests; Revision 10 records sixty after the review correction and twenty-five additional regression cases. Those results apply to the supplied logical abstraction, not to authenticators, storage hardware, an operational service, a human task or a native MBSE model. Appendix C.8 explains the discovered gap, correction and remaining limits.

CP-09 in Appendix D is a separate inspection-vehicle analysis. Its CP requirements and physical stopping distances do not amend the administration system or its SR timing criteria. The companion arithmetic can be reproduced without representing it as a physical experiment.

Appendix C — Completing a bounded engineering case

C.1 A completion branch, not a rewritten history

The earlier chapters intentionally stop where the available information stops. That is useful discipline, but it is not the end of an engineering project. A team must eventually resolve the matters necessary for a decision, evaluate the resulting arrangement and explain what reliance is justified. This appendix follows that work through one bounded branch of the administration case, designated CB-09. It demonstrates a completed line of reasoning rather than promising a complete production specification.

CB-09 is a new fictional branch. It does not change the original status of E-05-03, which remains an adverse hypothetical observation, or convert E-05-07 from a plan into an accomplished correction. Nor does it retrospectively select ARC-B for the earlier case. The branch stipulates additional stakeholder decisions, interface commitments and evaluation information so that the reader can examine an intelligible decision. A statement explicitly labelled “stipulated system evidence” is invented case material. The only executed evaluation supplied with this edition is the small Python abstraction described in C.8; that work did not exercise an operational system, authenticator, storage platform or human participant.

The question for the branch is whether a defined administration arrangement can support authorized normal changes and restricted restoration during an availability-only outage of enterprise identity. The system boundary remains the administration capability. The current management configuration is its consequential output. The operational dispatch consumers and the correctness of dispatch policy remain separate responsibilities. A successful management activation is not a declaration that all downstream work has adopted the change.

The route through the example is: settle the operating conditions; allocate the authority and state obligations; reconcile the timing arguments; resolve the recovery arrangement; evaluate the identified configuration; and make a decision that preserves its limits. The chapter methods, rather than a newly introduced framework, organize each step.

C.2 Freeze the branch's meaning before calculating

For this branch, the fictional service owner accepts a serialized administrative workload. Up to five staff may maintain authenticated sessions, but only one activation is outstanding across the capability at a time, and no more than ten complete activation offers occur in a rolling sixty-second interval. “Outstanding” includes pending execution and an outcome that has not yet been established. The ten-second interval starts when the capability receives the first complete offer, not when it later removes a request from a hidden queue. This resolves the original burst ambiguity by an explicit branch assumption; it does not claim that the earlier unconstrained arrival description already imposed serialization.

A conforming client presents the result of the preceding transaction before offering another activation. Diagnostic and outcome queries remain separately authorized and are included in the capacity evaluation. A new client that violates the offered-load agreement receives an explicit busy or invalid response without a state change. Such rejection protects the system but is not evidence of satisfying the positive activation requirement for that nonconforming workload. The

owner must choose a different design or a different assessed envelope if simultaneous urgent activations become necessary.

NET-C9 specifies, for one conforming normal operation, a delivered authority exchange taking no more than two seconds, at most one interruption of up to five seconds on the operation's critical path, and retry overhead included in those allocations. The remaining local contributions are bounded below. The service's fifteen-minute recovery obligation remains different from this ten-second operation. An interruption lasting longer than the stated envelope does not grant new authority or permit stale commits; it changes the capability that can be claimed.

TIME-C9 requires a conservative time-relationship allowance no greater than two seconds for the relevant source and receiving observations. The allowance includes the uncertainty being used in the decision, rather than a convenient estimate of average clock offset. After a reset, loss of time confidence or a clock correction that invalidates the relationship, normal activation remains prohibited until the relationship is re-established. A monotonic elapsed-time basis is needed between a local check and its effective action; an adjustable wall-clock display alone cannot supply that guarantee.

AUTH-C9 stipulates a truthful source observation time and a source that reflects each recorded identity revocation in every newly issued status observation no later than ten seconds after the authoritative event. The identity-service owner must substantiate that contribution. The receiver rejects invalid authenticity or bindings and detectable inconsistencies; it cannot infer source truthfulness from a valid signature or timely timestamp. A plausible but false statement from a compromised authority lies outside this availability-and-withdrawal argument and needs separate analysis.

Table C.1. Branch definitions and their evaluation consequences

Branch definition	Specific choice	Consequence for evaluation
ADM-C9	Five authenticated sessions; one outstanding activation capability-wide; at most ten offers per rolling minute.	Test the defined offered load and the refusal of excess work separately. No hidden queue resets the clock.
NET-C9	Two-second delivered exchange; at most one five-second interruption on the critical path; retry overhead included.	Repeated interruptions and simultaneous activation bursts are outside the positive-performance claim, not outside protective checks.
TIME-C9	Conservative additional allowance of at most two seconds; trustworthy event meaning; explicit reset response.	Show how source and receiving time relate and invalidate authority when that relation is unavailable.
AUTH-C9	Source status reflects revocation within ten seconds; record binds subject, action, target, mode, epoch and observation.	Source consistency and source authenticity are separate premises.
TX-C9	No execution-history eviction while its namespace remains active; bounded provisioned workload and controlled retirement.	Storage exhaustion is a service limitation; it cannot justify forgetting transactions that may still be retried.

These are sufficient definitions for the selected teaching argument. They are not universal settings. The additional design records still need concrete encodings, capacities, failure tolerances and implementation evidence before a real contract or release could use them. The important difference from the earlier case is that the branch does not conceal which meanings have now been chosen and which realization obligations follow from them.

C.3 Bind identity, permission and the particular operation

The branch uses a normal authentication service and a separately governed recovery authentication service. For each, the selected illustrative authentication pattern is a registered public-key authenticator requiring both possession and local user verification. The branch chooses a hardware-held credential unlocked by a PIN; this is an example configuration, not a claim that every passkey or every implementation has equivalent factor properties. Enrolment binds the credential to an identified person through an approved process; replacement, loss and withdrawal are explicit credential-lifecycle events. NIST's authentication guidance provides the relevant distinction between authenticators, authentication outcomes and management of their lifecycle. [56, Sections 3.1.7 and 4]

The WebAuthn relying party checks the expected ceremony, challenge, origin, relying-party binding, signature and required user-verification result using a maintained implementation. A replayed ceremony, unexpected origin or missing verification indication does not create an authenticated session. WebAuthn supplies authentication semantics; it does not decide whether the person may activate a particular package. That application decision remains separate. [57, Sections 7.1–7.2]

Backend exchanges use mutually authenticated service connections with a restricted peer identity and endpoint purpose. The branch selects TLS 1.3 and disables early data for state-changing exchanges. TLS 1.3 does not itself supply the application's transaction-identity or exactly-once-effect property; its early-data replay limitations are a further reason not to confuse transport acceptance with application authorization. [58, Sections 2.3 and 8]

The permission record binds the authenticated person, activation action, target D1, approved package, mode, epoch and observation time. A recovery grant additionally binds the transaction identifier and expected starting generation, so it authorizes one specified operation. The receiver obtains authority from its configured source; the gateway may relay the request but cannot supply an unchecked replacement decision. The branch still relies on the gateway and administrator workstation to represent the intended action accurately and does not claim resistance to their full compromise.

Credentials for backend service authentication identify the service, not the human whose action is being requested. A successful service connection therefore cannot replace the human permission record. Conversely, a valid human authentication cannot authorize a service to update the protected transaction store directly. The receiver must establish both the identity of the communicating service and the authority for the requested operation.

The trust-material inventory identifies authenticator registration records, service public-key trust anchors, authority-verification material and the participants permitted to change them. No live private keys are included in the book or its companion. For the fictional recovery site, the required certificates, trust material and verification services remain usable throughout the

stipulated outage without fetching a new secret from enterprise identity. Expiry, replacement and local-role withdrawal are tested as dependencies in their own right.

A production design must bind these requirements to actual supported algorithms, implementations and protocol profiles. The book does not invent a cryptographic protocol or certify a browser, token or TLS library. Its completed design choice is the separation and binding of the contributions, with a concrete standards-based example for their realization. An evaluator must still establish that the delivered configuration implements that choice.

C.4 Coordinate the effective change and preserve its history

CB-09 selects one logical receiving coordinator for target D1. Normal and recovery callers pass through that coordinator. A deployment service may install approved executable content during a controlled maintenance state, but it has no supported API that independently selects an active dispatch package. In the fictional branch's acceptance dossier, the installed privilege and path review reconciles those paths. That stipulated result belongs to CB-09; A-01 remains unresolved for the earlier developmental case.

The coordinator binds a transaction identifier to its complete activation content. A current-generation comparison, mode-epoch comparison, permitted package check and authority decision precede the effective change within the defined coordinated operation. The active package, activation generation, transaction outcome and relevant history change as one recoverable logical transaction. Describing that transaction as indivisible is an abstraction to realize, not a substitute for selecting and evaluating a storage implementation.

Suppose TX-41 activates P8 from generation 27 to generation 28. A lost reply leaves the caller uncertain but does not undo the commit. A matching retry returns the recorded outcome without creating another effect, subject to applicable authority for the interaction. Conflicting reuse of TX-41 is rejected. An independent, authorized outcome query can report both the historical result of TX-41 and the target's current generation. If TX-42 later activates P9 at generation 29, neither result erases the other.

A mode epoch identifies an authority context, not merely the string NORMAL or RECOVERY. Entry to a new context advances the epoch through the same controlled state boundary. Pending work with an earlier epoch cannot execute in the destination context without new evaluation. Returning to NORMAL advances the epoch again; it does not make old normal-mode grants current. An operation committed before the transition retains its actual history.

The branch rejects arbitrary snapshot rollback. Recovery of storage must match a retained, protected current-state anchor or reconstruct a state whose configuration, history and authority lineage are justified. An old image cannot be declared current simply because it contains the desired package. The physical implementation of the anchor, its failure behaviour and its joint consistency with stored state require design and evidence. If the anchor cannot be established, the branch refuses state-changing administration and reports a service limitation rather than guessing that no earlier transaction occurred.

TX-C9 retains complete transaction bindings and outcomes while the execution namespace is active. It neither evicts old identifiers after a short cache interval nor reuses a retired namespace. Historical readback remains available for the period during which the owner specifies that function. Provisioned capacity, protected archival access and retirement are part of the

realization record. When capacity or necessary history is unavailable, an activation is not silently admitted on weaker semantics. The resulting loss of useful capability must be reported and managed; protecting history does not make a full store an acceptable availability result.

This design connects SR-07, SR-08, SR-09, SR-10 and SR-12 without claiming that one database feature satisfies all of them. The receiver's persistent state, client retry behaviour, authority rules and operator feedback contribute different parts. A crash-consistency test at one boundary remains only one piece of the composed argument.

C.5 Reconcile freshness, withdrawal and useful performance

The freshness rule is evaluated for the consequential event. Let a be the age of the authoritative observation at the final check, d the justified bound from that check to commit, and u the additional conservative time allowance. For the predicted-commit approach, the branch permits a normal activation only when a + d + u is no greater than thirty seconds. If execution is suspended beyond its justified bound, the operation must obtain a new valid check or be rejected; a favourable earlier check is not a permanent licence to execute.

For this branch, expiry is exclusive: the conservatively predicted commit must occur strictly before the grant’s expiry time. A commit at expiry is refused. The thirty-second freshness limit remains inclusive. These are different predicates over different time quantities, and tests must exercise both exact boundaries. This convention is an explicit branch design choice, not a universal protocol rule.

The existing example a = 18, d = 8 and u = 2 gives 28 seconds. The separate proposed twelve-second branch gives 32 seconds and cannot proceed on the same record. CB-09 instead moves the consequential authority check into a bounded coordinator path for which the proposed delay contribution is 0.7 seconds. It does not simply declare that the previously hypothetical late commit has disappeared. The changed path requires new analysis and corresponding realization evidence.

A useful conditional withdrawal argument can now be derived. Under AUTH-C9, no newly issued active-status observation after t0 + 10 seconds may omit the identity's recorded revocation. Any usable earlier active observation can support a commit only while its true age, conservatively bounded by the receiving rule, is at most thirty seconds. The latest permitted commit under that observation is therefore no later than t0 + 40 seconds. That establishes analytical room within SR-03's sixty-second deadline under the explicit premises; it is not a new contractual forty-second requirement or a measured result.

The argument depends on the observation meaning, consistency bound, record binding, time relationship and complete enforcement path. A source that places a new timestamp around old membership data breaks the premise. A receiver that checks freshness before an unbounded queue breaks the premise. A path that commits without either check breaks the coverage argument. A source signature establishes neither source truthfulness nor complete receiver enforcement by itself.

The positive-performance model remains separate. Under the serialized offered load and NET-C9, the proposed critical path is 0.8 seconds for local preparation, two seconds for the authority exchange, up to five seconds for the one permitted interruption, 0.7 seconds for coordinated commit, and 0.9 seconds for making the authoritative outcome available: 9.4 seconds in total.

The remaining 0.6 seconds is margin within the ten-second requirement, not an independent allowance each component may consume.

The 0.7-second commit contribution is now shared deliberately between the positive-performance model and the branch's final-check model. Other intervals remain different. The earlier forty-five-second source-to-commit allocation is retained in the main text as development history; it is not simultaneously substituted into this branch's critical path. Distinguishing those models avoids both double counting and the false conclusion that a withdrawal bound shorter than sixty seconds automatically establishes a ten-second successful activation.

Cross-model source attribution — The proposed 0.7-second coordinated-commit allocation originates in the SR-00 positive-performance critical-path illustration in Chapter 11, Section 11.12, and is explicitly reused for the bounded CB-09 coordinator/final-check contribution here. The equality of that allocation does not make the models interchangeable: the SR-05 freshness guard remains a + d + u ≤ 30 seconds, including the TIME-C9 two-second uncertainty allowance, while SR-03 continues to bound withdrawal from t0. The 0.7-second contribution is not counted twice within one path. Analytical feasibility remains conditional on the stated source-consistency, workload, interruption and enforcement premises, and does not constitute physical-system measurement or acceptance evidence.

A final timing review asks whether every supposedly bounded step has the necessary input, priority and resource. Retry, status validation, journal completion and query processing cannot be assigned to nowhere. The fictional dossier below stipulates the required realization evidence for the teaching decision. The executable Python model does not measure these contributions and must not be used as their timing evidence.

The two timing requirements can be confused because both involve authority, but they bound different relationships. The timeline below shows the source-consistency contribution, the maximum true age permitted at commit and the unchanged withdrawal deadline on one axis.

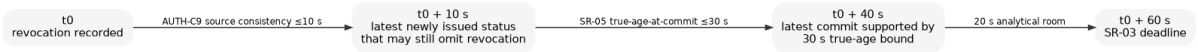


Figure C.1. Source consistency, freshness at commit and withdrawal deadline

Original timing schematic for CB-09. The 40-second point is an analytical consequence of the branch premises, not a new requirement or measured result; SR-03 remains 60 seconds.

C.6 Complete the recovery arrangement and its authority

For CB-09, the fictional policy owner permits an ARC-B arrangement with separately administered local recovery roles. This is an explicit case premise, not a legal or operational approval made by the author. Normal enterprise revocation and local recovery-role withdrawal are distinct decisions with defined relationships. The policy owner accepts that an unavailable communication path cannot convey an instantaneous new central withdrawal; it does not pretend that cached data solves that information constraint.

Two distinct designated people participate: an operator and an approver, identified by canonical person records rather than account aliases. Both authenticate through independently available registered MFA arrangements. The operator proposes an eligible, previously approved package; the approver confirms the transaction identifier, expected starting generation, action, target, package, mode and epoch. The resulting grant authorizes only that operation and expires twenty

minutes after issuance. The receiver requires both participants' applicable local roles at the consequential decision. These are fictional branch choices, not a general administrative shell or a reset of the fifteen-minute restoration clock.

Binding the grant to a transaction and starting generation gives 'single-operation' a precise meaning. Matching retransmissions may recover the same outcome while authority remains applicable; they cannot create another commit. A different transaction, different expected generation or changed target/package requires a new approval. After expiry or role withdrawal, execution remains prohibited, but a separately authorized outcome query can still explain an already completed transaction. Persistent history supplies the no-second-effect contribution; a grant label alone does not.

The local role service can withdraw either participant during the outage. A local withdrawal invalidates uncommitted grants under that role at the receiving decision. Required roster, approval, package, time and credential-verification information is held within the available recovery arrangement. The branch excludes reliance on an enterprise-authenticated approval portal. Normal synchronization does not silently reactivate a locally withdrawn role when enterprise connectivity returns; the two records require reconciliation.

A central-only withdrawal that is not yet known locally remains a stated limitation during communication loss. Physical access, separate roles, narrow package eligibility and local supervision reduce particular opportunities, but do not turn missing information into current knowledge. An organization whose policy requires instantaneous central revocation at every offline action must supply another sufficiently timely channel or reject this branch. It cannot retain that requirement and simultaneously claim the offline arrangement satisfies it.

The complete task begins with the service owner's declaration: two minutes to mobilize the two people, two for access and authentication, four to establish the instruction and approval, three for coordinated restoration, and two for confirmation. This sequence puts authentication before approval while retaining the thirteen-minute allocation and two-minute planned margin. The grant is issued within the task. Its twenty-minute lifetime is a separate authority bound, not evidence that the thirteen-minute task allocation has been achieved.

The granted action remains subject to package eligibility, target generation, current epoch and preserved history. An identity-service return midway through the task does not grant automatic normal authority. Staff may finish the permitted recovery action or make a controlled handover, provided the original fifteen-minute obligation and transition rules remain satisfied. Suspected compromise of the local controller, approval source or retained history requires the separate incident arrangement, not an automatic invocation of this availability-only recovery permission.

On return, the team reconciles the active management package, completed transactions, local withdrawals, pending work and normal authority basis. It closes temporary grants and advances the context. Completing those steps is different from concluding that every downstream dispatch consequence has been repaired. The operational service owner retains that broader responsibility.

C.7 Show correction and evaluation without rewriting the evidence

The earlier E-05-03 dossier records a late commit under revoked identity. For this branch, the fictional investigation attributes the relevant discrepancy to a request that passed an admission

check, then waited in a queue before becoming effective without a sufficiently current receiving-side decision. That is an added story fact used to demonstrate correction, not a cause proven for an actual service. The original observation and its uncertainty interval remain unchanged.

The proposed correction removes reliance on admission-time authority for later effective change. It introduces the bounded final decision, source-observation meaning, controlled queue behaviour and receiving coverage described above. The team then identifies what must be evaluated again and what earlier evidence can remain applicable. Gateway input-format results may still apply; the authority, queue, workload, time and corresponding integration conclusions require new support.

The following dossier is deliberately labelled as stipulated system evidence. It represents the kind of information a completed project would have to produce. It is not populated from the Python tests and does not claim that the author performed these activities.

Table C.2. Stipulated system-evaluation dossier for CB-09

Stipulated record	Content supplied by the fictional project	What it supports, and what it does not
SE-C9-01	Controlled path and privilege review for the named CB-09 configuration; normal and recovery activations reach the coordinator; deployment cannot select active packages through another supported path.	Supports branch-specific path coverage. Does not retrospectively resolve A-01 for the earlier case or establish immunity to platform compromise.
SE-C9-02	Source-interface review and bounded timing analysis, with the admitted source and receiving configurations identified.	Supports the conditional withdrawal and freshness composition. Does not cover a dishonest authority source.
SE-C9-03	Corrected-configuration revocation observations include a last permitted commit at 37.4 seconds with a conservative ±0.8-second bound, alongside boundary and delayed-work cases and the complete timing analysis.	The stated observation is compatible with 36.6–38.2 seconds. A single early commit is not evidence that no later one can occur; coverage and the timing argument are essential.
SE-C9-04	Authorized activations complete in the defined workload; the worst stipulated observation is 9.1 seconds with ±0.2-second measurement bound. Excess offered load is refused without changing state.	Supports the represented ten-second positive-operation criterion. Does not assert capacity for unrestricted simultaneous requests.
SE-C9-05	Storage, interruption, repetition, restart, mode and migration evaluations preserve the described current and historical state; an old snapshot is refused.	Supports the evaluated implementation and fault model. The actual storage/controller evidence must exist in a real project.

Stipulated record	Content supplied by the fictional project	What it supports, and what it does not
SE-C9-06	Representative designated staff complete the full availability-outage task in 13.2 minutes with a conservative ±0.3-minute observation bound; a separate exercise confirms rejection of self-approval and a locally withdrawn role.	Supports represented task completion within fifteen minutes, including people and access. Does not establish resilience to local-controller compromise or unavailable staff.
SE-C9-07	Reviewed model exchange retains the source allowance, selected branch, configuration identities and the separation of plans from results.	Supports the affected exchange and evidence lineage. It is not a new observation of the running system.

A real dossier would contain the procedures, raw observations needed for challenge, configuration records, coverage model, exceptions and competent reviews. The table is a concise worked interpretation, not permission to replace those records with a list of favourable assertions. An apparently acceptable result must still survive a check of whether the relevant event was actually observed.

SE-C9-06's stipulated 13.2 ± 0.3 minutes corresponds to 12.9–13.5 minutes. Its upper compatible value is below the fifteen-minute requirement, leaving 1.5 minutes of conservative observed headroom in that scenario. It does not substantiate the earlier thirteen-minute allocation: the interval crosses that value. Retain the successful requirement-level interpretation and review the planning allocation separately. Neither value is a real participant measurement.

C.8 Execute a deliberately smaller abstraction

The companion supplies activation_model.py, the original test module as revised, and test_review_regressions.py. They use the Python standard library. The model represents package, generation, mode epoch and transaction history, checks trusted authority/time inputs, and performs an indivisible logical change. Its snapshot fingerprint assumes a protected current-state anchor and joint atomic state; it is not a power-fail-safe persistence implementation. Mode transitions are privileged test-driver operations whose real authorization remains outside this abstraction.

Revision 9's thirty-five unit tests passed, but review found that one recovery grant could authorize two distinct transactions. A reproduced counterexample is retained in the companion. Revision 10 binds that grant to a transaction and expected generation and adds explicit approver-authentication, issuance, lifetime and boundary checks. The revised suite contains sixty passing tests, including twenty-five review regressions. A revised conflicting-identifier test also supplies otherwise valid package authority so that it actually reaches the intended history check. These results concern the supplied abstraction, not the stipulated system dossier.

The model tests exercise logical behaviour under trusted inputs. They do not authenticate either participant, verify WebAuthn or TLS, measure real time, exercise database power loss, discover missing writers or validate a human task. Authentication, active-role status, grant contents and times are supplied test values; real mechanisms must establish them. Arbitrary modification of Python objects is outside the model's trust arrangement.

The deliberately faulty variant restores an earlier generation and erases history while an external model observer retains the fact of the first effect. TX-41 then appears new and produces a second effect. The protected variant rejects that snapshot because it does not match its retained anchor. This counterexample makes the design obligation visible. It does not prove that the chosen anchor implementation will survive the failure or compromise conditions of a deployed system.

The example also demonstrates useful restraint in model claims. Unit tests sample behaviour; they are not exhaustive formal verification. The no-second-commit argument in Chapter 11 still requires its transition and history premises. Neither a passing test suite nor an induction over an incomplete state model establishes that every relevant real-world transition has been represented. The executable contribution is a repeatable teaching aid, not a new certification badge.

C.9 Build a decision from the completed branch

The fictional review separates requirement and interface adequacy, support within the stipulated realization/task dossier, and the narrower results actually obtained from the companion abstraction. Revision 10's grant correction improves the last contribution; it does not supply actual evidence for the fictional storage, authenticators or staff. A real decision needs those distinct contributions at the required maturity and must preserve their provenance.

The resulting fictional decision permits CB-09 for the stated administration function under its defined normal workload and availability-only recovery conditions. It does not authorize arbitrary package content, a general-purpose recovery shell, use without the designated local roles, or operation after protected state or timing confidence is lost. It is not an authorization of the original unresolved branch, and it does not establish the correctness of downstream dispatch rules. No actual person signs or grants this illustrative decision.

Continuing conditions are assigned to real kinds of work rather than slogans. The service owner maintains staffing and the allowed task. The identity participants preserve source semantics and local-role governance. The implementation owner controls the coordinator and storage configuration. The integrator retains writer coverage and model/evidence applicability. A changed source consistency bound, a newly introduced writer, a modified storage controller, repeated recovery-role workarounds or a failed readiness exercise reopens the affected conclusion.

If the fictional organization cannot supply one of those contributions, the decision changes. A shortage of recovery staff is not fixed by the controller's fast execution. An unavailable historical record cannot be replaced with an assertion that a retry is probably new. An unsupported source observation cannot be made current by receiving it over TLS. The completed example is useful precisely because each limit has a responsible response and an effect on the permitted use.

The general lesson is not that every open item must disappear. It is that the questions material to the chosen commitment must be resolved, constrained or knowingly retained through the proper decision. Technical evidence, residual limitations and authority remain distinct all the way to the conclusion. That is how an engineering case reaches a justified result without manufacturing certainty.

Appendix D — A cyber-physical inspection vehicle

D.1 Why this example is different

The administration case centres on a management-state change. A physical system adds something that cannot be undone by restoring a database: motion, energy and consequences already produced in the environment. This appendix develops the inspection-vehicle vignette from Chapters 9 and 12 into a separate example, CP-09. It is not a proposed medical-device design, a certified mobile robot or an account of a field trial.

The vehicle's useful task is to carry an inspection sensor along an approved indoor route and return usable observations. A remotely supplied route can be authentic and still be unsuitable for the vehicle's current load, location or surroundings. Refusing every route prevents the task; accepting every correctly signed route permits losses that authentication does not address. SSE therefore contributes to a joint engineering problem with motion control, safety, sensing, human factors, communications and support.

For the calculations below, the selected condition is straight-line motion toward a stationary obstacle on a level surface. The speed, detection distance, delay and minimum deceleration are invented teaching assumptions. They have not been measured on a vehicle. The example does not cover a moving pedestrian, a slope, a suspended load, every sensor failure or every form of brake degradation. Those exclusions matter to any transfer of the result. NIST's OT guidance explicitly calls for consideration of physical-system performance, reliability and safety needs; it does not supply these particular values or a complete vehicle design. [21, scope]

The work proceeds from a useful mission and losses through control relationships, requirements, allocation, analysis, evaluation and change. Security does not replace the physical disciplines. It asks how inappropriate commands, altered parameters, misleading feedback or maintenance authority could produce or prevent the physical loss, and helps the responsible disciplines design an adequate response.

D.2 Describe the control relationship and its losses

CP-09 distinguishes a route-planning service, an onboard mission controller, a local motion-protection function, sensing, the drive and braking contribution, and an operator/maintainer. The route-planning service proposes a path and permitted mission parameters. The onboard controller interprets those inputs and requests motion. The protection function determines whether the requested motion remains within its justified operating conditions. Actuation produces the physical result, and sensing supplies feedback. Each arrow must identify what is sent and what decision uses it.

The example's losses include collision with an obstacle, unintended departure from the permitted inspection area, loss of useful inspection data and inability to bring the vehicle into a controlled maintenance state. These losses are not identical and need not share one response. A command that prevents a route update may preserve integrity while leaving the moving vehicle dependent on its current control loop. Removing all electrical power might not provide the assumed braking behaviour. The actual mechanical and power arrangements decide that question.

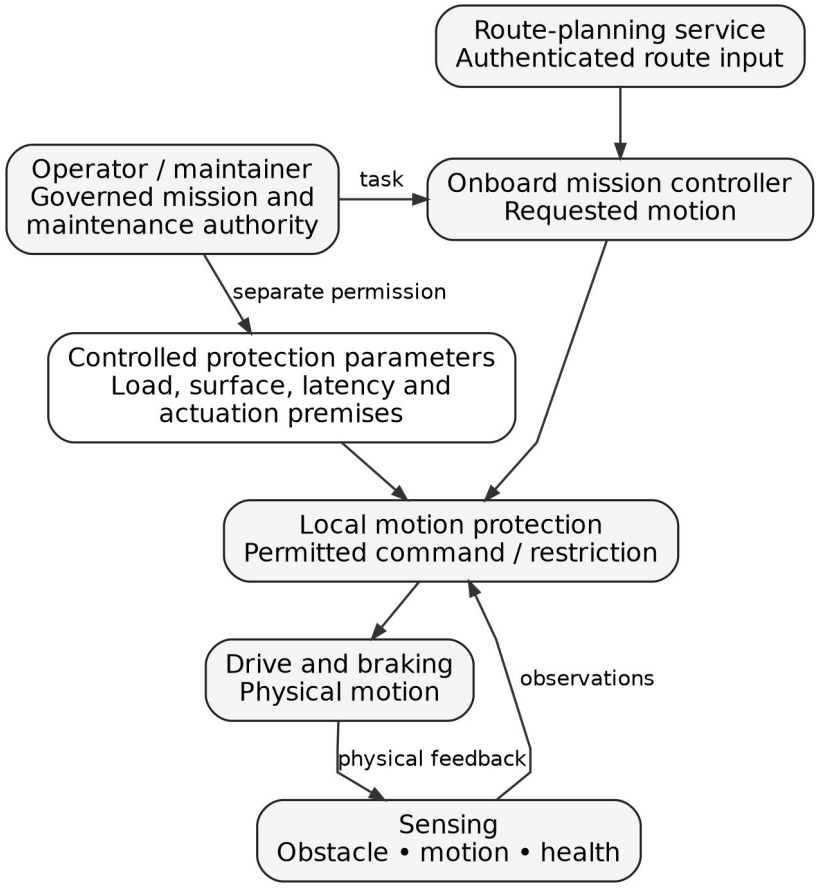
The STPA Handbook's control-structure treatment is useful for examining controller responsibilities, commands and feedback, including missing, inappropriate and mistimed control actions. [16, Chapter 2, control-structure and unsafe-control-action sections] CP-09 uses an original, limited control discussion; it is not a completed STPA analysis and does not reproduce its source diagrams.

Consider the request to continue at an indicated speed. It can be inappropriate because the speed is too high for the available stopping distance, because obstacle information is stale, because load changes have invalidated the deceleration assumption, or because a maintenance mode permits motion without the required protective function. Security-relevant causes include unauthorized parameter changes, replay of an earlier command, alteration of a route's coordinate frame and privilege that disables the protective function. Non-deliberate causes may lead to similar hazardous conditions and remain the concern of the relevant specialty analyses.

The resulting dependency map includes more than the radio link. A local protection controller may depend on a shared processor, the same compromised configuration source, common time information, power and sensors. Calling it independent because it is represented in another box does not establish the independence needed for a particular failure. The architecture must state which shared cause it addresses and what common dependencies remain.

The following schematic brings together the relationships developed here.

Figure D.1. Functional relationships in the inspection-vehicle example



Original CP-09 functional schematic, not a wiring diagram or a certified vehicle architecture. Control and feedback relationships require the detailed timing, authority and failure analysis in Appendix D.

D.3 Allocate positive and protective requirements

CP-09 introduces its own CP identifiers. None changes the administration case's SR requirements or time windows. The following statements are candidate teaching requirements, expressed at a level sufficient to direct the analysis; the vehicle's detailed operational specification and applicable standards would have to complete their real use.

Table D.1. Candidate cyber-physical requirements and evidence

Candidate requirement	Allocation and intent	Evidence needed
CP-01: The vehicle shall execute an authorized route within the defined inspection envelope and associate observations with the route and position context used.	Mission controller, sensing and data recording provide useful inspection, not merely denial of unsafe commands.	Representative route and data-use validation, configuration and interface checks.
CP-02: A route activation shall identify its coordinate frame, revision, permitted operating envelope and target vehicle, and shall not silently substitute the current physical pose with a stored prior pose.	Route interface and onboard state reconciliation distinguish information activation from physical state.	Incorrect-target/frame, replay, restart and interrupted-update cases.
CP-03: Motion permission shall depend on the current justified stopping-distance condition and required protection-function health, not solely on successful route authentication.	Local protection, sensors and actuation must establish the physical contribution.	Timing and braking evidence, fault analysis and tests under representative loads and conditions.
CP-04: Loss of remote communication shall invoke a locally defined continuation or stopping behaviour whose assumptions are explicit.	The onboard arrangement must not depend on a remote human responding within an unachievable interval.	Lost-link scenarios with actual dynamics, local resources and operator information.
CP-05: Changes to protective speed, sensing and braking parameters shall require separate authorized configuration control and shall not be available through ordinary route-upload authority.	Administration, configuration enforcement and maintenance arrangements restrict effective influence.	Privilege/path review, configuration-change tests and return-to-service evaluation.
CP-06: Return from maintenance shall require established current pose, applicable route and protective configuration, and a valid basis for motion permission.	Maintainer and onboard checks prevent restored files from becoming assumed physical truth.	Representative maintenance/restart tasks, conflicting-state and stale-route cases.

Several relationships deserve attention. CP-03 cannot be fulfilled by a software comparison when the inputs to that comparison have no adequate physical basis. CP-05 does not prevent every malicious change if the same administrator can alter the protective configuration through an undocumented maintenance port. CP-01 requires useful work within the permitted envelope; repeatedly refusing valid missions is not proof of a satisfactory vehicle. The requirement set must be reviewed together.

A safety function may be subject to domain-specific independence, integrity or evaluation requirements. This appendix does not assign such a level or infer conformity from the security treatment. The project must identify the actual application, hazard severity, governing standards and competent safety authority. The SSE contribution should make the security-relevant influence paths visible to that work.

D.4 Calculate a stopping condition with honest boundaries

Let v bound actual forward speed throughout the detection-to-braking delay and at brake onset, tau bound that delay, and a_min be a justified positive lower bound on the effective braking deceleration thereafter. A speed measured only at the start is insufficient if the vehicle may accelerate during tau. Let e be a conservative spatial uncertainty allowance and m an additional engineering margin. For straight-line motion toward a stationary obstacle under those bounds, the conservative required distance is:

$$D_{required} = v \times tau + v^2 / (2 \times a_{min}) + e + m.$$

The first term bounds travel before braking because speed remains no greater than v. The second bounds subsequent stopping distance when effective deceleration remains at least a_min; constant deceleration at that lower bound gives the limiting value in this simplified model. The uncertainty and margin terms address different contributions and must not double-count one error. This is an elementary-mechanics teaching calculation, not a vehicle standard or measured brake characteristic.

Assume v = 1.2 metres per second, tau = 0.35 seconds, a_min = 0.8 metres per second squared, e = 0.15 metres and m = 0.10 metres. Then D_required = 0.42 + 0.90 + 0.15 + 0.10 = 1.57 metres. If the assumed usable detection distance is two metres measured from the vehicle's leading edge to the obstacle along the path, the numerical room is 0.43 metres. That room is conditional on every contribution having the stated meaning and bound.

Now raise the speed to 1.8 metres per second without changing the other assumptions. The result becomes 0.63 + 2.025 + 0.15 + 0.10 = 2.905 metres. A two-metre detection arrangement cannot justify that speed under this model. An authorized configuration change can therefore produce an unacceptable physical relationship without any failure of route authentication.

A change in load or floor condition can be equally consequential. Keeping the original 1.2-metre-per-second speed but reducing the justified minimum deceleration to 0.4 metres per second squared gives 0.42 + 1.80 + 0.15 + 0.10 = 2.47 metres. The original speed is no longer supported by the same detection-distance argument. A software release identifier remaining unchanged does not preserve the lost physical premise.

Table D.2. Conditional stopping-distance calculations

Assumed condition	Calculated required distance	Relation to assumed 2.00 m usable distance
v = 1.2 m/s; a_min = 0.8 m/s²	1.570 m	0.430 m numerical room.
v = 1.8 m/s; a_min = 0.8 m/s²	2.905 m	Exceeds the available distance by 0.905 m.
v = 1.2 m/s; a_min = 0.4 m/s²	2.470 m	Exceeds the available distance by 0.470 m.

The model itself directs the questions for further work. Does tau include sensing, filtering, communication, scheduling, decision and actuator delay? Does v describe actual motion or merely the last requested speed? Does a_min remain justified under the permitted load, energy state, floor, wear and represented faults? Can the obstacle be detected at the stated distance under the intended lighting and reflectivity? Is the vehicle already turning, accelerating or approaching a moving object? Until those matters are addressed, the calculation is a conditional design analysis, not permission to operate.

A separate sensitivity case shows why the speed premise matters. If only the starting speed v0 is known and forward acceleration during tau is bounded above by b, use $D_{required} = v0 \times tau + 0.5 \times b \times tau^2 + (v0 + b \times tau)^2 / (2 \times a_{min}) + e + m$, under the same straight-line, stationary-obstacle and braking premises. With v0 = 1.2 m/s, b = 1.0 m/s² and the earlier delay, deceleration and allowances, the bound becomes approximately 2.233 m, exceeding the assumed two metres. Setting b = 0 recovers 1.57 m. The added b value is a hypothetical review assumption, not a measured acceleration. A verified speed bound throughout the delay would instead permit the original formula to be used directly.

The baseline stopping model is also useful as a sensitivity picture. The curve below holds the stated delay, minimum deceleration and allowances constant while varying the bounded speed. It therefore visualizes one relationship only; it does not replace the later questions about acceleration during the delay, moving obstacles, turning, surface or brake capability.

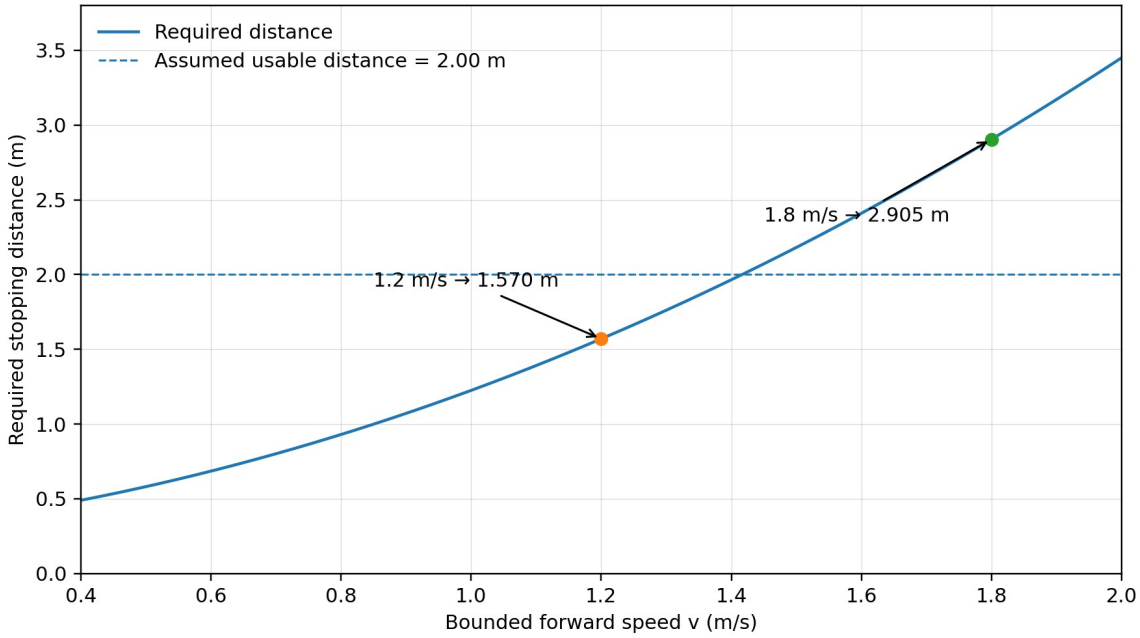


Figure D.2. Required stopping distance grows nonlinearly with bounded speed

Original plot of the Appendix D baseline stopping equation using its fictional assumptions. The dashed line is the assumed 2.00 m usable distance; points at 1.2 m/s and 1.8 m/s reproduce the worked calculations.

D.5 Engineer the protection and its own failures

One candidate architecture constrains motion locally using a separately controlled protective parameter set and a receiving enforcement point that ordinary route upload cannot bypass. This reduces dependence on round-trip communication and operator reaction for the immediate stopping condition. It does not eliminate the need for human authority, nor does it establish that a separate controller is always the right design.

The candidate must explain how it detects loss of its own necessary inputs and what physical response remains feasible. A controller that commands braking when power disappears requires a brake and energy arrangement capable of that response. A watchdog timer that detects missed execution requires a path to effective action and protection against the same failure of concern. A sensor plausibility check can reveal some inconsistent information without proving the absence of a common bias.

The operator's role is designed around the time and information available. The operator can authorize a mission, interpret a restriction, request recovery and manage maintenance. A remote operator is not credited with stopping a rapidly developing collision merely because a stop button exists on a dashboard. Detection, presentation, recognition, decision, transmission and actuation must fit the available physical interval. Where they do not, the immediate protective function needs another allocation.

A controlled stop is also a task with preconditions and consequences. It may be suitable in the isolated straight-line test scenario while inappropriate at another location or under another load. The operational concept should identify where the vehicle may stop, what it must report, how

other people avoid it, and what is required before movement resumes. A universal instruction to "fail closed" conceals those physical differences.

This is a joint trade rather than an argument that safety and security conflict by nature. Restricting route-upload privilege may simplify both analyses. Separating maintenance parameter authority may reduce unintended alteration while requiring additional staffing and access planning. A local protection function can improve response time while adding a significant trusted implementation. The engineering record should preserve these benefits, costs and remaining dependencies together.

D.6 Connect models without mistaking them for the vehicle

The descriptive model relates CP requirements to route reception, mission control, local protection, sensing, actuation and operator tasks. A behavioural view describes route activation, lost communication, restricted operation, stop, maintenance and return. A dynamics model evaluates selected motion and stopping conditions. A configuration record identifies the vehicle, load, protective parameters and the versions of the models and software. These are connected contributions, not interchangeable models.

The same quantity needs compatible meaning across the toolchain. A speed in kilometres per hour cannot enter a calculation expecting metres per second without the correct conversion. A simulation step interval is not automatically a bound on deployed control latency. A braking curve calibrated at one load does not establish the minimum deceleration for another. Model-based engineering helps expose those relationships when units, conditions and provenance are preserved.

The interface between models deserves evaluation as well. If the architecture model exports a maximum speed while omitting a changed load condition, an unchanged dynamics result can mislead the reviewer. If both the test oracle and controller are generated from the same incomplete model, they may agree while excluding a needed lost-sensor transition. Independently reasoned scenarios and physical observations remain necessary where those common omissions matter.

The companion reproduces the three original stopping calculations and the added acceleration-during-latency sensitivity case. These are arithmetic checks, not dynamics simulation, hardware-in-the-loop execution or robot validation. A richer model would need its own stated scope, solver behaviour, input evidence and correspondence with a physical article.

D.7 Evaluate the coupled task and make a bounded decision

A credible evaluation programme begins by validating the requirement set with the intended inspection use, physical environment and loss constraints. It then plans analytical checks, implementation review, controlled bench work, representative sensing and actuation tests, and carefully bounded integrated trials under the relevant safety arrangements. Intrusive failure injection on a moving system requires authorization, protection of participants and a method that does not create an uncontrolled hazard.

For CP-03, the evidence must establish the contributions to the stopping condition, not simply show that a software threshold exists. A measured latency distribution needs a justified interpretation for the required bound. A brake test needs identified load, surface, energy and

instrument conditions. A sensor result needs obstacle and environmental relevance. Integration tests must show that the actual sensing, decision and actuation chain preserves those relationships.

Verification of CP-05 examines whether ordinary route-upload authority can alter protective parameters, including maintenance and support paths. Validation asks whether legitimate personnel can perform the inspection and permitted maintenance tasks without unsafe or unmanageable workarounds. Both matter. A perfectly enforced maintenance restriction can leave a damaged vehicle stranded in an unsafe position if no usable recovery arrangement exists.

A release decision would combine those results with the supported operating envelope, residual limitations, trained roles, maintenance checks and ongoing change responsibilities. CP-09 does not supply a fictional all-passing physical dossier. It reaches a completed analytical finding: two proposed changes violate the stated stopping-distance condition, and the original condition has numerical room only under explicit physical premises. The justified next commitment is to obtain the domain evidence needed for the selected vehicle—not to import the administration case's logical test results as physical assurance.

D.8 Exercise: signed software, stale physics

A maintainer installs an authentic approved route package and increases the configured speed to 1.8 metres per second to finish inspections sooner. The software authenticity checks pass. The protective configuration still assumes a two-metre usable detection distance, 0.35-second total delay and 0.8-metre-per-second-squared minimum deceleration, with the stated uncertainty and margin. The approval record says “security checks passed.” What should the engineering review conclude?

The checks may establish authenticity within their evaluated scope. They do not establish that the faster operating condition satisfies CP-03. The stopping calculation now requires 2.905 metres, exceeding the assumed available distance by 0.905 metres. Preserve the valid authenticity result while rejecting the claimed physical adequacy under those assumptions. Review why route or maintenance authority could alter a parameter on which the protection argument depends, and determine the authorized design or operating change needed.

A different sensor arrangement, lower speed, reduced justified delay or stronger justified deceleration could change the result. None can be assumed merely to preserve the desired completion time. The new combination requires compatible evidence and intended-use review. This is the cross-domain lesson: the authenticity of an instruction, the correctness of its processing, the dynamics it produces and the authority to rely on it are different propositions within one engineered capability.

Appendix E — Protecting information flow without losing the support function

E.1 Start with the support task, not the encryption control

A confidentiality example should preserve useful work rather than treat secrecy as the absence of information exchange. Consider a supplier support team asked to diagnose repeated failures in the administration capability. The customer can export diagnostic records, but those records can contain privileged configuration details, person identifiers, internal addresses, package metadata and potentially authentication or secret material. The engineering question is what information the supplier actually needs, which information the customer is permitted to release, and how the exchange remains useful enough to resolve the defect.

The example is deliberately bounded. It does not define privacy law, classified-information handling, export controls or a universal support contract. Those obligations come from the real jurisdiction and agreement. The teaching purpose is to show how a familiar confidentiality requirement becomes a system and workflow design problem rather than a command to encrypt everything indiscriminately.

E.2 Define the permitted information outcome

Assume the fictional customer authorizes release of transaction timing, software and configuration versions, non-secret error identifiers and selected interface-state observations for a named incident. It does not authorize release of authentication secrets, unrelated operational content, full privileged configuration exports or personal data that is unnecessary to the diagnostic task. The support result must still preserve enough context to reproduce the failure and relate the supplier's finding to the affected configuration.

This creates both a positive requirement and a protective one: the supplier needs an adequate diagnostic view, and the export must not include information outside the permitted purpose and scope. Encrypting the transfer can protect data in transit while leaving the selection wrong. Conversely, removing every sensitive field can make the support package useless. The design therefore includes collection, selection, filtering, export protection, supplier use, customer evidence retention and eventual disposition.



Figure E.1. Information-flow protection begins with permitted content and purpose

Original information-flow schematic for the support example. The filtering/enforcement function determines what may cross the organizational boundary; transport protection then protects an already permitted export rather than deciding its purpose.

E.3 Turn the outcome into reviewable requirements

Table E.1. Bounded information-flow requirements for the supplier-support example

Teaching requirement	Engineering contribution	Evidence / challenge
IF-01: The exported support package shall contain the information required for the approved diagnostic task and identify the affected configuration and incident scope.	Collector and export format preserve selected timing, version, error and interface context.	Representative diagnostic task; supplier can reproduce the issue without hidden customer intervention.
IF-02: The export shall exclude authentication secrets and information not authorized for the approved support purpose.	Selection and filtering occur before release; alternate export and debug paths are included.	Field/path review; seeded prohibited values; inspection of generated package and retained logs.
IF-03: The approved export shall be protected against unauthorized disclosure or alteration during transfer and storage.	Transport, access, storage and integrity mechanisms are selected for the actual exchange.	Configuration and access review; transfer/storage evidence within scope.
IF-04: Copies, retention and disposal shall remain traceable across customer and supplier responsibilities.	Agreement identifies retained copies, support case closure, customer evidence and disposition responsibilities.	Records of transfer, retained copies, expiration/disposition and unresolved exceptions.

E.4 Follow the information after the first transfer

A release decision applies to an identified package and purpose, not to every later copy. A supplier may import the package into a case-management system, copy it into a test environment or include excerpts in a defect report. Each transformation can alter the audience, retention period or ability to remove information later. The receiving-system analysis therefore follows copies and derived records that remain material to the protection claim.

This is where existing IT practices become valuable engineering inputs. Data-loss-prevention rules, secure-transfer mechanisms, access logs, retention schedules and supplier handling procedures can provide real contributions. Their existence does not prove the complete flow is correct. The SSE task is to connect them to the defined information, purpose, interfaces, copies and evidence needed for the receiving-system claim.

E.5 Exercise: encrypted, authenticated and still wrong

Situation. A diagnostic archive is encrypted in transit and delivered to the authenticated supplier contact. It contains a full privileged configuration export, including an unnecessary credential-recovery value, because the exporter selected “collect everything.” The support engineer needed only timing, version and transaction-state information. The project reports the confidentiality requirement as satisfied because encryption and access control worked. What should the review conclude?

Worked answer. Preserve the valid evidence that the approved transfer path protected the package in transit and restricted the receiving account. Do not extend that result to the selection of information. The package contained data outside the stated diagnostic need and permitted

release. Correct the collection/filtering path, determine whether copies or derived records already exist, apply the governing response and retention arrangements, and obtain evidence for the corrected flow. The failure is not evidence that encryption is useless; it shows that transport protection and permitted information flow answer different questions.

Appendix F — Resilience when a trusted source becomes suspect

F.1 A functioning dependency can still be unusable for a claim

Availability-only recovery asks what happens when a service cannot be reached. A compromise scenario is different: a service can remain responsive while its answers are no longer a justified basis for consequential action. This appendix develops a bounded extension of the recurring case in which credible evidence indicates that the package-approval source may have been compromised. The scenario is separate from CB-09 and does not retroactively change its availability-only premises.

The immediate question is not "is the source up?" but "which claims still depend on information from that source, and what independent basis exists for the capability that must continue?" A cryptographically valid package can still be unsuitable if the approving authority or its process is suspect. A successful network connection cannot restore that lost basis.



Figure F.1. Resilience after loss of confidence in a previously trusted source

Original compromise-response schematic. It separates containment, independent retained capability and reconstitution; it does not imply that every compromise permits continued operation.

F.2 Define what may continue and what must stop

Table F.1. Bounded capability decisions when the approval source is suspect

Capability / contribution	Provisional treatment under the scenario	Reason / evidence needed
Read-only diagnosis using locally verified state	May continue if it does not depend on suspect approval data and remains properly authorized.	Preserves useful understanding without creating a new configuration effect.
Activation of a newly approved package from the suspect source	Prohibit pending re-establishment of approval trust or another authorized independent basis.	Availability of the source does not establish trustworthiness of its decisions.
Use of a previously approved recovery package	Consider only if its identity, retained approval basis, eligibility, state assumptions and local authority remain independently supportable.	A prior approval may be useful evidence; its continuing applicability must be justified.
Return to normal approval service	Require reconstitution and evidence for restored source integrity, credentials, configuration and decision lineage.	A service restart alone does not restore the previous assurance basis.

F.3 Reconstitute the basis for reliance, not only the service

Reconstitution may involve replacing or validating the affected approval service, rotating credentials or trust material, establishing a known-good configuration, reconciling packages approved during the suspect interval, and determining whether downstream systems acted on untrustworthy approvals. The exact work follows the incident facts and governing authorities; the example does not prescribe a universal rebuild sequence.

An independent recovery arrangement is useful only to the extent that it avoids or contains the compromised dependency. If the “independent” recovery controller accepts the same suspect package-eligibility feed, its independence is incomplete for this scenario. If it relies on a protected local catalogue whose provenance and eligibility were established before the compromise, that catalogue may provide a bounded basis for a restricted task, subject to freshness, revocation and operational limitations. The systems-engineering question is which assumptions survive the new condition.

F.4 Exercise: the approval service is online again

Situation. The package-approval service was suspected of compromise. After an hour, its administrators restart the service and report that it is reachable. The administration capability can again retrieve signed approval records. A manager proposes returning immediately to normal package activation because “availability is restored.” What should the engineering and assurance review require?

Worked answer. Reachability closes only the availability observation. The decision still depends on whether the approval service, its trust material, relevant configuration and approval lineage have been re-established as a justified basis for package eligibility. Determine which approvals were created or changed during the suspect interval, whether downstream actions require reconciliation, and which evidence supports the restored source. Until that basis is established, retain only the capability whose claims do not depend on the suspect contribution, or impose another authorized restriction. Reconstitution restores reliance through evidence and decisions, not through uptime alone.

Appendix G — Practitioner worksheets

These worksheets extract information structures already used in the book. They are optional aids, not mandatory SSE documents. A project should place the same information in its existing requirements, architecture, risk, test, configuration or decision systems when those systems can preserve the necessary meaning, ownership and evidence. Completed examples appear throughout the chapters; the blank structures below are intended for adaptation.

G.1 Protection need and loss scenario

Field	Project entry
Useful capability / outcome	
System boundary and affected stakeholders	
Loss or unacceptable consequence	
Protection need	
Relevant adversity / initiating condition	
Key assumptions and dependencies	
Next engineering question / owner	

Use the second column to record the project-specific answer and link to the controlled source where appropriate. A blank or unknown entry is a visible work item; do not convert it into a favourable assumption merely to complete the form.

G.2 Requirement and allocation

Field	Project entry
Source need / obligation	
Requirement statement and subject	
Definitions / profile references	
Allocated functions / elements / people	
Positive capability preserved	
Evaluation method and criterion	
Open assumptions / authority decisions	

Use the second column to record the project-specific answer and link to the controlled source where appropriate. A blank or unknown entry is a visible work item; do not convert it into a favourable assumption merely to complete the form.

G.3 Architecture decision

Field	Project entry
Decision question	
Alternatives considered	
Binding conditions / gates	
Principal trade criteria	
Selected direction for this maturity	

Field	Project entry
Consequences and new dependencies	
Evidence still required	
Reconsideration triggers	

Use the second column to record the project-specific answer and link to the controlled source where appropriate. A blank or unknown entry is a visible work item; do not convert it into a favourable assumption merely to complete the form.

G.4 Evidence applicability

Field	Project entry
Claim / requirement	
Evidence item and provenance	
Evaluated configuration / representation	
Method and conditions	
Result / observation	
Coverage and limitations	
Contrary evidence / defeaters	
Applicable conclusion and decision use	

Use the second column to record the project-specific answer and link to the controlled source where appropriate. A blank or unknown entry is a visible work item; do not convert it into a favourable assumption merely to complete the form.

G.5 Change-impact review

Field	Project entry
Proposed / observed change	
Affected capability / interfaces / assumptions	
Requirements and claims potentially affected	
Evidence retained with rationale	
New analysis / evaluation required	
Operational / supplier / human impacts	
Decision and resulting baseline	
Triggers for further reconsideration	

Use the second column to record the project-specific answer and link to the controlled source where appropriate. A blank or unknown entry is a visible work item; do not convert it into a favourable assumption merely to complete the form.

Subject index

Selected methods and examples, indexed by section rather than page. These locators remain stable within this edition after repagination. The glossary supplies definitions; this index directs practical reading.

Acceptance, contractual and operational. 17.2; 20.2; 20.7; C.9

Acquirer contributions. 7.9; 17.6

Activation, commit and outcome. 10.2; 10.3; C.4

Adversity and causal scenarios. 6.1; 6.3; 12.2

AI-assisted engineering. 28.1; 28.3; 28.12

AI-enabled capability. 28.1; 28.5; 28.10

Allocation of requirements and functions. 7.7; 9.2; 16.2

Architecture alternatives. 9.5; 11.15; 11.16

Assumptions, model and operating. 2.5; 8.8; 11.7; C.2

Assurance cases and contrary evidence. 19.2; 19.5; 19.7

Authentication and application authority. 7.3; C.3; C.6

Authority, decision and action boundaries. 20.1; 20.6; 28.5

Baselines and configuration identity. 22.2; 25.6

Binding requirements versus preferences. 11.15; 11.16

Black-box and inherited-system evidence. 23.2; 23.3

Change-impact analysis. 11.6; 22.5

Cloud responsibility and exit. 23.6; 23.8

Competence and supervised practice. 26.4; 26.5

Compliance in engineering. 3.1; 3.2; 18.2

Composition and common dependencies. 10.7; 10.8; 27.4

Configuration, intended and installed. 13.2; 22.2; 25.6

Controlled degradation. 12.6; D.5

Controls and constraints. 3.2; 9.4

Cryptographic erasure and retained keys. 24.5; 24.6

Customer and supplier evidence exchange. 17.8; 18.6

Cyber-physical effects. 9.9; 12.9; D.1; D.4

Decision briefs and conditions. 20.6; 20.9; C.9

Defeaters and independent challenge. 19.7; 19.9

Design, implementation and enabling systems. 9.1; 13.1; 13.9

Digital thread and federated information. 11.8; 27.6

Drift and controlled evolution. 22.1; 22.3

Emergency maintenance. 22.6; 22.7

Evidence applicability and reuse. 18.6; 19.6; 22.5

Evidence, plans and performed results. 8.2; 19.4; C.7; C.8

Exit capability and information transfer. 23.8; 24.4

Fallback, time and state limits. 22.7; 28.11

Freshness versus revocation. 7.4; 11.11; C.5

Human factors and complete task timing. 16.2; 16.3; C.6

Human–AI Engineering functions. 28.3; 28.4

Idempotence and repeated transactions. 10.5; C.4

Independent review and evidence dependence. 19.6; 19.9

Inherited and COTS systems. 23.1; 23.2; 23.4

Integration and emergent behaviour. 14.1; 14.2

Interface contracts and semantic meaning. 10.1; 10.2; 17.8

Maintenance, migration and rollback. 22.4; 22.7; 24.4

MBSE, method, language and tools. 11.3; 11.9

Model credibility and implementation correspondence. 11.7; 11.14; C.8

Model exchange and source units. 11.8; 17.8; D.6

Monitoring and runtime protection. 21.3; 21.4

Normal operating envelope. 7.3; 11.12; C.2

Organizational integration and capability. 26.1; 26.2; 26.6

Positive capability and protective requirements. 5.1; 7.4; D.3

Procurement engineering-work obligations. 17.2; 17.4

Provenance and bills of materials. 18.7

Recovery authority and local roles. 12.3; C.6

Recovery readiness and operational tasks. 12.4; 16.3; 21.8

Requirements validation. 7.10; 8.10

Residual risk, current and intended. 20.3; 20.4

Resource contention and capacity. 25.4; 27.5

Retirement and continuing obligations. 24.2; 24.5; 24.8

Risk acceptance versus a passing result. 20.7; 20.11

Sanitization and domain-specific assurance. 24.6; 24.8

Separation of duties and effective influence. 9.6; C.6

Source records and scope of authority. 3.1; 19.4

Statistical trials, uncertainty and denominators. 15.6; 15.7; 25.5; 28.8

Stopping distance and physical parameters. D.3; D.4; D.8

Supplier substitutions and changing platforms. 18.8; 22.5

Systems-of-systems and local versus joint claims. 27.1; 27.3; 27.5

Technical-management work packages. 25.2; 25.3

Testing a logical model. 11.14; C.8

Time uncertainty and conservative bounds. 8.7; 15.6; C.5

Transaction history and mode epochs. 10.4; 10.5; 10.6; C.4

Trust allocation and containment. 9.6; 10.8; 27.7

Verification versus validation. 8.4; 8.6; 15.2; D.7

Work-system alternatives and AI evaluation. 28.8; 28.9

Writer coverage and privileged paths. 9.2; 17.4; 19.5; C.4

Information-flow protection. 5.8; Appendix E

Resilience under suspected compromise. 12.2; Appendix F

System context and trust boundaries. 2.2; Figure 2.1; 9.3

Causal analysis and branch coverage. 6.5; Figure 6.1; 19.7

Practitioner worksheets. Appendix G

Glossary

These are the book's concise teaching formulations, consolidated from Parts I–VII. They are not verbatim standards definitions or substitutes for controlling definitions in an applicable publication, contract or project. Case-specific terms retain their stated scope and proposed or hypothetical status. Repeated entries have been reconciled without treating distinct concepts—such as allocation and allocated requirement, or semantic exchange and its check—as synonyms.

A

Acquirer contribution. Information, services, facilities, personnel or decisions the acquiring organization must provide for the agreed work and resulting capability. Specifying supplier obligations does not establish that these customer contributions already exist. [1, Appendix K.1]

Acquisition package. The connected requirements, work obligations, information deliverables, acceptance provisions and customer contributions used to obtain a defined capability or development result. AP-05 is the book's teaching arrangement, not a prescribed contract format.

Adversity. Relevant adverse events or conditions against which useful capability and protection are evaluated. In the example these include deliberate misuse, incorrect actions, dependency loss and interrupted transactions. The set must be selected for the actual system and purpose. [1, Chapter 3; 2, Chapter 1]

AI-assisted engineering. Use of an AI contribution to help perform engineering work or produce engineering information, such as candidate requirements, analysis or an evidence summary. The AI's output remains distinguishable from actual system observations, performed evaluations and accountable decisions. See Sections 28.1 and 28.3.

AI-enabled system. A system in which an AI contribution participates in the delivered capability. Its required function, interfaces, action authority, operating conditions and evidence must be established as part of the whole system. A model-level result does not automatically establish the receiving-system outcome. See Section 28.1.

Allocated requirement. A requirement assigning a necessary contribution to an element, interface or participant while retaining its relationship to the system-level result. Allocation establishes an obligation or proposed contribution; it does not demonstrate successful composition. [1, Appendix H.3; 4, Sections 4.2–4.3]

Allocation. Assignment of a requirement, function or responsibility to one or more elements or participants, with the relationships needed to support the system-level result. Allocation does not by itself establish that the assigned contribution has been implemented. [4, Section 4.2]

Architecture. The significant structures, relationships and organizing principles of a system. An architecture description represents selected aspects for communication and analysis; that representation can be incomplete or out of date. See also Architecture description. [4, Sections 4.3–4.4]

Architecture decision record. A controlled account of a design issue, alternatives, direction, rationale, consequences and reconsideration conditions. A decision to investigate or prototype does not imply production selection or authorization. [4, Section 4.4; 6, Section 6.8]

Architecture description. A representation of an architecture organized to address relevant concerns. Multiple views can expose functional, behavioural, information, deployment and operational

relationships; they must remain mutually consistent. The book's selected views are original teaching arrangements. [19, public scope]

As-built configuration. The identified composition and settings of the realized item, distinguished from intended design and later installed state. Differences require explanation, not automatic rewriting of design intent. [6, Section 6.5]

Asset. Something valued by stakeholders or required by the system to provide its capability. Its significance can follow from the consequence of losing or misusing it, rather than its purchase or replacement cost. [1, Sections 3.4–3.7]

Assumption. A proposition used as a basis for reasoning that requires explicit treatment of its status, significance and continued applicability. The management response depends on what relies on it. [6, Section 6.8]

Assurance. The basis for justified confidence in specified claims. It connects relevant evidence and reasoning to the context, assumptions and limits of those claims. [1, Appendix F.2]

Assurance argument. Reasoning that connects evidence and supporting propositions to a specified claim, including context, assumptions and limitations. An argument can support a limited or negative conclusion; it need not advocate proceeding. [1, Appendix F.2]

Assurance case. A structured, reviewable account of claims, supporting reasoning, evidence and relevant context concerning a system or contribution. Its quality depends on the adequacy of the reasoning and support, not only completeness of its structure. It can identify insufficient support rather than necessarily conclude that a desired claim is justified. [1, Appendix F.2–F.2.3; 38, public scope]

Authoritative outcome, in the case. The established result of a particular activation transaction, distinguished from receipt or queuing. TE-03 proposes separating the transaction's committed package and generation from the target's current package and generation, which may have changed later.

Authority boundary. The extent of a participant's permitted decision or action under the governing arrangements. Technical expertise, ownership of a component and authority to accept operational consequences are different matters.

Authority record versus usable authority. A historical record can explain who was entitled to perform an action without retaining a credential that permits a new action. Retirement and archival arrangements should distinguish those different functions. See Section 24.5.

Autonomy, working use. The discretion and action a system can exercise within its assigned role. Autonomy is distinct from whether the system uses a learned model, generates content or updates its parameters during use. See Section 28.2.

B

Baseline. An identified, agreed reference configuration or set of information under change control, against which work, differences, evidence and decisions can be understood. Requirements, design, realization and installed configurations can have distinct baselines and controlled relationships. [6, Section 6.5; 43, Section 3.2]

Binding condition. An obligation or necessary feasibility condition that an alternative must satisfy before preferences can determine selection. An unresolved condition is not a pass. Changing it requires the appropriate source or decision authority, not a compensating score.

C

Capability charter. An account of the organizational support provided to projects and services, including users, scope, responsibilities, resources, access and limits. CAP-07 illustrates one such arrangement without prescribing an organizational chart. See Section 26.2.

Change-impact analysis. Examination of how a proposed or observed change affects requirements, interfaces, assumptions, useful capability, evidence and decisions. Its result determines necessary work; a trace query assists but does not decide the disposition. See Section 22.5.

Commit coordinator, in the case. A proposed receiving-side responsibility that coordinates authority, package, target, generation, mode and transaction checks at effective configuration change. It is a logical design contribution, not a prescribed product or a claim of a verified implementation.

Commit, in the teaching case. The event at which the authoritative management interface designates the requested package as active for its target. It differs from receiving, staging or queuing a request. Its meaning at the receiving operational interface must be established separately.

Competence evidence. Information supporting a person's ability to perform a specified task in a relevant context and at an identified level of responsibility. Education, credentials and experience can contribute, but their scope and the actual task remain important. COMP-07 is the book's teaching arrangement. [54, public overview; Section 26.4]

Completion branch. A separately identified continuation that supplies explicit assumptions, design decisions and evidence status for a bounded worked example. CB-09 does not rewrite the base case’s earlier findings or approvals. See Appendix C.

Compliance. Satisfaction of applicable obligations within a defined scope. The conclusion depends on what was required and assessed; it does not automatically establish every property of a system. [8–10]

Composition. The system behaviour produced by interacting contributions. An argument for a system property must address those interactions and relevant common conditions, not merely collect acceptable component claims. [1, Appendix E.7]

Conditional independence. In the Chapter 11 model, independence of the gateways' local unavailability events given that the shared service is available. It is an explicit mathematical assumption, not a property inferred from having two gateways.

Constituent system. A participating system that contributes to a wider capability while retaining relevant operational and managerial identity. Not every connected component is therefore a constituent system in the sense used in Chapter 27. [53, pp. 1–4]

Constraint. A restriction on permitted design choices, behaviour or use. It may be imposed by an external obligation or established through engineering and stakeholder decisions; changing it requires attention to its source and authority. [4, Section 4.2]

Continuous monitoring. An ongoing approach to obtaining security-relevant information at frequencies appropriate to its purpose. It is not necessarily real-time observation of every property, and does not replace a specified runtime protective mechanism. [41, Executive Summary and Section 3.2.2]

Contribution agreement. The defined function, interface meaning, conditions, evidence and continuing responsibilities supplied across a participating-system boundary. CA-07 is an engineering record; the binding commitment depends on actual agreements and authority. See Section 27.3.

Contribution-centred review. The book's method of examining a supplier or service through the functions, lifecycle influence and evidence on which the receiving system relies. It directs review depth according to consequence and relevant uncertainty rather than supplier size alone.

Control. A safeguard or countermeasure used to address security or privacy concerns. It can be technical, physical or procedural; a selected control statement can become an applicable requirement. [8]

Controlled degradation. A defined reduction or alteration of capability under specified adverse conditions, with explicit protection, information and transition obligations. The case's proposed modes describe the retained functions; they do not create permission to bypass source obligations.

COTS, descriptive use in Chapter 23. Commercial off-the-shelf products supplied for use as contributions to systems. No jurisdictional procurement definition or exemption is asserted. Configuration, modification and receiving-system integration still require explicit treatment. See Section 23.1.

Coverage. The extent to which specified requirements, relevant scenarios, paths, states and conditions are addressed by the selected analysis or evaluation. Coverage claims require an identified model of what is to be covered and a reasoned treatment of exclusions. [1, Appendix F.2; 5, Sections 5.3–5.4]

Cryptographic erase. A sanitization approach relying on treatment of the key material needed to recover appropriately encrypted information, subject to the method's necessary conditions. It is not equivalent to disabling an account or deleting one key reference. [47, Section 3.2]

Current residual position. The presently assessed exposure or uncertainty after measures actually in place, distinguished from the intended position after planned treatment. The specific risk method and rating require the relevant organization's definitions and evidence.

Cutover. Controlled change to a new receiving or operational arrangement, including relevant software, state, interfaces, authority and support. Reverting executable content is not automatically a valid rollback of persistent state. [5, Section 5.5]

Cyber resilience. The ability, in relation to cyber-dependent capability, to anticipate, withstand, recover from and adapt to adversity. A useful claim specifies the capability, conditions, consequences and limits concerned. [2, Section 2.1.1]

D

Decision baseline. The identified combination of requirements, profiles, models, realized configurations, evidence and conditions used for a particular decision. A collection of links to the latest records does not necessarily preserve it. See Section 25.6.

Defeater. A reason capable of undermining an inference or support for a claim. It may challenge an assumption, identify an alternative explanation or provide contrary evidence. A possible defeater is not automatically a demonstrated product defect.

Deliverable acceptance. A decision about whether an identified delivery meets its agreed acceptance provisions. Acceptance of a satisfactory analysis can coexist with rejection of the design that the analysis found inadequate. The actual contractual meaning depends on the governing agreement.

Dependency. A relationship in which a required function, property or conclusion relies on another element, service, resource or condition. Ownership of the dependency does not determine its engineering relevance. [1, Section 2.1.2]

Dependency condition. A relevant state or behaviour of a contribution on which a system function or claim relies. It can affect the result even when the dependent service is outside the selected system boundary or contractual responsibility.

Digital thread. In this book, maintained relationships among controlled engineering records across life-cycle states and repositories. It is not a claim that every record is complete, synchronized or an observation of the deployed system.

Drift, working book usage. A developing mismatch in the relevant system definition, realized or operating arrangement, context or basis for reliance. The Chapter 22 lenses are author-defined investigative categories, not physical laws or a universal taxonomy of failures.

E

Emergence. Behaviour or a property arising from relationships among system elements, rather than from an element considered alone. Emergence can be intended and useful as well as unintended or harmful. [3, introductory discussion]

Enabling system. A system providing services needed to create, evaluate, deploy, support or retire the selected system of interest. Development, production, training and maintenance arrangements are examples. Modelling repositories, build environments, production fixtures and test rigs can affect both the product and its evidence. [1, Section 2.1.2 and Appendices H.7–H.11]

Engineering margin. Room between a proposed or established contribution and a limiting criterion, under stated assumptions. The fifteen-second timing margin in Chapter 7 is an illustrative allocation result, not demonstrated system performance. [6, Section 6.8; original case arithmetic]

Evaluation scope. The proposition, item or representation, configuration, method and conditions to which a result applies. Component, model and task results do not silently establish broader system claims. [5, Sections 5.3–5.4]

Evidence. Information used to support or challenge a claim. Its value depends on provenance, applicability, method, coverage, conditions and limitations, not merely on its format. [1, Appendix F; 5, Sections 5.3–5.4]

Evidence applicability. The justified relationship between an evidence item's identity, configuration, criteria, method, conditions and limitations and the claim or conclusion for which it is used. Reuse needs justification where the receiving context differs; a matching product name is insufficient. A change may limit one conclusion while leaving other evidence useful. [1, Appendix F.2; Sections 18.6, 21.9 and 22.5]

Evidence dependence. Shared inputs, observations, tools or assumptions that limit the independence of apparently separate evidence items. Three summaries of the same incomplete export do not constitute three independent observations of the system.

Executable teaching abstraction. A runnable representation of selected logical relationships with stated omissions and assumptions. Its test results concern that representation, not an operational implementation. See Section C.8.

Exit capability. The ability to leave a supplied or hosted arrangement while obtaining the required receiving capability, information, authority and support, and addressing continuing obligations. Exporting bytes is one possible contribution, not the complete task. See Section 23.8.

F

Fallback decision point. The latest time or state at which a specified fallback remains technically valid and can be completed within the applicable constraints. Time remaining and reversibility of state are distinct conditions. See Section 22.7.

Federated engineering information. Controlled information maintained across several responsible repositories or organizations, with identified authoritative sources and relationships. Federation does not imply that every participant exposes its complete internal model. See Sections 26.6 and 27.6.

Fencing, in the transfer example. Preventing a formerly active writer from making further effective changes after a defined transition. The word names an intended control effect; it does not prescribe or verify a particular implementation. See Section 24.4.

Flow-down. Allocation of relevant obligations through actual supplier agreements according to each party's contribution, with the information needed to perform and evaluate that contribution. The receiving-system integration responsibility remains explicit. [32, Appendix A, control SR-3(3)]

Function allocation. Assignment of necessary contributions to elements, people or their interaction, accounting for information, resources and limitations. Performing a function does not itself confer decision authority. [33, Section 5.2.2.2]

G

Generation, in the case. A proposed identity for an activation state, distinct from the package content identifier. Restoring an earlier package creates a new activation context rather than making the earlier context current again. Persistence and recovery of generation information require design and evidence.

H

Human–AI Engineering arrangement. The author's proposed organization of Frame, Ground, Allocate, Challenge and Assure as interacting functions, with learning feeding all five. It is a proportionate teaching synthesis, not a statistically validated claim of superiority over simpler workflows. See Section 28.3.

Hypothetical finding. A stipulated observation used to teach interpretation or disposition, not a report of an actual experiment or incident. The D-04 examples have this status.

I

Idempotence. A property concerning the effect of repeated operations. For an HTTP method, RFC 9110 defines it in terms of the intended server effect; the case separately requires no more than one commit for an identified transaction and a dependable outcome. Network delivery count and application effects are different questions. [20, Section 9.2.2]

Inconclusive result. Evidence insufficient to establish satisfaction or violation of the particular criterion. It differs from failure and from an evaluation that was blocked or not performed. See Section 15.6.

Inherited-system reconstruction. Establishing enough of an existing system's functions, interfaces, configuration, dependencies and evidence to support a stated decision. It need not imply complete recovery of all internal design information. LEG-06 is a separate hypothetical vignette.

Integration increment. A planned combination of contributions selected to establish capability or resolve an interaction question. INT-01–INT-05 are teaching arrangements, not mandatory stages. [5, Section 5.2]

Intended residual position. A proposed state of exposure after planned treatment has been implemented and its effectiveness evaluated. It is not the current state merely because a plan is approved or funded.

Interface. A relationship or boundary through which relevant exchanges or interactions occur. Its definition may include physical, informational, behavioural, timing and human aspects. [6, Section 6.3]

Interface contract. The specified meaning and obligations of an interaction, including inputs, outputs, checked conditions, state, authority, timing and adverse behaviour. A schema is one possible part of the contract, not the entire behavioural agreement. [6, Section 6.3; original IC-01]

Invariant. A property intended to hold across the relevant states and transitions of a model or system. The limited induction argument in Section 11.14 establishes an abstract property only under its stated transition and history assumptions.

L

Life-cycle stage, process and project. A stage organizes a period of the system's life around a purpose or maturity. A process organizes activities that produce outcomes. A project is a bounded effort that contributes to the system or its environment. [12, public scope; 3, Section 2.1]

Loss scenario. A structured account of how events, states, interactions and conditions can lead to a stakeholder loss. In this book the selected scenario record is an author-created teaching aid, not a substitute for a prescribed risk or safety method. [15, Sections 2.3 and 3.2, for related risk concepts]

M

MBSE. See Model-based systems engineering (MBSE).

Measurement uncertainty. A quantified account of the limitation in a measurement result's relation to the quantity being evaluated. The conservative bounds in Chapter 8 are invented assumptions illustrating a decision rule, not confidence intervals obtained from experiments.

Metamodel. A description of the kinds of model elements and relationships used to organize a representation. MS-01 uses an original, limited information agreement rather than prescribing a new systems modelling language. [23, Section 7]

Mission. See Mission and mission assurance.

Mission and mission assurance. A mission is the operational, business or other purpose being served. Mission assurance concerns protecting and sustaining the relevant mission-essential capabilities and assets. The security engineering contribution is bounded by the capabilities, dependencies and conditions actually considered. [2, Chapter 1 and Appendix A, "mission assurance"]

Mission assurance. See Mission and mission assurance.

Mission Protection Needs method. The book's teaching arrangement for connecting purpose, assets, losses, protection needs and subsequent engineering questions. Equivalent reasoning can be performed within established mission, stakeholder and requirements processes. It is not presented as a standardized or exclusive method.

Mode epoch, in the case. A proposed persistent identifier for a particular authority context. Returning to a mode named NORMAL does not reactivate grants or queued work belonging to an earlier NORMAL epoch.

Mode, state and transition. A mode is an intended operating arrangement; a state is a relevant condition at a point in time; a transition is movement between conditions. The model should distinguish them sufficiently to specify and evaluate permitted behaviour. [1, Appendix F.1.2]

Model configuration. The identified combination of model content, referenced records, selected alternatives, parameters and relevant tool/library versions used for a purpose. A repository version is not the same event as the administration case's activation commit.

Model credibility. The justified basis for relying on a model and its results for a specified purpose, considering its representation, inputs, evaluation and limits. An internal-consistency check alone does not establish that basis. [30, public scope]

Model view. A selected presentation of modelling information for particular concerns. Its content, filters and configuration must be understood before drawing a conclusion from what is visible. [19, public scope; original application]

Model, context distinction. An MBSE model represents selected engineering concepts and relationships. A learned AI model supplies a computational contribution developed through training. One may describe or help develop the other, but their evidence and validation questions differ. See Section 28.2.

Model-based systems engineering (MBSE). Formalized use of modelling to support requirements, design, analysis, verification and validation across the life cycle. It is an approach to engineering, not a synonym for a tool, notation or simulation. [22, Mission]

Monitoring-chain health. The availability and adequacy of the sources, collection, timing, transport, processing and presentation needed for an observation to support its intended use. A silent failed channel does not establish healthy system behaviour. See Sections 21.3–21.4.

O

Operational readiness condition. A maintained technical, human, support or authority condition necessary for the permitted task. Its failure can limit readiness without changing the application binary. See Sections 21.2 and 21.8.

Operational thread. A coherent account of an activity and the contributions, decisions, exchanges and actions needed for its outcome. SOS-07 uses a repair-coordination thread to expose consequential interfaces and shared resources. See Section 27.2.

Organizational capability. The maintained ability to supply the people, methods, infrastructure, information and decision support needed for work across projects and through change. It does not by itself establish the trustworthiness of every delivered system. [1, Appendix J; Section 26.1]

P

Prepared restoration capsule, in the case. An author-defined label for a protected, prepared instruction binding restoration content, target, eligibility and bounded authority information. It is neither a standardized security object nor an approved mechanism. Permitted authentication and current eligibility remain necessary at use.

Profile. In the teaching case, a controlled collection of definitions or parameters referenced by requirements. A profile change can alter requirement meaning even when the requirement's identifier and sentence remain unchanged.

Protection need. A stakeholder need concerning the protection of assets or capability against unacceptable loss. It informs requirements but is not automatically a complete requirement or a selected mechanism. [1, Section 3.7]

Protective stopping distance. In CP-09, the distance required by the stated speed, delay, deceleration and spatial allowances. The equation's applicability depends on the physical conditions represented; it is not a universal vehicle clearance. See Section D.4.

Provenance. Information about a contribution's origin and relevant history through development, production, transformation, delivery or change. Provenance supports particular origin and integrity claims; it does not establish every property of the delivered system. [32, Appendix A, control SR-4]

Q

Quality assurance. The planned examination of applicable processes and work products against agreed quality expectations, with findings and improvement. It is distinct from the claim–argument–evidence structure of a system assurance case, although it can support confidence in that case's inputs. [1, Appendix I.8]

R

Realization manifest. An identified account linking requirements, design, realized items, configuration, tools and records. RZ-01 proposes its information content; it is not an actual release manifest.

Recovery readiness. The availability and applicability of the people, authority, credentials, packages, interfaces and support needed to perform the required restoration under its adverse conditions. A successful earlier demonstration does not establish that all those contributions remain ready.

Representation fidelity. How adequately a model or evaluation arrangement represents characteristics relevant to its intended use. Detail alone does not establish credibility; significant omissions must be considered. [30, public scope]

Requirement. A statement of a necessary capability, characteristic, condition or constraint on a specified subject. Useful requirements need clear meaning, rationale, allocation and a means of evaluation appropriate to their level. [4, Section 4.2; 13]

Requirement validation. Examination of whether the requirements are an adequate, coherent, feasible and evaluable expression of the relevant stakeholder needs and obligations. It differs from verification that an implementation satisfies those requirements. [4, Section 4.2; 18, Appendix C]

Residual risk. See Risk and residual risk.

Resource contention. Competition for a contribution that cannot satisfy all demands simultaneously under the assumed conditions. Local feasibility does not establish joint feasibility where tasks share crews, equipment, approval personnel or other resources. See Section 27.5.

Retirement boundary. The specified use, functions, assets and authority being withdrawn, distinguished from what remains in service or under continuing obligations. A bounded completion statement identifies both completed actions and remaining responsibilities. See Sections 24.2 and 24.8.

Retrieval collection. The set of documents or records made available for selection into an AI-supported task. Its scope, version, permissions and selection process can affect the output even when the learned model remains unchanged. See Sections 28.2 and 28.4.

Risk. See Risk and residual risk.

Risk and residual risk. Risk concerns uncertainty and potential adverse effects relevant to objectives and decisions. Current residual risk is what remains after the responses actually in place, distinguished from the intended position after planned treatment. An accountable decision to accept a position does not itself prove those responses effective. [6, Section 6.4; 10; Section 20.4]

Risk disposition. An accountable decision about treatment or retention of an identified risk under the applicable authority, scope and conditions. It does not alter the truth of an assessment result or grant powers the decision-maker does not possess. [10, Section 3.6]

S

Sanitization verification and validation. Within NIST SP 800-88 Revision 2, checks on sanitization outcome and the decision about its adequacy for disposition. These domain-specific activities are not synonymous with every system-level verification or intended-use validation in the book. [47, Sections 4.5.1–4.5.2]

Satisfaction assertion. A statement that a proposed or represented contribution satisfies a requirement. Its encoding in a model is not independent proof that a delivered implementation satisfies the requirement. [25, Section 7.21]

Scenario hypothesis. A proposed causal explanation to investigate. Recording the hypothesis does not establish that an implementation defect exists, that a particular event is likely or that a proposed response is effective.

Scope exclusion. An identified condition or contribution not included in a claim or evaluation. It needs a defensible relationship to the remaining claim; excluding an unexamined path does not establish that all relevant paths are controlled.

Security. In this book, protection of assets and useful capability against unacceptable loss, considered in the system's actual context and across relevant forms of adversity. It is not limited to information confidentiality or to externally initiated attacks. [1, Sections 3.1 and 3.4–3.7]

Semantic exchange. Transfer preserving relevant engineering meaning, including relationships, units, conditions, alternatives and evidence status. Successful file import alone does not establish preservation. See Sections 11.8 and 13.9.

Semantic exchange check. A project-defined comparison of whether exchanged modelling information preserves the meanings needed for the receiving task, including conditions, units, identities, relationships and status. It is not established solely by successful file import.

Sensitivity analysis. Examination of how a model's result or an alternative's ranking changes when inputs, assumptions or preferences change. In Chapter 11 it exposes the importance of uncertain task time and declared value weights. [6, Section 6.8]

Source consistency bound. In CB-09, an explicit upper limit on how long newly issued status observations may fail to reflect an authoritative revocation. It is different from the age of the observation and from its authenticity. See Sections C.2 and C.5.

Stakeholder requirement. A requirement expressing a needed capability or condition from the stakeholder perspective, providing a basis for the technical requirements of the selected system. It retains purpose and context rather than merely naming a preferred product. [1, Appendix C.2 and Appendix H.2]

State-aware recovery. Restoring usable capability while preserving or reconciling the configuration, transaction history, authority and other state needed for the required properties. Reinstalling an earlier executable is not necessarily a valid recovery of the whole arrangement. See Sections 21.7 and 22.7.

Stipulated system evidence. Invented observations or review information supplied as premises of a fictional teaching dossier. It must remain distinguishable from evidence actually produced by executing a test or model. See Section C.7.

Supplier organizational assurance. Support concerning the supplier entity, activities and criteria within an identified organizational assessment. It can be relevant to product development and support but is not automatically delivered-product or receiving-system assurance. [11, public scope; 39, p. 1, Overview]

System of interest. The system selected as the focus of an engineering question. Its boundary and context determine the scope of analysis without erasing relevant external dependencies. [1, Section 2.1]

Systems engineering. The integrating engineering effort that develops and sustains a coherent system by coordinating technical contributions, requirements, alternatives, realization and evaluation across its life cycle. [3, introductory discussion]

Systems Security Engineering. The security specialty within systems engineering, contributing protection-needs analysis, requirements, design reasoning, evaluation and support for justified reliance throughout the system life cycle. [1, Section 3.10]

T

Task analysis. Examination of the actions, information, people, resources and conditions needed for an activity. The restoration case includes waiting, authority, access and confirmation—not only command execution. [33, Section 5.2.2.4]

Technical-management record. Information connecting planned technical work, resources, achieved results, discrepancies and decisions. TM-07 is an original case application of technical-management purposes, not an additional mandatory document. [1, Appendix I; Chapter 25]

Temporary workaround. A bounded, controlled response pending correction or another decision. It needs a purpose, owner, permitted scope, observation and reconsideration or ending condition; repeated use may reveal a changed operating concept. See Sections 21.8 and 22.6.

Test article. The item or identified representation evaluated. Its configuration and differences from the intended operational system determine which conclusions can be transferred. [5, Sections 5.3–5.4]

Test oracle. The justified rule or reference used to decide whether an observation meets a criterion. The teaching example requires a dependable observation of authoritative configuration and commit, not merely an unexamined success message; a client timeout alone cannot establish absence of a receiving-side commit. This is a general testing term applied through original case reasoning. See Sections 8.7 and 15.3.

Traceability. Controlled relationships connecting sources, needs, requirements, design contributions, scenarios, evidence and decisions. A reference establishes a link to be explained; it does not itself establish adequacy or equivalence. [1, Appendices H.2–H.3; 18, Appendices C–E]

Transaction identity lifetime, in the case. The period and namespace conditions during which an identifier can be accepted for execution or queried for its outcome. Suppression, retention, retirement and restoration must preserve the requirements attached to that identity; a short cache lifetime does not silently narrow SR-07.

Transition readiness. Adequate support for a specified commitment in an identified receiving context, including technical, human, support and authority conditions. TR-04 organizes these questions without declaring readiness. [1, Appendix H.10]

Trust. See Trust and trustworthiness.

Trust and trustworthiness. Trust is reliance that an entity will meet an expectation; trustworthiness concerns whether that reliance is merited. Evidence can support a trustworthiness judgement, while an actual reliance decision can still be made with incomplete support. [1, Appendix F.1]

Trustworthiness. See Trust and trustworthiness.

V

Validation. See Validation and verification.

Validation and verification. Validation evaluates suitability for stakeholder needs and intended use; verification evaluates satisfaction of specified requirements. Both need appropriate methods, conditions and evidence, and can occur at different levels of maturity and system structure. [5, Sections 5.3–5.4]

Value function. An explicit representation of preferences over a defined attribute range. Chapter 11's linear cost and time functions are invented for a small illustrative trade; their outputs are not security measurements or probabilities.

Verification. See Validation and verification.

W

Work package. A bounded assignment with defined inputs, participants, outputs and a completion criterion appropriate to its purpose and maturity. Completion of a competent investigation does not necessarily establish that its investigated design is acceptable. See Section 25.3.

Information-flow protection. Engineering the permitted movement, transformation, use, retention and disposition of information in relation to a defined function and audience. Cryptographic protection can contribute to a flow without determining whether the content or recipient is permitted. See Appendix E.

Suspect source. A source whose availability may continue while its truthfulness, integrity, authority or provenance is not a justified basis for the claim being made. Treatment depends on what relies on the source and which independent evidence or capability remains. See Appendix F.

References

References [1]–[55] retain their identities from the consolidated draft; [56]–[58] support the explicit authentication and transport choices in Appendix C. A 9 October 2026 source-record pass checked identities, editions and accessible status information, with deeper inspection of selected technical passages. It was not a complete line-by-line attribution or rights audit. The companion source register records the level of access and unresolved limits for each entry.

The book's explanations, tables, case records, scenarios, calculations and exercises are original teaching applications unless explicitly identified otherwise. A cited publication does not automatically govern the reader's project. Public scope or abstract-only access is not a complete review of licensed or unavailable technical text, and is not evidence of conformity, product evaluation or institutional endorsement.

[1] Ross, R., Winstead, M., and McEvilley, M. Engineering Trustworthy Secure Systems. NIST Special Publication 800-160, Volume 1, Revision 1, November 2022. DOI: 10.6028/NIST.SP.800-160v1r1. Locations used across the book: Sections 2.1, 3.1, 3.4–3.7 and 3.10; Appendix C.2; D.3–D.4.2; E.7, E.12, E.16, E.19–E.20 and E.24; Appendix F, particularly F.1–F.2.3; and the process material in Appendices G–K. The chapter citations identify the narrower passage used, including H.1–H.3 and H.7–H.14, I.1 and I.5–I.8, J, and K.1–K.2. Source use comprises selected concepts, assurance relationships and process passages, not a claim that every passage was audited. The process framework contains attributed standards material; the book supplies original explanations rather than reproducing normative blocks or asserting conformity.

[2] Ross, R., Pillitteri, V., Graubart, R., Bodeau, D., and McQuaid, R. Developing Cyber-Resilient Systems: A Systems Security Engineering Approach. NIST Special Publication 800-160, Volume 2, Revision 1, December 2021. DOI: 10.6028/NIST.SP.800-160v2r1. Locations used: Chapter 1; Sections 2.1.1–2.1.3, cyber-resiliency goals, objectives and techniques; Sections 3.1.4–3.1.5, conflicts, synergies and existing investments; and Appendix A, entry "mission assurance". Used for resilience framing and selective technique consideration. The case's modes, recovery arrangements, credentials and numerical targets are original teaching material, not prescribed by this source.

[3] National Aeronautics and Space Administration. NASA Systems Engineering Handbook, online edition, Section 2.0, "Fundamentals of Systems Engineering." Principal locators used: introductory discussion and Section 2.1. Public source locator: nasa.gov/reference/2-0-fundamentals-of-systems-engineering/. NASA-specific organizational arrangements are not adopted as universal requirements.

[4] National Aeronautics and Space Administration. NASA Systems Engineering Handbook, online edition, Section 4.0, "System Design Processes." Locations used: Sections 4.1–4.4, including 4.1.1.2 and 4.2.1.2, stakeholder expectations, technical requirements, logical decomposition and design-solution definition. Public source: nasa.gov/reference/4-0-system-design-processes/. Used for iterative requirements and alternative-design reasoning. NASA-specific governance, work-product conventions and templates are not imposed as universal requirements.

[5] National Aeronautics and Space Administration. NASA Systems Engineering Handbook, online edition, Section 5.0, "Product Realization." Locations used: Sections 5.1–5.5, implementation, integration, verification, validation and transition. Public source: nasa.gov/reference/5-0-product-realization/. Used for process distinctions, realization strategies, evaluation purpose and maturity, and preparation of the receiving context. The worked strategies, plans and procedures are original, not NASA templates or assertions of NASA compliance.

[6] National Aeronautics and Space Administration. NASA Systems Engineering Handbook, online edition, Section 6.0, "Crosscutting Technical Management." Locations used: the introductory framing and Sections 6.1, Technical Planning; 6.3, Interface Management; 6.4, Technical Risk Management; 6.5, Configuration Management; 6.6, Technical Data Management; 6.7, Technical Assessment; and 6.8, Decision Analysis. Public source: nasa.gov/reference/6-0-crosscutting-technical-management/. Used for planning, resources, interfaces,

configuration, information, technical measures, review, alternatives, criteria and uncertainty. The book's records, equations and numerical assumptions are original. NASA-specific organizational, contractual and decision authorities are not assigned to other projects.

[7] National Institute of Standards and Technology. The NIST Cybersecurity Framework (CSF) 2.0. NIST Cybersecurity White Paper 29, February 2024. DOI: 10.6028/NIST.CSWP.29. Scope description based on the official publication abstract: high-level outcomes without prescribing implementation.

[8] Joint Task Force. Security and Privacy Controls for Information Systems and Organizations. NIST Special Publication 800-53, Revision 5, September 2020, with the December 2020 update. DOI: 10.6028/NIST.SP.800-53r5. Scope and mapping cautions based on the official publication record, which also identifies Release 5.2.0 of August 2025. No individual control is quoted or assigned to the example as an external requirement.

[9] Joint Task Force. Assessing Security and Privacy Controls in Information Systems and Organizations. NIST Special Publication 800-53A, Revision 5, January 2022. DOI: 10.6028/NIST.SP.800-53Ar5. Official publication record and abstract used for lifecycle assessment and tailored assessment planning. The source records identify Release 5.2.0 of August 2025. No release-specific assessment procedure is reproduced or imposed on the fictional case.

[10] Joint Task Force. Risk Management Framework for Information Systems and Organizations: A System Life Cycle Approach for Security and Privacy. NIST Special Publication 800-37, Revision 2, December 2018. DOI: 10.6028/NIST.SP.800-37r2. Source use includes the official abstract and selected passages in Section 3.6, particularly Tasks R-3 and R-4; Section 3.7, particularly Task M-3; and Appendix F, authorization-related information. Used for lifecycle integration, accountable decisions, risk response, retained findings, authorization conditions and continuing review. Later chapters reuse those relationships rather than claim a fresh full audit. The source's United States governmental roles and powers are not assigned to the fictional case or readers in other jurisdictions.

[11] International Organization for Standardization and International Electrotechnical Commission. ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems — Requirements. Public ISO scope and overview: iso.org/standard/27001. Used to identify the subject of an information security management system and distinguish organizational assessment from product or receiving-system evidence. The complete licensed normative text was not reviewed or reproduced, and no certificate is evaluated by the book.

[12] ISO/IEC/IEEE. 15288:2023 — Systems and software engineering — System life cycle processes. Public ISO scope and publication record. Public source locator: iso.org/standard/81702.html. Used for the distinction between processes and life-cycle models and for iterative, concurrent and recursive applicability; not a clause-by-clause conformance claim.

[13] ISO/IEC/IEEE. 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering. Public ISO scope and publication record: iso.org/standard/72089.html. Used to identify the requirements-engineering processes and information-product subject area. This is not a review or reproduction of the complete licensed normative text or a conformance assessment. The candidate requirements and drafting examples are original teaching material.

[14] Souppaya, M., Scarfone, K., and Dodson, D. Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST Special Publication 800-218, February 2022. DOI: 10.6028/NIST.SP.800-218. Source use includes the official abstract and selected Table 1 practices PO.3, supporting toolchains; PS.2–PS.3, release integrity and protected release information; and PW.6–PW.8, build configuration, code analysis and executable testing. Used for secure-development integration and producer–acquirer communication. Applications to RZ-01 and MS-01 are original, not a complete SSDF assessment. Reference [37] retains the separately identified draft edition; it does not replace this practice basis.

[15] Joint Task Force Transformation Initiative. Guide for Conducting Risk Assessments. NIST Special Publication 800-30, Revision 1, September 2012. DOI: 10.6028/NIST.SP.800-30r1. Principal locators: Section 2.3, risk models and assessment approaches; Sections 3.1–3.4, preparation, conduct, communication and maintenance. Used for risk factors, assessment purpose, assumptions and complementary starting points. No particular risk matrix or numerical rating is imposed by the book.

[16] Leveson, N. G., and Thomas, J. P. STPA Handbook, March 2018, with the MIT Partnership for Systems Approaches to Safety and Security public handbook outline. Sources: psas.scripts.mit.edu/home/books-and-handbooks/ and psas.scripts.mit.edu/home/get_file.php?name=STPA_Handbook.pdf. The earlier Chapter 6 treatment used the outline. For Revision 9, selected Chapter 2 passages on control structures, unsafe control actions and loss scenarios, with the relevant figure page, were examined for Appendix D. The book does not reproduce the handbook figures, tables or method text, or claim that CP-09 is a complete STPA analysis. The handbook retains its own copyright and reuse conditions.

[17] National Institute of Standards and Technology. Using Business Impact Analysis to Inform Risk Prioritization and Response. NIST Interagency Report 8286D, February 2025 update. DOI: 10.6028/NIST.IR.8286D-upd1. Official abstract and publication record consulted for the broader use of business impact analysis across forms of loss and mission-essential assets. No detailed procedure from the report is reproduced.

[18] National Aeronautics and Space Administration. NASA Systems Engineering Handbook, online edition, "System Engineering Handbook: Appendix." Principal locators: Appendix C, requirements-writing guidance; Appendix D, requirements verification matrix; Appendix E, validation planning and matrix. Public source locator: nasa.gov/reference/system-engineering-handbook-appendix/. Used as public teaching guidance for clear requirements and distinct evaluation planning; the book's tables and wording are original.

[19] ISO/IEC/IEEE. 42010:2022 — Software, systems and enterprise — Architecture description. Public ISO scope and publication record. Public source locator: iso.org/standard/74393.html. Used only for the subject of architecture descriptions and views addressing concerns. This is not a review or reproduction of the complete licensed normative text, a prescribed set of viewpoints or a conformance assessment.

[20] Fielding, R., Nottingham, M., and Reschke, J. HTTP Semantics. Internet Engineering Task Force, RFC 9110, June 2022. DOI: 10.17487/RFC9110. Principal locators: Section 9.2.2, Idempotent Methods; Section 13.1.1, If-Match; Section 13.2.1, When to Evaluate; and Section 15.3.3, 202 Accepted. Public source locator: rfc-editor.org/rfc/rfc9110.html. Used for narrowly scoped protocol examples. HTTP is not selected as the case's protocol, and its semantics are not claimed to establish the full transaction or recovery properties.

[21] National Institute of Standards and Technology. Guide to Operational Technology (OT) Security. NIST Special Publication 800-82, Revision 3, September 2023. DOI: 10.6028/NIST.SP.800-82r3. Official publication record and abstract, used for the need to consider OT performance, reliability and safety characteristics. The inspection-vehicle vignette is original, is not a detailed application of this guide and is not a validated safety or control design. No detailed physical-system deployment procedure is derived from the abstract.

[22] International Council on Systems Engineering. "MBSE Initiative Working Group," Mission. Public source: incose.org/group/mbse-initiative/. Consulted 8 October 2026 for the scope of MBSE. The book's MS-01 workflow is an original teaching arrangement, not an INCOSE-prescribed method.

[23] National Aeronautics and Space Administration. NASA Systems Modeling Handbook for Systems Engineering. NASA-HDBK-1009A, 12 March 2025. Principal locators: Sections 5, Model Planning; 6, Setting Up the Model; 7, The Metamodel; and 9, Generating Diagrams and Tables from the Model to Support Systems Engineering Products. Public source: standards.nasa.gov/standard/NASA/NASA-HDBK-1009. Selected text and the relationship figures were examined; no source figure or companion model is reproduced. NASA-specific work-product conventions are not imposed on the case. The related use in Chapters 13–16 and 21–24 carries

forward this selected conceptual basis; it is not an additional complete handbook audit or an executed native-model exercise.

[24] Object Management Group. “What is SysML?”, sections “SysML v1” and “SysML Diagram Summary”; and the SysML 1.7 official specification record. Public sources: omg.org/sysml/what-is-sysml.htm and omg.org/spec/SysML/1.7. Consulted 8 October 2026 for a high-level language orientation. The complete v1.7 normative PDF was not reviewed for this update; no v1 syntax or model-conformance claim is made.

[25] Object Management Group. Systems Modeling Language (SysML), Version 2.0, Part 1: Language Specification. Document formal/2026-03-02, March 2026, and official specification record. Principal locators: Sections 6.1–6.2; 7.19–7.24, calculations, constraints, requirements and cases, particularly 7.21 and 7.24. Public source: omg.org/spec/SysML/2.0. The record lists formal adoption in September 2025. Selected concepts were consulted; the book does not provide executable SysML code, reproduce normative diagrams or assert complete conformance.

[26] Object Management Group. Systems Modeling Application Programming Interface (API) and Services, Version 1.0. Document formal/2026-03-04, March 2026. Principal locators: Clauses 1–2, scope and conformance; 7, platform-independent model and services; and 8, platform-specific mappings. Public source: omg.org/spec/SystemsModelingAPI/1.0. Consulted for narrowly scoped exchange and integration concepts, not as evidence that any named product implements every service. Chapter 17 reuses the scope and conformance relationship; no interoperability test or renewed full specification audit is claimed.

[27] Eclipse Capella project website. “Capella | Open Source MBSE Tool,” including the Arcadia-method and ecosystem descriptions. Public source: mbse-capella.org/. Consulted 8 October 2026 for the limited companion-note description. No installed product, extension, interoperability or usability assessment was performed.

[28] Dassault Systèmes. “Cameo Systems Modeler,” product overview and listed modelling, traceability and analysis capabilities. Public source: 3ds.com/products/catia/no-magic/cameo-systems-modeler. Consulted 8 October 2026. Source-described capabilities are not an independent performance comparison; the page identifies additional products for some functions.

[29] MathWorks. “System Composer,” product overview and architecture/behaviour integration descriptions. Public source: mathworks.com/products/system-composer.html. Consulted 8 October 2026. Used for the distinction and connection between architecture models and behavioural simulation; no product, licence configuration or model exchange was tested.

[30] National Aeronautics and Space Administration. Standard for Models and Simulations. NASA-STD-7009B, 5 March 2024. Public scope and publication record: standards.nasa.gov/standard/NASA/NASA-STD-7009. Used for model/simulation credibility and intended-use acceptance. Source use is limited to public scope and publication-record material, not a complete detailed qualification process or conformance assessment.

[31] Regenscheid, A. Platform Firmware Resiliency Guidelines. NIST Special Publication 800-193, May 2018. DOI: 10.6028/NIST.SP.800-193. Official abstract and publication record consulted for firmware protection, detection and secure recovery. The reset and state-history examples are original; the chapter does not assert a complete platform-firmware assessment.

[32] Boyens, J., Smith, A., Bartol, N., Winkler, K., Holbrook, A., and Fallon, M. Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations. NIST Special Publication 800-161, Revision 1, May 2022, including updates as of 1 November 2024. DOI: 10.6028/NIST.SP.800-161r1-upd1. Principal locators: overview and Appendix A, particularly controls SR-3(3), SR-4, SR-6 and SR-8. Selected text and control-table pages were examined for subtier contributions, provenance, assessment and notification. These NIST control identifiers are distinct from the book's SR requirement identifiers. No complete control assessment or source-table reproduction is claimed. The official abstract and record also inform the product/service visibility discussion.

Later notification and sustainment examples reuse these concepts; no additional supplier assessment is claimed.

[33] National Aeronautics and Space Administration. Human Systems Integration Handbook. NASA/SP-20210010952, 2021. Public record: ntrs.nasa.gov/citations/20210010952. Principal locators: Section 5.2.2.1, Concept of Operations; 5.2.2.2, Function Allocation; and 5.2.2.4, Task Analyses. Selected text used for integrating human contributions into engineering. The logistics roles, tasks, time allocations and exercises are original, not NASA requirements or results of a human-factors study.

[34] Taylor, B. N., and Kuyatt, C. E. Guidelines for Evaluating and Expressing the Uncertainty of NIST Measurement Results. NIST Technical Note 1297, 1994 edition. Principal locator: Section 6, Expanded Uncertainty, in the official online reproduction. Public source: nist.gov/pml/nist-technical-note-1297/nist-tn-1297-6-expanded-uncertainty. Used to distinguish uncertainty measures and coverage interpretation. This historical source is not represented as a newly issued standard; the book's conservative-bound examples are original and are not calibrated measurement results.

[35] Kuhn, D. R., Kacker, R. N., and Lei, Y. Practical Combinatorial Testing. NIST Special Publication 800-142, October 2010. DOI: 10.6028/NIST.SP.800-142. Official abstract and publication record consulted for t-way interaction testing, tool support and practical limits. No ACTS execution, generated test suite or exhaustive interaction claim is made for the case.

[36] NIST/SEMATECH. e-Handbook of Statistical Methods, Section 7.2.4.1, “Confidence intervals,” particularly “Exact Intervals for Small Numbers of Failures and/or Small Sample Sizes.” Public source: itl.nist.gov/div898/handbook/prc/section2/prc241.htm. Used for exact binomial-tail confidence-limit context. Section 15.7 derives the zero-failure one-sided special case explicitly; its numerical examples are not measurements or estimates for the logistics service or adversarial attack success.

[37] Booth, H., Ogata, M., Kent, K., Souppaya, M., and Dodson, D. Secure Software Development Framework (SSDF) Version 1.2: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST Special Publication 800-218, Revision 1, Initial Public Draft, 17 December 2025. Public record: csrc.nist.gov/pubs/sp/800/218/r1/ipd. Used only to identify the draft's publication status at the source check; it is not substituted for the published Version 1.1 practice basis in [14].

[38] ISO/IEC/IEEE. 15026-2:2022 — Systems and software engineering — Systems and software assurance — Part 2: Assurance case. Public ISO scope and publication record: iso.org/standard/80625.html. Used for the subject of assurance-case structure and terminology. The complete licensed requirements were not reviewed or reproduced; CASE-05 does not claim conformity to them.

[39] Cybersecurity and Infrastructure Security Agency and Federal Bureau of Investigation. Secure by Demand Guide: How Software Customers Can Drive a Secure Technology Ecosystem, August 2024. Full four-page official joint guide retrieved through the FBI Internet Crime Complaint Center on 9 October 2026: ic3.gov/CSA/2024/240806.pdf. Principal locator used: p. 1, Overview, distinguishing enterprise and product security and locating product-security questions across procurement. This replaces the earlier incomplete-retrieval note; the book does not reproduce the checklist or claim an executed supplier assessment.

[40] Boyens, J., McWhite, R., and Calloway, L. NIST Cybersecurity Supply Chain Risk Management: Due Diligence Assessment Quick-Start Guide. NIST Special Publication 1326, July 2026. DOI: 10.6028/NIST.SP.1326. Public source: csrc.nist.gov/pubs/sp/1326/final. The official publication record, introductory material and assessment-dimension outline were consulted, including foreign ownership, control or influence; provenance; resilience; foundational cyber practices; and supply-chain tiers. The book’s contribution-centred review is original and is not a reproduced assessment procedure or a supplier-acceptability determination under this guide.

[41] National Institute of Standards and Technology. Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations. NIST Special Publication 800-137, September 2011. DOI: 10.6028/NIST.SP.800-137. Principal locators: Executive Summary, including the explanation of ongoing and continuous; Section 3.2.2, monitoring and assessment frequencies. Used for purposeful, frequency-appropriate and event-driven monitoring, not as a prescribed monitoring architecture or interval for the case.

[42] Nelson, A., Rekhi, S., Souppaya, M., and Scarfone, K. Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile. NIST Special Publication 800-61, Revision 3, April 2025. DOI: 10.6028/NIST.SP.800-61r3. Principal locators: Sections 2.1–2.2, response within wider risk management and roles. Selected text and the relevant lifecycle figure were examined; the source figure and profile tables are not reproduced. The case's operational and engineering interactions are original applications.

[43] National Institute of Standards and Technology. Guide for Security-Focused Configuration Management of Information Systems. NIST Special Publication 800-128, August 2011, including updates as of 10 October 2019. DOI: 10.6028/NIST.SP.800-128. Principal locators: Section 2, fundamental concepts, and Sections 3.2–3.4, baseline implementation, change control and monitoring. Selected passages and the official scope support the integration of security with configuration management. CR-06 is not a prescribed NIST form or a completed assessment.

[44] Souppaya, M., and Scarfone, K. Guide to Enterprise Patch Management Planning: Preventive Maintenance for Technology. NIST Special Publication 800-40, Revision 4, April 2022. DOI: 10.6028/NIST.SP.800-40r4. Principal locators: Sections 3.3–3.5, maintenance planning, scenarios and maintenance groups. Used for the preventive-maintenance framing and differentiated patching situations, not to prescribe a universal deployment deadline or reduced evaluation scope.

[45] Mell, P., and Grance, T. The NIST Definition of Cloud Computing. NIST Special Publication 800-145, September 2011. DOI: 10.6028/NIST.SP.800-145. Principal locator: Section 2, Service Models. Used for the distinction among infrastructure, platform and software services and corresponding consumer control. The historical taxonomy is not a current product specification, contractual allocation or universal responsibility matrix.

[46] Microsoft. “Shared responsibility in the cloud.” Microsoft Learn, Azure security fundamentals; page updated 24 August 2026. Public source: learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility. The introduction, responsibility discussion and explicit distinction from a legal agreement were consulted. This provider's explanation is an illustrative scoped source, not an independent product assessment or a contract governing LEG-06. No provider-specific deployment was examined.

[47] Chandramouli, R., and Hibbard, E. Guidelines for Media Sanitization. NIST Special Publication 800-88, Revision 2, September 2025. DOI: 10.6028/NIST.SP.800-88r2. Principal locators: Sections 3.1–3.2, methods and cryptographic erase, particularly 3.2.2–3.2.3; and Section 4.5, sanitization assurance, particularly 4.5.1–4.5.2. Selected text and relevant PDF pages were examined. The book gives a bounded conceptual treatment, not a device-specific sanitization procedure, legal retention rule or certificate of completed treatment.

[48] National Institute of Standards and Technology. “Frequently Asked Questions (FAQ) for NIST SP 800-88r2,” document dated 16 July 2026, published as supplemental material on the SP 800-88r2 record. Principal locators: Questions 12–13, scope and virtual/cloud media. Public source: csrc.nist.gov/files/pubs/sp/800/88/r2/final/docs/sp800-88r2-faq.pdf. Used only for the limits of applying physical-media methods directly to virtual resources and the need to examine provider and key-management arrangements. No generalized physical-destruction prescription is adopted.

[49] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, January 2023. DOI: 10.6028/NIST.AI.100-1. Principal locators: Executive Summary; Section 1.2.4, Organizational Integration and Management of Risk; Section 3, AI Risks and Trustworthiness;

Section 5, AI RMF Core; and Appendix C, AI Risk Management and Human-AI Interaction. Selected framing and human-interaction material was consulted. The identified 2023 edition is used as a foundation, not an assertion that no subsequent development exists or that the book implements every framework outcome.

[50] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1, July 2024. DOI: 10.6028/NIST.AI.600-1. Principal locators: Section 2, Overview of Risks Unique to or Exacerbated by Generative AI; and Section 3, particularly the MEASURE 2.5 suggested actions. Selected text and the relevant evaluation table page were examined. The case's source checks, function allocation, scorecard and evaluation plan are original; no profile table or complete implementation claim is reproduced.

[51] Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X., and Hamin, M. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. NIST AI 100-2 E2025, March 2025. DOI: 10.6028/NIST.AI.100-2e2025. Principal locators: Chapters 2–3, particularly Sections 3.4, Indirect Prompt Injection, and 3.5, Security of Agents. Selected taxonomy, scope and agent-related passages were consulted. The book develops defensive system questions, not a comprehensive mitigation implementation, product vulnerability finding or claim that prompt filtering eliminates these risks. The official record also carries a potential-updates notice; a final publication audit should recheck the cited sections against any subsequently issued correction or revision.

[52] Booth, H., Souppaya, M., Vassilev, A., Ogata, M., Stanley, M., and Scarfone, K. Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile. NIST Special Publication 800-218A, July 2024. DOI: 10.6028/NIST.SP.800-218A. Principal locators: Sections 1.1, Purpose, and 1.2, Scope, with the official publication abstract. Used to connect AI model-development practices to the SSDF. Deployment and operation are outside the stated scope of this profile; it is not presented as the entire lifecycle or operational assurance basis for HAI-07 or AI-07.

[53] U.S. Department of Defense, Office of the Deputy Under Secretary of Defense (Acquisition and Technology), Systems and Software Engineering. Systems Engineering Guide for Systems of Systems, Version 1.0, August 2008. Public source: cto.mil/wp-content/uploads/2024/06/DoD-SE-for-SoS-2008.pdf. Principal locators used: the introductory definition of systems of systems, distinctions from single-system engineering, and the discussion of constituent-system management and relationships. The full guide and the official 2010 summary were checked for the Chapter 27 explanatory treatment. The source is historical guidance and does not allocate current authority for SOS-07.

[54] International Council on Systems Engineering. Systems Engineering Competency Framework (2nd Edition), official public overview. Public source: incose.org/resources-publications/publish-with-incose/competency-framework/. Consulted for its tailorable knowledge, skills, abilities and behaviours orientation. The complete framework and competency-rating material were not applied to any person or reproduced. COMP-07 is an original teaching record, not an INCOSE certification or competence assessment.

[55] National Institute of Standards and Technology. Development and Implementation of the Community Resilience Planning Guides, official project and planning-guide overview. Public source: nist.gov/community-resilience/planning-guide. Principal locators: Purpose and the overview of Volumes I and II of the Community Resilience Planning Guide for Buildings and Infrastructure Systems. Used for the relationship among social and economic functions, infrastructure dependencies and cascading consequences. The full guide's planning method was not applied; SOS-07 is not a community resilience assessment or a prescribed regional recovery design.

[56] Temoshok, D., Fenton, J., Choong, Y.-Y., Lefkovitz, N., Regenscheid, A., Galluzzo, R., and Richer, J. Digital Identity Guidelines: Authentication and Authenticator Management. NIST Special Publication 800-63B-4, July 2025. DOI: 10.6028/NIST.SP.800-63b-4. Principal locators: Section 3.1.7, multi-factor cryptographic authentication; Section 3.2.5, phishing resistance; and Section 4, authenticator event management. Official

record and selected passages examined on 9 October 2026. Appendix C selects an illustrative pattern; it does not assign an authenticator assurance level or claim complete conformity.

[57] World Wide Web Consortium. Web Authentication: An API for accessing Public Key Credentials — Level 2. W3C Recommendation, 8 April 2021. Public source: w3.org/TR/webauthn-2/. Principal locators: Sections 7.1–7.2, registration and assertion verification, with related definitions. Selected checks examined on 9 October 2026. This explicitly identified edition supplies an authentication example, not a claim that it is the newest WebAuthn level or that its use determines application authorization. No normative algorithm or source code is reproduced.

[58] Rescorla, E. The Transport Layer Security (TLS) Protocol Version 1.3. Internet Engineering Task Force, RFC 8446, August 2018. DOI: 10.17487/RFC8446. Public source: rfc-editor.org/rfc/rfc8446.html. Principal locators: Sections 2.3 and 8, early data and replay implications, with the protocol scope. Selected text examined on 9 October 2026. The illustrative branch disables early data for state-changing traffic; transport protection does not establish transaction-level authorization, source truthfulness or exactly-once effect. No RFC code component is reproduced.

Table and figure directory

The chapter tables and figures are listed in manuscript order. Entries identify the rendered page in this revision and the original teaching caption. Figures use one consistent monochrome technical style; their editable sources are included in the companion package.

Table 1.1. Decision framing for the remote-administration example 14

Figure 2.1. System context and consequential trust relationships 17

Table 2.1. Selected context relationships for the remote-administration capability 18

Table 2.2. Assumptions that affect the architecture choice 19

Table 3.1. Distinguishing the scope of familiar security instruments 22

Table 3.2. Putting engineering results back into existing work 28

Figure 4.1. Lifecycle work and continuing responsibility 30

Table 4.1. Distinguishing contributions and decision responsibilities 30

Table 5.1. Stakeholder contributions to the administration capability 38

Table 5.2. Protection needs for the selected teaching case 40

Figure 5.1. From useful capability to protection and evidence 42

Table 6.1. Choosing an analytical view 47

Figure 6.1. One loss, several causal pathways and discriminating evidence 48

Table 6.2. Scenario-to-work connections 49

Table 7.1. TE-02 definitions used by the normal-mode requirements 54

Table 7.2. Contributions needed to support SR-03 59

Table 7.3. Selected trace relationships for TE-02 61

Table 8.1. Candidate claims and the questions they require 65

Table 8.2. Selected planned verification work 67

Table 9.1. Views needed to challenge the administration architecture 75

Figure 9.1. Recovery architecture alternatives differ in authority and dependency 78

Table 9.2. Architecture alternatives and their principal unresolved contributions 78

Table 9.4. From secure-design principle to a reviewable engineering question 80

Table 9.3. Proposed TE-03 design directions for further evaluation 80

Table 10.1. IC-01: candidate receiving-side activation contract 84

Figure 10.1. Request, authority, commit, lost reply and matching retry 86

Table 10.2. Selected transaction transitions and interruption obligations 87

Table 10.3. Dependency questions for restricted restoration 89

Table 11.1. MS-01: a connected, tool-neutral model slice 95

Figure 11.1. One concern across the model and its evidence 97

Table 11.2. Model-supported impact review for the proposed delay change 98

Table 11.3. Original freshness-budget illustrations 103

Table 11.4. Shared-dependency calculation for invented snapshot probabilities 105
Table 11.5. Initial architecture screening against the selected obligations 106
Figure 11.2. Illustrative architecture preference changes with the declared weighting 108
Table 12.1. Decisions needed to complete the candidate recovery profiles 112
Table 12.2. Illustrative end-to-end restoration task allocations 113
Table 12.3. Proposed controlled-degradation and restoration states 115
Figure 12.1. Controlled-degradation and recovery-mode transitions 116
Table 13.1. RZ-01 realization information to establish 121
Table 13.2. Enabling-system influence and corresponding challenge 126
Table 14.1. Proposed integration increments for RZ-01 130
Table 14.2. Hypothetical integration findings and disciplined responses 135
Table 15.1. Evaluation purpose and limits in the teaching case 138
Figure 15.1. Verification and validation answer different questions 139
Table 15.2. DP-01: proposed transaction-evaluation sequence 139
Table 15.3. Hypothetical result interpretations—not executed evaluations 145
Table 16.1. Task and resource questions for the candidate ARC-B restoration 148
Table 16.2. TR-04: proposed transition-readiness record 153
Table 17.1. AP-05: distinguish the obligations before drafting the agreement 157
Table 17.2. Proposed evidence packages and review purposes 160
Table 17.3. Model-delivery questions for AP-05 162
Table 18.1. SC-05: contribution-centred supply-chain view 166
Table 18.2. Questions before reusing supplier evidence 169
Table 19.1. CASE-05: candidate claims and their necessary support 177
Figure 19.1. Evidence supports an argument, not a preferred answer 178
Table 19.2. Hypothetical dossier items for evidence interpretation 178
Figure 19.2. Instantiated authority claim with supporting and contrary evidence 181
Table 19.3. CASE-05 challenge ledger 182
Table 19.4. Preserve the meaning of disposition status 185
Table 20.1. Different decisions that may use the same evidence 188
Table 20.2. RR-05: a scoped risk record for the hypothetical late-commit finding 190
Table 20.3. Turn vague conditions into reviewable obligations 192
Table 21.1. OP-06: information an operating team must be able to use 198
Table 21.2. MON-06: relate signals to claims without enlarging their meaning 199
Table 21.3. Route operational learning without rewriting the evidence 204
Table 22.1. Working drift lenses and their different responses 207
Figure 22.1. A change loop that preserves the evidence history 209
Table 22.2. CR-06: focused impact and evidence-applicability review 210

Table 22.3. CR-06 completion record: different questions need different answers 213
Table 23.1. LEG-06: reconstruct the inherited contribution 217
Table 23.2. A contribution-based cloud responsibility record 220
Table 23.3. Separate exit-window illustration, using invented planning inputs 222
Table 24.1. RET-06: identify which use and obligation are ending 226
Table 24.2. RET-06 evaluation: distinguish the action from the conclusion 231
Table 24.3. Separate retained knowledge from obsolete execution capability 232
Table 25.1. Technical-management contributions to the continuing case 235
Table 25.2. TM-07: proposed work packages for the next development commitment 236
Table 26.1. Organizational support that makes the project work possible 244
Table 26.2. COMP-07: task-based routes into integrated SSE work 246
Table 27.1. SOS-07: a repair-coordination thread and its protection questions 253
Table 27.2. CA-07: a reviewable constituent contribution 254
Table 27.3. Resource-aware interpretation of the two-site example 256
Figure 27.1. Two locally feasible tasks can share one limiting resource 257
Table 28.1. Distinguish changes in an AI-supported arrangement 262
Table 28.2. HAI-07: a proposed work-system arrangement for evidence review 264
Figure 28.1. Human–AI work and action boundary 266
Table 28.3. AE-07: distinct evaluation questions for AI contributions 270
Table M.1. Representative modelling environments — dated orientation 277
Table B.1. Development of the recurring case 279
Table B.2. Principal record families and where to find them 280
Table B.3. Requirement and profile continuity 281
Table B.4. Numerical context and branch guide 282
Table C.1. Branch definitions and their evaluation consequences 285
Figure C.1. Source consistency, freshness at commit and withdrawal deadline 289
Table C.2. Stipulated system-evaluation dossier for CB-09 291
Figure D.1. Functional relationships in the inspection-vehicle example 295
Table D.1. Candidate cyber-physical requirements and evidence 296
Table D.2. Stopping-distance calculations 298
Figure D.2. Required stopping distance grows nonlinearly with bounded speed 299
Figure E.1. Information-flow protection begins with permitted content and purpose 302
Table E.1. Bounded information-flow requirements for the supplier-support example 303
Figure F.1. Resilience after loss of confidence in a previously trusted source 305
Table F.1. Bounded capability decisions when the approval source is suspect 305

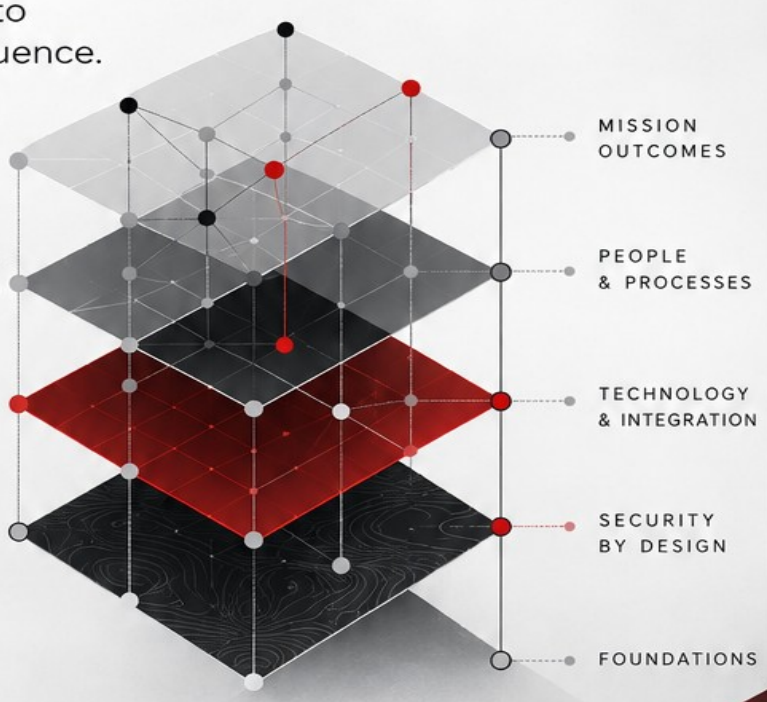
Security must be engineered, not merely appended.

PEOPLE
SYSTEMS
SECURITY
INTEGRATION
ASSURANCE
OUTCOMES

The Security-Engineering Convergence argues that trustworthy systems do not emerge from compliance activity alone, nor from engineering performed without adequate attention to security, resilience, and mission consequence. This book brings those worlds together.

Written for engineers, cybersecurity practitioners, acquisitiom professionals, and decision-makers, it shows how systems security engineering connects stakeholder needs, adversity, design choices, implementation, evaluation, assurance, and accountable decisions across the life cycle.

Through worked examples, practical methods, model-based reasoning, cyber-physical analysis, and reusable worksheets, the book helps readers move from controls and assessments toward disciplined engineering and justified confidence.



A PRACTICAL TECHNICAL REFERENCE FOR ENGINEERING TRUSTED CAPABILITIES.

COMPLEX SYSTEMS. TRUSTED CAPABILITIES.

Openly licensed under CC BY 4.0
Author: Peter J. Hillier
Hillier Information Protection Solutions